Transformari Geometrice 2D

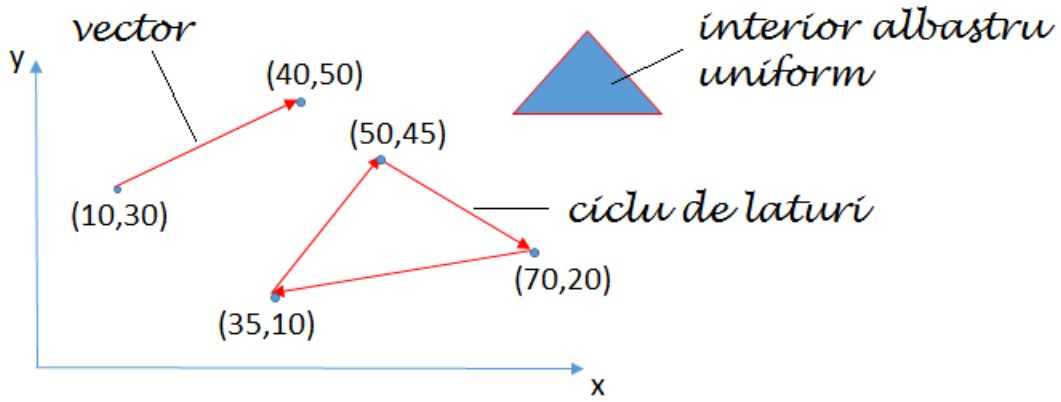
Prof. univ. dr. ing. Florica Moldoveanu
Conferentiar dr. ing. Anca Morar

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2018-2019

Transformari geometrice(1)

Obiectele 2D/3D sunt reprezentate prin:

- Date geometrice: coordonatele varfurilor, raportate la un sistem de coordonate carteziene 2D sau 3D;
- Atribute topologice, care definesc modul in care sunt conectate varfurile (laturi, ciclul de laturi al unei feţe, ş.a.);
- Atribute de aspect: culoarea, tipul de interior pentru suprafete 2D, atribute de material pentru suprafete 3D (ex: reflexia/refractia luminii de catre suprafata), s.a.



Transformarile geometrice se aplica (coordonatelor) varfurilor obiectului si nu afecteaza atributele sale!

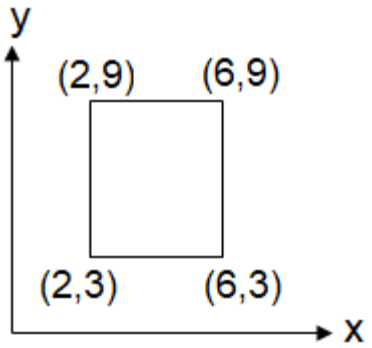
Transformari geometrice (2)

- Sunt operatii fundamentale in sinteza imaginilor
- Folosite pentru:
  - Redarea desenelor la diferite marimi
  - Compunerea desenelor sau a scenelor 3D
  - Realizarea animatiei
  - Transformarea obiectelor dintr-un spatiu de coordinate logic, in care sunt definite, in spatiul fizic de afisare
  - Etc.

Transformari geometrice 2D (1)

Sunt transformari ale obiectelor definite in sistemul de coordonate carteziene XOY, numite şi obiecte 2D. Fiecare vârf al unui obiect 2D este definit printr-o pereche de coordonate (x,y).

Transformari geometrice elementare



- Translaţia
- Scalarea faţă de origine
- Rotaţia faţă de origine
- Forfecarea faţă de origine
- Oglindirile faţă de axele principale şi faţă de origine

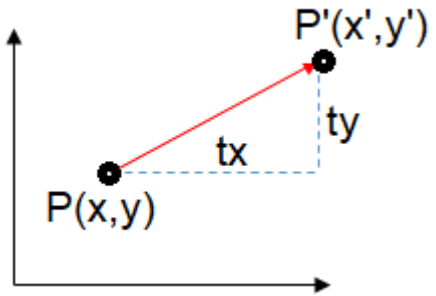
Orice alta transformare se obtine prin compunrea a 2 sau mai multe transformari elementare

- Scalarea /rotaţia fata de un punct oarecare din plan
- Scalare compusa cu translatie, etc

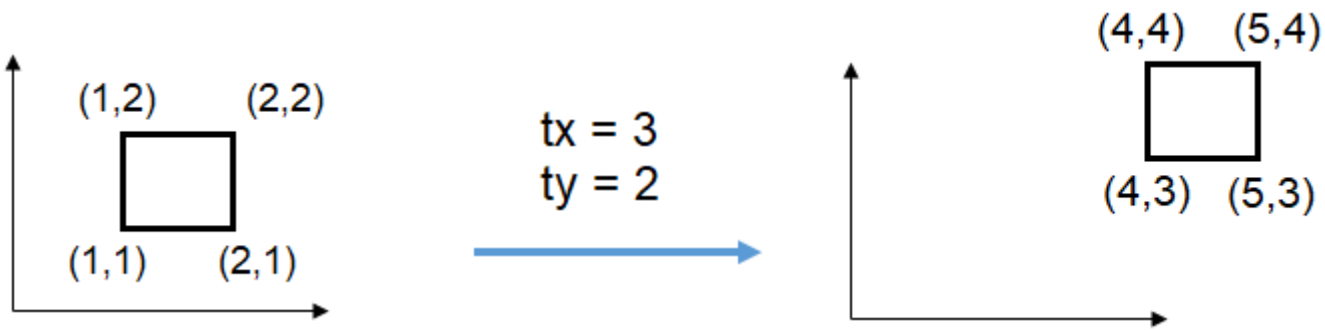
Transformari geometrice 2D elementare (1)

- Translatia (1)

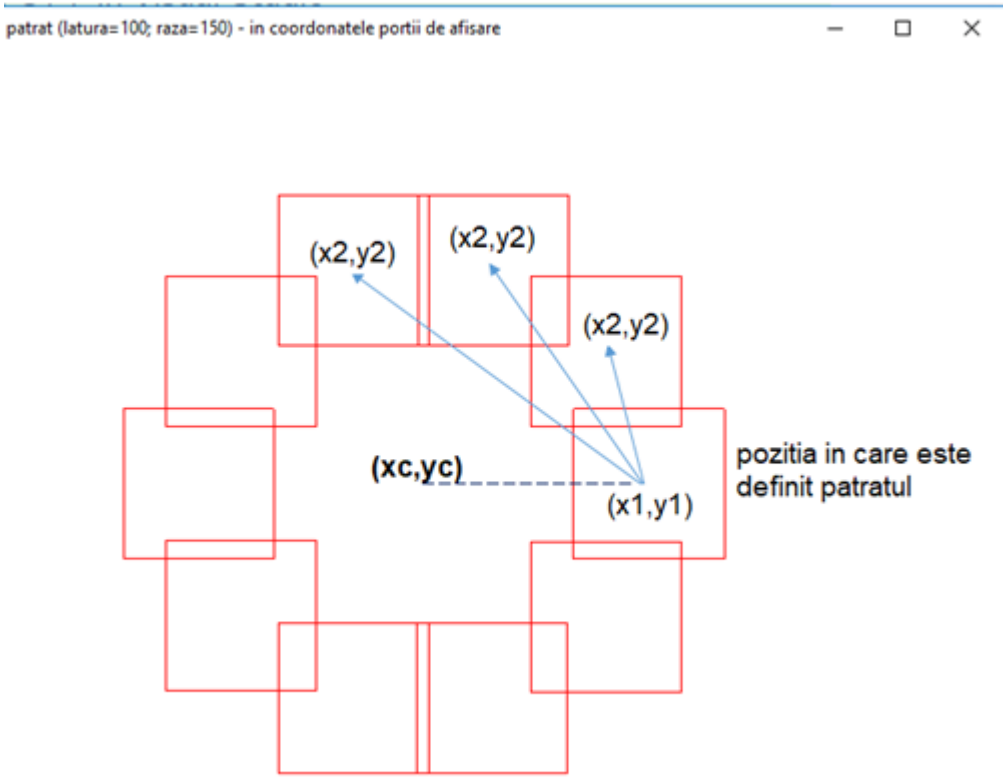
  Este definita printr-un vector, T[tx,ty].

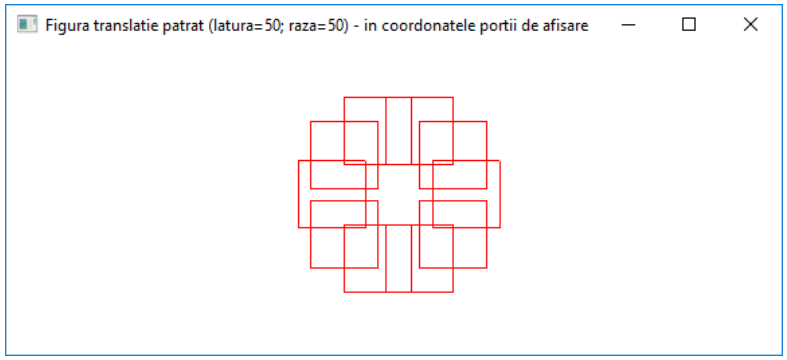


$$x' = x + tx$$
$$y' = y + ty$$



Translatia unui patrat pe circumferinta unui cerc






```
//campuri ale clasei DesenTranslatiePatrat
int raza, xc, yc, latura;

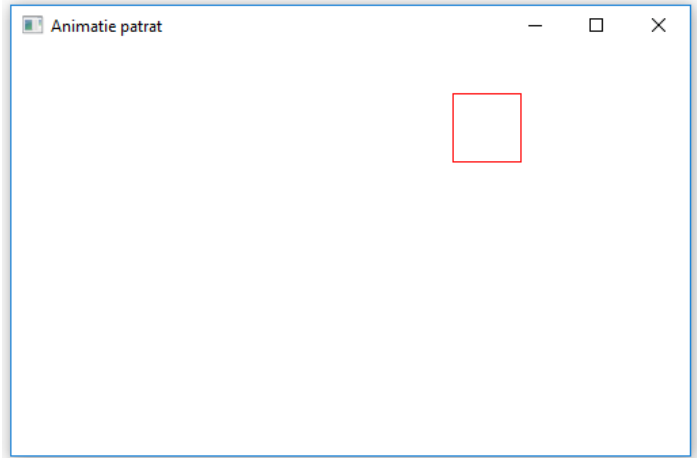
void DesenTranslatiePatrat::Update(float  Seconds)
{ //functia deseneaza cadrul imagine curent
  glm::ivec2 resolution = window->GetResolution();
  xc = resolution.x / 2;
  yc = resolution.y / 2;

  float x1 = xc + raza, y1 = yc, x2, y2;
  for (int i = 0; i < 10; i++)
  { // distanta unghiulara intre puncte: 6.28/10
      x2 = xc + raza * cos(0.628f*i);
      y2 = yc + raza * sin(0.628f *i);

    // seteaza matricea de transformare a varfurilor
    modelMatrix =
            Transform2D::Translate(x2 - x1, y2 - y1);

    // se cere desenarea patratului
     RenderMesh2D(meshes["square"],
            shaders["VertexColor"], modelMatrix);
  }
}
```

Animatia translatiei patratului



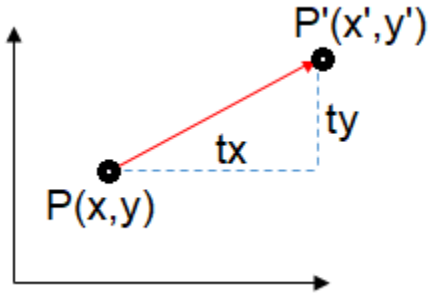

```
//campuri ale clasei DesenTranslatiePatrat
int raza, xc, yc,latura, angle;

void AnimatieTranslatiePatrat::Update(float deltaSeconds)
{ //functia deseneaza cadrul imagine curent

  glm::ivec2 resolution = window->GetResolution();
  xc = resolution.x / 2;
  yc = resolution.y / 2;
  float x1 = xc + raza, y1 = yc, x2, y2;
  angle += 0.5 * deltaSeconds;
  x2 = xc + raza * cos(angle); y2 = yc + raza * sin(angle);
   // seteaza matricea de transformare a varfurilor
   modelMatrix = Transform2D::Translate(x2 - x1, y2 - y1);
  //  se cere desenarea patratului translatat cu unghiul angle
 // pe circumferinta cercului
  RenderMesh2D(meshes["square1"],
                    shaders["VertexColor"], modelMatrix);
}
```

Transformari geometrice 2D elementare (2)

- Translatia (2)



$$x' = x + tx$$
$$y' = y + ty$$

Se doreste o reprezentare matriciala a transformarii, pentru compunerea sa cu alte transformari: compunerea transformarilor se realizeaza prin inmultirea matricilor.

Un punct din plan, P(x,y), se reprezintă în coordonate carteziene printr-un vector [x, y] sau $\begin{bmatrix} x \\ y \end{bmatrix}$

→ Nu exista o matrice M, de 2x2, astfel incat sa exprimam translatia prin:

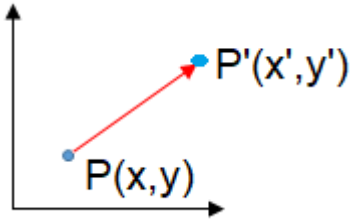
[x', y'] = [x, y] * M sau $\begin{bmatrix} x' \\ y' \end{bmatrix} = M * \begin{bmatrix} x \\ y \end{bmatrix}$

Transformari geometrice 2D elementare (3)

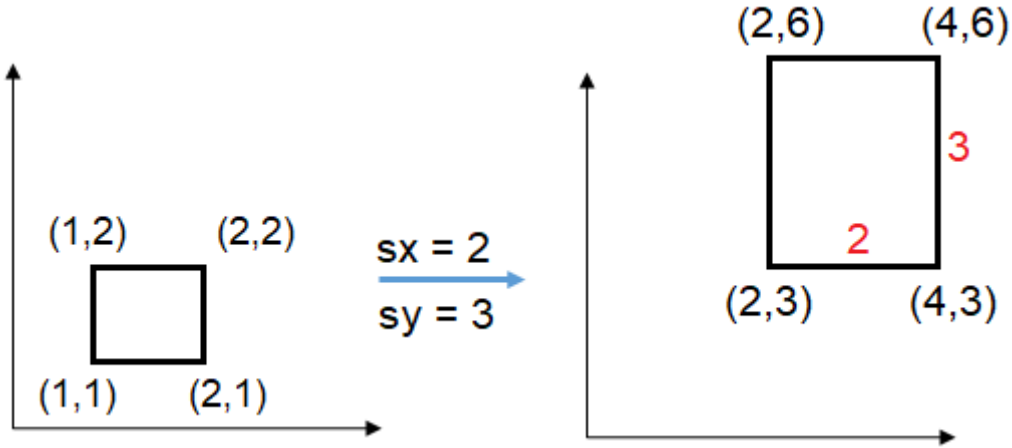
Scalarea fata de origine

Este definita prin 2 numere reale, de regula pozitive:

- sx - scalarea pe axa OX
- sy - scalarea pe axa OY



x' = x * sx
y' = y * sy



Efecte:

- Daca sx sau sy sau ambii sunt >1:.....
- Daca sx sau sy sau ambii sunt <1:.....
- ➢ sx = sy , scalare uniforma

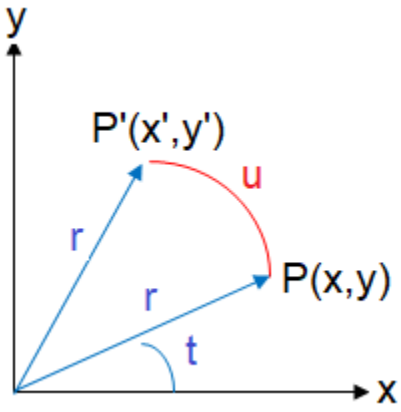
Reprezentarea matriciala

$$[x' \;\; y'] = [x \;\; y] * \begin{bmatrix} sx & 0 \\ 0 & sy \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} sx & 0 \\ 0 & sy \end{bmatrix} * \begin{bmatrix} x \\ y \end{bmatrix}$$

Transformari geometrice 2D elementare (3)

Rotaţia fata de origine



x = r*cos(t)
y = r*sin(t)

Relatia dintre coordonatele carteziene si coordonatele polare ale unui punct P(x,y)

x' = r*cos(t+u) = r*(cos(t)*cos(u) – sin(t)*sin(u)) = x*cos(u) – y*sin(u)
y' = r*sin(t+u) = r*(cos(t)*sin(u) + sin(t)*cos(u)) = x*sin(u) – y*cos(u)

Rotatia fata de origine

x' = x*cos(u) - y*sin(u)
y' = x*sin(u) + y*cos(u)

$$[x' \;\; y'] = [x \;\; y] * \begin{bmatrix} \cos(u) & \sin(u) \\ -\sin(u) & \cos(u) \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos(u) & -\sin(u) \\ \sin(u) & \cos(u) \end{bmatrix} * \begin{bmatrix} x \\ y \end{bmatrix}$$

Transformari geometrice 2D fata de un punct oarecare din plan(2)

Scalarea fata de un punct oarecare din plan

- Punctul fix al transformării este un punct oarecare F(xf,yf) – coordonatele sale nu se modifica prin aplicarea transformării.
- P(x,y) este punctul transformat
- Scalarea se aplica vectorului FP [x-xf, y-yf]:

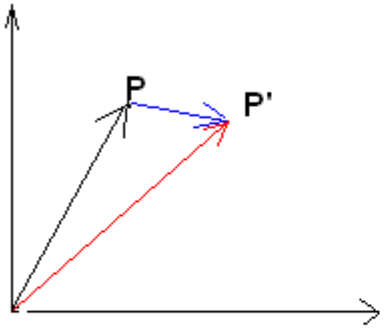
x’ – xf = sx*(x-xf)

y’ – yf = sy*(y – yf)

Rezultă:

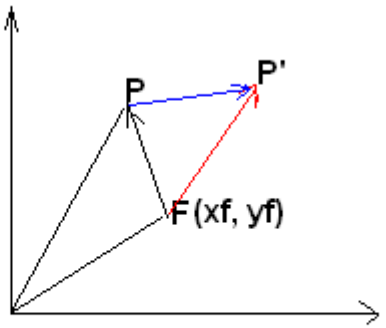
x’ = x*sx + xf –xf*sx

y’ = y*sy + yf –yf*sy



Scalarea fata de origine

x' - 0 = sx (x - 0)
y' - 0 = sy (y - 0)



Scalarea fata de un punct F din plan

x' - xf = sx (x - xf)
y' - yf = sy (y - yf)

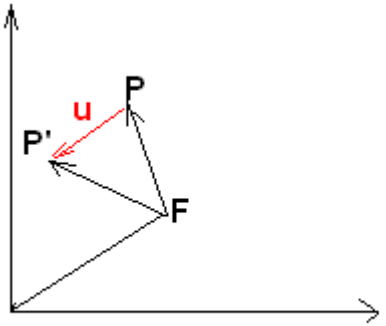
Nu poate fi exprimata în coordonate carteziene printr-o matrice de 2x2!

Transformari geometrice 2D fata de un punct oarecare din plan(2)

Rotatia fata de un punct oarecare din plan

- Punctul fix al transformarii este F(xf,yf)
- P(x,y) este punctul transformat
- Rotatia se aplica vectorului FP [x-xf, y-yf], in jurul lui F:

$$x' - xf = (x-xf)* \cos(u) - (y - yf) * \sin(u)$$
$$y' - yf = (x-xf)* \sin(u) + (y - yf) * \cos(u)$$

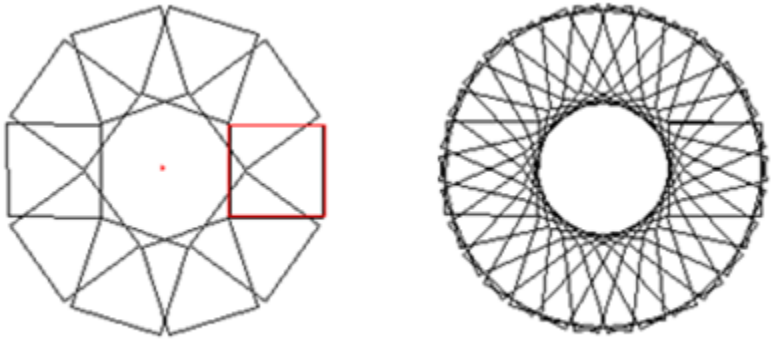


Rotatia fata de un punct F din plan

Rezultă:

$$x' = x*\cos(u) - y*\sin(u) + xf - xf*\cos(u) + yf*\sin(u)$$
$$y' = x*\sin(u) + y*\cos(u) + yf - xf*\sin(u) - yf*\cos(u)$$

Nu poate fi exprimata in coordonate carteziene printr-o matrice de 2x2!


Rotatia unui patrat in jurul unui punct din plan

Compunerea transformarilor geometrice 2D

De ce este necesară?

- Pentru a putea aplica o singura transformare care înglobeaza o secvenţă de transformari elementare, în locul aplicării unei succesiuni de transformări elementare; de ex., se aplică tuturor vârfurilor o transformare care înglobează scalare (S), rotatie (R) si translatie (T) în loc să se aplice fiecarui varf secventa de transformari elementare

  Pt = M * P în loc de P' = S * P; P" = R * P'; Pt = T * P" unde M = T * R * S

- Matricea unei transformari compuse se obtine prin înmultirea matricilor transformarilor elementare. Exemplu:

RS # SR

SR = S*R =

$$\begin{bmatrix} \cos(u) & \sin(u) \\ -\sin(u) & \cos(u) \end{bmatrix} \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$$

RS = S*R =

$$\begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} \cos(u) & \sin(u) \\ -\sin(u) & \cos(u) \end{bmatrix}$$

- **Translatia nu poate fi reprezentata în coordonate carteziene printr-o matrice!**
- **Aceasta impune reprezentarea transformarilor în coordonate omogene.**

Reprezentarea transformarilor geometrice 2D în coordonate omogene(1)

Coordonate omogene:

Un punct din plan, P(x,y), se reprezinta in coordonate omogene printr-un vector

[xw, yw, w] sau $\begin{bmatrix} xw \\ yw \\ w \end{bmatrix}$ xw = x * w; yw = y * w; w – orice numar real

Exemplu: P(2, 0.5) → [2, 0.5, 1], [4, 1, 2], [20, 5, 10]

Transformarea din coordonate omogene in coordonate carteziene:

[xw yw w] → P(x, y), unde:

- pentru w #0, x = xw/w, y = yw/w
- pentru w =0, P este un punct la infinit

Reprezentarea transformarilor elementare în coordonate omogene

Translația

$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ t_x & t_y & 1 \end{bmatrix} \quad \text{sau} \quad \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

In OpenGL se folosesc vectori coloană pentru a reprezenta puncte din plan sau din spaţiu.

Scalarea față de origine

$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad \text{sau} \quad \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Rotația fața de origine

$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} \cos(u) & \sin(u) & 0 \\ -\sin(u) & \cos(u) & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad \text{sau} \quad \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos(u) & -\sin(u) & 0 \\ \sin(u) & \cos(u) & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Transformarile inverse ale transformarilor elementare

T(tx, ty): matricea translatiei, in coordonate omogene

S(0, 0, sx, sy): matricea scalarii fata de origine, in coordonate omogene

R(0,0,u): matricea rotatiei fata de origine, in coordonate omogene

Transformarile inverse sunt:

$T(tx, ty)^{-1} = T(-tx, -ty)$

$S(0, 0, sx, sy)^{-1} = S(0, 0, 1/sx, 1/sy)$

$R(0,0, u)^{-1} = R(0, 0, -u)$

Transformari geometrice 2D compuse(1)

Exemple de transformari compuse: scalarea/rotatia fata de un punct oarecare din plan.

- Expresiile matematice ale scalării şi rotaţiei faţă de un punct oarecare din plan se pot obţine prin compunerea următoarelor transformări:
  - Translaţia prin care punctul fix al transformării ajunge în origine: T(-xf, -yf);
  - Scalarea / rotaţia faţă de origine: S(0,0,sx,sy)/R(0,0,u);
  - Translaţia inversă celei de la punctul 1: T(xf, yf).

Transformari geometrice 2D compuse(2)

Scalarea faţă de punctul F(xf,yf)

$$[x'\ y'\ 1] = [x\ y\ 1] \underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -x_f & -y_f & 1 \end{bmatrix}}_{T(-xf,\ -yf)} \underbrace{\begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{S(0,\ 0,\ sx,\ sy)} \underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ x_f & y_f & 1 \end{bmatrix}}_{T(xf,\ yf)}$$

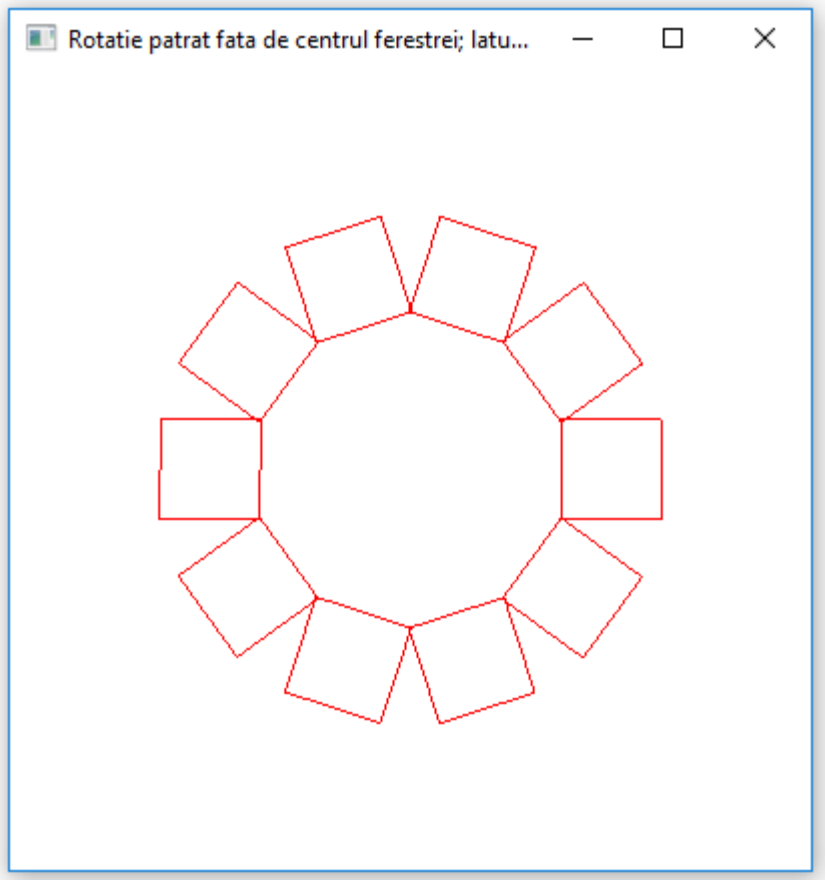
Matricea transformarii compuse:

M = T(-xf, -yf) * S(0, 0, sx, sy) * T(xf, yf)

Aplicarea transformarii compuse:

[x' y' 1] = [x y 1] * M

Rotatia unui patrat in jurul centrului ecranului




```
//campuri ale clasei DesenTranslatiePatrat
int raza, xc, yc, latura;

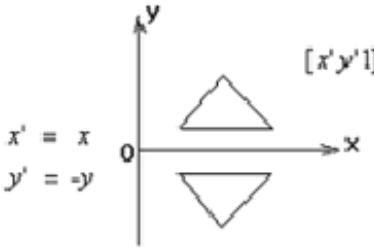
void DesenRotatiePatrat::Update(float  Seconds)
{ //deseneaza cadrul imagine curent
  glm::ivec2 resolution = window->GetResolution();
  xc = resolution.x / 2;
  yc = resolution.y / 2;

for (int i = 0; i < 10; i++)
  {
    // seteaza matricea de transformare a varfurilor
    modelMatrix = Transform2D::Translate(xc, yc) *
                  Transform2D::Rotate(0.628f * i) *
                  Transform2D::Translate(-xc, -yc);
   //  deseneaza patratul rotit fata de (xc,yc)
    RenderMesh2D(meshes["square1"],
                 shaders["VertexColor"], modelMatrix);
  }
 }
```

Alte transformari geometrice 2D(1)

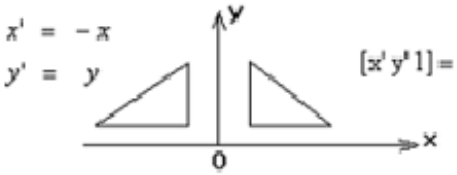
Oglindirea

Fata de axa OX



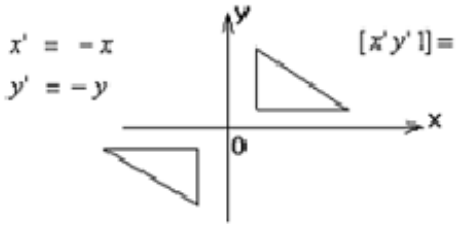
$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ sau } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Fata de axa OY



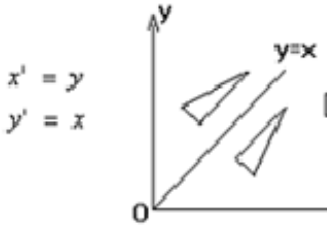
$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ sau } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Fata de origine



$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ sau } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Fata de dreapta x=y



$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ sau } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Alte transformari geometrice 2D(2)

Oglindirea faţă de o dreaptă oarecare

Se exprimă ca transformare compusa prin înmulţirea matricilor care exprima urmatoarele transformari:

1. O translaţie, astfel încât dreapta sa treaca prin origine.

2. O rotaţie faţă de origine, a.î. dreapta să se suprapună peste una dintre axele principale.

3. Oglindirea faţă de axa principală peste care a fost suprapusă dreapta.

4. Rotaţia inversă celei de la punctul 2.

5. Translaţia inversă celei de la punctul 1.

În notaţie matricială:

M = T * R* O* R^{-1} *T^{-1} (folosind vectori linie) **sau M = T^{-1} * R^{-1} *O *R *T** (folosind vectori coloana)

Deduceti elementele matricilor T, R, O, atunci cand dreapta este data printr-un punct, (xd, yd) si o directie, D[a, b].

Alte transformari geometrice 2D(3)

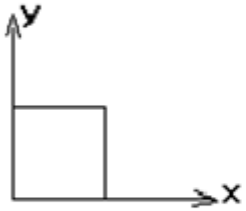
Forfecarea

Este definita prin 2 numere reale:

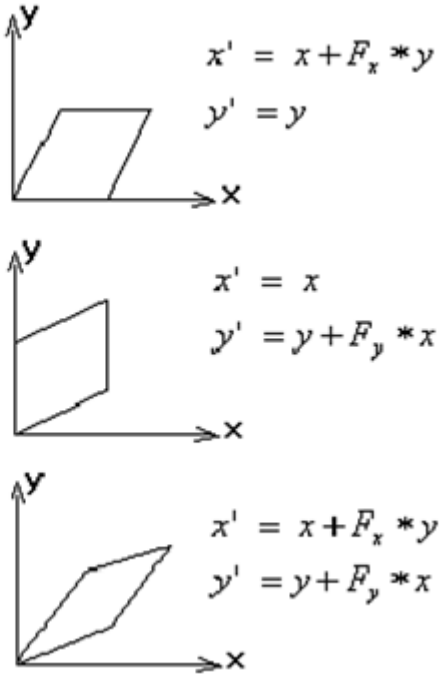
Fx: factorul de forfecare pe axa OX

Fy: factorul de forfecare pe axa OY

Deduceti formele matriciale ale transformarilor de forfecare



Forfecarea fata de origine



Forfecarea fata de un punct oarecare din plan, (xf,yf) , exprimata ca transformare compusa:

1. Translatie prin care punctul (xf, yf) ajunge in origine
2. Forfecarea fata de origine
3. Translatia inversa celei de la pasul 1

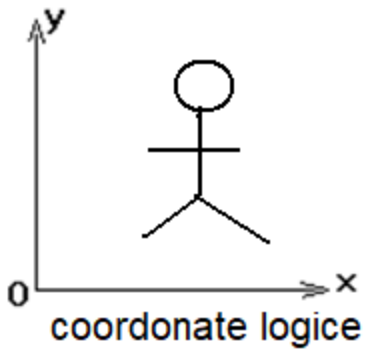
Transformarea de vizualizare 2D

Prof. univ. dr. ing. Florica Moldoveanu
Conf. dr. ing. Anca Morar

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2018-2019

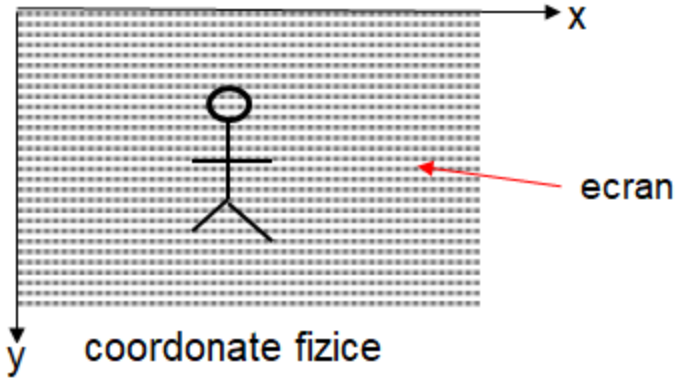
Coordonate logice şi coordonate fizice

- Desenele reprezentate într-un program de aplicatie grafica 2D sunt, de regula, raportate la un sistem de coordonate diferit de cel al suprafetei de afisare.
- Exemple: planul unei case, un desen tehnic, graficul unei functii, etc.



(metri, milimetri, viteza, timp, etc.)

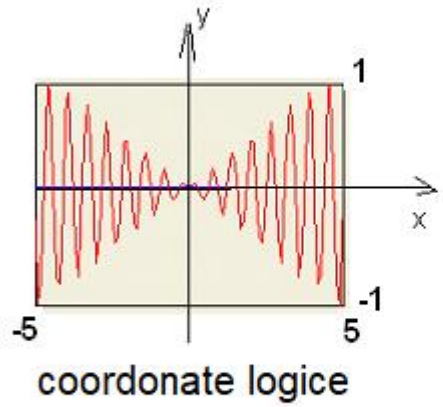
Sistemul de coordonate utilizator

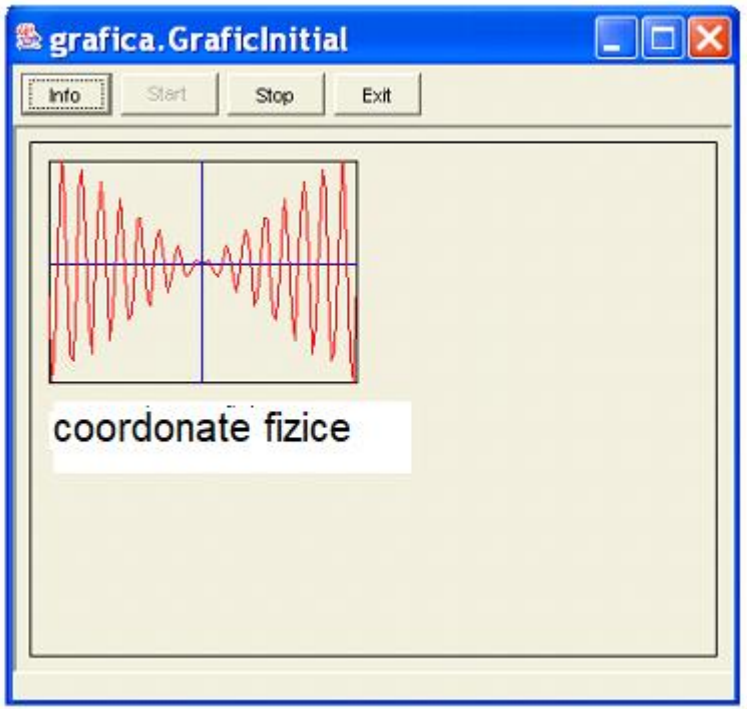


(pixeli, milimetri, s.a.)

Sistemul de coordonate dispozitiv

Transformarea coordonate logice - coordonate fizice



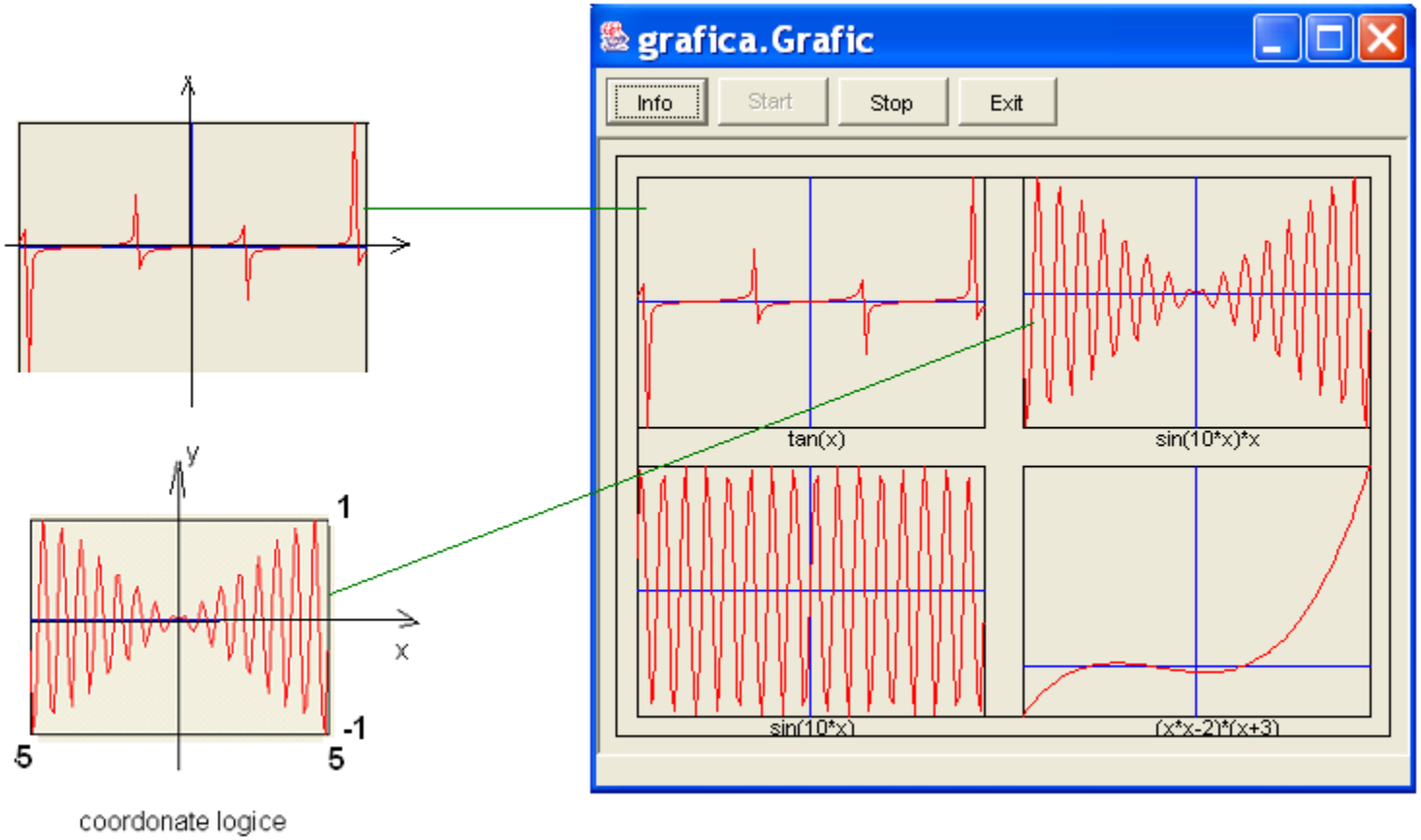


Aplicatia defineste graficul functiei in sistemul de coordonate utilizator (logice).

Coordonatele punctelor de pe graficul afisat pe ecran sunt adrese de pixeli.

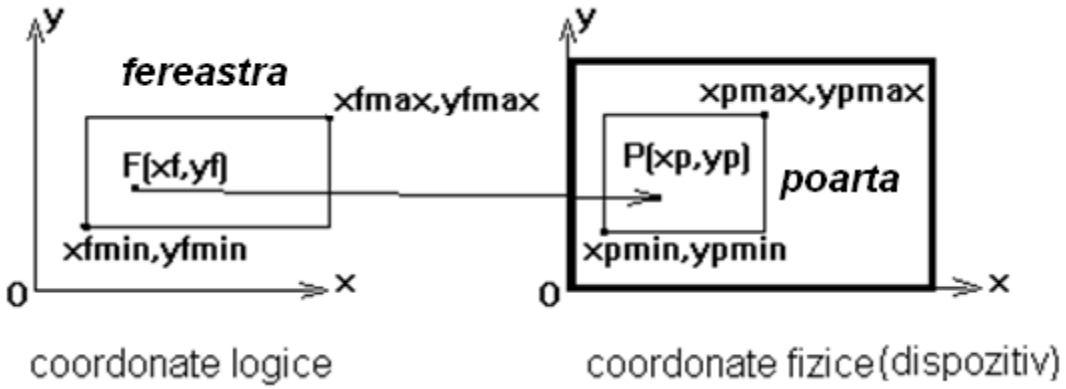
Coordonatele logice trebuie transformate in coordonate fizice.

Transformarea de vizualizare 2D (1)



Este necesara o transformare generala, prin care un desen definit in coordinate logice sa poata fi afisat intr-o zona oarecare a ecranului.

Transformarea de vizualizare 2D (2)
- transformarea "fereastra-poarta"-



- fereastra: zona din spatiul coord. logice al carei continut se va afisa
- poarta: zona din spatiul coord. fizice in care se va afisa continutul ferestrei

▪ Transformarea este definita folosind 2 dreptunghiuri, definite in cele 2 sisteme de coordonate, numite: ***fereastra (de vizualizare)*** si ***poarta (de afisare).***

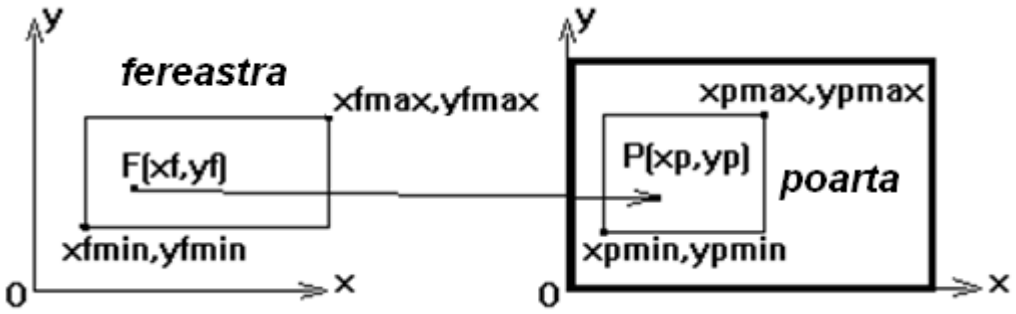
• Transformarea se mai numeste ***transformarea fereastra-poarta.***

F: un punct din fereastra.
P: punctul in care se transforma F prin transformarea fereastra-poarta.

➢Transformarea fereastra-poarta trebuie sa asigure ca:

" pozitia relativa a lui P in poarta de afisare este aceeasi cu pozitia relativa a lui F in fereastra".

Transformarea de vizualizare 2D (3)



$$\frac{xp - xp\,\min}{xp\,\max - xp\,\min} = \frac{xf - xf\,\min}{xf\,\max - xf\,\min}$$

$$\frac{yp - yp\,\min}{yp\,\max - yp\,\min} = \frac{yf - yf\,\min}{yf\,\max - yf\,\min}$$

$$sx = \frac{xp\,\max - xp\,\min}{xf\,\max - xf\,\min}$$

$$sy = \frac{yp\,\max - yp\,\min}{yf\,\max - yf\,\min}$$

- sx, sy reprezinta factorii de scalare ai transformarii
- tx, ty depind de pozitiile celor 2 ferestre fata de originea sistemului de coord. in care sunt definite

şi $tx = xpmin - sx*xfmin$ $ty = ypmin - sy*yfmin$

$$xp = xf * sx + tx$$
$$yp = yf * sy + ty$$

← Exprimarea matematica a transformarii fereastra – poarta, considerand o aceeasi orientare a axelor celor 2 sisteme de coordonate.

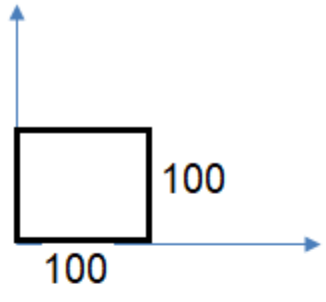
Transformarea de vizualizare 2D (4)

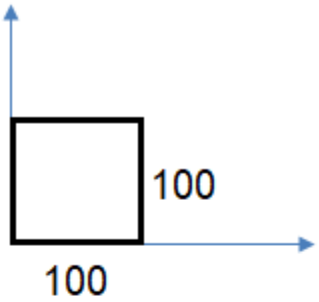
$$sx = \frac{xp\max - xp\min}{xf\max - xf\min}$$

$$sy = \frac{yp\max - yp\min}{yf\max - yf\min}$$

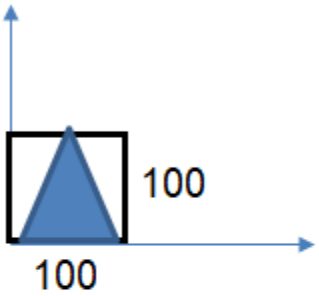
*tx = xpmin- sx*xfmin*

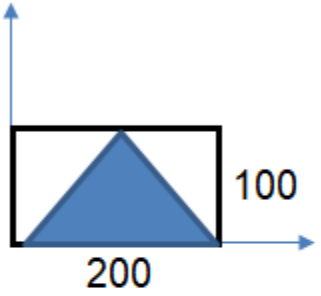
*ty= ypmin- sy*yfmin*



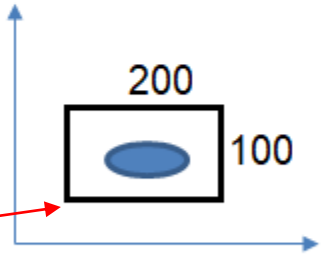


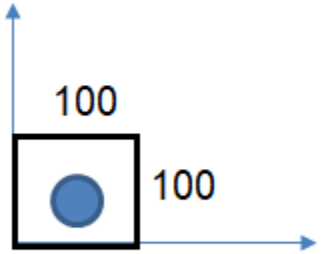
sx=1, sy = 1
tx = 0, ty =0



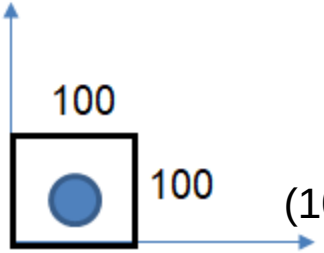


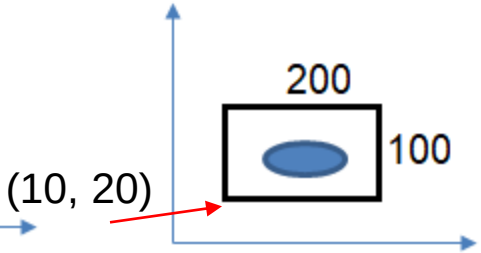
sx=2, sy = 1
tx = 0, ty =0





sx=0.5, sy = 1
tx = 0 - 0.5*10 = - 5
ty = 0 – 1*20 = -20

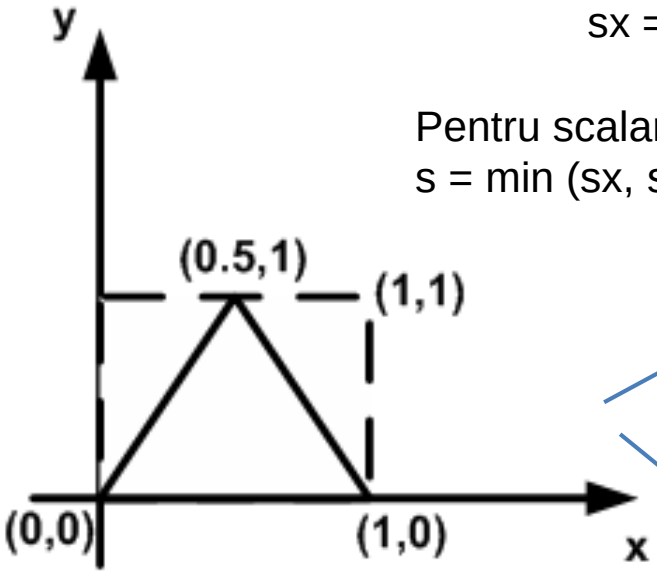




sx=2, sy = 1
tx = 10
ty = 20

Transformarea de vizualizare 2D (5)

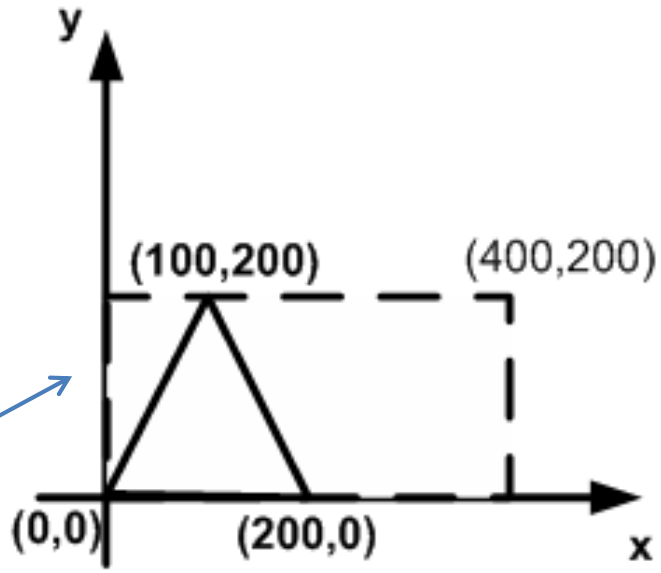
Scalare uniforma
cu afisare centrata in poarta de vizualizare



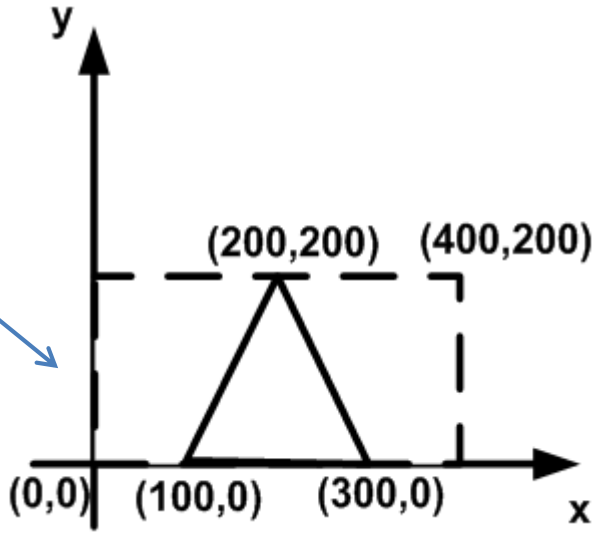
sx = 400, sy = 200

Pentru scalare uniforma:
s = min (sx, sy)

tx=0, ty =0



tsx = 100,
tsy =0



tsx =(xpmax – xpmin – s*(xfmax – xfmin))/2

= (400 – 200*1) /2 = 100

tsy =(ypmax – ypmin – s*(yfmax – yfmin))/2

= (200 – 200*1) /2 = 0

Transformarea de vizualizare 2D (7)

Efectele transformarii

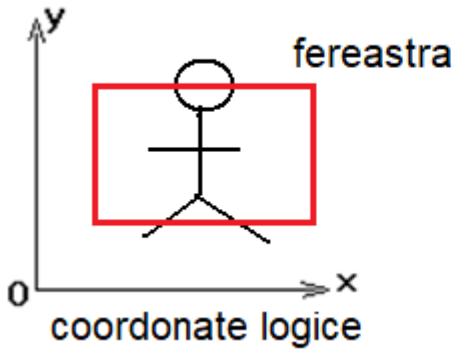
- Marire / micsorare, in functie de dimensiunile ferestrei si ale portii
- Deformare: fereastra si poarta nu sunt dreptunghiuri asemenea
- Pentru scalare uniforma: s = min(sx,sy)
- Afisare centrata in poarta: translatie suplimentara pe axa ox sau pe axa oy:

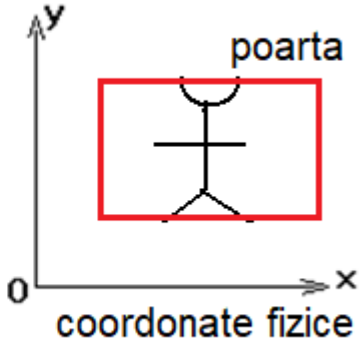
  tsx =(xpmax – xpmin – s*(xfmax – xfmin))/2

  tsy =(ypmax – ypmin – s*(yfmax – yfmin))/2

  Translatia totala: tx+tsx, ty+tsy

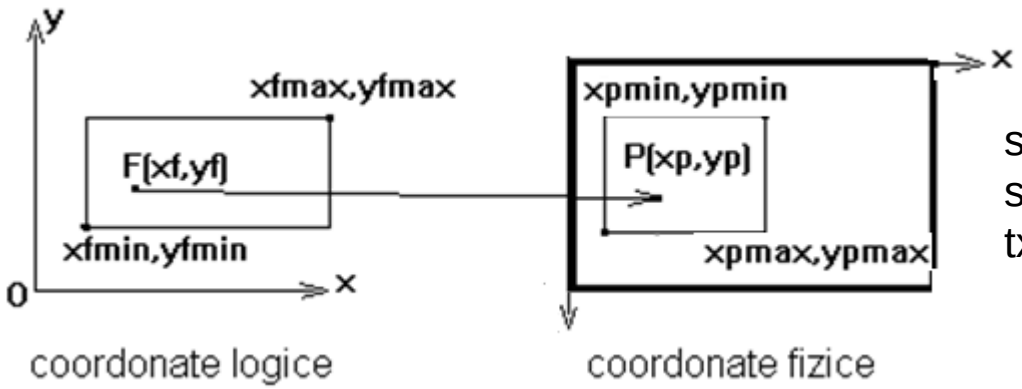
- Decuparea primitivelor aflate in afara ferestrei vizuale

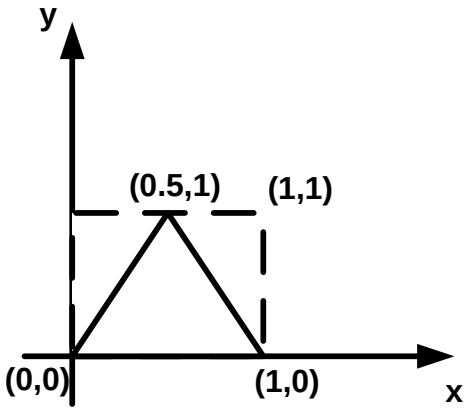




Transformarea de vizualizare 2D (8)

Corectia coordonatei yp
tinand cont de orientarea axelor sist. coord fizice





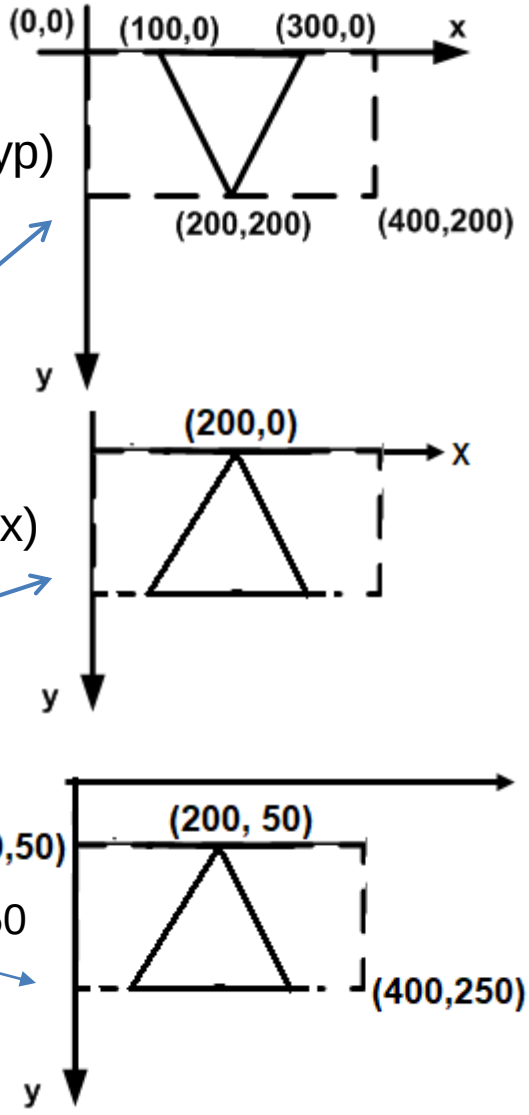
(xp, yp)

sx = 400, sy = 200
s=smin = 200
tx = tsx =100, ty =0

(xp, yp'= -yp+ypmax)

ty = 50

yp' = -yp + ypmax + ypmin



Transformarea de vizualizare 2D (9)

Formulele transformarii fereastra – poarta atunci cand axa OY a sistemului de coordinate fizice este orientata in jos:

xp = xf * sx + tx
yp = – (yf * sy + ty) + ypmin + ypmax = -yf * sy + ypmin + ypmax -ty

Exprimarea matriciala a transformarii fereastra-poarta in cazul corectiei coordonatei y

$$\begin{bmatrix} xp \\ yp \\ 1 \end{bmatrix} = M * \begin{bmatrix} xf \\ yf \\ 1 \end{bmatrix}$$

$$M = \begin{bmatrix} sx & 0 & tx \\ 0 & -sy & yp\min + yp\max - ty \\ 0 & 0 & 1 \end{bmatrix}$$

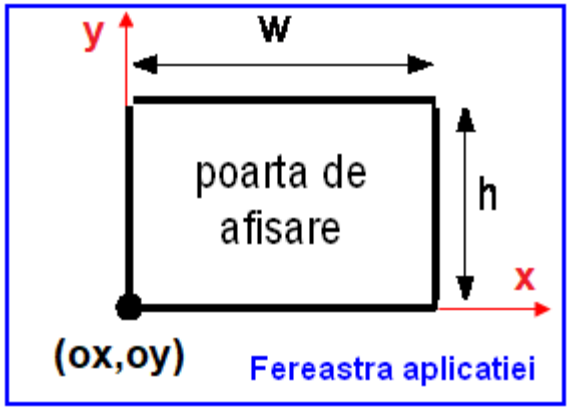
Implementarea transformarii de vizualizare în OpenGL (1)

- Pentru setarea zonei de afisare, în OpenGL se apeleaza

  glViewport(ox, oy, width, height);

unde (ox,oy) sunt coord. coltului stanga jos al viewportului, in coordonatele ferestrei aplicatiei.

- Orientarea axelor viewportului este aceeasi cu a sistemului coordonatelor logice:
  - Nu este necesara corectia coordonatei y → matricea transformarii de vizualizare este:



$$M = \begin{bmatrix} sx & 0 & tx \\ 0 & sy & ty \\ 0 & 0 & 1 \end{bmatrix}$$

- Se poate defini un desen in coordonate logice.
- Pentru obtinerea desenului in coordonate ecran se foloseste ca matrice de transformare a vârfurilor matricea transformării de vizualizare

Implementarea transformarii de vizualizare in OpenGL (2)

```
class TransformareaFereastraPoartaPatrate : public SimpleScene
{
……..
    protected:
    float length;
    ViewportSpace viewSpace; // dreptunghiul “poarta de afisare”
    LogicSpace logicSpace; // dreptunghiul “fereastra”
    glm::mat3 modelMatrix,
          visMatrix; //matricea transformarii de vizualizare
    bool uniform; // transformare uniforma/neuniforma
 // figura desenata se obtine prin rotatia unui patrat  in jurul centrului ferestrei
    float raza; // raza cercului pe care se roteste patratul
    float cx, cy; // centrul cercului
 };
```

Implementarea transformarii de vizualizare in OpenGL (3)

```
void TransformareaFereastraPoartaPatrate::Init()
{ ……..
   logicSpace.x = -5; // logic x
   logicSpace.y = -5; // logic y
   logicSpace.width = 10; // logic width
   logicSpace.height = 10; // logic height
   length = 2.0f; //latura patratului
   raza = 3.6f;

   cx = logicSpace.x + logicSpace.width / 2;
   cy = logicSpace.y +  logicSpace.height / 2;


   glm::vec3 corner = glm::vec3(cx - length / 2 + raza , cy - length / 2, 0);

   Mesh* square = Object2DPatrate::CreateSquare("square", corner, length, glm::vec3(1, 0, 0));

   AddMeshToList(square); // patratul care se roteste


   glm::vec3 cornerw = glm::vec3(logicSpace.x, logicSpace.y, 0);
   Mesh* window =
        Object2DPatrate::CreateSquare("window", cornerw, logicSpace.width, glm::vec3(0, 0, 0));
   AddMeshToList(window); // cadrul ferestrei

   uniform = true;
   }
```

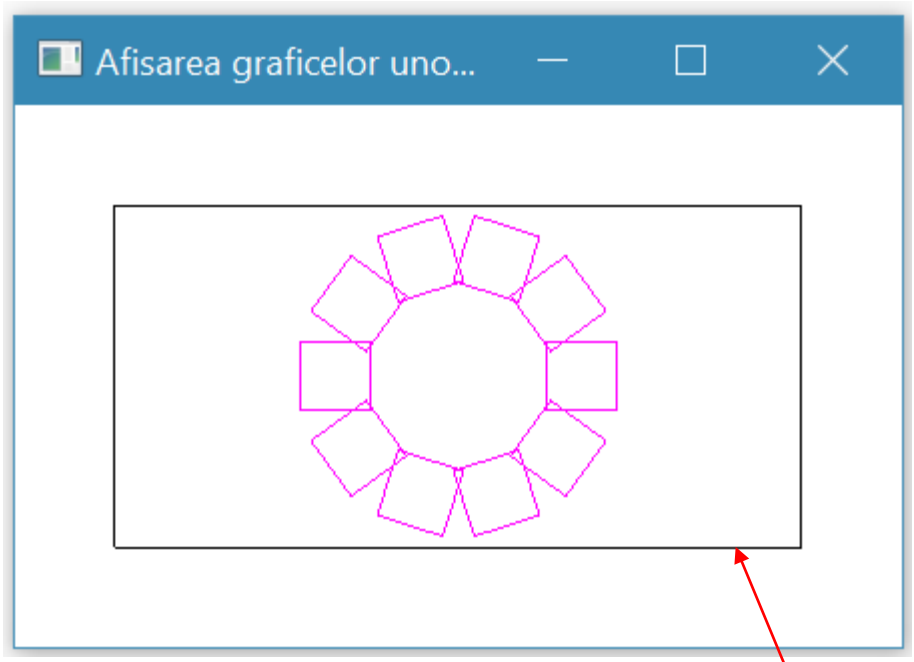
Implementarea transformarii de vizualizare in OpenGL (4)

Calculul matricei transformarii de vizualizare

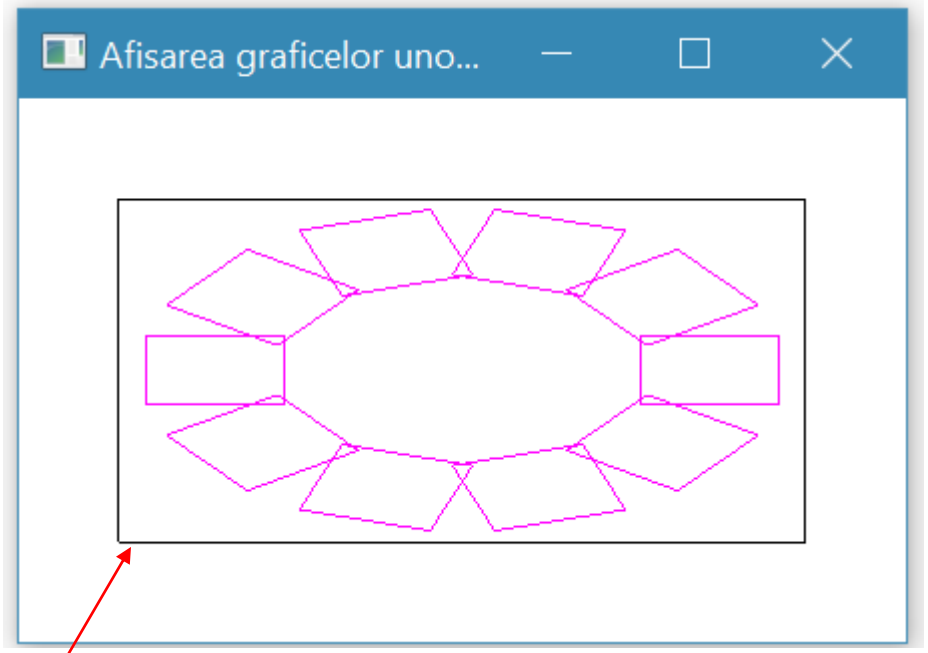
```
glm::mat3 TransformareaFereastraPoartaPatrate::VisualizationTransf2D
                (const LogicSpace & logicSpace, const ViewportSpace & viewSpace)
{   float sx, sy, tx, ty;
    sx = viewSpace.width / logicSpace.width;  sy = viewSpace.height / logicSpace.height;
    tx = viewSpace.x - sx * logicSpace.x;  ty = viewSpace.y - sy * logicSpace.y;
    return glm::transpose(glm::mat3(sx, 0.0f, tx,0.0f, sy, ty,0.0f, 0.0f, 1.0f));
}

glm::mat3 TransformareaFereastraPoartaPatrate::VisualizationTransf2DUnif(const LogicSpace
                                        & logicSpace, const ViewportSpace & viewSpace)
{   float sx, sy, tx, ty, smin;
    sx = viewSpace.width / logicSpace.width; sy = viewSpace.height / logicSpace.height;
    if (sx < sy) smin = sx;
        else smin = sy;
    tx = viewSpace.x - smin * logicSpace.x + (viewSpace.width - smin * logicSpace.width) / 2;
    ty = viewSpace.y - smin * logicSpace.y + (viewSpace.height - smin * logicSpace.height) / 2;
    return glm::transpose(glm::mat3(smin, 0.0f, tx,0.0f, smin, ty,.0f, 0.0f, 1.0f));
}
```

Transformare uniformă/neuniformă



Desenul cu transformare uniforma



Desenul cu transformare neuniforma

Cadrul ferestrei mapat pe poarta de afisare
(neafectat de transformarea uniforma)

Transformare uniformă/neuniformă - implementare

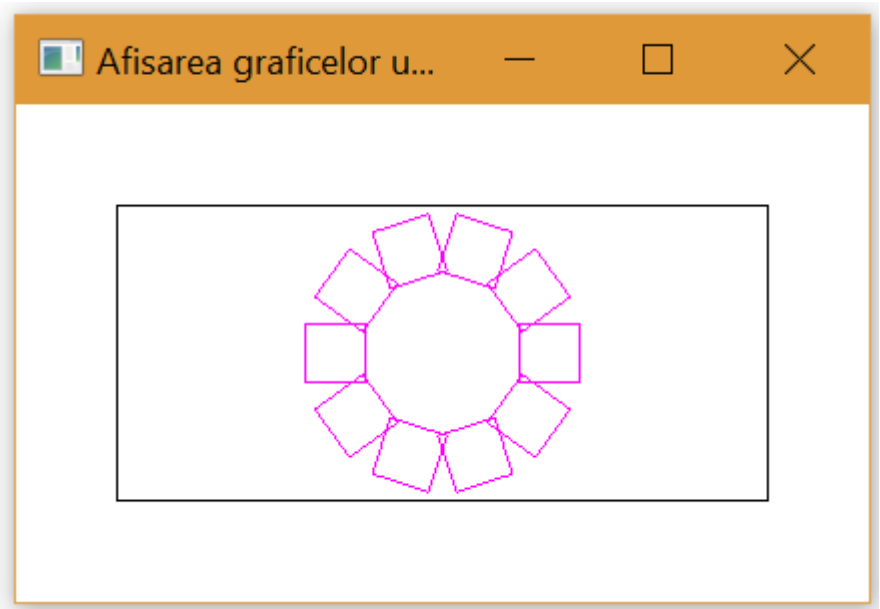
```
void TransformareaFereastraPoartaPatrate::Update(float deltaTimeSeconds)
{   glm::ivec2 resolution = window->GetResolution();
//setare poarta de vizualizare
    viewSpace = ViewportSpace(50, 50, resolution.x - 100, resolution.y - 100);
// seteaza zona ecran in care se va desena
    SetViewportArea(viewSpace, glm::vec3(1), true);
    visMatrix = VisualizationTransf2D(logicSpace, viewSpace);
//deseneaza cadrul ferestrei
    RenderMesh2D(meshes["window"], visMatrix, glm::vec3(0, 0, 0));
    if (uniform)   //modificata la apasarea tastei “s”
      visMatrix = VisualizationTransf2DUnif(logicSpace, viewSpace);
    else
      visMatrix = VisualizationTransf2D(logicSpace, viewSpace);
//deseneaza patratele rotite
   DrawRotatedSquares(visMatrix, glm::vec3(1, 0, 1), 10);
}
```

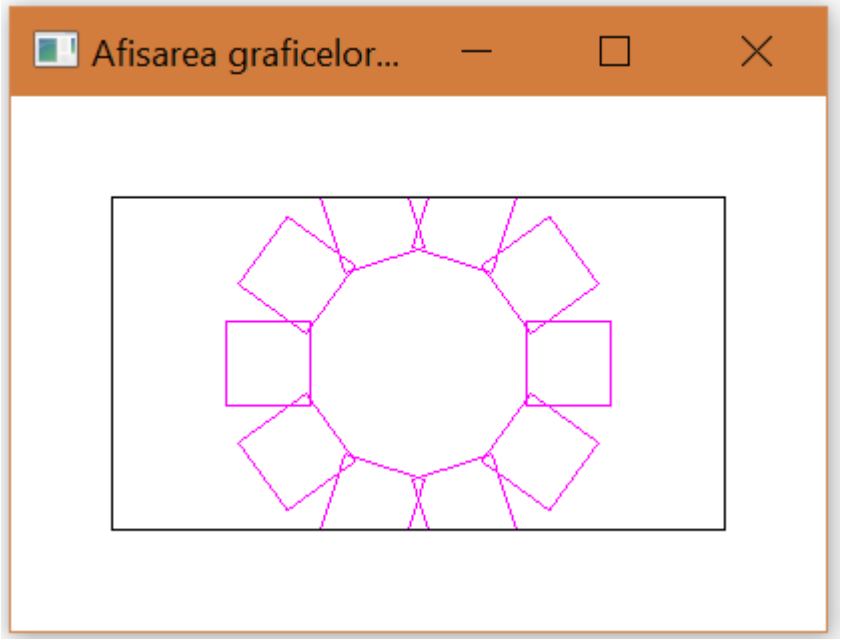
Transformare uniformă/neuniformă - implementare

```
void TransformareaFereastraPoartaPatrate::DrawRotatedSquares(glm::mat3
            visMatrix, glm::vec3 color, int iterations)
{
    float delta = 2 * 3.14159 / iterations;

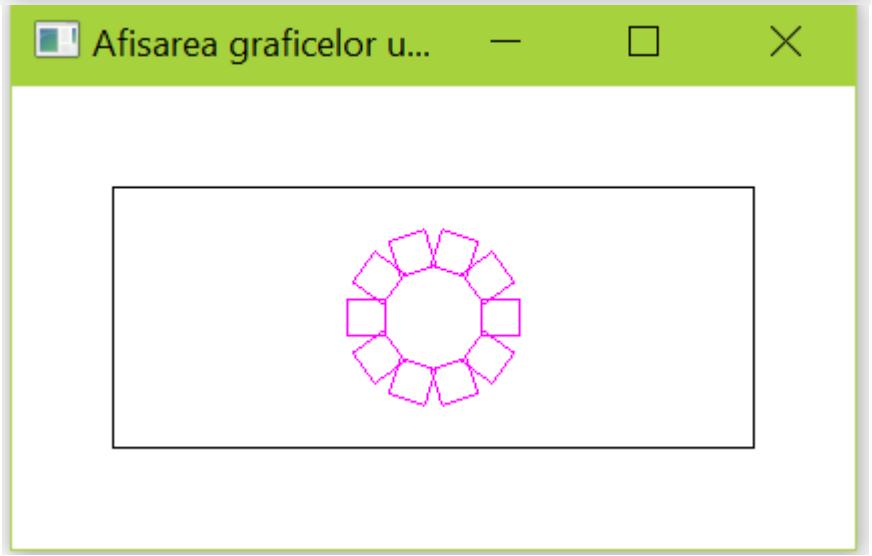
    for (int i = 0; i < iterations; i++)
   {
       modelMatrix = visMatrix *
                          Transform2D::Translate(cx,cy ) *
                          Transform2D::Rotate(delta * i) *
                          Transform2D::Translate(-cx,-cy);
       RenderMesh2D(meshes["square"], modelMatrix, color);
   }
}
```

Mărirea/Micşorarea ferestrei





Fereastra micşorată.
Efectul de mărire a desenului şi decupare la marginile ferestrei



Fereastra mărită.
Efectul de micsorare a desenului

Mărirea/Micşorarea ferestrei - implementare

```
void TransformareaFereastraPoartaPatrate::Update(float deltaTimeSeconds)
{
   glm::ivec2 resolution = window->GetResolution();
  // logicSpace.x = logicSpace.x - 2; logicSpace.y = logicSpace.y -2;
  // logicSpace.width = logicSpace.width + 4; logicSpace.height = logicSpace.height +4;
  //Micsorare
   logicSpace.x = logicSpace.x + 1; logicSpace.y = logicSpace.y +1;
   logicSpace.width = logicSpace.width – 2; logicSpace.height = logicSpace.height -2;

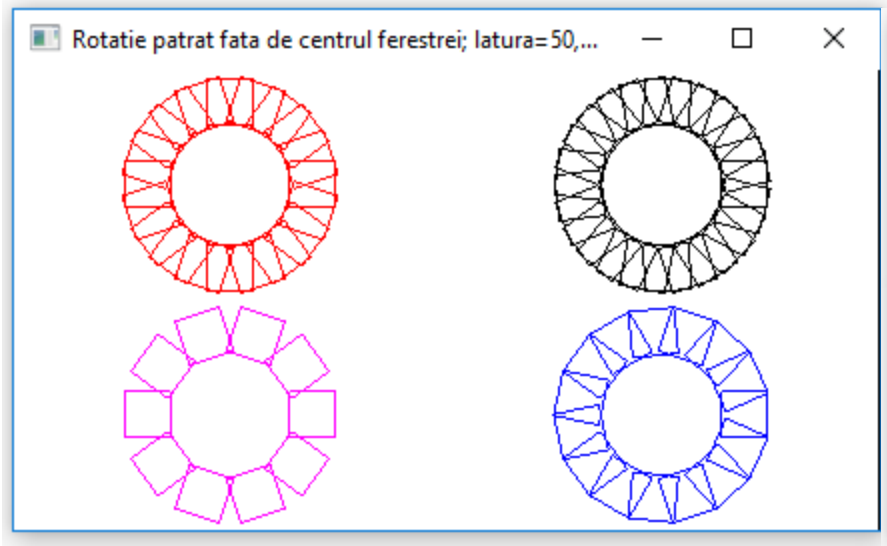
   glm::vec3 cornerw1 = glm::vec3(logicSpace.x, logicSpace.y, 0);
   Mesh* window1 = Object2DPatrate::CreateSquare("window1", cornerw1, logicSpace.width,
                           glm::vec3(0, 0, 0));
    AddMeshToList(window1);

   viewSpace = ViewportSpace(50, 50, resolution.x - 100, resolution.y - 100);
   SetViewportArea(viewSpace, glm::vec3(1), true);

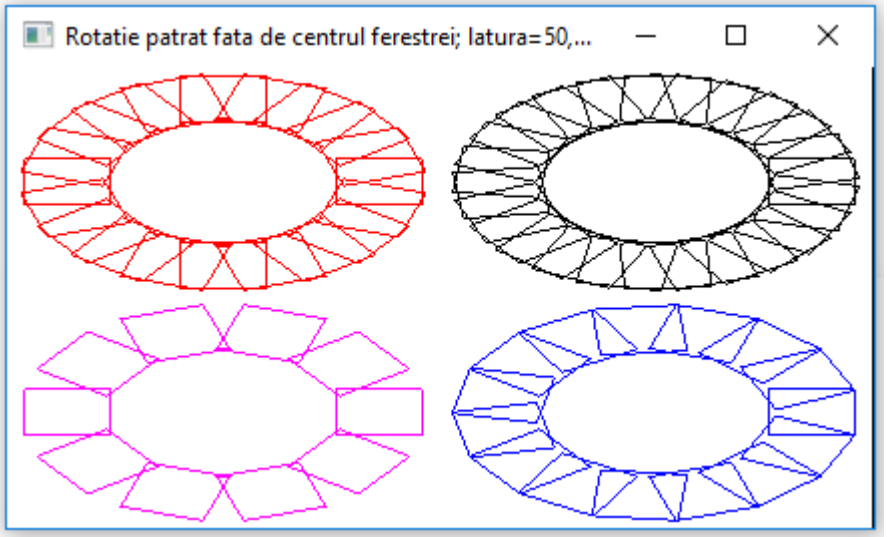
   visMatrix = VisualizationTransf2D(logicSpace, viewSpace);
   RenderMesh2D(meshes["window1"], visMatrix, glm::vec3(0, 0, 0));
   if (uniform)
              visMatrix = VisualizationTransf2DUnif(logicSpace, viewSpace);
   else     visMatrix = VisualizationTransf2D(logicSpace, viewSpace);

   DrawRotatedSquares(visMatrix, glm::vec3(1, 0, 1), 10);
}
```

Desenare în 4 porţi de afişare



Transformare uniforma



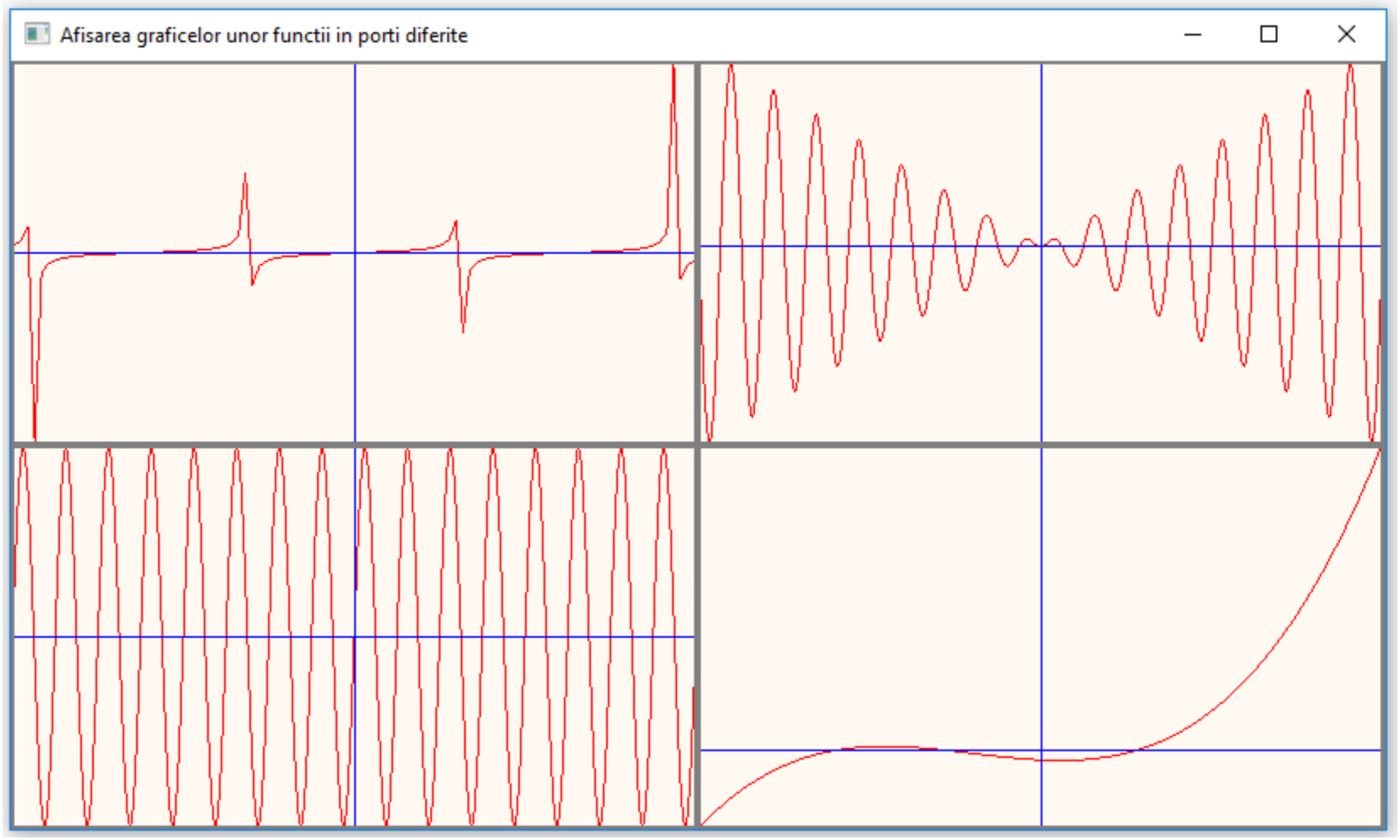
Transformare neuniforma

Desenare în 4 porţi de afişare - implementare

```
void TransformareaFereastraPoartaPatrate::Update(float deltaTimeSeconds)
{
   glm::ivec2 resolution = window->GetResolution(); //fereastra aplicatiei

   // desenare in partea stanga-jos a ferestrei aplicatiei
   viewSpace = ViewportSpace(0, 0, resolution.x / 2, resolution.y / 2);
   SetViewportArea(viewSpace, glm::vec3(1), true);
    if (uniform) visMatrix = VisualizationTransf2DUnif(logicSpace, viewSpace);
     else  visMatrix = VisualizationTransf2D(logicSpace, viewSpace);
   DrawRotatedSquares(visMatrix, glm::vec3(1, 0, 1), 10);
    // desenare in partea dreapta-jos a ferestrei aplicatiei
   viewSpace = ViewportSpace(resolution.x / 2, 0, resolution.x / 2, resolution.y / 2);
   SetViewportArea(viewSpace, glm::vec3(1), true);
   if (uniform) visMatrix = VisualizationTransf2DUnif(logicSpace, viewSpace);
   else visMatrix = VisualizationTransf2D(logicSpace, viewSpace);
   DrawRotatedSquares(visMatrix, glm::vec3(0, 0, 1), 15);
// analog pentru desenare in partea stanga-sus si dreapta-sus a ferestrei
}
```

Afişarea graficelor unor funcţii în porţi diferite



Afişarea graficelor unor funcţii în porţi diferite – implementare (1)

```
Mesh* Object2DGrafic::CreateFunctionPlot(std::string name,
        float &x1, float &x2, float &y1, float &y2, float pas, float(*f)(double f))
{   // calculeaza punctele de pe graficul functiei matematice y = f(x) in intervalul x1 – x2,
    //in coordonate logice
   // întoarce în (x1,y1), (x2,y2) încadrarea graficului incluzand si axele sistemului de coordonate
    float x, y, xmin=x1, xmax=x2;

   // calculeaza incadrarea graficului pe axa OY( [y1-y2]) in intervalul xmin - xmax
      y1 = y2 = f(xmin);
      for (x = xmin + pas; x<xmax; x += pas)
            {
                  if ((y = f(x))<y1) y1 = y;
                  if (y>y2)y2 = y;
            }

      if ((y = f(xmax))<y1) y1 = y;
      if (y>y2) y2 = y;
```

Afisarea graficelor unor functii in porti diferite – implementare (2)

```
// Include axele OX, OY în încadrarea graficului
   if (0<xmin)x1 = 0; else x1 = xmin;
   if (0<y1)y1 = 0;
   if (0>xmax)x2 = 0; else x2 = xmax;
   if (0>y2)y2 = 0;

Mesh* plot = new Mesh(name);
plot->SetDrawMode(GL_LINES); // se vor uni câte 2 varfuri succesive
std::vector<VertexFormat> vertices;
// memoreaza varfurile axei OX – se asociaza varfurilor culoarea albastru
    vertices.push_back(VertexFormat(glm::vec3(x1, 0, 0), glm::vec3(0, 0, 1)));
    vertices.push_back(VertexFormat(glm::vec3(x2, 0, 0), glm::vec3(0, 0, 1)));
// memoreaza varfurile axei OY – se asociaza varfurilor culoarea albastru
   vertices.push_back(VertexFormat(glm::vec3(0, y1, 0), glm::vec3(0, 0, 1)));
   vertices.push_back(VertexFormat(glm::vec3(0, y2, 0), glm::vec3(0, 0, 1)));
   std::vector<unsigned short> indices;
   indices.push_back(0); indices.push_back(1); indices.push_back(2);indices.push_back(3);
```

Afisarea graficelor unor functii in porti diferite – implementare (3)

```
// se adauga varfurile care aproximeaza graficul
// se asociaza varfurilor culoarea roşu
   int contor = 4;
   for (x = xmin; x<xmax - pas; x += pas)
    { vertices.push_back(VertexFormat(glm::vec3(x, f(x), 0), glm::vec3(1, 0, 0)));
      indices.push_back(contor); indices.push_back(contor+1);
      contor++;
     }
// se adauga ultimul varf
    vertices.push_back(VertexFormat(glm::vec3(xmax, f(xmax), 0), glm::vec3(1, 0, 0)));

    plot->InitFromData(vertices, indices);
   return plot;

//CreateFunctionPlot
}
```

Afisarea graficelor unor functii in porti diferite – implementare (4)

```
float f1(double x)
{
return (float)(sin(10 * x)*x);
}

float f2(double x)
{
return (float)(tan(x));
}

float f3(double x)
{
return (float)(sin(10 * x));
}

float f4(double x)
{
return (float)((x * x - 2) * (x + 3));
}
```

Afisarea graficelor unor functii in porti diferite – implementare (5)

```
void TransformareaFereastraPoartaGrafic::Init()
{
//setare camera
……….

// se deseneaza graficele celor 4 functii in intervalul xmin=-5, xmax=5
        float y1, y2, x1=-5.0f, x2 = 5.0f;
// se creaza un mesh cu graficul functiei f1; se seteaza spatiul logic pentru graficul functiei f1
        Mesh* mesh_f1 = Object2DGrafic::CreateFunctionPlot("mesh_f1", x1, x2, y1, y2, 0.01f, f1);
        AddMeshToList(mesh_f1);
        logicSpace1.x = x1;
        logicSpace1.width = x2 - x1;
        logicSpace1.y = y1;
        logicSpace1.height = y2 - y1;
//se creaza, in mod analog, mesh_f2, mesh_f3, mesh_f4 şi logicSpace2, logicSpace3, logicSpace4
……..
}
```

Afisarea graficelor unor functii in porti diferite – implementare (6)

```
void TransformareaFereastraPoartaGrafic::Update(float deltaTimeSeconds)
{
    glm::ivec2 resolution = window->GetResolution(); // fereastra aplicatiei

// seteaza zona de desenare – partea stanga-sus
   viewSpace = ViewportSpace(0, resolution.y / 2, resolution.x / 2, resolution.y / 2);
   SetViewportArea(viewSpace, glm::vec3(1, 0.98, 0.95), true);
   visMatrix = VisualizationTransf2D(logicSpace2, viewSpace);
   DrawFunctionPlot(visMatrix, 2);//deseneaza gaficul functiei f2
// partea dreapta-sus a ferestrei aplicatiei
   viewSpace = ViewportSpace(resolution.x / 2, resolution.y / 2, resolution.x / 2, resolution.y / 2);
   SetViewportArea(viewSpace, glm::vec3(1,0.98,0.95), true); //culoarea fondului pt viewport: bej
   visMatrix = VisualizationTransf2D(logicSpace1, viewSpace);
   DrawFunctionPlot(visMatrix, 1); //deseneaza graficul functiei f1
// analog pentru desenarea celorlalte functii
}
```

Afisarea graficelor unor functii in porti diferite – implementare (7)

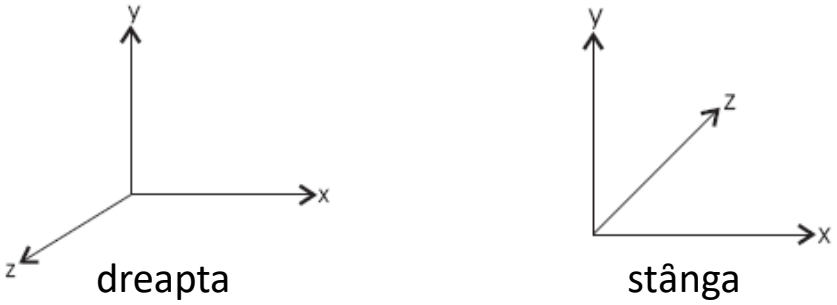
```
void TransformareaFereastraPoartaGrafic::DrawFunctionPlot(glm::mat3 visMatrix, int nr_functie)
{
    modelMatrix = visMatrix;
    switch (nr_functie)
    {
     case 1:
        RenderMesh2D(meshes["mesh_f1"],shaders["ShaderGrafic"],modelMatrix);
        break;
     case 2:
        RenderMesh2D(meshes["mesh_f2"], shaders["ShaderGrafic"], modelMatrix);
        break;
     case 3:
        RenderMesh2D(meshes["mesh_f3"], shaders["ShaderGrafic"], modelMatrix);
        break;
     case 4:
        RenderMesh2D(meshes["mesh_f4"], shaders["ShaderGrafic"], modelMatrix);
        break;
     default:
        break;
    }
}
```

Transformari geometrice 3D

Prof. univ. dr. ing. Florica Moldoveanu
Conf. dr. ing. Anca Morar

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2018-2019

Sisteme de coordonate carteziene 3D



Sisteme de coordonate carteziene 3D

i, j, k: versorii directiilor axelor sistemului de coordonate

dreapta: $k = (i \times j)/|i \times j|$

stanga: $k = (j \times i)/|j \times i|$

Sistemul de coordonate in care este descrisa scena intr-o aplicatie OpenGL: sistem de coordonate carteziene 3D dreapta.

Transformările geometrice 3D elementare

- Translaţia
- Scalarea faţă de origine
- Rotaţiile în jurul axelor principale ale sistemului de coordonate
- Oglindirea faţă de un plan al sistemului de coordonate
- Forfecarea faţă de originea sistemului de coordonate

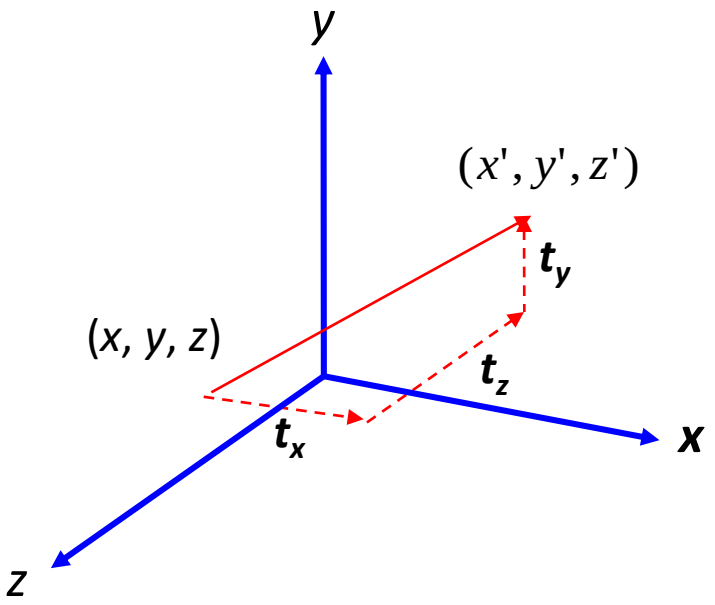
Considerăm punctele din spaţiu reprezentate prin vectori coloană, pentru a respecta conventia din OpenGL:

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = M \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

M este matricea de transformare a punctului (x, y, z), in coordinate omogene

Translaţia

Definita printr-un vector T[tx,ty,tz]



$$x' = x + t_x$$

$$y' = y + t_y$$

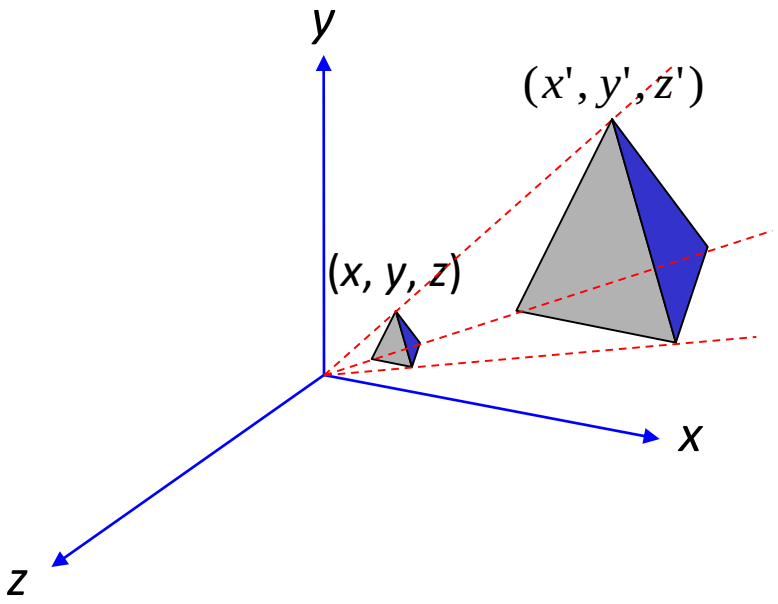
$$z' = z + t_z$$

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

$T(t_x, t_y, t_z)$

Scalarea faţă de origine

Definita prin 3 numere reale: Sx,Sy, Sz – factorii de scalare pe cele 3 axe



$S_x = S_y = S_z \quad \Rightarrow \quad$ scalare uniforma

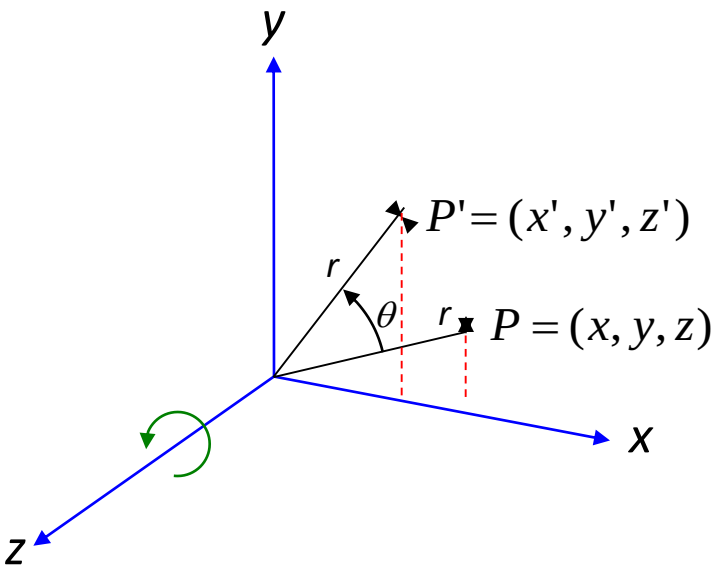
altfel, scalare neuniforma

$$x' = x \cdot s_x$$
$$y' = y \cdot s_y$$
$$z' = z \cdot s_z$$

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

$S(s_x, s_y, s_z)$

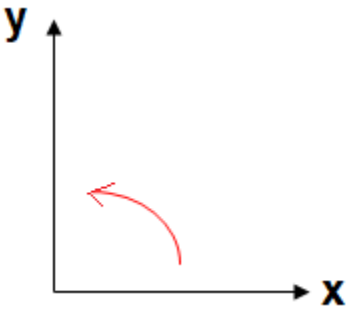
Rotaţia pozitivă (trigonometrica) în jurul axei oz



Este o rotaţie într-un plan de z constant (prin rotaţie nu se modifică coordonata z).

Rotatia in planul XOY:

$$x' = x\cos\theta - y\sin\theta$$
$$y' = x\sin\theta + y\cos\theta$$
$$z' = z = 0$$



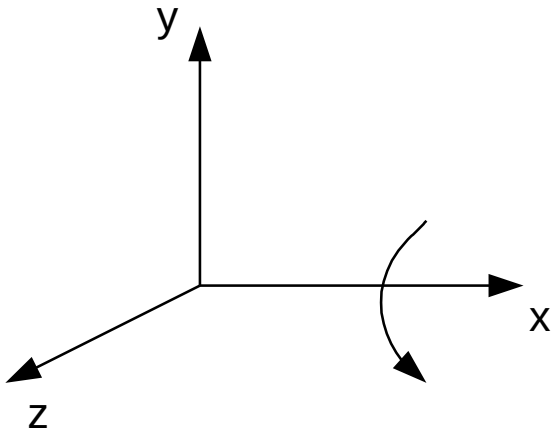
$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

$R_z(\theta)$

Rotatia punctului P(x,y,z) in jurul axei Oz cu unghiul θ:

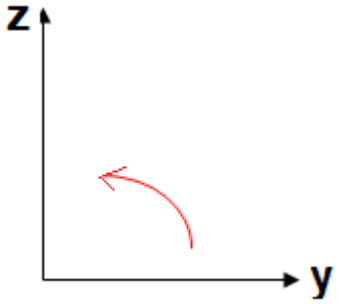
$$x' = x\cos\theta - y\sin\theta$$
$$y' = x\sin\theta + y\cos\theta$$
$$z' = z$$

Rotaţia pozitivă în jurul axei ox



Rotatia in planul Y0Z

$$y' = y\cos\theta - z\sin\theta$$
$$z' = y\sin\theta + z\cos\theta$$
$$x' = x = 0$$



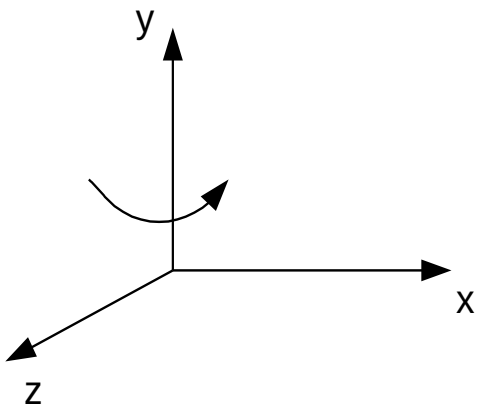
Rotatia punctului P(x,y,z) in jurul axei Ox cu unghiul θ:

$$x' = x$$
$$y' = y\cos\theta - z\sin\theta$$
$$z' = y\sin\theta + z\cos\theta$$

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

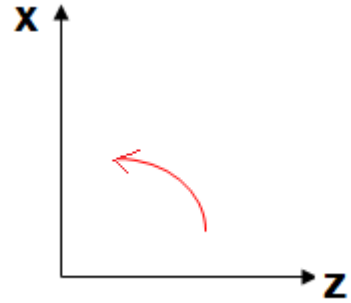
$R_x(\theta)$

Rotaţia pozitivă în jurul axei oy



Rotatia in planul ZOX

$$z' = z\cos\theta - x\sin\theta$$
$$x' = x\cos\theta + z\sin\theta$$
$$y' = y = 0$$



Rotatia punctului P(x,y,z) in jurul axei Oy cu unghiul θ:

$$x' = x\cos\theta + z\sin\theta$$
$$y' = y$$
$$z' = -x\sin\theta + z\cos\theta$$

$$R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$R_y(\theta)$

Alte transformari geometrice 3D elementare

Oglindirea faţă de un plan al sistemului de coordonate

$$[O_z] = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Forfecarea faţă de origine

- **De-a lungul axei OZ: modifica x si y proportional cu z**

  x’ = x + Fx*z

  y’ = y + Fy*z

  z’ = z



- **Analog pentru forfecarea de-a lungul axei OX si a axei OY**

- **Cazul general (forfecarea pe toate cele 3 axe):**

  x’ = x + y* d + z*g

  y’ = x*b + y + z*i

  z’ = x*c + y*f + z

$$[x' \; y' \; z' \; 1] = [x \; y \; z \; 1] * \begin{vmatrix} 1 & b & c & 0 \\ d & 1 & f & 0 \\ g & i & 1 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

Transformari geometrice 3D compuse

Exemple:

1. **Scalarea faţă de un punct oarecare, F(xf, yf, zf):**

   T(xf,yf,xf)*S(sx,sy,sz)*T(-xf,-yf,-zf)

2. **Rotatia in jurul unei axe paralele cu o axa a sistemului de coordonate**:
   1. Translatia obiectului astfel incat axa de rotatie sa se suprapuna peste o axa a sistemului de coordonate.
   2. Rotatia obiectului in jurul axei sistemului de coordonate.
   3. Translatia inversa celei din pasul 1.

   Rezulta:

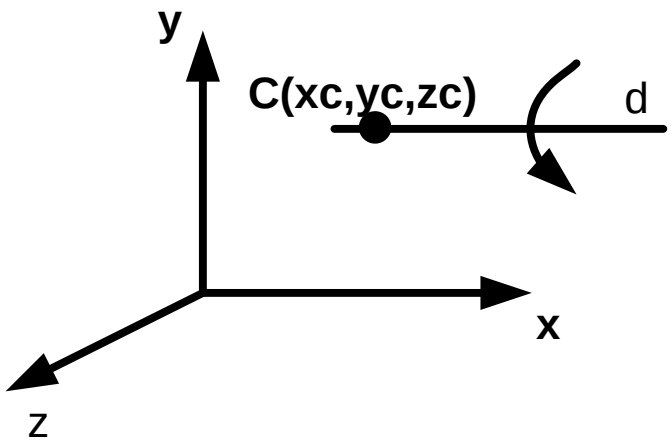
   M = T(xd, yd, zd) * R(u) * T(-xd, -yd, -zd)

   u : unghiul de rotatie

   xd, yd, zd : un punct de pe axa de rotatie

$$\begin{bmatrix} \mathbf{x'} \\ \mathbf{y'} \\ \mathbf{z'} \\ \mathbf{1} \end{bmatrix} = M * \begin{bmatrix} \mathbf{x} \\ \mathbf{y} \\ \mathbf{z} \\ \mathbf{1} \end{bmatrix}$$

Rotaţia în jurul unei drepte paralele cu axa OX



$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = T(xc, yc, zc)\, R_X(u)\, T(-xc, -yc, -zc) \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Rotatia cu în jurul unei drepte oarecare (1)

Se considera dreapta data printr-un punct (xd, yd, zd) si directia sa, D[a, b, c].

1. Translatie prin care dreapta va trece prin origine: T(-xd, -yd, -zd)
2. Alinierea dreptei cu una dintre axele principale, de ex. cu axa OZ:

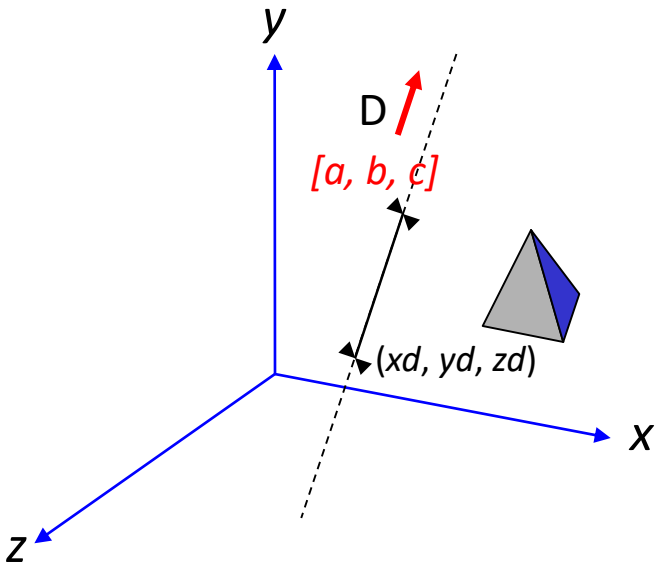
   2.1. Rotatie in jurul axei OX, cu un unghi ux, prin care dreapta ajunge in planul XOZ: Rox(ux)

   2.2. Rotatie in jurul axei OY, cu un unghi uy, prin care dreapta se suprapune pe axa OZ: Roy(uy)
3. Rotatia cu unghiul dat, u, in jurul axei pe care s-a aliniat dreapta: rotatie in jurul axei OZ : Roz(u)
4. Transformarea inversa celei din pasul 2:

   4.1. Rotatie in jurul axei OY, cu unghiul –uy: Roy(-uy)

   4.2. Rotatie in jurul axei OX, cu unghiul –ux: Rox(-ux)
5. Transformarea inversa celei de la pasul 1: T(xd, yd, zd)

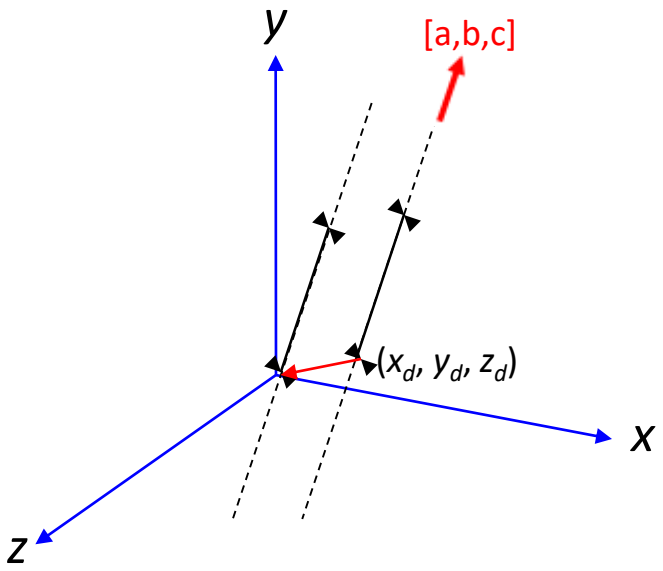
Rotatia in jurul unei drepte oarecare (2)



1. Translatie astfel incat dreapta sa treaca prin origine
2. Aliniere dreapta cu una din axe
3. Rotatia propriu-zisa.
4. Transformarea inversa de la punctul 2
5. Transformarea inversa de la punctul 1

Rotatia in jurul unei drepte oarecare (3)

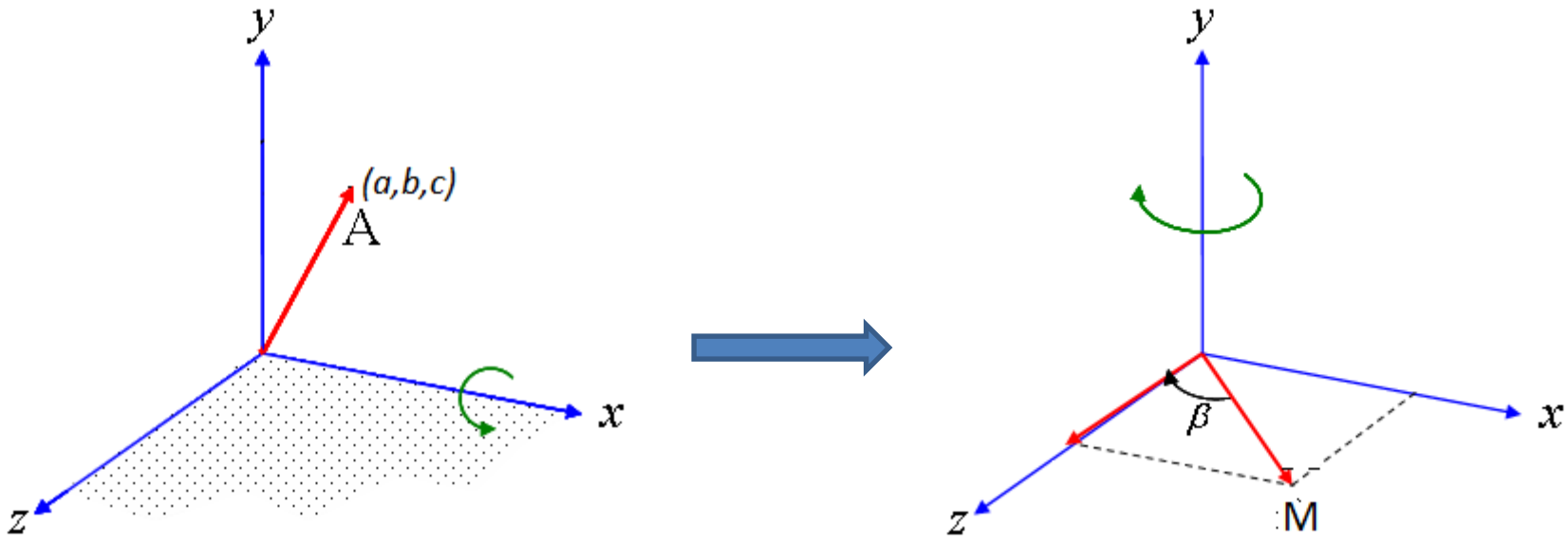
1. Translaţie



$$T = \begin{bmatrix} 1 & 0 & 0 & -xd \\ 0 & 1 & 0 & -yd \\ 0 & 0 & 1 & -zd \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotatia in jurul unei drepte oarecare (4)

2. Aliniere *D* cu axa OZ



Rotatie in jurul axei OX cu un unghi ux, prin care dreapta ajunge in planul XOZ: Rox(ux)

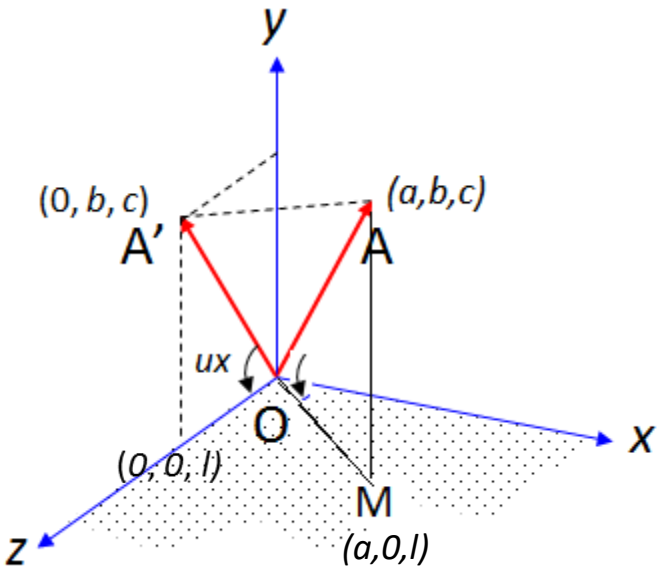
Rotatie in jurul axei OY, cu un unghi uy,

prin care dreapta se suprapune pe axa OZ:

Roy(uy)

Rotatia in jurul unei drepte oarecare (5)

2. 1. Rotatia in jurul axei OX



A' – proiectia lui A in planul YOZ

Lungime (OA) = L; lungime (OA') = l;

$$L = \sqrt{a^2 + b^2 + c^2}$$

$$l = \sqrt{b^2 + c^2}$$

b = l*sin(ux)
c = l*cos(ux)

$$\sin(ux) = \frac{b}{l}$$

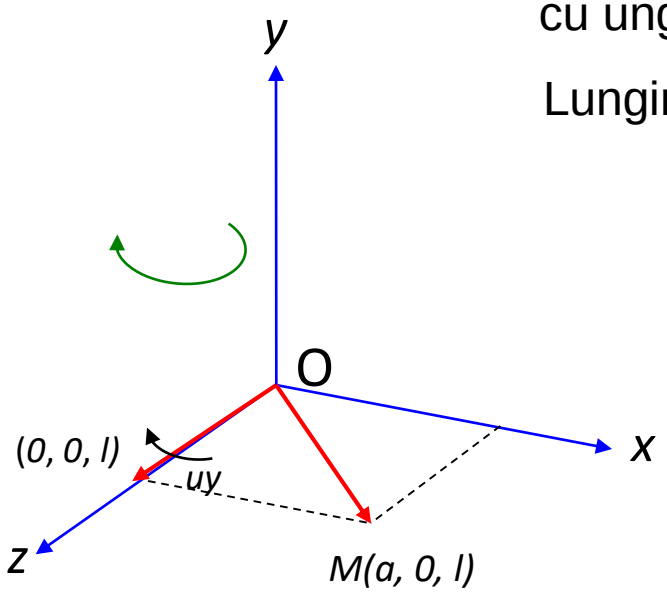
$$\cos(ux) = \frac{c}{l}$$

$R_x(ux)$

Prin rotatia lui OA in jurul axei Ox cu unghiul ux, acesta va ajunge in planul XOZ

Rotatia in jurul unei drepte oarecare (6)

2. 2. Rotatia in jurul axei OY



R_y(uy)

M este punctul obtinut prin rotatia lui A in jurul axei Ox, cu unghiul ux.

Lungime (OM) = Lungime (OA) = L

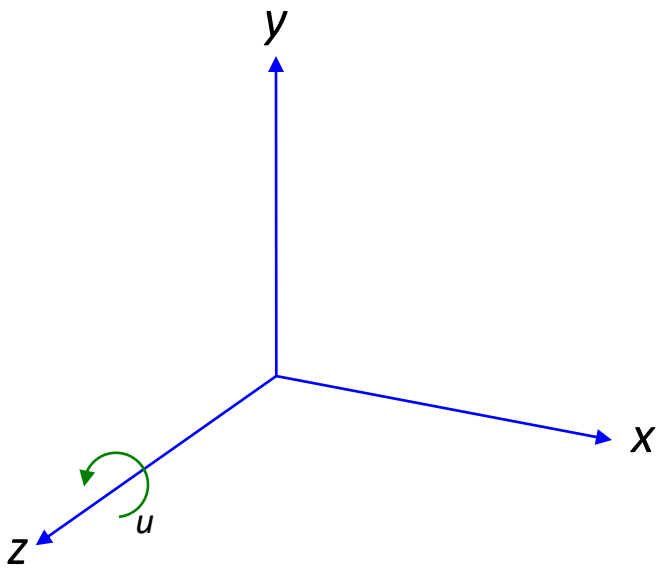
a = L*sin(-uy)

l = L*cos(-uy)

$$\sin(-uy) = a/L -> \sin(uy) = -a/L$$
$$\cos(-uy) = \cos(uy) = l/L$$

Rotatia in jurul unei drepte oarecare (7)

3. Rotatia in jurul axei OZ cu unghiul dat

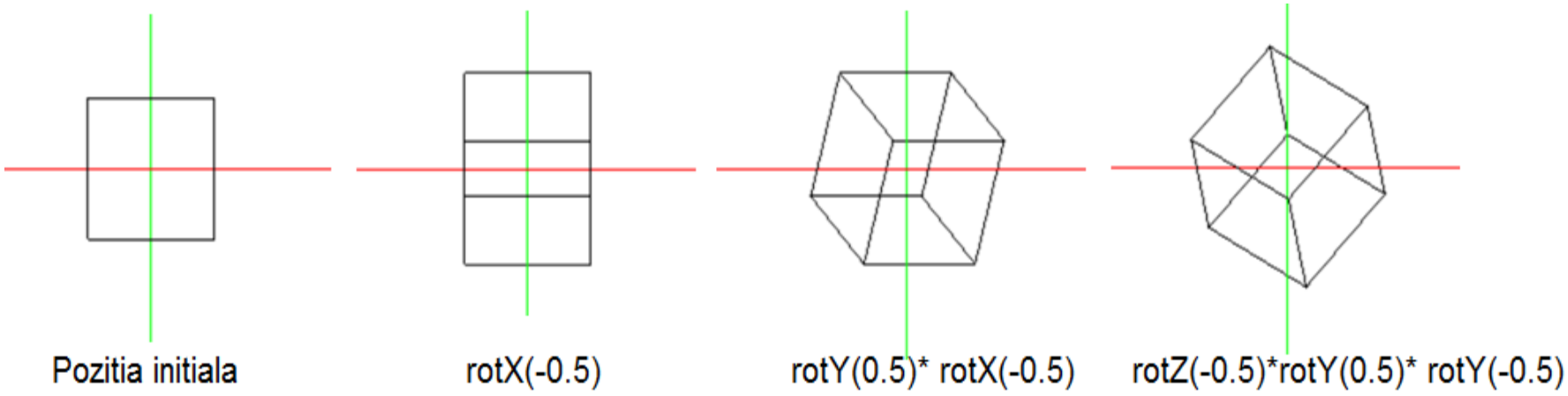


$$R_z(u) = \begin{bmatrix} \cos(u) & -\sin(u) & 0 & 0 \\ \sin(u) & \cos(u) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Exemplu: rotatia unui cub (1)

Cub centrat in originea sistemului de coordonate XYZ

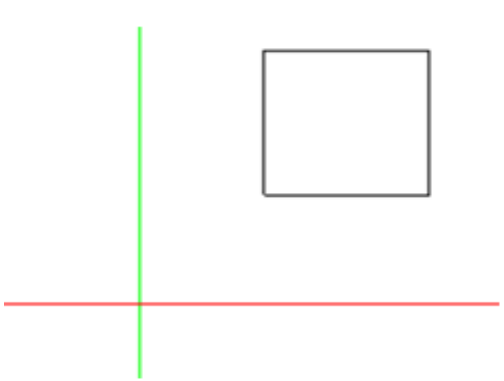
Proiectie ortografica in XOY: (x,y,z) → (x' = x; y' = y)



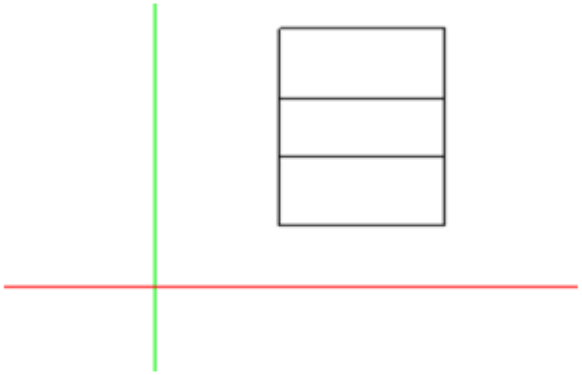
Exemplu: rotatia unui cub (2)

Cub cu centrul in (100, 100, 0)

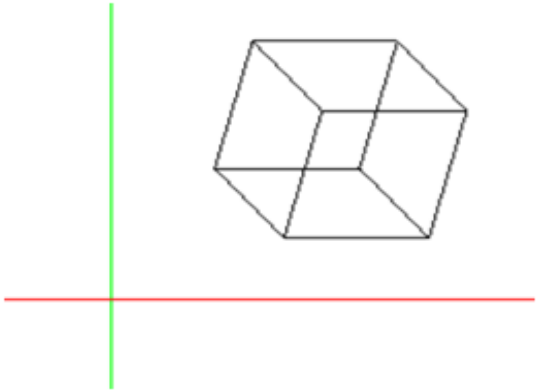
Proiectie ortografica in XOY



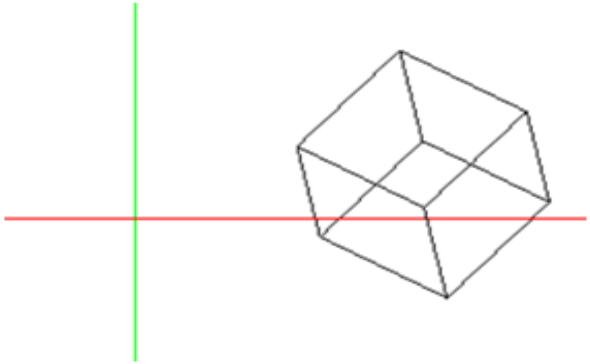
Pozitia initiala



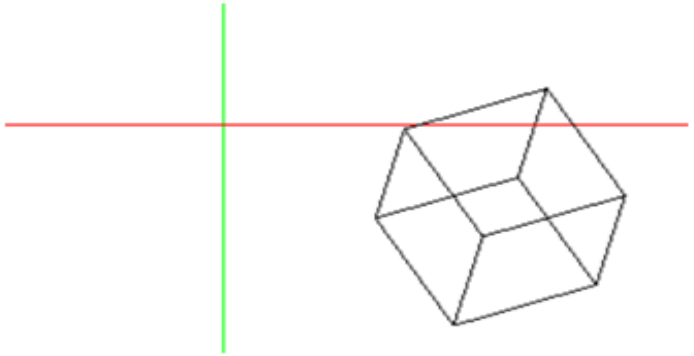
rotX(-0.5)



rotY(0.5)*rotX(-0.5)



rotZ(-0.5)*rotY(0.5)*rotX(-0.5)



rotZ(-0.5)*rotZ(-0.5)*rotY(0.5)*rotX(-0.5)

Proiectii

Prof. univ. dr. ing. Florica Moldoveanu

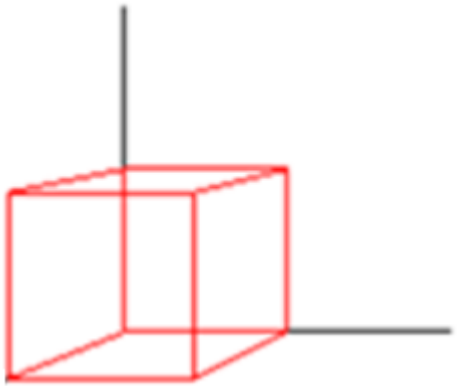
Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2018-2019

Proiectii

❖ Transformari dintr-un spatiu n-dimensional intr-un spatiu k-dimensional, k<n.

Proiectii $R^3 \rightarrow R^2$

- Transformari din spatiul tri-dimensional intr-un spatiu bi-dimensional
- Se aplica varfurilor obiectelor
- Nu modifica legaturile dintre varfuri (topologia)

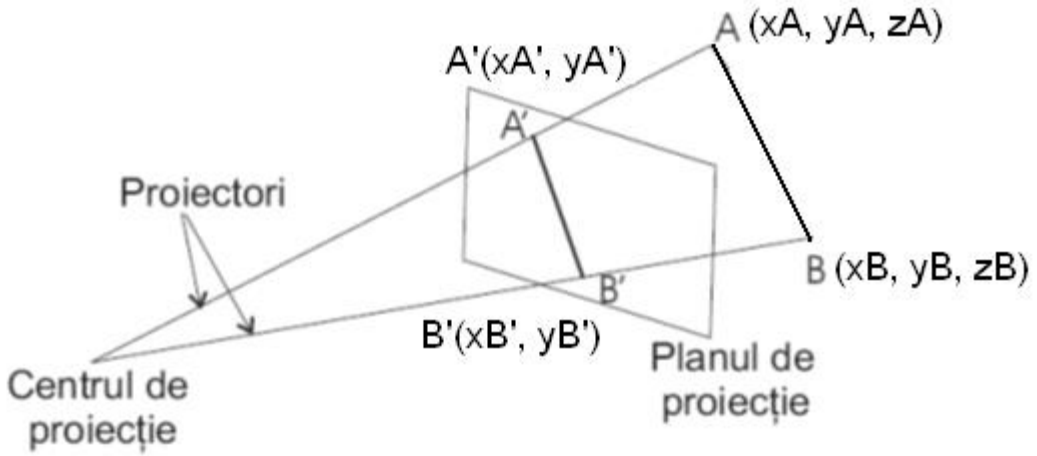


Exemplu:

- Proiectia unui cub in planul XOY
- Legaturile dintre varfurile 2D sunt aceleasi cu legaturile dintre varfurile 3D proiectate

Proiectii $R^3 \rightarrow R^2$

- Proiectia unui varf 3D într-un plan 2D este punctul de intersectie dintre plan si proiectorul care pleaca din ***centrul de proiectie*** si trece prin varf.



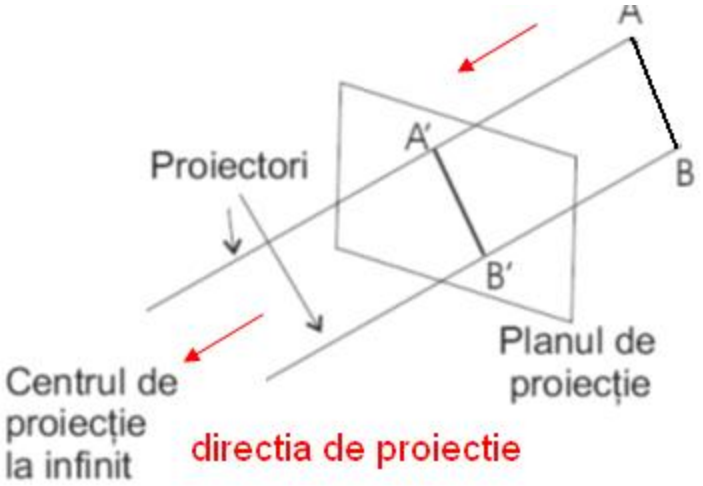
Segmentul A'-B' este proiectia segmentului A-B in planul de proiecţie XOY (z=0).

Exista 2 clase de proiectii:

- Proiectii perspectiva: centrul de proiectie este situat la distanta finita fata de planul de proiectie (ca in figura de mai sus); proiectorii sunt drepte convergente in CP.

Proiectii $R^3 \rightarrow R^2$

- Proiectii paralele - centrul de proiectie este la infinit; proiectorii sunt linii paralele care trec prin varfurile obiectului proiectat si au directia specificata (vectorul ***directie de proiectie***)



Proiectii perspectiva(1)

Produc imagini asemanatoare celor obtinute cu aparatele de fotografiat.

Caracteristici:

1. Efectul de micsorare.

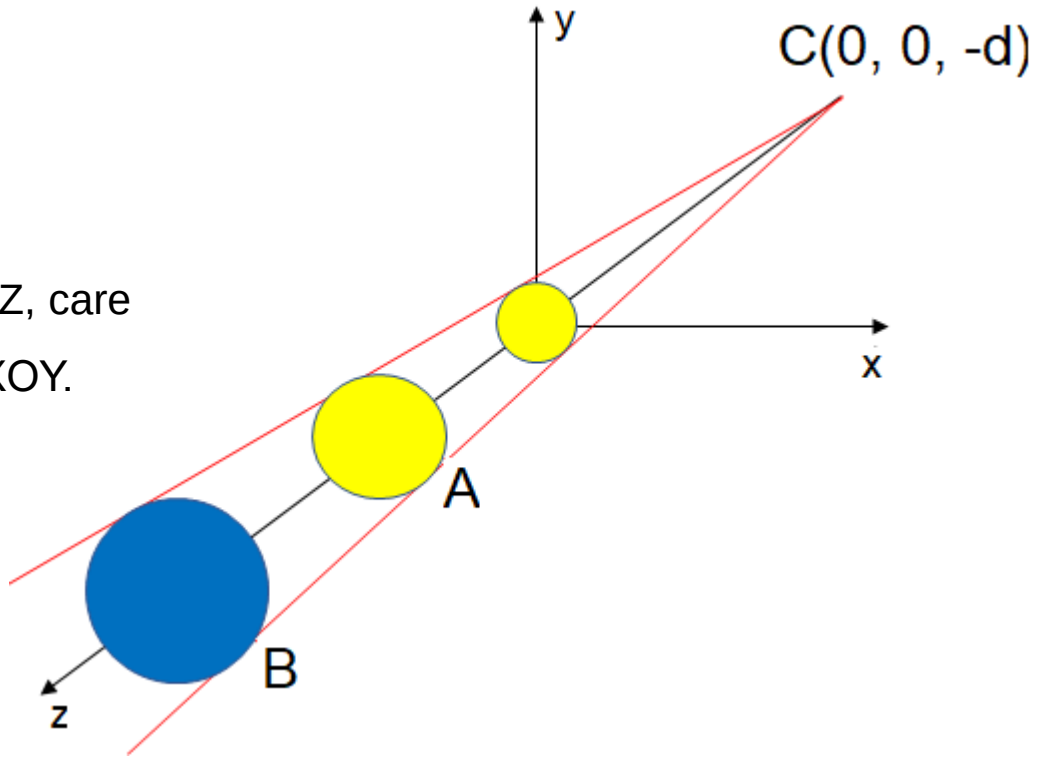
Dimensiunea obiectului in proiectia 2D este invers proportionala cu distanta de la centrul de proiectie la obiect.

Exemplu:

A si B sunt 2 sfere, cu centrele pe axa OZ, care se proiecteaza în acelaşi disc în planul XOY.

raza sferei B = 2* raza sferei A

Dist(centru_B-C) = 2* Dist(centru_A-C)

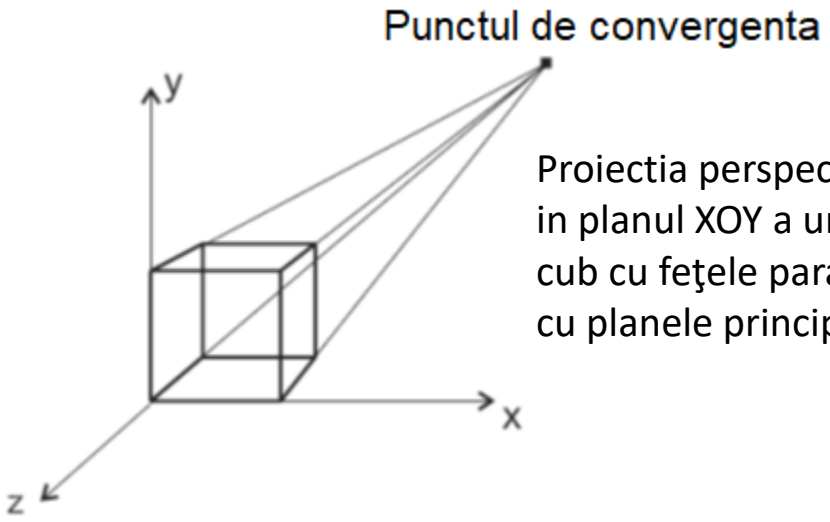


Proiectii perspectiva(2)

2. **Modifica unghiurile dintre dreptele care nu sunt paralele cu planul de proiectie**

Proiectiile liniilor paralele care nu sunt paralele cu planul de proiectie converg catre un punct din plan, numit punct de convergenta

- Punctul catre care converg liniile paralele cu una dintre axele principale: ***punct de convergenta principal***



Proiectia perspectiva in planul XOY a unui cub cu feţele paralele cu planele principale

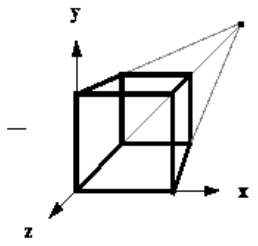


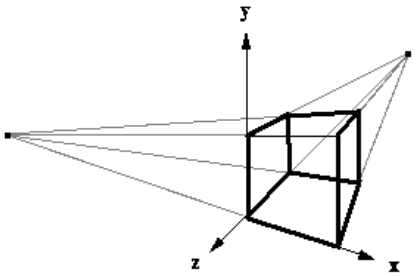
Pot fi efectuate proiectii cu unul, doua sau trei *puncte de convergenta principale.*

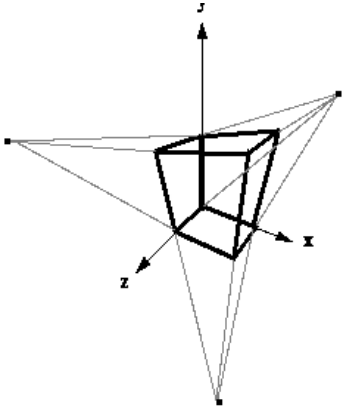
Proiectii perspectiva(3)

Pot fi efectuate proiectii cu unul, doua sau trei *puncte de convergenta principale*.

Numarul de puncte de convergenta principale = numarul de axe principale intersectate de planul de proiectie.

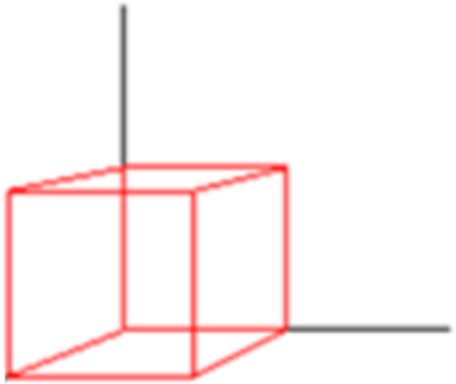




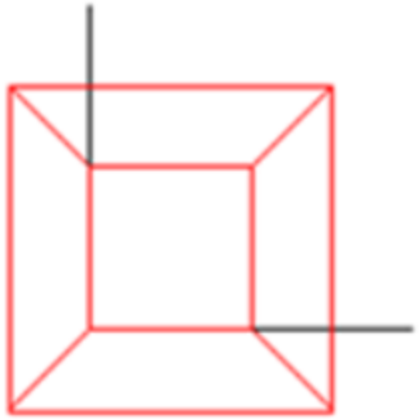


Exemple de proiectii perspectiva

Proiectii ale unui cub cu feţele paralele cu planele principale ale sistemului de coordonate si coltul de (xmin,ymin,zmin) in originea sistemului de coordonate.

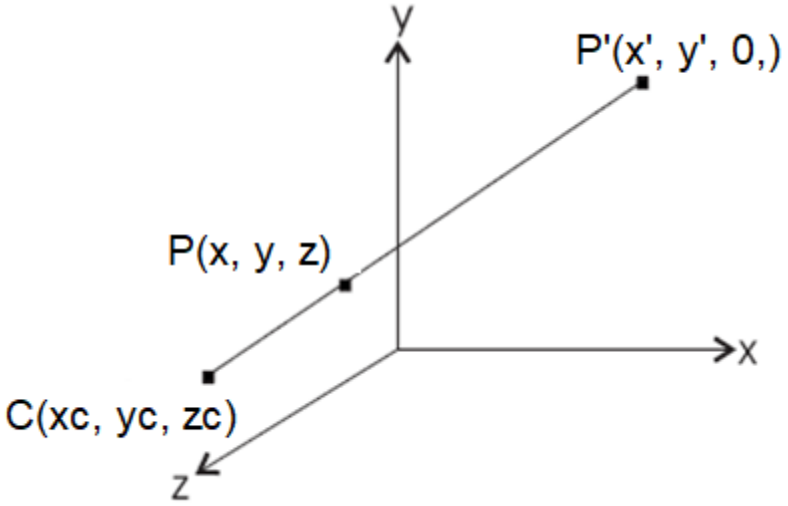


Proiectie perspectiva cu centrul in (5,2,8)



Proiectie perspectiva cu centrul in (0.5,0.5,2)

Proiectia perspectiva in planul XOY (1)



C – centrul de proiectie
P – punctul proiectat
P’ – proiectia lui P in XOY

Ecuatiile parametrice ale proiectorului care trece prin P:

x = xc + (x – xc)*t
y = yc + (y – yc)*t 0<= t <= 1, pentru punctele aflate pe segmentul C-P
z = zc + (z – zc) *t

0 = zc + (z –zc) *t’ → t’ = – zc / (z – zc) → **x’ = (xc*z – zc*x)/(z – zc)**

y’ = (yc*z – zc*y)/(z – zc)

Proiectia perspectiva in planul XOY (2)

Reprezentarea matricială a transformării de proiectie

$$\begin{vmatrix} x' \\ y' \\ z' \\ 1 \end{vmatrix} = Pperspectiva \begin{vmatrix} x \\ y \\ z \\ 1 \end{vmatrix}$$

$$Pperspectiva = \begin{vmatrix} \frac{-zc}{z-zc} & 0 & \frac{xc}{z-zc} & 0 \\ 0 & \frac{-zc}{z-zc} & \frac{yc}{z-zc} & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

Matricea proiectiei depinde de coordonata z a punctului proiectat:

→ nu poate fi folosita pentru toate varfurile aceluiaşi obiect

$$\begin{vmatrix} xw \\ yw \\ zw \\ w \end{vmatrix} = Pperspectiva \begin{vmatrix} x \\ y \\ z \\ 1 \end{vmatrix}$$

$$Pperspectiva = \begin{vmatrix} -zc & 0 & xc & 0 \\ 0 & -zc & yc & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & -zc \end{vmatrix}$$

Coordonatele omogene ale punctului P'

xw = -zc*x + xc*z

yw = -zc*y + yc*z

zw = 0

w = z – zc

Coordonatele carteziene ale punctului P'
- împărţirea perspectivă -

x' = xw/w = (-zc*x + xc*z)/(z-zc)

y' = yw/w = (-zc*y + yc*z)/(z-zc)

z' = zw/w = 0

Proiectia perspectiva in planul XOY (3)

Reprezentarea matricială a transformării de proiectie

$$\begin{vmatrix} xw \\ yw \\ zw \\ w \end{vmatrix} = Pperspectiva \begin{vmatrix} x \\ y \\ z \\ 1 \end{vmatrix}$$

$$Pperspectiva = \begin{vmatrix} 1 & 0 & \frac{-xc}{zc} & 0 \\ 0 & 1 & \frac{-yc}{zc} & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{-1}{zc} & 1 \end{vmatrix}$$

Coordonatele omogene ale punctului P'

xw = x + (-xc/zc)*z

yw = y + (-yc/zc)*z

zw = 0

w = (-1/zc)*z +1 = -(z – zc)/zc

Coordonatele carteziene ale punctului P'

x' = xw/w = -(x + (-xc/zc)*z)*zc/(z – zc)

= (-zc*x + xc*z)/(z-zc)

y' = yw/w = (-zc*y + yc*z)/(z-zc)

z' = zw/w = 0

Coordonatele omogene ale punctului obtinut prin proiectie perspectiva au w ≠ 1.

Proiectia perspectiva in planul XOY(4)

Pentru conservarea informatiei de adancime (necesara pentru eliminarea din imagine a partilor nevizibile ale scenei 3D) se foloseste matricea :

Coordonatele omogene ale punctului P'

xw = -zc*x + xc*z → x' = (-zc*x + xc*z)/(z-zc)

yw = -zc*y + yc*z → y' = (-zc*y + yc*z)/(z-zc)

zw = z → z' = z/(z – zc)

w = z – zc

$$Pperspectiva = \begin{vmatrix} -zc & 0 & xc & 0 \\ 0 & -zc & yc & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & -zc \end{vmatrix}$$

Proiectia perspectiva standard

Xc = 0, Yc = 0, Zc = - d

xw = x → x' = x*d/(z+d)

yw = y → y'= y*d/(z+d)

zw = z → z' = z/(z+d)

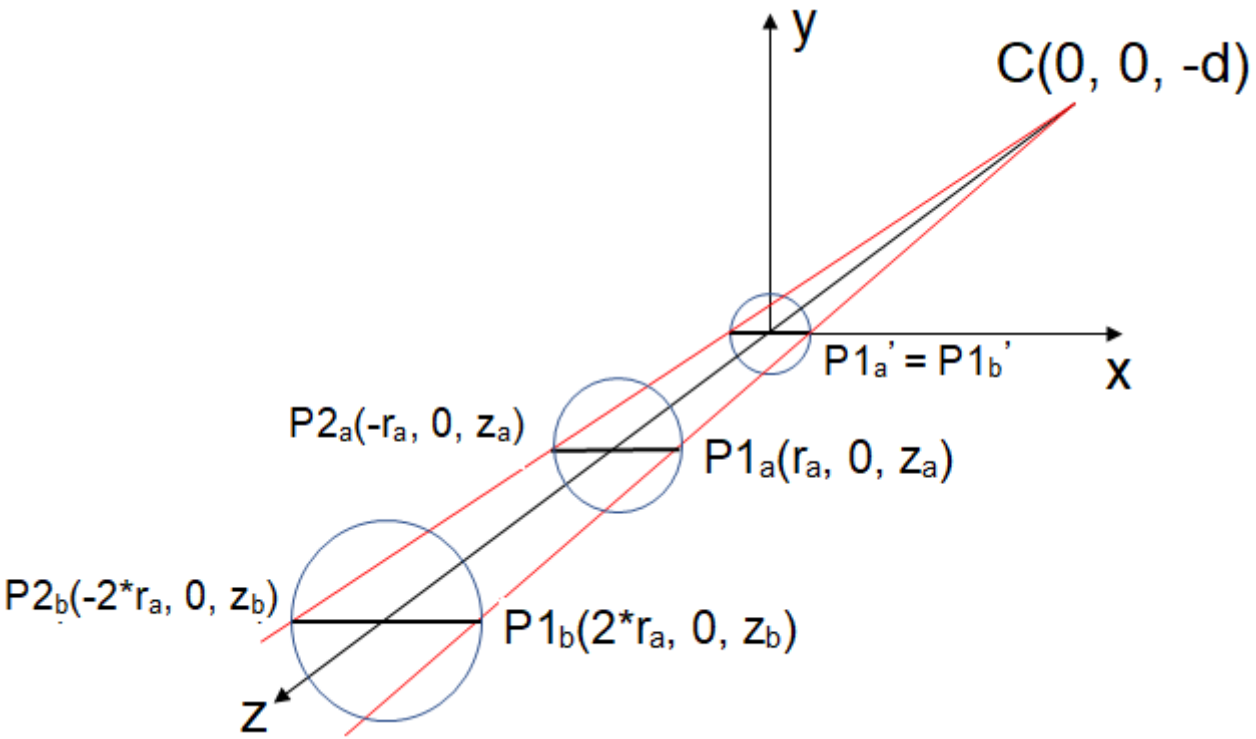
w = z/d + 1 = (z+d)/d

$$Ppers_standard = \begin{vmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & \frac{1}{d} & 1 \end{vmatrix}$$

Proiectia perspectiva in planul XOY - exemplu

A şi B sunt 2 sfere cu centrele pe axa OZ, A de rază r_a şi B de rază $2*r_a$.

Distanța de la centrul sferei B la centrul de proiecție C este de 2 ori mai mare decât distanta de la centrul sferei A la C.



Proiecția perspectivă standard

xw = x → x' = x*d/(z+d)

yw = y → y'= y*d/(z+d)

zw = z → z' = z/(z+d)

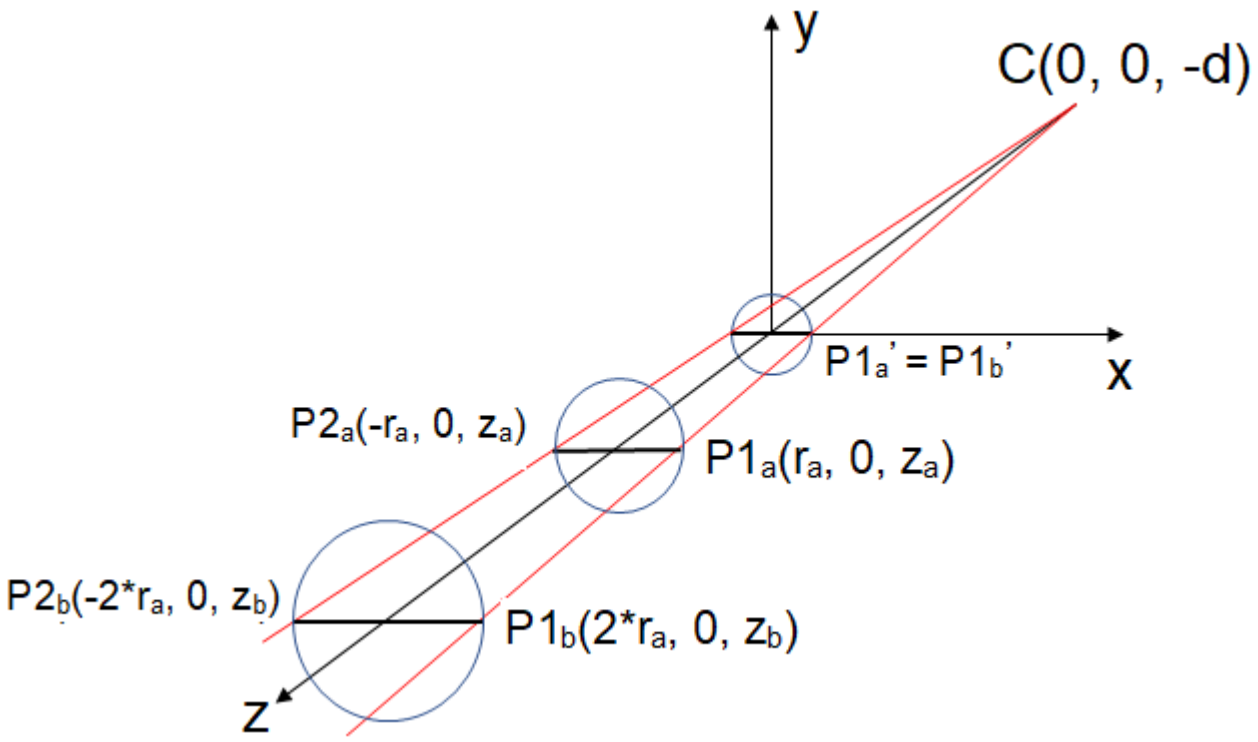
w = z/d + 1 = (z+d)/d

$x_P1_a' = r_a*d/(z_a + d)$; $y_P1_a' = 0 = y_P1_b'$

$x_P1_b' = 2*r_a*d/(z_b + d) = 2*r_a*d/(2*(z_a + d)) = r_a*d/(z_a + d) = x_P1_a'$

Analog se demonstrează că $P2_b' = P2_a'$

Utilizarea coordonatei de adâncime în eliminarea suprafetelor nevizibile din imagini



Proiecția perspectivă standard

zw = z → z' = z/(z+d)

$z_P1_a' = z_a / (z_a + d)$

$z_P1_b' = z_b / (z_b + d)$

Consideram:

d = 50; z_a =100;

$z_b + d = 2(z_a + d) \rightarrow z_b = 250$

$z_P1_a' = 100/150 = 0.666$;

$z_P1_b' = 250/300 = 0.833$

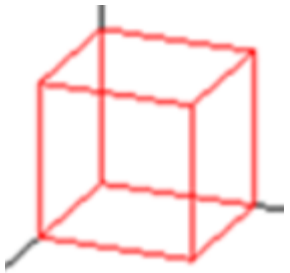
Daca 2 puncte apartinand la 2 suprafete 3D se proiecteaza in acelasi punct, se va reda in imagine culoarea punctului mai apropiat de observator (centrul de proiectie):

$z_P1_a' < z_P1_b'$ → discul in care se proiecteaza cele 2 sfere va fi afisat in culoarea sferei A

Proiectii paralele(1)

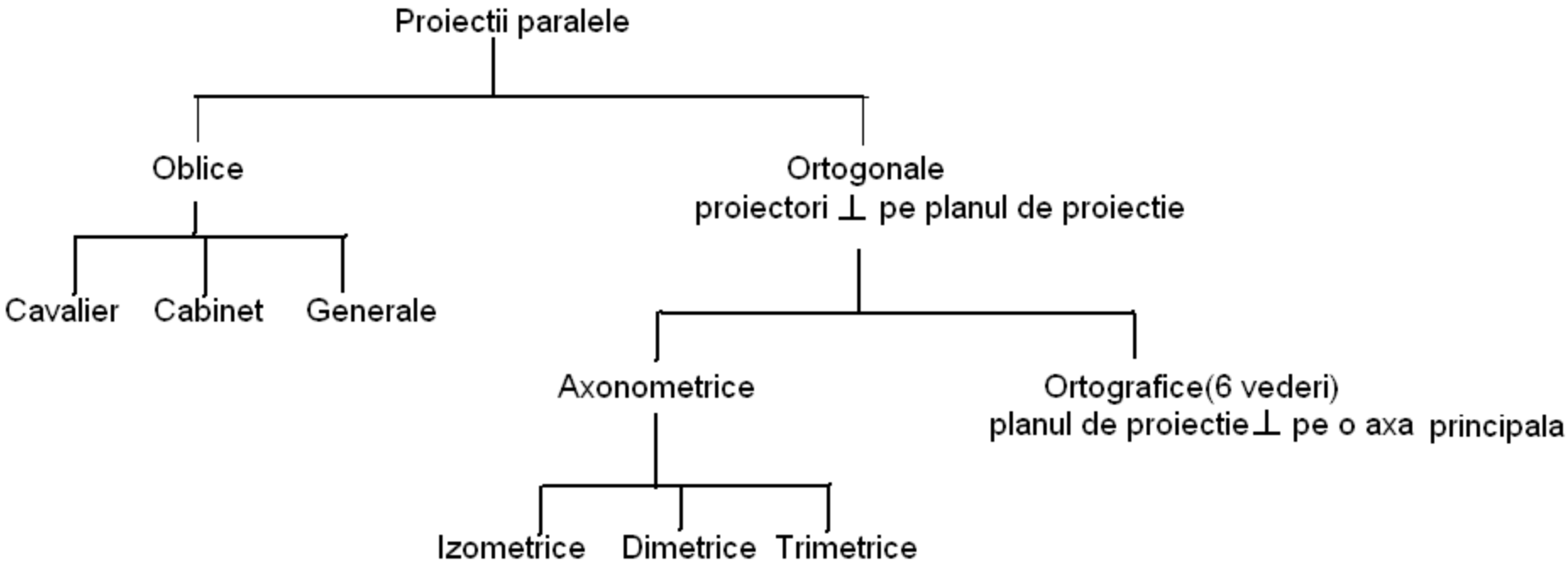
Caracteristici

- **Proiectorii sunt drepte paralele de directie data: directia de proiectie**
- **Sunt transformari afine (conserva paralelismul liniilor)**
- **Unghiurile se conserva doar pentru feţele obiectului paralele cu planul de proiectie.**



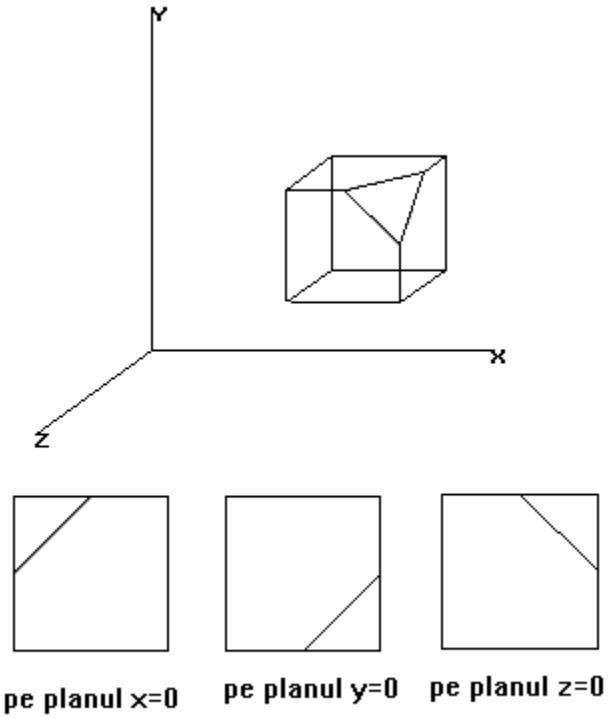
- Clasificare dupa unghiul dintre proiectori si planul de proiectie:
  - *Proiectii **ortogonale**:* proiectorii sunt perpendiculari pe planul de proiectie.
    - Ortografice: planul de proiectie este II cu un plan principal (⊥ pe o axa principala)
    - Axonometrice: planul de proiectie este oarecare
  - *Proiectii **oblice**:* proiectorii nu sunt perpendiculari pe planul de proiectie – cazuri particulare:
    - proiectii Cavalier (unghiul dintre proiectori si plan = 45 grade)
    - proiectii Cabinet (unghiul dintre proiectori si plan = 63.43 grade)

Proiectii paralele(2) - clasificare



Proiectii paralele(3)

1. Proiectii ortogonale - ortografice



Proiectii ortografice ale unui cub cu un colt taiat

- Proiectii in planele sistemului de coordonate (planul de proiectie ⊥ pe o axa principala) → numite şi **vederi**

- în YOZ: *vederea din stanga, vederea din dreapta,*
- în XOZ : *vederea de sus, vederea de jos,*
- în XOY : *vederea din fata, vederea din spate.*

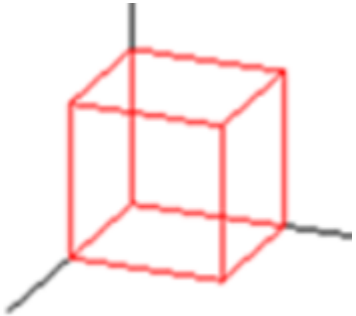
- Conserva lungimile laturilor si unghiurile dintre laturi
- Utile in desenul tehnic

Proiectii paralele(4)

2. Proiectii ortogonale - axonometrice

- Planul de proiectie nu este perpendicular pe nici una din axele sistemului de coordonate.
- Redau mai multe feţe ale obiectului proiectat (ca si cele perspectiva): alegand planul de proiectie se poate controla scalarea laturilor.





- In functie de unghiurile pe care planul de proiectie le face cu axele sistemului de coordonate:

- proiectii *izometrice*, planul face unghiuri egale cu toate cele trei axe → prin proiectie, laturile sunt scalate cu factori egali pe cele 3 axe: sx=sy=sz

- proiectii *dimetrice*, planul face unghiuri egale cu doua dintre axe → laturile sunt scalate cu factori de scalare egali pe 2 axe: sx=sy sau sx=sz sau sy=sz

- proiectii *trimetrice*, unghiurile dintre cele trei axe si plan sunt diferite → factorii de scalare ai laturilor pe cele 3 axe sunt diferiti: sx # sy # sz

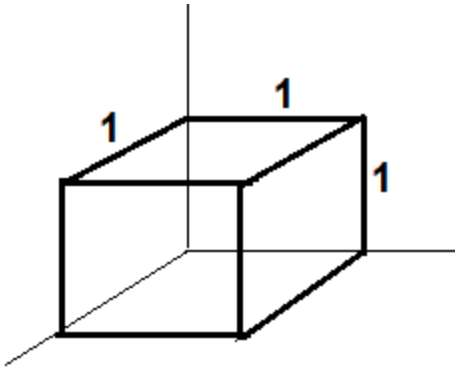
Proiectii paralele(5)

3. Proiectii oblice



- Planul de proiectie este perpendicular pe o axa principala.
- Directia de proiectie nu este perpendiculara pe planul de proiectie.
- Fețele II cu planul de proiectie se proiecteaza fara modificarea unghiurilor si a marimii laturilor
- Alegand directia de proiectie se obtin cazurile particulare:

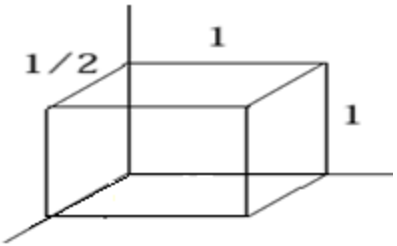
Proiectii oblice in XOY



❑ **Proiectii Cavalier**

(unghiul proiectorilor cu planul de proiectie = 45 grade)

- Lungimea proiectiei unei laturi perpendiculare pe plan este egala cu lungimea laturii 3D (conserva lungimea laturilor perpendiculare pe planul de proiectie)



❑ **Proiectii Cabinet**

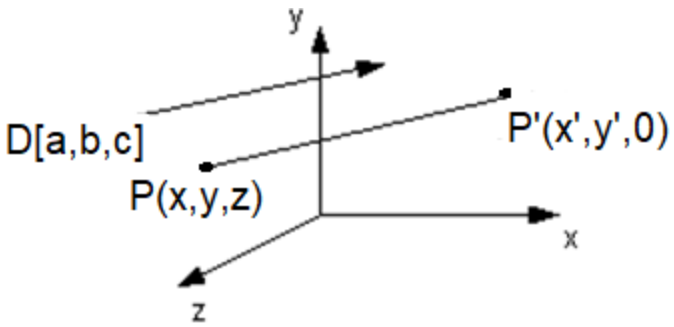
(unghiul proiectorilor cu planul de proiectie = 63.43 grade)

- Lungimea proiectiei unei laturi perpendiculare pe plan este egala cu jumatate din lungimea laturii 3D

Proiectii paralele in planul XOY(6)

Fie punctul P(x,y,z) şi direcţia de proiectie D[a,b,c].

Proiectia lui P in XOY este punctul P' (x',y',0).



Din conditia D || PP', rezulta:
PP' = s*D, unde s este un scalar oarecare
x' – x = s*a
y' – y = s*b
0 – z = s*c → s = z/c → x' = x – (a/c)*z
y' = y – (b/c) * z

Proiectii paralele in planul XOY(7)

Reprezentarea matriciala a proiectiilor paralele:

$$\begin{vmatrix} x' \\ y' \\ z' \\ 1 \end{vmatrix} = Pparalela \begin{vmatrix} x \\ y \\ z \\ 1 \end{vmatrix}$$

$$Pparalela = \begin{vmatrix} 1 & 0 & \frac{-a}{c} & 0 \\ 0 & 1 & \frac{-b}{c} & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

Ca si in cazul proiectiilor perspectiva, pentru conservarea informatiei de adancime se foloseste matricea →

$$Pparalela = \begin{vmatrix} 1 & 0 & \frac{-a}{c} & 0 \\ 0 & 1 & \frac{-b}{c} & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

Cazuri particulare

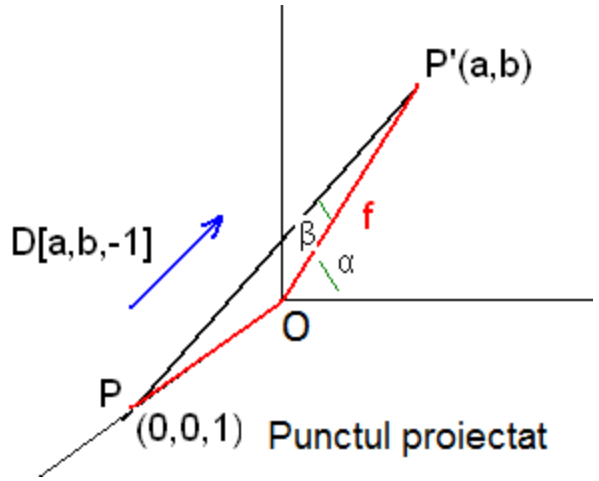
1. Proiectia ortografica in XOY (proiectorii perpendiculari pe XOY):

D[a, b, c]
a= 0, b = 0 →

$$P_{O\text{-}XOY} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Proiectii paralele in planul XOY(8)

2. Proiectii oblice:



c = -1 →

$$P_{OBl} = \begin{bmatrix} 1 & 0 & a & 0 \\ 0 & 1 & b & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$P_{OBl} = \begin{bmatrix} 1 & 0 & f*\cos(\alpha) & 0 \\ 0 & 1 & f*\sin(\alpha) & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

OP de lungime 1 se proiecteaza in OP' de lungime f: → f este factorul de scalare al laturilor perpendiculare pe XOY (OP este perpendicular pe XOY).

$tg(\beta) = 1/f$

$f = 0 \rightarrow \beta = 90^0$: proiectie ortografica

$f = 1 \rightarrow \beta = 45^0$: proiectie Cavalier

$f = 0.5 \rightarrow \beta = 63.43^0$: proiectie Cabinet

α este un parametru liber

in mod uzual, $\alpha = 30^0$ sau 45^0

Exemple de proiectii oblice



Proiectie Cavalier: f=1, beta=45, alfa=45

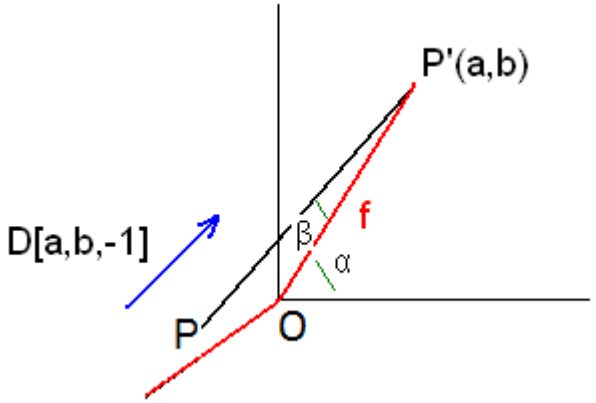
Directia de proiectie: [cos(45),sin(45),-1]



Proiectie Cabinet: f=0.5, beta=63.43, alfa=45

Directia de proiectie:

[0.5*cos(45),0.5*sin(45),-1]



Proiectii paralele in planul XOY(9)

3. Proiectii axonometrice(1)

2 posibilitati de exprimare matematica:

- Proiectie ortogonala intr-un plan oarecare

  - obiect fix in spatiu, alegere plan astfel incat sa obtinem cele 3 cazuri (izometrice, dimetrice, generale):

$$\begin{vmatrix} x' \\ y' \\ z' \\ 1 \end{vmatrix} = Pax(N,R) \begin{vmatrix} x \\ y \\ z \\ 1 \end{vmatrix}$$

- Transformare geometrica a obiectului (TA) urmata de proiectie ortografica in XOY (Po-xoy)– cu efect echivalent

  – plan de proiectie fix (XOY), pozitionare obiect in spatiu:

$$\begin{vmatrix} x' \\ y' \\ z' \\ 1 \end{vmatrix} = Po\text{-}xoy * TA \begin{vmatrix} x \\ y \\ z \\ 1 \end{vmatrix}$$

TA consta din rotatii in jurul axelor OX si OY. Particularizand unghiurile de rotatie obtinem cazurile de proiectii axonometrice: izometrice, dimetrice, generale.

Proiectii paralele in planul XOY(10)

Proiectii axonometrice(2)

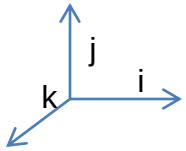
Exemplu:

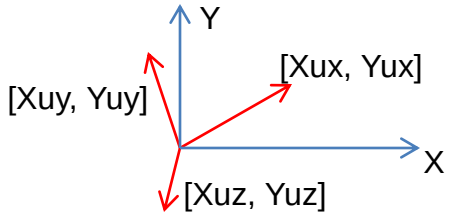
TA = Rx(ux) * Ry(uy) PA = Po-xoy* TA =

$$\begin{bmatrix} \cos(uy) & 0 & \sin(uy) & 0 \\ \sin(ux)^*\sin(uy) & \cos(ux) & -\cos(uy)^*\sin(ux) & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Se aplica transformarea PA versorilor axelor principale:

$$\begin{bmatrix} Xux & Xuy & Xuz \\ Yux & Yuy & Yuz \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix} = PA \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$





Factorii de scalare a laturilor pe cele 3 axe sunt lungimile proiectiilor celor trei vectori unitate:

$$sx = (Xux^2 + Yux^2)^{0.5}$$

$$sy = (Xuy^2 + Yuy^2)^{0.5}$$

$$sz = (Xuz^2 + Yuz^2)^{0.5}$$

Proiectii paralele in planul XOY(11)

Proiectii axonometrice(3)

Se deduc unghiurile de rotatie in jurul axelor pentru obtinerea celor 2 cazuri particulare de proiectii axonometrice: izometrice si dimetrice.

1. *Proiectii izometrice:*

- Se impune conditia: sx = sy = sz
- Rezulta: ux = +/- 35.26 grade ; uy = +/- 45 grade

2. *Proiectii dimetrice:* doi dintre factorii de scalare sunt egali

- Daca se impune conditia: sx = sy, rezulta:

$$ux = \arcsin(+/- sz/\sqrt{2}), \quad uy = \arcsin(+/- sz/\sqrt{(2 - sz^2)})$$

  - sz poate fi ales intre 0 si 1;
  - pentru fiecare valoare a lui sz exista 4 proiectii dimetrice; de exemplu:
  - pentru sz = 0.5 : ux = +/- 20.705 grade, uy = +/- 22.208 grade

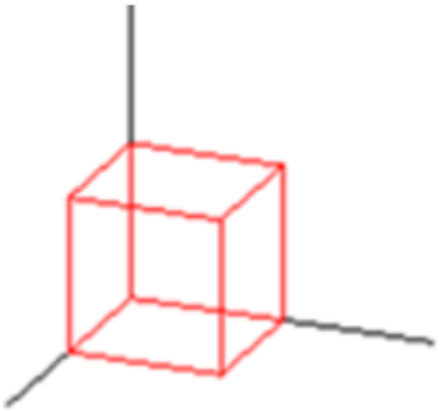
Exemple de proiectii axonometrice



Proiectie izometrica:

P_ort_XOY * R_Ox(35.26)* R_Oy(45)

sx = sy = sz



Proiectie dimetrica:

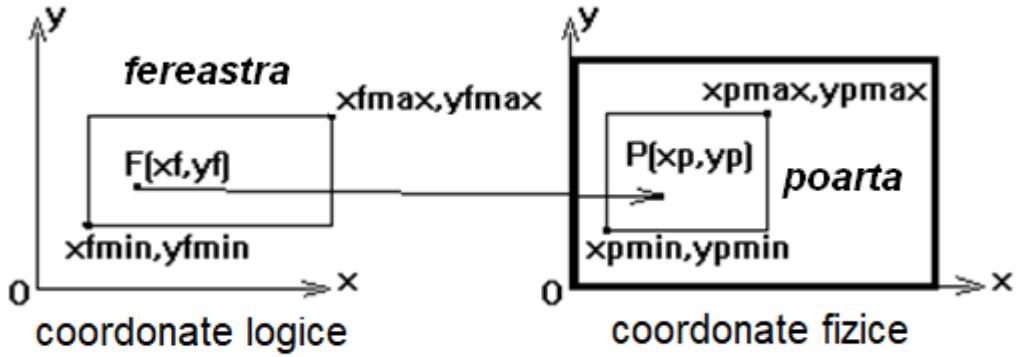
P_ort_XOY * R_Ox(20.705)* R_Oy(22.208)

sx = sy # sz

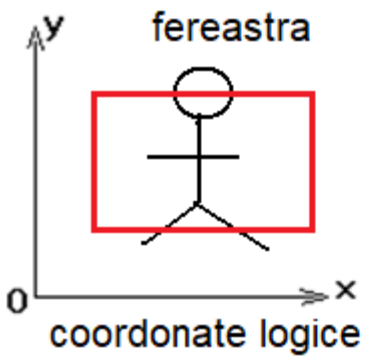
Implementarea proiectiilor într-o aplicatie OpenGL (1)

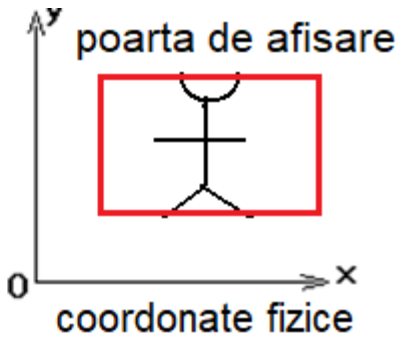
- Proiectia varfurilor primitivelor 3D este inclusa in **transformarea de vizualizare 3D.**

Analogie intre transformarea de vizualizare 2D si transformarea de vizualizare 3D



- Desenul este descris in coordonate logice.
- Coordonatele logice sunt transformate in coordonate fizice folosind matricea transformarii de vizualizare 2D

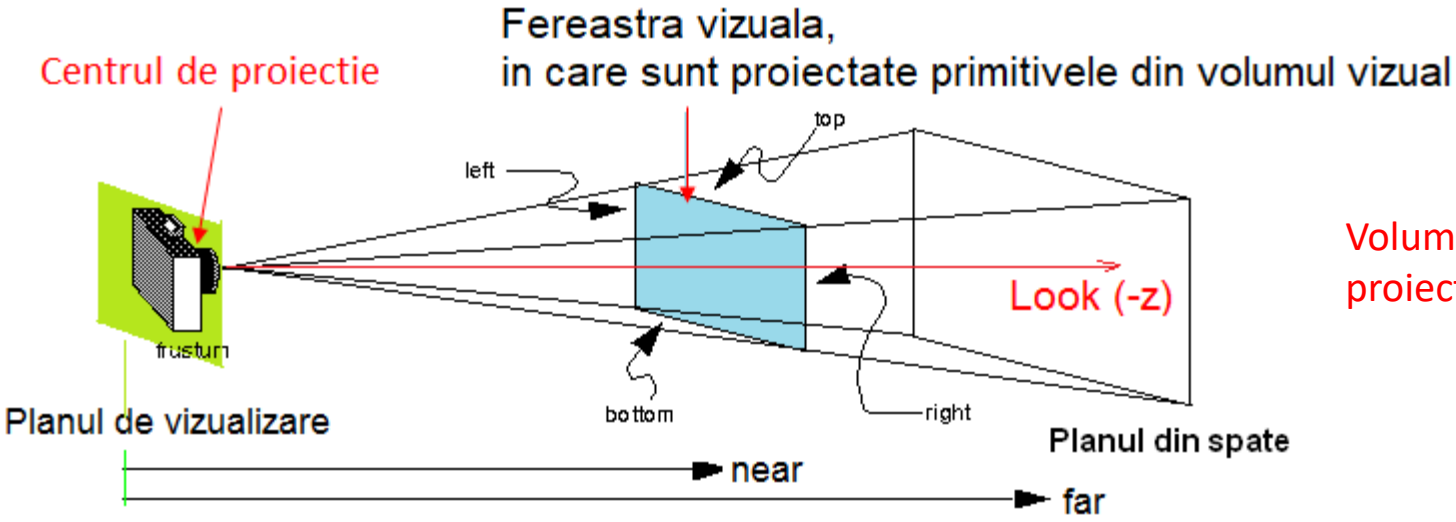




- Partile din desenul descris in coordonate logice care nu sunt incluse in fereastra sunt decupate (nu sunt afisate in poarta de afisare)

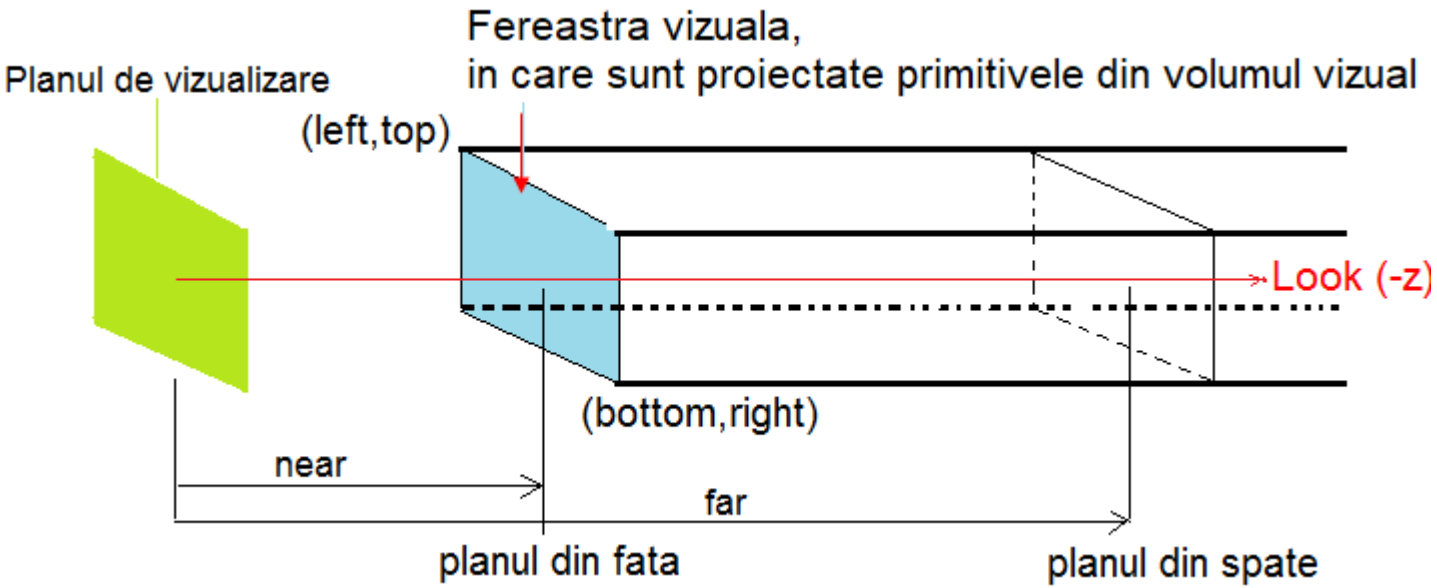
Implementarea proiectiilor într-o aplicatie OpenGL (2)

- In OpenGL pot fi specificate 2 tipuri de proiectii:
  - Proiectia perspectiva
  - Proiectia ortografica
- Obiectele 3D sunt reprezentate in coordonate logice (sistemul de coordonate carteziene 3D).
- Coordonatele logice sunt transformate in coordonate fizice prin transformarea de vizualizare 3D care include o transformare de proiectie.
- Partile din scena 3D, descrisa in coordonate logice, care nu sunt incluse in volumul vizual, sunt decupate la marginile volumului visual (nu vor fi redate in imagine).



Volumul vizual pentru proiectia perspectiva

Implementarea proiectiilor într-o aplicatie OpenGL (3)



Volumul vizual pentru proiectia ortografica

Cum putem implementa intr-un program OpenGL:

- O proiectie axonometrica?

Se foloseste proiectia ortografica asupra varfurilor transformate cu matricea TA

Transformarea de vizualizare din spatiul 3D

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2018-2019

Transformarea de vizualizare din spatiul 3D

- Are ca scop obtinerea unei vederi a modelului unei scene 3D pe o suprafaţă de afişare.
- Poate fi comparată cu transformarea efectuată de o cameră foto asupra lumii reale pentru a se obţine imaginea dintr-o fotografie.
  - Imaginea obţinută depinde de:
    - poziţia camerei (observatorului)
    - orientarea camerei
    - ceea ce vede ochiul prin vizorul camerei foto (volumul vizual)
- Este o transformare compusă din mai multe transformări, între care şi o proiectie $R^3 \rightarrow R^2$
- Transformările se efectuează asupra vârfurilor obiectelor din scena 3D.
- Include şi operaţia de decupare (clipping) a primitivelor grafice la marginea volumului vizual (eliminarea părţilor din scena 3D care sunt în afara volumului vizual).

Transformarea varfurilor in OpenGL

Spatiul obiect – Object space (Sistemul coordonatelor obiect)

Matricea M — **Transformarea de modelare**

Spatiul scenei 3D – World space (Sistemul coordonatelor globale)

Matricea V — **Transformarea de vizualizare**

Spatiul observator – Eye space (Sistemul coordonatelor observator)

Matricea P — **Transformarea de proiectie (volumul vizual)**

Spatiul de decupare – Clip space (Sistemul coordonatelor de decupare)

Impărţirea perspectivă

Sistemul coordonatelor dispozitiv normalizate

Transformarea în poarta de afişare (viewport)

Sistemul coordonatelor ecran ataşat porţii de afişare

Matricile transformarii varfurilor obiectelor 3D

M – matricea de modelare: difera de la obiect la obiect

V – matricea de vizualizare: aceeasi pentru intreaga scena vizualizata

P – matricea de proiectie: aceeasi pentru intreaga scena vizualizata

MV = V· M (matricea “model-view”)

MVP = P · V · M = P· MV (matricea “model-view-projection”)

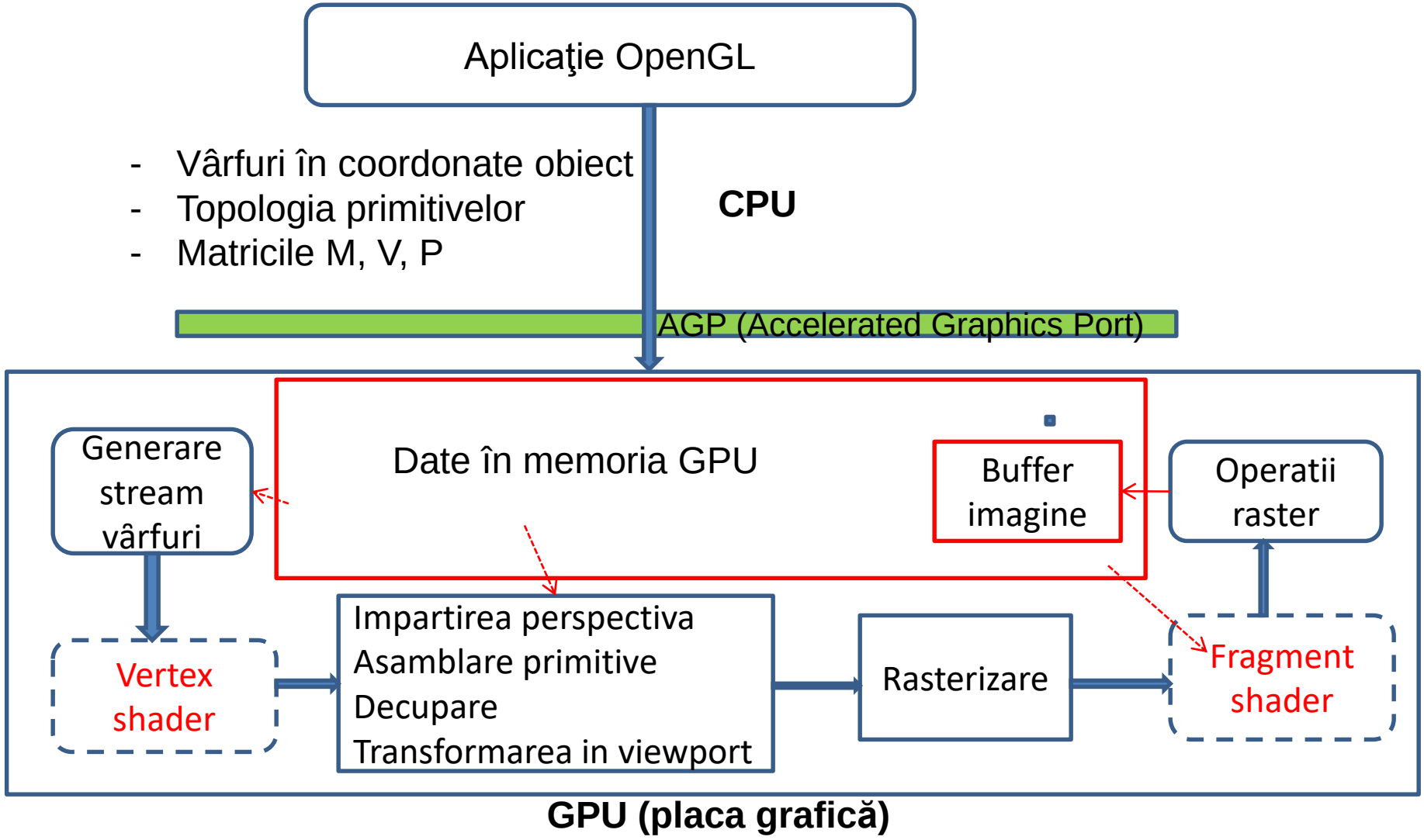
Transformarea vârfurilor din coordonate obiect în coordonate de decupare (clip):

$$[xc, yc, zc]^T = MVP \cdot [xo, yo, zo]^T$$

- Matricile sunt calculate în programul OpenGL folosind functii ale bibliotecii **glm** şi transferate la GPU
- Vârfurile sunt transformate în programul ***Vertex Shader executat pe GPU,*** folosindu-se matricea MVP
- Dupa asamblarea primitivelor, folosindu-se varfurile transformate şi topologia transmisa la GPU de programul OpenGL, primitivele sunt decupate la frontiera volumului vizual.

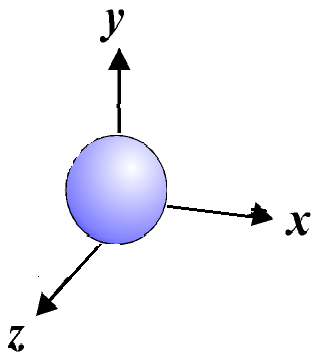
Banda grafică- a V-a generaţie GPU

➢Incepand cu OpenGL 3.0:



Transformarea de modelare (1)

Spatiul obiect (Sistemul coordonatelor obiect)



- In mod uzual, fiecare obiect este definit in propriul sau sistem de coordonate (sistemul coordonatelor locale – coordonate obiect)
- Sistem de coordonate carteziene 3D « dreapta »
- Exemple: cubul, sfera, scheletul unui personaj animat, elemente de mobilier, etc.

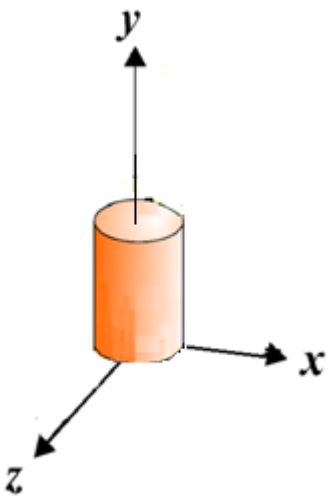
  De regula, obiectul este definit centrat in originea sistemului de coordonate.
- Avantaje: usurinta modelarii şi reutilizarea modelelor obiectelor 3D

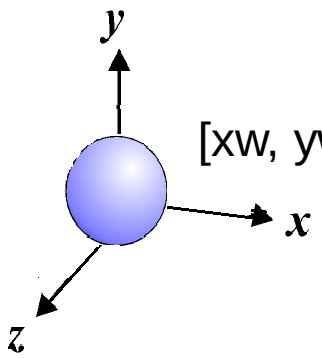
Spatiul scenei 3D -World space (Sistemul coordonatelor globale)

- Spaţiul in care este compusă scena 3D
- Raportat la un sistem de referinţă global – sistem de coordonate carteziene 3D “dreapta”
- Scena redată într-un cadru imagine este reprezentată în sistemul coordonatelor globale

Transformarea de modelare (2)

- Scop: dimensionarea si poziţionarea obiectelor scenei 3D într-un cadru imagine
- Este o transformare geometrica compusa.

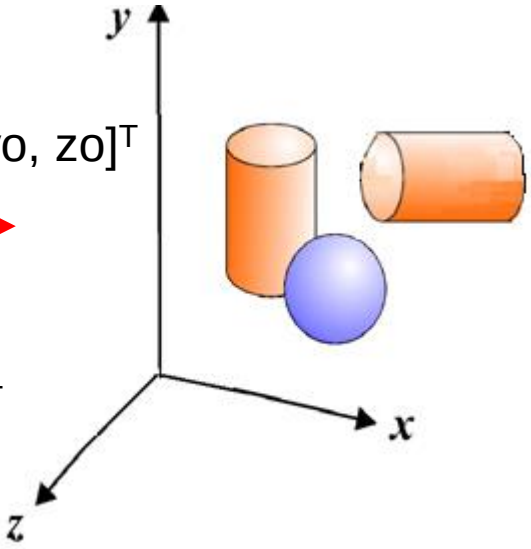




$[xw, yw, zw]^T = M1 \cdot [xo, yo, zo]^T$

$[xw, yw, zw]^T = M2 \cdot [xo, yo, zo]^T$

$[xw, yw, zw]^T = M3 \cdot [xo, yo, zo]^T$



Spatiul scenei 3D

Coordonatele obiect ale vârfurilor obiectelor sunt transformate pentru modelarea scenei 3D

Transformarea de modelare

↓

Rotatie, Scalare, Translatie

Obiecte definite în sistemul coordonatelor globale

Transformarea de vizualizare

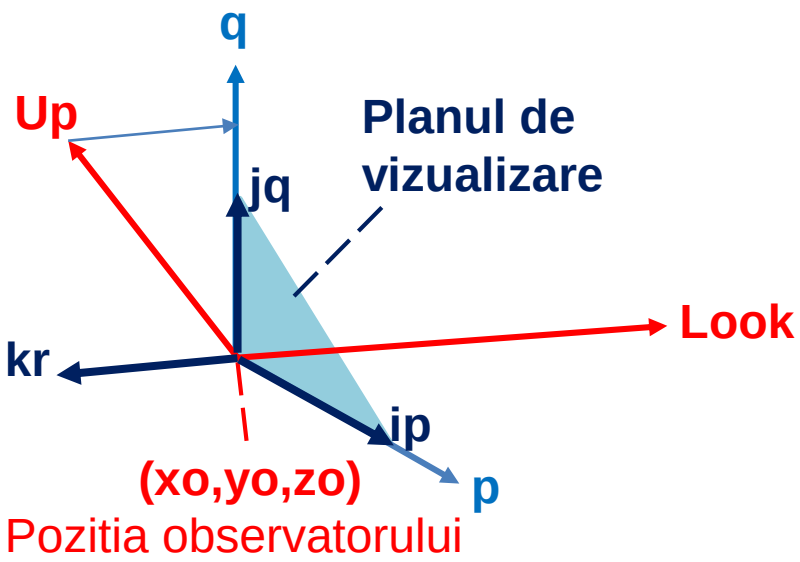
din sistemul de coordonate globale → în sistemul de coordonate observator

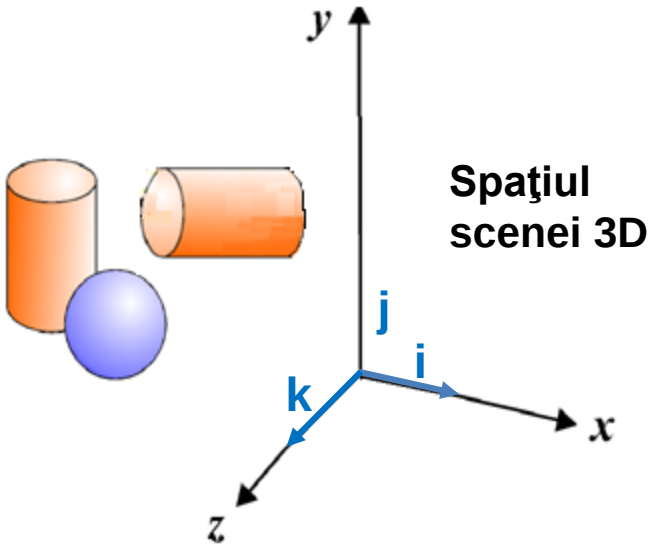
- O imagine este o vedere obtinuta dintr-un punct al spatiului 3D, similara unei fotografii:
- Depinde de:
  - Pozitia camerei (observatorului) in spatiul lumii reale.
  - Orientarea camerei: directia in care priveste observatorul si rotatia camerei fata de axa sa.

- Spaţiul observator este spaţiul 3D văzut din pozitia observatorului pe direcţia în care priveşte observatorul.
- Obiectele 3D din spatiul observator sunt raportate la un sistem de coordonate cu originea in pozitia observatorului (camerei virtuale)
- **Sistemul de coordonate observator este definit prin 3 parametri**:

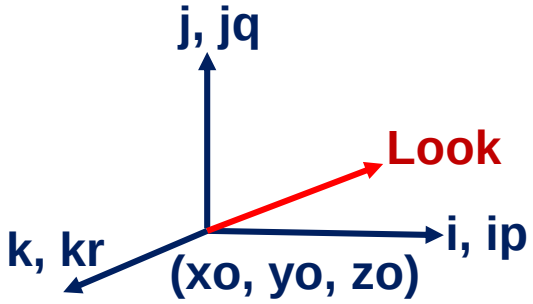
- Originea sa: **pozitia camerei virtuale** in spatiul coordonatelor globale **(xo, yo, xo).**
- **Directia in care priveste observatorul** (spre scena 3D) – **vectorul Look**, reprezinta normala la planul de vizualizare (**N**).
- Directia “sus” a planului de vizualizare (**vectorul Up**), care determina directia axei OY a sist. de coordonate observator: reprezinta **rotatia camerei in jurul axei privirii (Look)**

Sistemul de coordonate observator



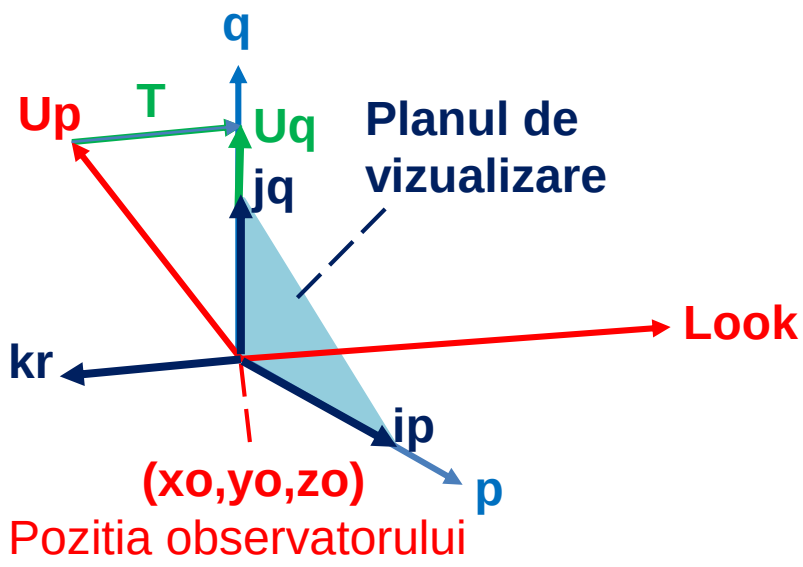


- (xo, yo, zo): originea sistemului de coordonate observator
- Look – normala la planul de vizualizare
- axa q: axa verticală a planului de vizualizare – are directia data de proiectia vectorului UP în planul de vizualizare
- (ip,jq,kr) formeaza un sistem de coordonate 3D dreapta



ip, jq, kr: versorii axelor sistemului de coordonate observator coincid cu cei ai sist. coordonatelor globale

Determinarea axelor sistemului de coordonate observator



kr = - Look/|Look|

Uq: proiectia ortogonala a vectorului Up in planul de vizualizare

Uq dă direcţia axei verticale a planului de vizualizare

Uq = Up + T

T || (-kr) → T = s*(-kr) = - s*kr (s un scalar oarecare) → Uq = Up - s*kr

Uq perpendicular pe kr → Uq·kr = 0 → (Up - s*kr)·kr =0 → Up·kr - s*kr·kr = 0 → s = Up·kr

Uq = Up – (Up·kr)·kr

jq = Uq/|Uq| este versorul axei verticale a planului de vizualizare

ip = (jq x kr) / |(jq x kr)| este versorul axei orizontale a planului de vizualizare

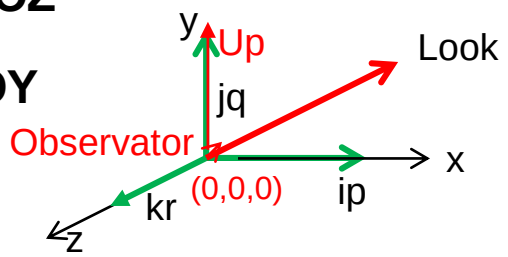
(ip, jq, kr) formeaza un sistem de coordonate carteziene 3D dreapta

Calculul matricei transformarii de vizualizare

Sistemul de coordonate observator implicit in OpenGL

- poziţia observatorului: **originea sistemului de coordonate globale**
- direcţia în care priveşte observatorul: **direcţia negativa a axei OZ**
- direcţia sus a planului de vizualizare: **direcţia pozitivã a axei OY**

Cu aceste valori implicite, transformarea de vizualizare este transformarea identica.



Sistemul de coordonate observator implicit in OpenGL

Pentru calculul matricei transformarii de vizualizare poate fi folosita functia **lookAt:**

glm::mat4 **V** =

glm::lookAt (glm::vec3 (xo,yo,zo), glm::vec3 (Px,Py,Pz), glm::vec3 (Upx, Upy, Upz))

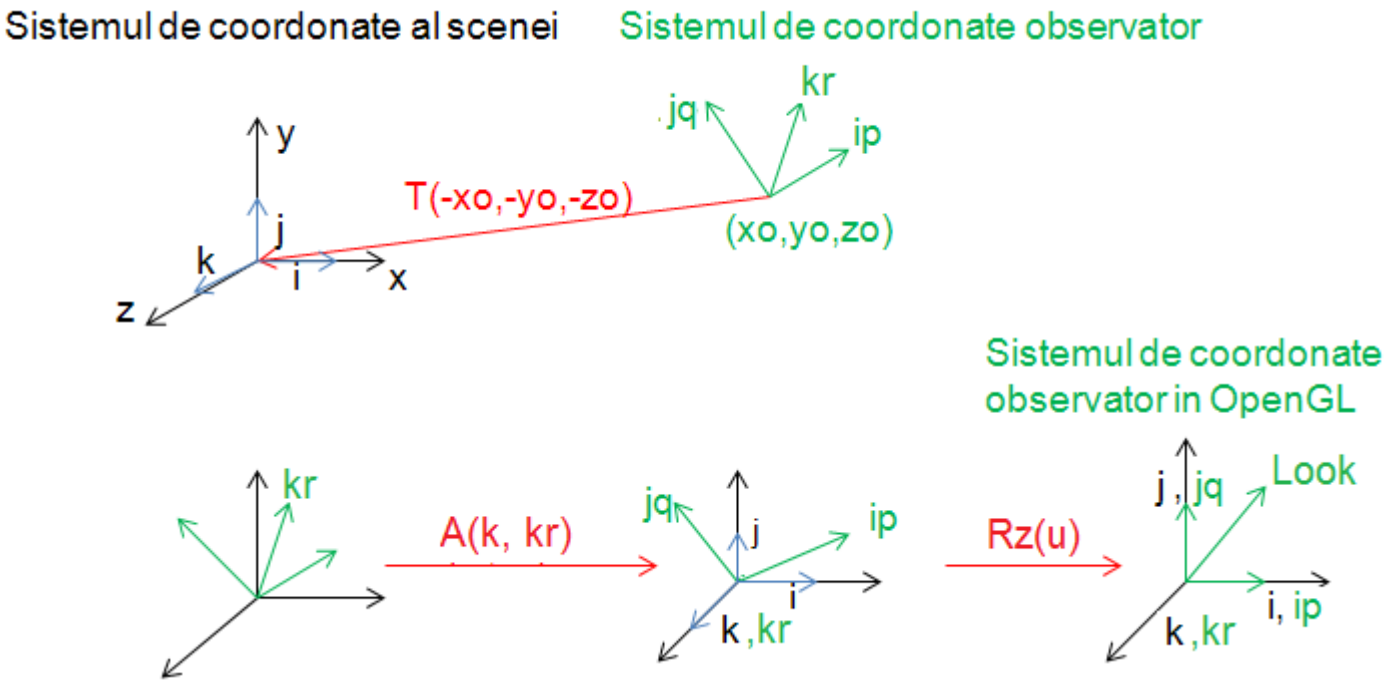
(xo, yo, zo): **poziţia observatorului**

(Px,Py,Pz): punctul din scena 3D catre care priveste observatorul;

vectorul (xo,yo,zo) – (Px,Py,Pz) reprezinta **vectorul Look**

(Upx, Upy, Upz): **vectorul « sus » al planului de vizualizare (Up)**

Matricea transformarii de vizualizare



- A(k,kr) este o transformare prin care se aliniaza axele z ale celor 2 sisteme de coordonate: consta din 2 rotatii – o rotatie in jurul axei X si una in jurul axei Y.
- Prin rotatia in jurul axei z (Rz) se aliniaza celelalte 2 axe.

Matricea transformarii de vizualizare: V = Rz(u) * A(k,kr) * T(-xo,-yo,-zo)

Transformarea de modelare si vizualizare

din sistemul de coordonate obiect → in sistemul de coordonate observator

- Transformare compusa (model-view) : MV = V•M

  M – matricea de modelare folosita pentru transformarea unui obiect in spatiul scenei

  V – matricea de vizualizare a scenei

  Transformarea varfurilor din coordonate obiect in coordonate observator (*eye*):

  $[xe\ ye\ \ ze\ we]^T = MV \bullet [xo\ yo\ zo\ wo]^T$

- Pentru calculele de iluminare a obiectelor scenei 3D se folosesc vectori normali atasati varfurilor obiectelor. Calculele de iluminare se fac, de regula, in sistemul coordonatelor observator.

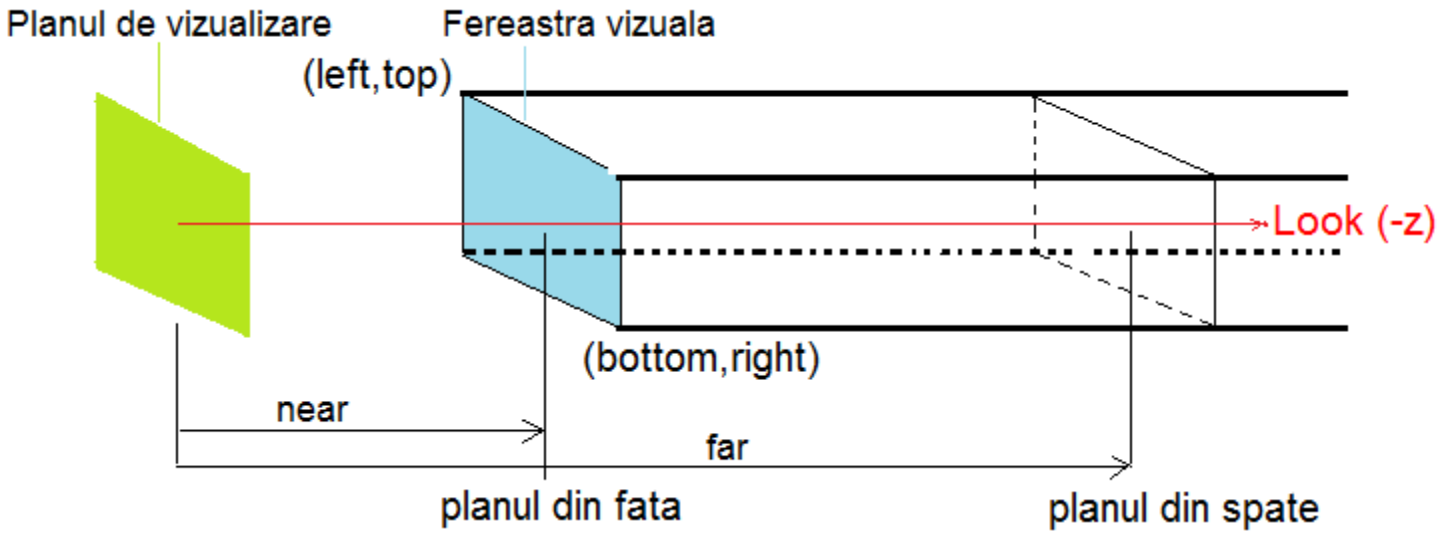
  Transformarea normalelor din coordonate obiect in coordonate observator:

  $[nxe\ nye\ \ nze\ we]^T = (MV^{-1})^T \bullet [nxo\ nyo\ nzo\ nwo]^T$

Transformarea de proiectie (1)

Determinata de **volumul vizual** definit de programator

Volumul vizual al proiectiei ortografice: paralelipiped dreptunghic cu laturile paralele cu axa z a sistemului de coordonate observator (SCO), delimitat in adancime de planul din fata (near) si planul din spate (far) al vederii; near si far sunt distanţe (pozitive) faţă de planul de vizualizare masurate pe axa z negativa a SCO.

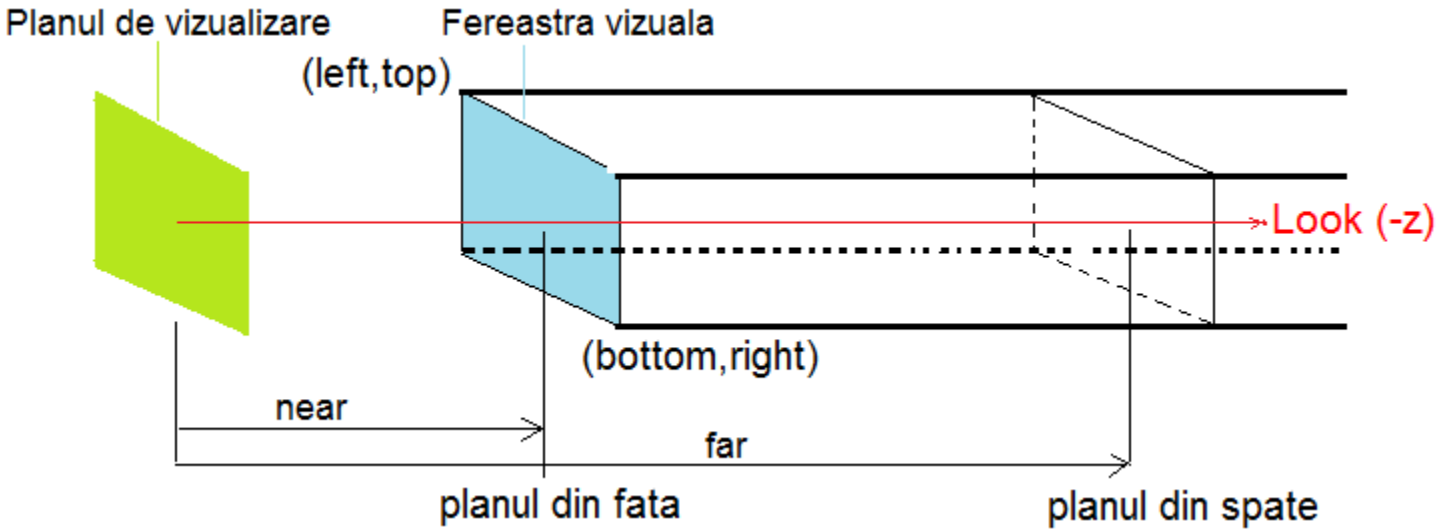


- Coordonatele colturilor ferestrei vizuale sunt raportate la sistemul de coordonate al planului din faţă, care are axele paralele cu cele ale planului de vizualizare.

Transformarea de proiectie (2)

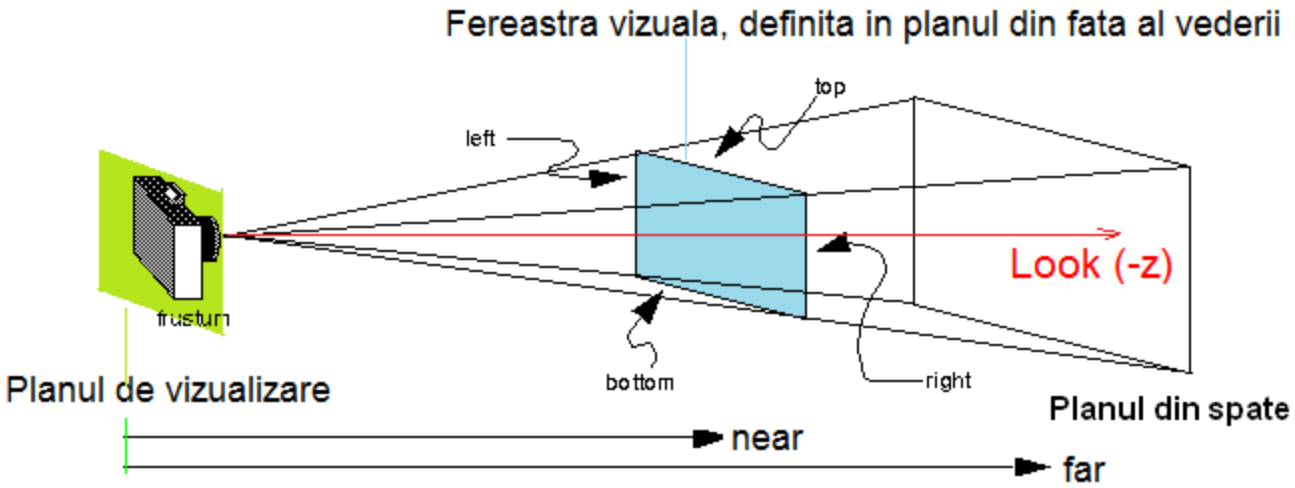
Proiectia ortografica

- Se efectueaza proiectia ortografica în fereastra vizuala a obiectelor cuprinse in volumul vizual. Primitivele grafice situate complet in afara volumului vizual sunt eliminate iar cele care intersecteaza volumul sunt decupate la frontiera volumului creindu-se noi primitive.



Transformarea de proiectie (3)

Volumul vizual al proiectiei perspectiva (View frustum): trunchi de piramida delimitat in adancime de planul din faţă (near) si planul din spate (far) al vederii, cu varful in pozitia observatorului.

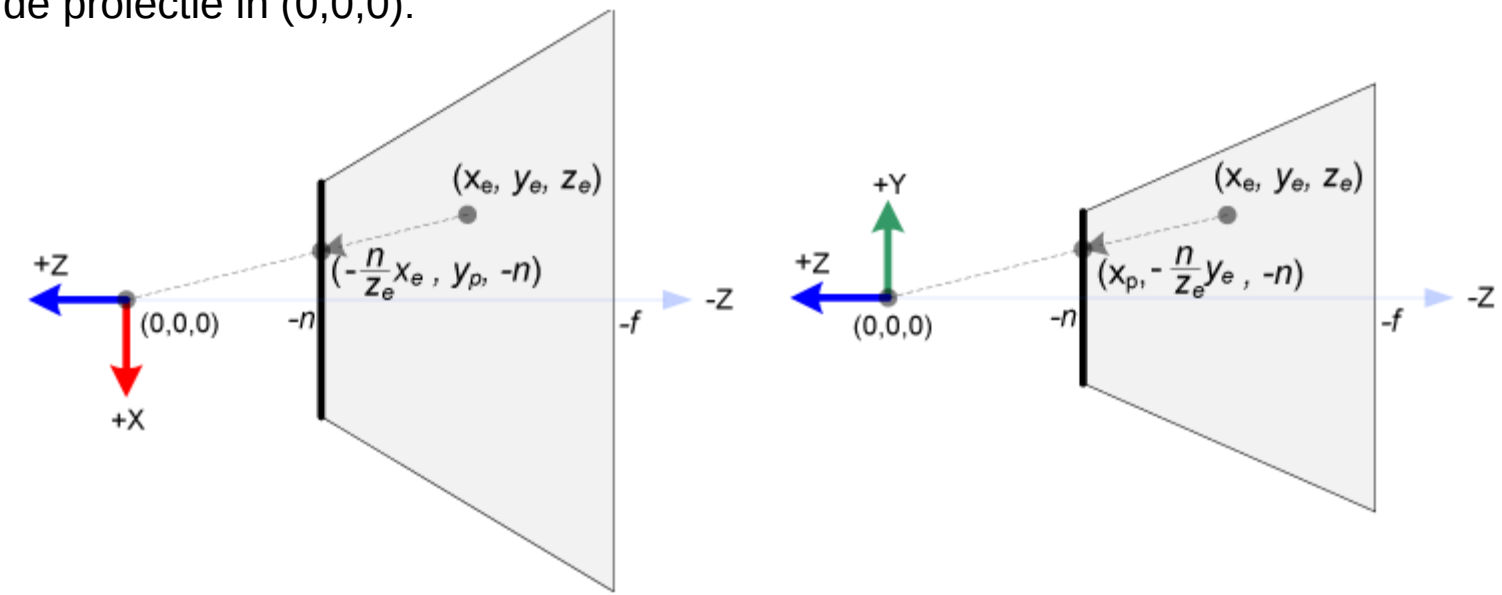


Se efectueaza proiectia perspectiva în planul din faţă (in fereastra vizuala), cu centrul de proiecţie în poziţia observatorului, a obiectelor cuprinse in volumul vizual. Obiectele situate in afara volumului vizual sunt eliminate sau decupate la frontiera volumului vizual.

Transformarea de proiectie (4) - proiectia

❖ **Proiectia se efectueaza in fereastra vizuala din planul din** faţă: planul z =-near (in SCO).

Proiectia perspectiva a unui punct din SCO, (xe,ye,ze), pe planul din fata al vederii, cu centrul de proiectie in (0,0,0):



xp = (-near/ze)*xe, yp =(-near/ze)*ye – se pot obtine aplicand formulele proiectiei perspectiva cu centrul de proiectie in (0,0,0) si planul de proiectie este z=-near.

Proiectia ortografica a unui punct din sist. coordonatelor observator, (xe,ye,ze), in fereastra vizuala:

xp = xe, yp = ye, zp = -near.

Transformarea de proiectie (5)

- Transformarea de proiectie este insotita de operatia de decupare a primitivelor grafice la marginile volumului vizual.
- Pentru simplificarea calculelor de decupare, volumul vizual definit de programator este transformat in volumul vizual canonic: cub cu latura de 2 unitati, centrat in originea sistemului coordonatelor de decupare, cu laturile paralele cu axele.
- Transformarea prin care volumul vizual este transformat in volumul vizual canonic se numeste transformarea de normalizare.

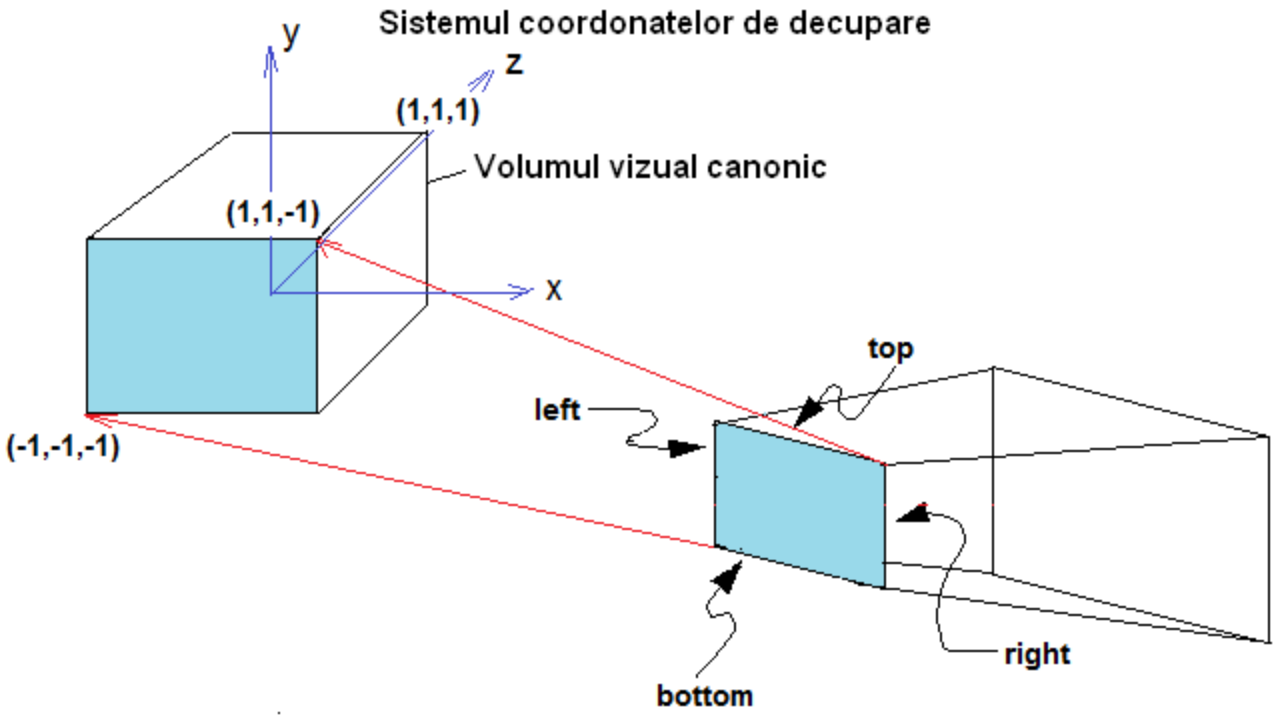
- In OpenGL transformarea de proiectie include: proiectia definita prin volumul de vizualizare compusa cu transformarea de normalizare.

  = transformarea coordonatelor observator in coordonate de decupare.

- Dupa transformarea varfurilor prin transformarea de proiectie, varfurile sunt unite pe baza topologiei definite de programator, formand primitive grafice 3D. Acestea sunt decupate la marginile volumului vizual canonic.

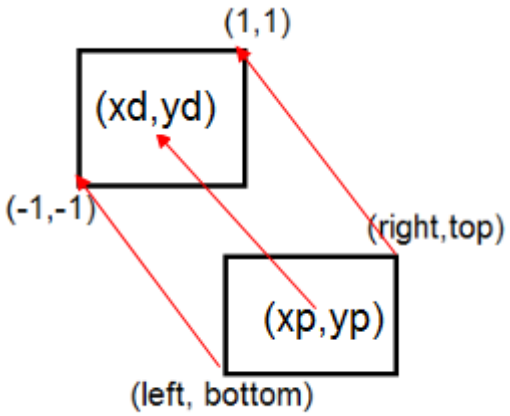
Transformarea de proiectie (6) – normalizarea

- *Sistemul coordonatelor de decupare*: sistem de coordonate carteziene 3D stanga.
- Prin transformarea de normalizare:
  - colţurile ferestrei 2D din planul de proiecţie, (left, bottom,-near) şi (right, top,-near) sunt mapate pe colţurile stânga-jos şi dreapta-sus ale ferestrei 2D din sistemul coordonatelor de decupare, adică (-1,-1,-1) şi (1,1,-1);
  - planul z= -near se mapeaza pe z= -1, iar z= -far se mapeaza pe z= 1



Transformarea de proiectie (7) calculul coordonatelor de decupare

- Impunand conditia (left,bottom) → (-1,-1) si (right,top) → (1,1), rezulta transformarea punctelor din planul din faţă, (xp,yp), in sistemul coordonatelor de decupare – este o transformare freastra-poarta:



xd = 2*xp/(right – left) – (right + left)/(right – left)

yd = 2*yp/(top – bottom) – (top + bottom)/(top-bottom)

Inlocuind xp si yp rezultate din proiectia perspectiva in planul -near:

xd = (xe*2*near/(right – left) + ze*(right + left)/(right – left))/(-ze)

yd = (ye*2*near/(top – bottom) + ze*(top + bottom)/(top-bottom))/(-ze)

Pentru transformarea coordonatelor z se impun conditiile: -near → -1, -far → 1; rezulta:

zd = (-ze*(far+near)/(far-near) – 2*far*near/(far-near)) / (-ze)

(xd,yd,zd) sunt coordonate carteziene in sistemul coordonatelor de decupare (clip coordinates); folosind matricea transformarii de proiectie rezulta coordonate omogene:

[xc, yc, zc, w] → xd = xc/w; yd = yc/w; zd = zc/w ; w = - ze

Obs: w trebuie sa fie ≠ 0 → punctul proiectat nu poate fi in planul camerei (centrul de proi.)

Transformarea de proiectie (8) matricea transformarii

Matricea transformarii de proiectie pentru proiectia perspectiva:

$$P = \begin{bmatrix} \frac{2*near}{right - left} & 0 & A & 0 \\ 0 & \frac{2*near}{top - bottom} & B & 0 \\ 0 & 0 & C & D \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

$$A = \frac{right + left}{right - left} \qquad B = \frac{top + bottom}{top - bottom}$$

$$C = -\frac{far + near}{far - near} \qquad D = -\frac{2*far*near}{far - near}$$

Matricea transformarii de proiectie pentru proiectia ortografica:

xp=xe si yp=ye → xc = 2*xe/(right – left) – (right + left)/(right – left)

yc = 2*ye/(top – bottom) – (top + bottom)/(top-bottom)

$$P = \begin{bmatrix} \frac{2}{right - left} & 0 & 0 & -A \\ 0 & \frac{2}{top - bottom} & 0 & -B \\ 0 & 0 & \frac{-2}{far - near} & C \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$[xc\ yc\ \ zc\ w]^T = P \bullet [xe\ ye\ ze\ we]^T$

Transformarea de proiectie (9)

Matricile de proiectie pot fi obtinute apeland urmatoarele functii din biblioteca glm:

glm::ortho(float left, float right, float bottom, float top, float zNear, float zFar);

glm::frustum (float left, float right, float bottom, float top, float zNear, float zFar);

glm::perspective (float fov, float aspect, float znear, float zfar);

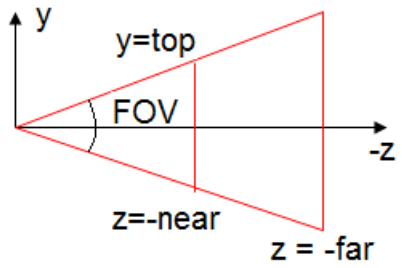
Creaza o matrice de proiectie pentru un volum perspectiva simetric:

- fov: Field of View

  tan(FOV/2) = top/near

- aspect: aspect ratio

  aspect ratio = (right-left)/(top-bottom)



= raportul dintre latimea si inaltimea ferestrei vizuale

Transformarea de proiectie (10)

din coordonate observator → in coordonate de decupare

$[xc\ yc\ zc\ w]^T = P \bullet [xe\ ye\ ze\ we]^T$ (clip coordinates) ← eye coordinates)

Impartirea perspectiva

Coordonate omogene → coordonate carteziene: xd = xc/w , yd = yc/w , zd= zc/w

(xd, yd, zd): coordonate dispozitiv normalizate

- **Varfurile rezultate din impartirea perspectiva sunt asamblate în primitive grafice care sunt decupate la frontiera volumului vizual canonic.**

Transformarea de modelare-vizualizare-proiectie

❖ Prin inmultirea matricilor transformarilor de modelare, vizualizare si proiectie se obtine matricea numita “model-view-projection”, a.i. asupra varfurilor definite in sistemul coordonatelor obiect poate fi efectuata o singura transformare:

MVP = P•V•M (matricea ModelViewProjection)

Transformarea in poarta de afisare

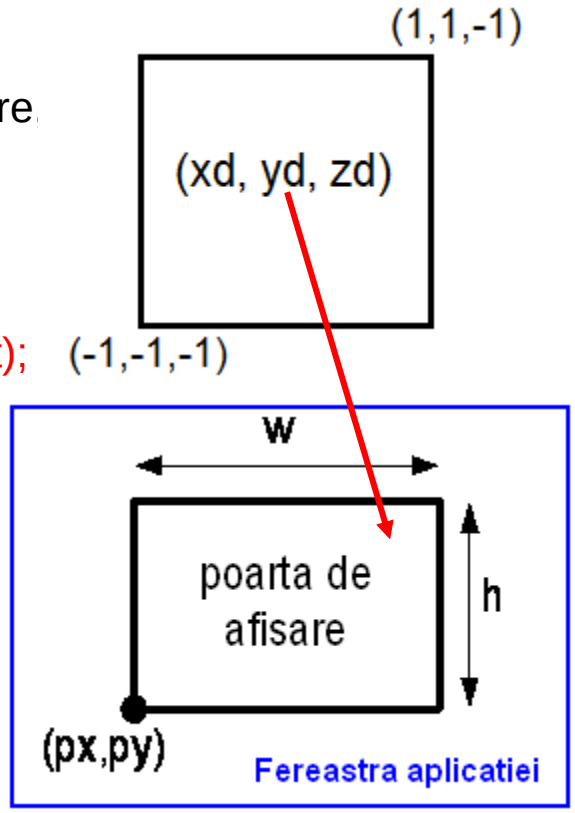
- Este o transformare fereasta-poarta, in care:
- Fereastra este fereastra 2D din sistemul coordonatelor de decupare,

  având colţurile în (-1, -1, -1) - (1, 1, -1).
- Poarta este definita de programator folosind functia:

  void **glViewport**(GLint px, GLint Py, GLsizei width, GLsizei height);

  (*px, py*) reprezintă coordonate relative la coltul stanga-jos al ferestrei aplicatiei (în pixeli). Valorile implicite : (0,0).

  width, height reprezintă lăţimea/ înălţimea porţii de afişare. Valorile implicite sunt: lăţimea şi înălţimea ferestrei aplicatiei.



- **Transformarea in poarta de afisare este definita astfel:**

  xv = (width/2)xd + ox , ox = px+ width/2

  yv = (height/2)yd + oy, oy = py + height/2

  zv = ((f-n)/2)zd + (n+f)/2 // conserva coordonata z pentru eliminarea partilor nevizibile

  n (near) şi f (far) au valorile implicite 0.0 respectiv 1.0, pentru care 0<=zv<=1

  n si f pot fi modificate la valori cuprinse în intervalul [0,1] folosind funcţia ***DepthRange()***

Eliminarea partilor nevizibile ale scenelor 3D din imagini - 1

Prof. univ. dr. ing. Florica Moldoveanu

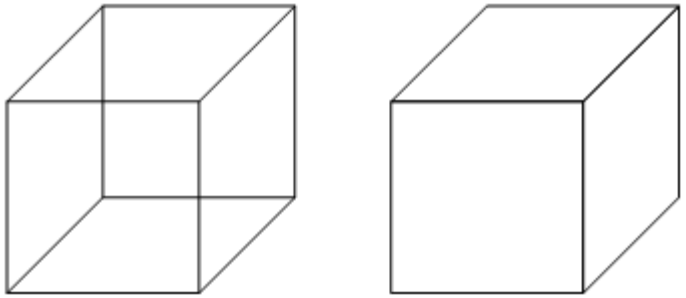
Eliminarea partilor nevizibile ale scenelor 3D, din imagini (1)

Vizibilitatea obiectelor dintr-o scena 3D depinde de:

– Pozitia observatorului si directia in care priveste

– Pozitionarea obiectelor din scena unul fata de celalalt

– Volumul vizual



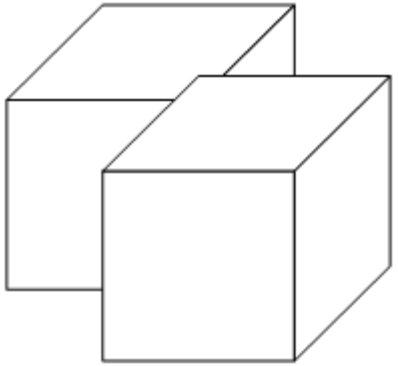
Partile unei scene 3D nevizibile într-o imagine sunt:

– Părți aflate total sau parțial în afara volumului vizual

– Fețe auto-obturate

– Părți obturate (ascunse) de obiecte ale scenei,

aflate în fața lor în raport cu observatorul



Eliminarea partilor nevizibile ale scenelor 3D, din imagini (2)

Efectuata la nivel de:

1. Obiecte sau grupuri de obiecte - **Object culling**:

   - **Frustum Culling**: eliminarea din banda grafica a obiectelor sau grupurilor de obiecte care sunt complet in afara volumului vizual
   - **Occlusion Culling**: – eliminarea din banda grafica a obiectelor sau grupurilor de obiecte care sunt complet obturate de alte obiecte

   - nu este efectuata de OpenGL/GPU
   - poate fi efectuata prin algoritmi implementati intr-o biblioteca folosita de aplicatia grafica 3D (motorul grafic)

2. Fețe ale obiectelor - ***Back face culling:*** eliminarea din banda grafica a fețelor auto-obturate ale obiectelor (este efectuata pe GPU)**.**

Eliminarea partilor nevizibile ale scenelor 3D, din imagini (3)

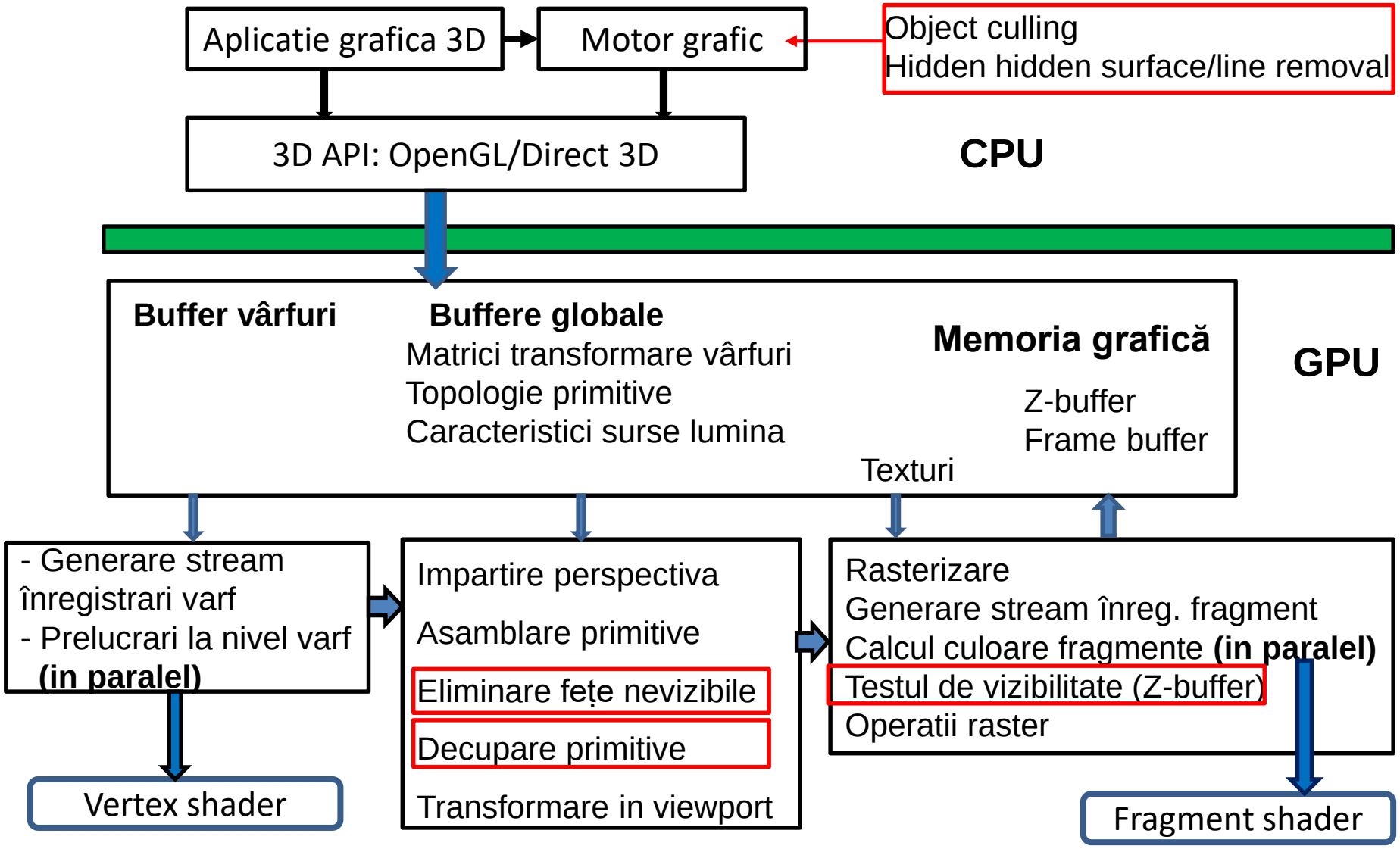
3. Primitive grafice (poligoane, linii):

- Eliminarea primitivelor aflate total sau partial in afara volumului vizual prin operatia de **decupare (clipping)** efectuata de GPU

- Eliminarea partilor obturate (ascunse) ale primitivelor grafice: ***hidden surface/ hidden line removal***

  - poate fi efectuata prin algoritmi implementati intr-o biblioteca folosita de aplicatia grafica 3D (motorul grafic): BSP, Pictorului, ş.a.
  - este efectuata de GPU, ca operatie raster la nivel de fragment (algoritmul Z-buffer)

Eliminarea partilor nevizibile in banda grafica



Inregistrare varf: intrare Vertex shader

Inregistrare fragment: intrare Fragment shader

Eliminarea fetelor auto-obturate pentru poliedre convexe (Back face culling)(1)

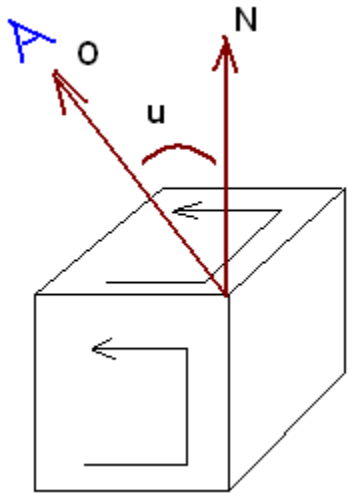
- Poate reduce substantial numarul de poligoane rasterizate.

Algoritmul:

- Observatorul si obiectul sunt raportati la acelasi sistem de coordonate.

- Conturul fiecarei fețe a obiectului este orientat in sens trigonometric atunci cand poliedrul este vazut din exterior.

- Observatorul este situat in afara obiectului.



Fața este vizibila daca : $0 <= u < 90^o$ sau $0 < \cos(u) <= 1$

N: normala la față

O: vectorul orientat catre observator

Produsul scalar: $N \bullet O = |N| * |O| * \cos(u) \rightarrow \cos(u) = N \bullet O / (|N| * |O|) = Nu \bullet Ou$

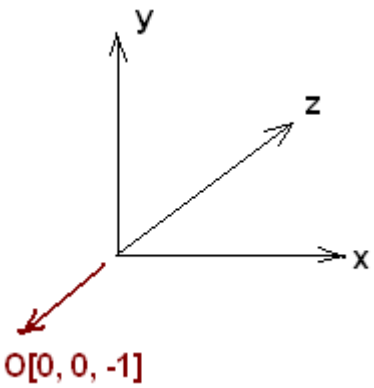
Conditia de vizibilitate: Nu • Ou > 0 (produsul scalar al versorilor)

Determinarea fetelor auto-obturate pentru poliedre convexe(2)

Determinarea feţelor auto-obturate poate fi efectuata:

1. In sistemul coordonatelor globale (in care este definit si observatorul – camera virtuala)
2. In sistemul de coordonate observator
3. In sistemul coordonatelor de decupare (dupa aplicarea transformarii de proiectie)

In cazul 3



Sistemul coordonatelor de decupare

Nu • Ou = [nx, ny, nz] • [0, 0, -1] = - nz

Conditia de vizibilitate devine: nz <0

Dupa transformarea de proiectie pozitia observatorului este la infinit, pe axa z negativa.

Vectorul orientat catre observator este acelasi pentru orice punct al unui obiect

Determinarea fetelor auto-obturate pentru poliedre convexe(3)

Calculul normalei la o faţă a poliedrului

Fie V1(x1, y1, z1), V2(x2, y2, z2), V3(x3, y3, z3), 3 varfuri succesive ale conturului feţei.
Normala la faţă se poate obtine calculand produsul vectorial: (V1-V2)x(V2-V3)

$$\mathbf{N} = (V1-V2)\times(V2-V3) = \begin{vmatrix} i & j & k \\ x2-x1 & y2-y1 & z2-z1 \\ x3-x2 & y3-y2 & z3-z2 \end{vmatrix}$$

$$= [(z3-z2)*(y2-y1) - (z2-z1)*(y3-y2)]*\mathbf{i} - [(x2-x1)*(z3-z2) - (x3-x2)*(z2-z1)]*\mathbf{j} + [(x2-x1)*(y3-y2) - (x3-x2)*(y2-y1)]*\mathbf{k}$$

unde **i**, **j**, **k** sunt versorii directiilor axelor sistemului de coordonate carteziene 3D

Normalele:

- Pot fi calculate in sistemul coordonatelor obiect si atasate varfurilor in aplicatia OpenGL → sunt transformate in sistemul coordonatelor de decupare, in vertex shader.
- Nu sunt atasate varfurilor: sunt calculate pe GPU dupa asamblarea primitivelor, folosind varfurile transformate in sistemul coordonatelor de decupare.

Determinarea fetelor auto-obturate pentru poliedre convexe(4)

Pentru eliminarea feţelor auto-obturate se apeleaza urmatoarele funcţii OpenGL:

glCullFace(GL_FRONT/ GL_BACK/GL_FRONT_AND_BACK);

- eliminare fete: din faţă / din spate / toate (nu vor fi afisate fețe, ci numai alte primitive: puncte, linii)

glFrontFace(GL_CCW/GL_CW);

- specifica orientarea feţelor din faţă: trigonometrica/sensul acelor de ceas

➢ Daca ciclul de varfuri al fiecarei fete este orientat trigonometric atunci cand obiectul este privit din exteriorul său:

- atunci cand observatorul este in spatiul exterior obiectului se va apela glFrontFace(GL_CCW)
- atunci cand observatorul este in spatiul interior obiectului se va apela glFrontFace(GL_CW)

(vazute din interior, feţele obiectului au conturul orientat anti-trigonometric).

Algoritmul z-buffer(1)

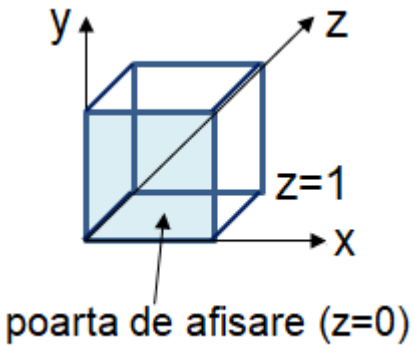
- Algoritm de eliminare a fragmentelor nevizibile ale primitivelor grafice.

- Executat de GPU, fiind integrat in procesul de rasterizare a primitivelor

- Observatorul este la infinit, pe axa z negativa

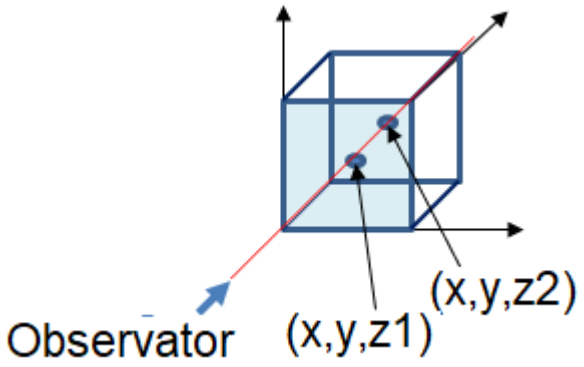
- Asupra primitivelor se efectueaza proiectie ortografica

  in planul XOY (poarta de afisare)



Daca doua fragmente aflate pe acelasi proiector, rezultate din rasterizarea a doua primitive diferite, au coordonata z diferita:

- Fragmentul avand coordonata z mai mica va fi afisat în pixelul (x,y).



Algoritmul z-bufer(2)

- Primitivele grafice (triunghiuri, linii) in care a fost descompusa scena 3D sunt rasterizate in ordinea in care au fost transmise din programul de aplicatie (nu sunt sortate).

- Buffer-ul imagine (frame buffer) este actualizat pe masura rasterizarii primitivelor, astfel:

daca (fragmentul curent se proiecteaza in pixelul (x,y) si
z-fragment < z-fragment afisat in pixelul (x,y))
atunci
actualizeaza culoarea pixelului (x,y) in bufer-ul imagine, la culoarea fragmentului curent

Rezulta: este necesar sa se memoreze coordonatele z ale fragmentelor afisate in pixelii imaginii



Z-buffer: - tablou bidimensional cu numar de elemente egal cu numarul de pixeli ai suprafetei de afisare
- în memoria grafica

Algoritmul z-bufer(3)

Algoritmul z-buffer (integrat in procesul de rasterizare):

*Initializeaza buffer-ul imagine la culoarea de fond
*Initializeaza Z-buffer la coordonata z maxima (z=1)

Pentru fiecare fragment f(x,y,z) rezultat din rasterizarea unei primitive

* calculeaza culoarea fragmentului (in fragment shader)

//testul de vizibilitate (adancime)

daca z< Z-buffer[y][x] atunci

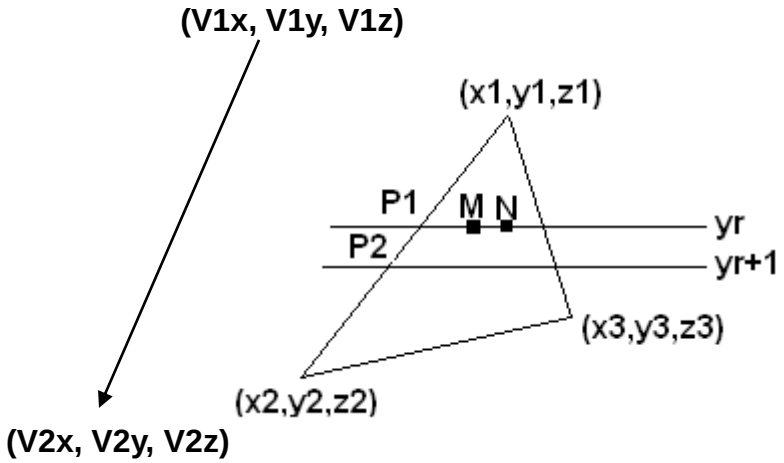
{ Z-buffer[y][x] = z

*actualizeaza culoarea pixelului (x,y) in buffer-ul imagine folosind culoarea fragmentului f

}

Algoritmul z-bufer(4)

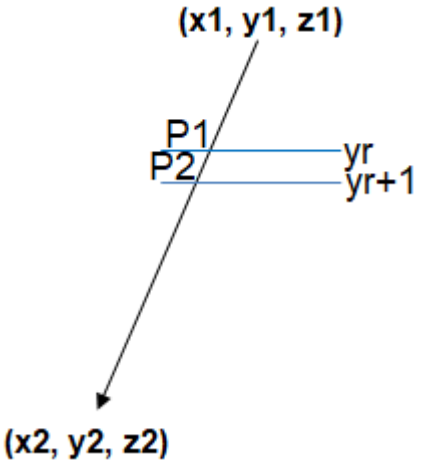
Calculul coordonatei z a unui fragment



Coordonatele z ale fragmentelor rezultate la rasterizarea vectorilor si a suprafetelor triunghiulare se obtin prin calcul incremental, in algoritmul de rasterizare.

Algoritmul z-bufer(5)

Calculul coordonatei z la rasterizarea vectorilor



x = x1 + t(x2-x1)
y = y1 + t(y2-y1) Ecuatiile parametrice ale vectorului
z = z1 + t(z2-z1)

yr, yr+1: ordonatele a 2 linii raster (ecran) care intersecteaza vectorul

y(P1) = yr = y1 + t'(y2-y1) → t' = (yr-y1)/(y2-y1)

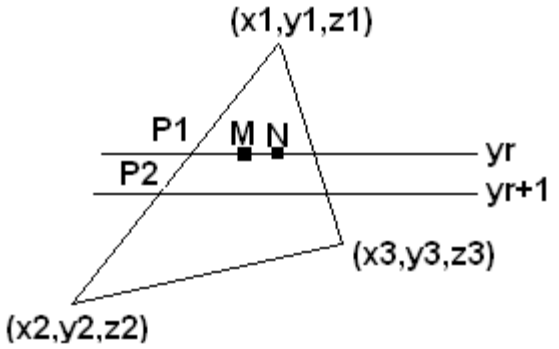
z(P1) = z1 + (yr-y1)(z2-z1)/(y2-y1) = z1 + (yr-y1)*K,
K este o constanta la rasterizarea vectorului

z(P2) = z1 + (yr+1-y1)*K = z1 + (yr-y1)*K + K

Z(P2) = z(P1) + K → coordonatele z ale fragmentelor de pe vector se obtin prin calcul incremental.

Algoritmul z-bufer(5)

Calculul coordonatei z la rasterizarea poligoanelor



P1(xP1, yr, zP1), P2(xP2, yr+1, zP2)

m- panta laturii (x1,y1,z1) – (x2,y2,z2)

xP2 = xP1 + 1/m (vezi curs - rasterizare)

A*x + B*y + C*z + D = 0 ec planului poligonului

zP1 = (-A*xP1 - B*yr –D)/C

zP2 = (-A*xP2 - B*(yr +1) – D)/C = (-A(xP1 + 1/m) - B*yr –B – D)/C = zP1 + (-A/m –B) → **zP2 = zP1 + K1**

M(xM, yr, zM), N(xM + 1, yr, zN)

zN = (-A*(xM + 1) – B*yr – D)/ C = zM + (-A/C) → **zN = zM + k2**

Observatie: atunci cand un vector se suprapune pe suprafata unui poligon pot aparea defecte datorita modului diferit in care se calculeaza coordonatele z ale fragmentelor care se afiseaza in aceiasi pixeli!

Algoritmul z-bufer(6)

Aprecieri asupra algoritmului z-Buffer

1) Numarul de comparatii de valori z pentru un pixel = numarul de fragmente care se proiecteaza in acel pixel.

   Timpul de calcul tinde sa devina independent de numarul de poligoane: in medie, numarul de pixeli acoperiti de un poligon este invers proportional cu numarul de poligoane.

2) "Depth complexity": numarul de suprascrieri ale unui pixel in buffer-ul imagine, la generarea unui cadru imagine

   - ❖ calcul culoare fragment: model iluminare, utilizare texturi

3) Pentru o precizie buna, este necesara o reprezentare pe 32 biţi a valorilor din z-buffer.

Algoritmul BSP (Binary Space Partitioning)(1)

- Initial, arborele BSP a fost folosit numai pentru operatia de eliminare a obiectelor (grupurilor de primitive) nevizibile din banda grafica (object culling).
- Ulterior a devenit algoritm pentru reprezentarea si afisarea scenelor 3D cu "eliminarea" partilor nevizibile.
- Este eficient pentru eliminarea partilor nevizibile ale poligoanelor (hidden surface removal) dar poate fi folosit si pentru eliminarea partilor nevizibile ale liniilor dintr-o scena compusa din poligoane si linii.

Intrarea: lista primitivelor care compun scena 3D
(nu are importanta din care obiect face parte fiecare primitiva)

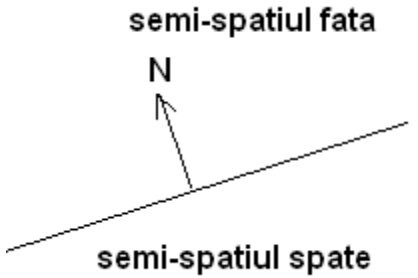
–Scena 3D este reprezentata printr-un arbore binar: arborele BSP

–Arborele BSP al scenei este independent de pozitia observatorului (reprezinta scena in sistemul coordonatelor globale).

Algoritmul BSP (2)

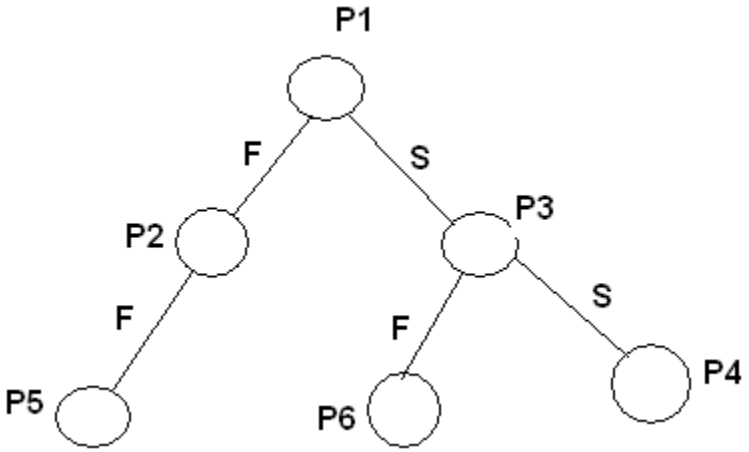
Construirea arborelui BSP al unei scene

▪Fiecare nod al arborelui corespunde unui plan de partitionare a spatiului 3D (de regula, planul unui poligon al scenei)



- Fiecare plan de partitionare imparte spatiul in 2 semi-spatii:
  - –cel din faţa planului (de aceeasi parte cu normala la plan)
  - –cel din spatele planului

1. Se incepe cu un poligon oarecare din lista (de regula primul), pentru care se creaza nodul radacina al arborelui



- Primitivele din semi-spatiul “faţă” formeaza “lista-faţă”, care va genera subarborele “faţă” al nodului
- Primitivele din semi-spatiul “spate” formeaza “lista-spate”, care va genera subarborele “spate” al nodului.

2. Se alege un poligon din lista-faţă si se creaza nodul radacina al subarborelui “faţă, etc.

Algoritmul BSP (3)

Algoritmul de creare a arborelui BSP:

arbore * creareBSP (Poligon * LP)

{ Poligon P, * ListaFata, * ListaSpate, * ListaNod;

daca (LP este vida) return NULL;

ListaFata = ListaSpate = ListaNod = NULL;

* alege un poligon P din LP;

* elimina P din LP; adauga P la ListaNod;

pentru (fiecare primitiva Q din LP) executa

{ **daca** (Q este in semispatiul față al planului lui P) **atunci**

* adauga Q in ListaFata

altfel

Algoritmul BSP (4)

daca (Q este in semispatiul spate al planului lui P) **atunci**

* adauga Q in ListaSpate

altfel

daca Q este in acelasi plan cu P **atunci**

* adauga Q in ListaNod

altfel // Q este intersectata de planul lui P

* divizeaza Q cu planul lui P

* adauga fiecare primitiva rezultata in ListaFata sau ListaSpate, in functie de pozitia sa

} // pentru fiecare primitiva

arbore * radacina = combina(creareBSP(ListaFata), creareBSP(ListaSpate));

*ataseaza ListaNod la radacina;

return radacina;

}

Algoritmul BSP (5)

Afişarea arborelui BSP al unei scene

-Tine cont de pozitia observatorului

- Afisare "back-to-front": se incepe cu primitiva cea mai indepartata de observator

- Se porneste din radacina arborelui si se avanseaza in arbore pana la frunze, in functie de pozitia observatorului fata de planul atasat fiecarui nod.

- In fiecare nod: se avanseaza in subarborele nodului care este opus observatorului fata de plan.

```
void afisareBSP(arbore * A, Pozitie Observator)
{ daca (! A) return;
  daca (Observator este in semispatiul fata al planului radacinii arborelui A) atunci
    { afisareBSP(A->spate); *afisare primitive din nodul radacina; afisareBSP(A->fata);}
  altfel
    { afisareBSP(A->fata); *afisare primitive din nodul radacina; afisareBSP(A->spate);}
}
```

Algoritmul BSP (6)

- **Eliminarea obiectelor aflate in afara volumului visual (Frustum culling):**

➢ Daca planul de partitionare al unui nod nu intersecteaza volumul vizual, atunci numai subarborele aflat de aceeaşi parte cu volumul vizual va fi afişat, celalalt subarbore fiind eliminat din banda grafica. La fel şi primitivele ataşate nodului (fiind în afara volumului vizual).

```
void afisareBSPcull (arbore * A, Pozitie Observator)
{ daca (! A) return;
  daca (volumul vizual este în fața planului radacinii arborelui A) atunci
        afisareBSPcull(A→fata); return;
  daca (volumul vizual este in spatele planului radacinii arborelui A) atunci
        afisareBSPcull(A→spate); return;
  //volumul vizual este intersectat de planul radacinii
    daca (Observator este in semispatiul fata al planului radacinii) atunci
     { afisareBSP(A->spate); *afisare primitive din nodul radacina; afisareBSP(A->fata);}
   altfel
    { afisareBSP(A->fata); *afisare primitive din nodul radacina; afisareBSP(A->spate);}
}
```

Algoritmul BSP (7)

Aprecieri

- Arborele nu trebuie sa fie construit pentru fiecare cadru imagine: avantaj pentru scenele statice
- Modificarea pozitiei observatorului → executia functiei de afisare (daca scena nu s-a modificat)
- Permite eliminarea din banda grafica a unui numar mare de poligoane care sunt in afara volumului vizual (Object culling)
- Pentru obiectele dinamice se pot face diverse adaptari ale algoritmului; de ex:

- pentru fiecare cadru imagine se insereaza in arborele BSP cu obiectele statice, noduri corespunzand obiectelor dinamice;

- se transmite arborele BSP static nemodificat si obiectele dinamice vizibile, eliminarea partilor obturate de acestea fiind realizata in algoritmul z-buffer (la rasterizare).

Algoritmul BSP (8)

Aprecieri (cont):

- Daca primitivele grafice sunt trimise in banda grafica in ordinea “din spate in faţă”, nu mai este necesar testul de adâncime la rasterizarea primitivelor (fragmentele care se proiecteaza in acelasi pixel sunt suprascrise).
- Testul de adancime efectuat asupra fragmentelor la rasterizare poate fi activat/dezactivat:

  glEnable(GL_DEPTH_TEST) / glDisable(GL_DEPTH_TEST)

- Daca se foloseste algoritmul BSP cu afisare «back-to-front» impreuna cu testul de adancime, numarul de suprascrieri ale pixelilor este mai mare decat in cazul unei ordini aleatoare a poligoanelor (algoritmul z-buffer fara BSP).
- Daca se foloseste BSP cu afisare «front-to-back» impreuna cu testul de adancime, afisarea poate fi mai rapida decat folosind BSP «back-to-front» fara testul de adancime, in functie de amplasarea poligoanelor in scena.
- Construirea si afisarea arborelui BSP sunt functii de management al scenei 3D, implementate de regula intr-un motor grafic 3D.

Algoritmul pictorului(1)

– Algoritm din categoria “Hidden surface removal”.

– Intrare: lista poligoanelor care alcatuiesc scena 3D, transformate in spatiul de afisare

– Ideea: eliminarea partilor nevizibile se rezolva prin afisare “din spate in fata”, la fel ca in pictura.

➢ Nu este necesar testul de adancime la rasterizarea poligoanelor.

Forma generala a algoritmului

1. Se calculeaza “extensia” fiecarui poligon din listă pe axele OX, OY, OZ (xmin,ymin,zmin – xmax, ymax, zmax): paralelipipedul incadrator al poligonului, cu feţele paralele cu planele principale ale sistemului de coordonate.

2. Se ordoneaza poligoanele crescator dupa coordonata zmin a fiecarui poligon: primul in lista va fi cel mai apropiat de observator.

3. Se descompun poligoanele ale caror extensii pe axa OZ se suprapun, astfel incat extensiile lor pe axa OZ sa fie disjuncte.

4. Se transmit in banda grafica poligoanele incepand cu ultimul din lista (cel mai indepartat de observator).

Algoritmul pictorului(2)

Pasul 3. Se descompun poligoanele ale caror extensii pe axa OZ se suprapun

- Sunt multe aplicatii in care acest pas nu este necesar, poligoanele fiind amplasate in plane de Z-constant: cartografie, generarea straturilor circuitelor imprimate, etc.
- Poate fi optimizat, stiind ca nu intotdeauna atunci cand extensiile pe axa OZ se suprapun este necesara descompunerea poligoanelor; descompunerea nu este necesara daca:

  - extensiile pe axa OX nu se suprapun

  - extensiile pe axa OY nu se suprapun

  - proiectiile poligoanelor nu se suprapun in imagine

- Testele se efectueaza progresiv, in functie de complexitatea calculelor presupuse.

- **Principalul efort de calcul:** sortarea listei de poligoane si descompunerea, daca este necesara.

Eliminarea partilor nevizibile ale scenelor 3D din imagini -2

Prof. univ. dr. ing. Florica Moldoveanu

Tehnici de eliminare a obiectelor nevizibile - Frustum culling -

❖ Scopul: eliminarea din banda grafica a obiectelor sau grupurilor de obiecte care sunt complet in afara volumului vizual

❖ Se bazeaza pe reprezentarea scenei printr-o structura de date spatiala.

Structuri de date spatiale pentru managementul scenei 3D

- Ierarhie de volume incadratoare (BVH)
- Arbore BSP
- Arbore AA-BSP
- Arbore octal

- Managementul scenei este realizat in cadrul unui motor grafic 3D, implementat peste OpenGL / Direct 3D.

- Structurile de date sunt utile si pentru alte operatii: mai sunt numite "**structuri de date de accelerare**".

1. Scena- Ierarhie de volume incadratoare (1) (Bounding Volume Hierarchy - BVH)

- **Scena este reprezentata printr-un arbore care are in noduri volume încadratoare de obiecte sau grupuri de obiecte**: sfera, paralelipipedul aliniat cu axele (Axis-Aligned minimum Bounding Box: AABB), poliedrul convex minimal incadrator, cilindrul, elipsoidul.

Construirea arborelui:

1. Top-down – radacia arborelui este asociata cu intreaga scena

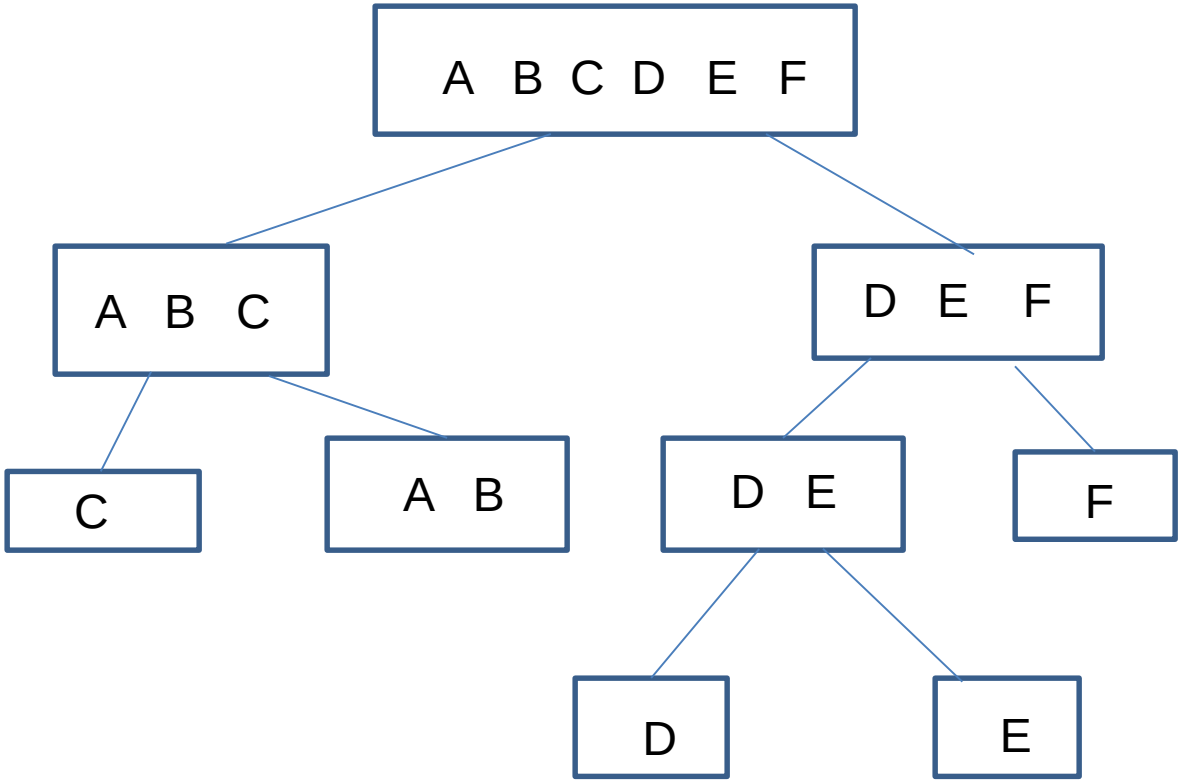
- Se partitioneaza setul de obiecte in 2 sau mai multe subseturi si se aloca fiecare subset unui volum incadrator – rezulta primul nivel al arborelui
- Se continua procedura recursiv la nivelul fiecarui grup de pe nivelul anterior pana cand se ajunge la un singur obiect (sau grup de obiecte) pe nod – nod frunza
- Metoda este simpla dar, in general, nu produce arbori optimi.

2. Bottom-up

- Se pleaca de la setul de obiecte si se defineste un volum incadrator pentru fiecare obiect sau grup de obiecte – nodurile frunza.
- Se grupeaza 2 sau mai multe volume nod frunza, rezultand alte noduri ale arborelui.
- Se continua gruparea pana cand totul este grupat intr-un singur volum – nodul radacina

- Construirea bottom-up produce arbori mai eficienti decat cea top-down.

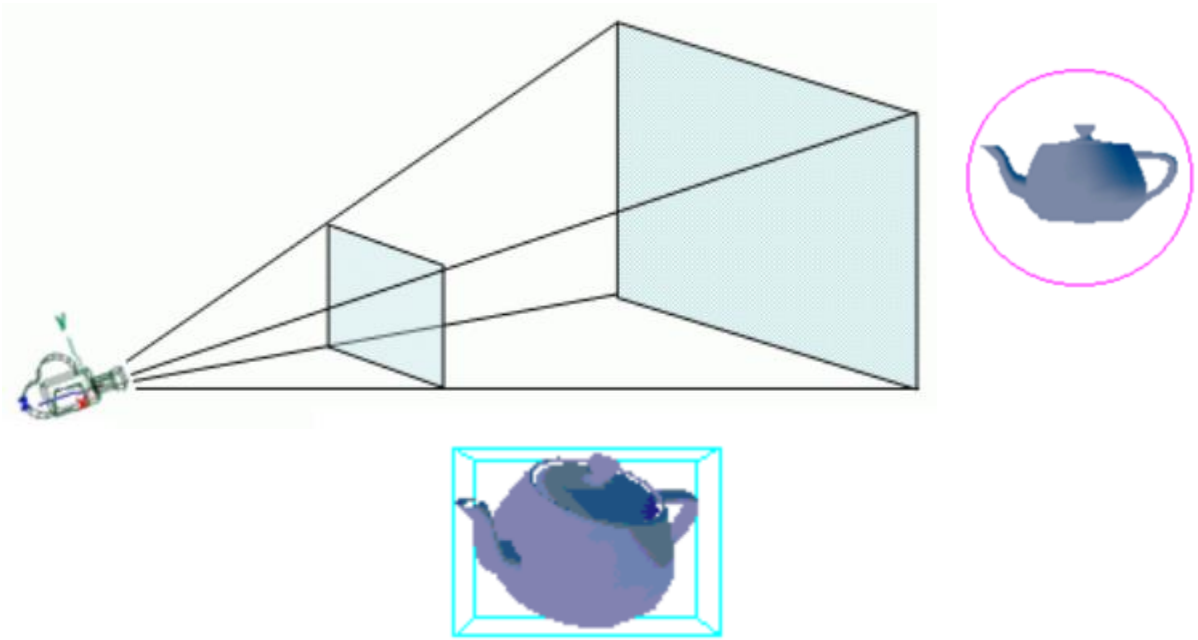
Ierarhie de volume incadratoare (2)



Ierarhie de volume incadratoare (3)

- **Afisarea scenei 3D folosind arborele BVH**

Daca volumul incadrator al unui nod este in afara volumului vizual atunci obiectele din subarborele său **NU** sunt trimise in banda grafica.



Ierarhie de volume incadratoare (4)

❖ **Afisarea scenei 3D folosind arborele BVH**

- testele de vizibilitate incep cu radacina arborelui;

- **daca** volumul incadrator al nodului curent este complet in afara volumului vizual

atunci se revine pe nivelul anterior al arborelui (intregul grup de obiecte din subarborele nodului curent este exclus din banda grafica)

altfel se coboara in arbore;

- **daca** se ajunge la o frunza al carei volum incadrator nu este in afara volumului vizual,

atunci - grupul de primitive (obiectul) asociat frunzei este trimis in banda grafica;

- se revine pe nivelul anterior al arborelui;

➢ **Pentru simplitate si eficienţă, testele de vizibilitate se pot face in spatiul coordonatelor de decupare, transformand volumele incadratoare prin matricea MVP si testand intersectia cu volumul vizual canonic (cub cu laturile paralele cu axele)**

❖ Structura BVH este frecvent utilizata pentru detectia coliziunilor dintre obiectele unei scene dinamice.

Ierarhie de volume incadratoare (5)

❖ **Eficienta structurii BVH privind eliminarea obiectelor nevizibile din banda grafica**

- depinde mult de felul in care sunt grupate obiectele in volume incadratoare; poate fi eficienta pentru anumite pozitii ale observatorului si ineficienta pentru alte pozitii.

Alegerea tipului de volum incadrator pentru un nod:

- Sa aiba o forma simpla, a.i. testele de intersectie cu volumul vizual sa fie simple si rapide
- Sa incadreze minimal obiectul/obiectele

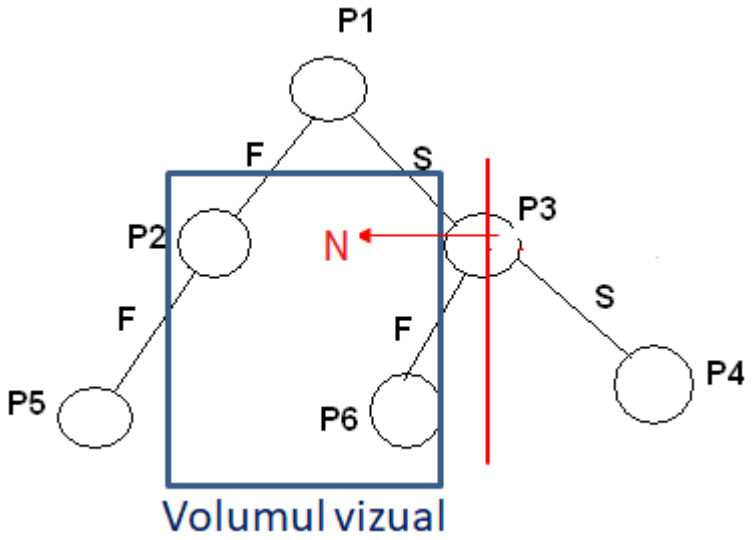
Proprietati dorite ale ierarhiei de volume incadratoare:

- Volumele nodurilor dintr-un subarbore trebuie sa fie appropriate spatial, cu atat mai apropiate cu cat sunt situate mai jos in arbore
- Volumul fiecarui nod sa incadreze minimal obiectul/grupul de obiecte corespunzator
- Volumul incadrator al intregii scene trebuie sa fie minimal
- Suprapunerea volumelor nodurilor vecine sa fie minimala

2. Scena - Arbore BSP (Polygon Aligned BSP tree) (1)

- **Planele de partitionare sunt planele poligoanelor (Polygon Aligned - PA BSP tree)**
  - Arborele scenei se construieste ca in algoritmul BSP pentru eliminarea suprafetelor nevizibile
- **Eliminarea grupurilor de primitive nevizibile**

Daca planul de divizare atasat unui nod nu intersecteaza volumul vizual, atunci subarborele atasat nodului care este situat de partea opusa a planului fata de volumul vizual este complet nevizibil si poate fi eliminat din procesul de redare. La fel şi primitivele ataşate nodului.



Eliminarea se efectueaza in spatiul coordonatelor globale, folosind volumul vizual definit de aplicatie (trunchi de piramida sau paralelipiped dreptunghic).

3. Scena - Axis Aligned BSP Tree (1)

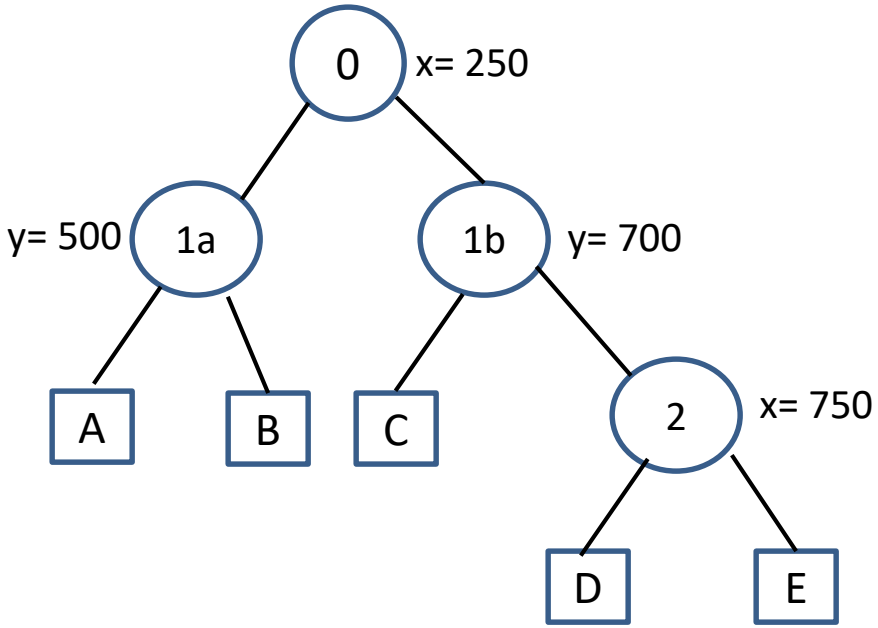
- **Planele de partitionare sunt aliniate cu axele sist de coord 3D -**perpendiculare pe cele 3 axe: Axis-Aligned BSP Tree (AA-BSP)
- AA-BSP tree este un k-d tree (*k-dimensional tree):* arbore binar in care fiecare nod corespunde unei coordonate intr-un spatiu k-dimensional. Pentru k=3, un nod va corespunde uneia dintre coordonatele x, y sau z. De exemplu, daca planul de partitionare al unui nod este perpendicular pe axa OX, atunci nodul este asociat coordonatei x a planului.
- **Construirea arborelui:**

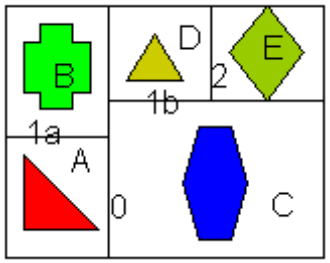
  - Scena este divizata recursiv prin plane paralele cu planele sistemului de coordonate, intr-o ordine fixa, tinand cont si de geometria scenei; de exemplu, in pasul 1 se alege un plan perpendicular pe OX, in pasul 2 un plan perpendicular pe OY, in pasul 3 un plan perpendicular pe OZ, in pasul 4 un plan perpendicular pe OX, s.a.m.d.
- **Alegerea planelor:**

  - numarul de obiecte din fiecare semispatiu sa fie acelasi sau diferit cu 1, pentru ca arborele sa fie echilibrat

  - numarul de intersectii dintre planele de partitionare si obiectele scenei sa fie minimizat.

3. Scena- Axis Aligned BSP Tree (2)





Exemplu de divizare
cu plane aliniate cu axele, in 2D:

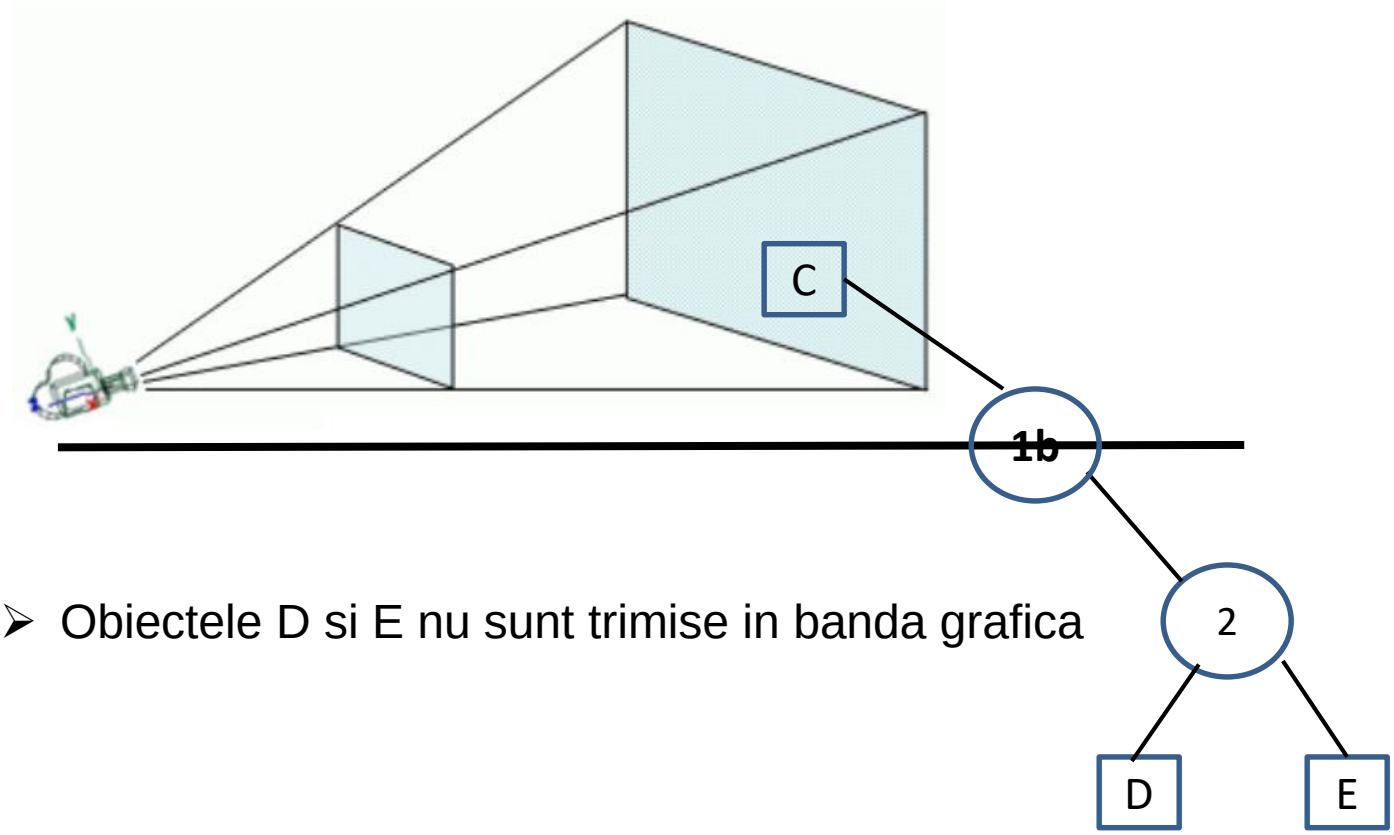
In fiecare pas planul de divizare este ales a.i. sa se evite intersectia sa cu obiectele scenei.

❖ **Eliminarea obiectelor nevizibile: la fel ca in cazul arborelui PA- BSP:** daca planul de partitionare al unui nod nu intersecteaza volumul vizual, intregul grup de obiecte din subarborele aflat de partea opusa volumului este exclus din banda grafica.

3. Scena- Axis Aligned BSP Tree (3)

❖ **Afisarea scenei 3D folosind arborele AABSP**

Daca planul atasat unui nod nu intersecteaza volumul vizual atunci întregul grup de obiecte din subarborele aflat de partea opusa volumului **NU** este trimis în banda grafica.

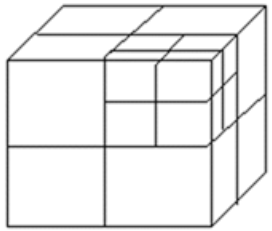


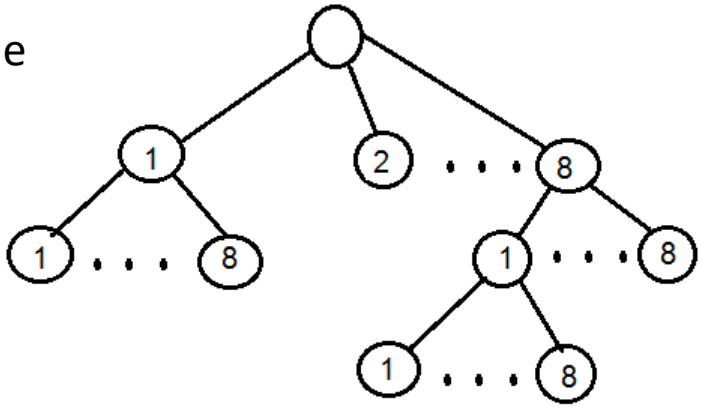
➢ Obiectele D si E nu sunt trimise in banda grafica

4. Scena - Arbore octal (1)

❖ **Construirea arborelui octal al scenei**

- Se pleaca de la volumul incadrator al scenei, aliniat cu axele, care se asociaza nodului radacina al arborelui.
- Se divizeaza volumul incadrator in 8 subvolume egale
- Se impart primitivele intre cei 8 octanti





- Se continua cu divizarea recursiva a subvolumelor si asocierea primitivelor cu subvolumele;

fiecare primitiva este asociata cu cel mai mic octant care o incadreaza complet.

- Un subvolum nu este divizat daca s-a ajuns la un nivel de divizare prestabilit sau subvolumul contine suficient de putine primitive sau nici o primitiva.
- Rezulta un arbore in care fiecare nod intern are 8 copii.

Scena - Arbore octal (2)

❖ **Fiecare nod al arborelui octal are ataşat un volum incadrator**

➢ **Eliminarea obiectelor nevizibile folosind arborele octal al scenei : la fel ca in cazul BVH**

- Se efectueaza testand intersectiile subvolumelor arborelui cu volumul vizual (cub, in sistemul coordonatelor de decupare), incepand cu nodul radacina.

- Daca volumul asociat nodului curent intersecteaza volumul vizual, se coboara in arbore, testand intersectia dintre fiecare copil al său si volumul vizual.

- Daca subvolumul asociat unui nod este in afara volumului vizual, atunci se revine in arbore pe nivelul anterior (se elimina din banda grafica toate obiectele din subarborele nodului).

Scena - Arbore octal (3)

❖**Eficienta arborelui octal al scenei**

- Subdivizarea stricta a spatiului impreuna cu metoda de asociere a fiecarei primitive cu cel mai mic octant care o incadreaza complet, poate conduce la ineficienţă;

de ex., daca o primitiva este situata in apropierea centrului volumului incadrator al scenei, primitiva va fi asociata cu acest volum, chiar daca este foarte mica in raport cu volumul!

- O alternativa: o primitiva poate fi asociata cu 2 sau mai multi octanti vecini care impreuna o incadreaza complet formeaza un volum minimal (creste complexitatea construirii arborelui).

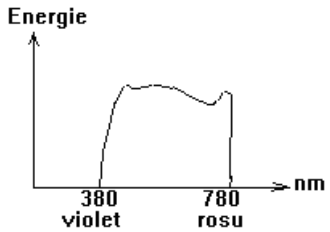
Proprietatile luminii. Modele de culoare

Prof. univ. dr. ing. Florica Moldoveanu

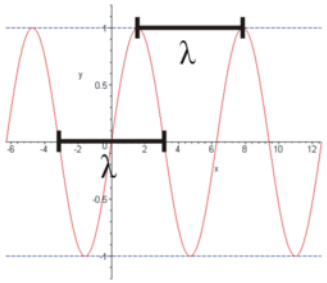
Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2018-2019

Lumina

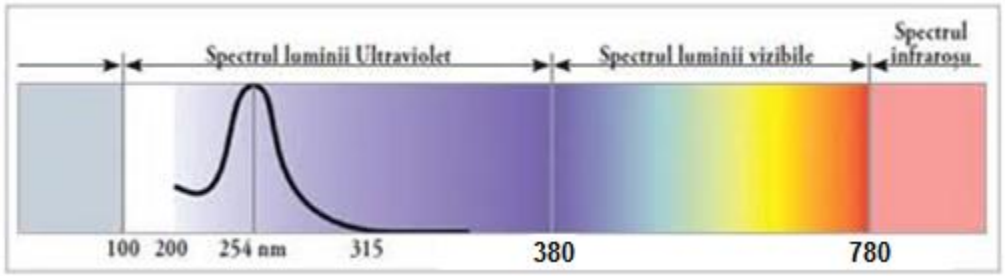
- Lumina este energie electromagnetică: spectrul vizibil
- Spectrul vizibil este alcatuit din undele cu lungimi cuprinse intre 380-780 nanometri.



Spectrul vizibil



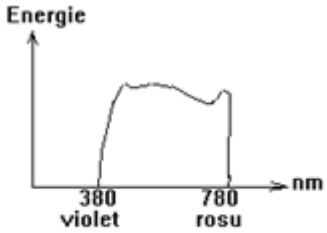
Lungimea de unda: distanța dintre două puncte din spațiu între care unda a parcurs o perioada de repetitie.



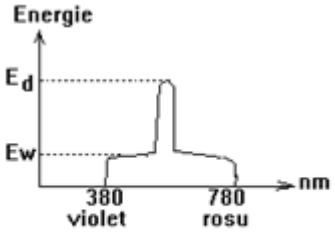
Proprietatile luminii(1)

- Atunci când cade pe suprafaţa unui obiect, lumina poate fi: absorbită, reflectată sau transmisă

- **Sistemul vizual uman percepe: - lumina provenita direct de la o sursa şi**
  - lumina reflectata sau transmisa de obiectele din mediu.



Lumina acromatica



Lumina cromatica

- Lumina care conţine toate lungimile de undă din spectrul vizibil în proporţii aproximativ egale se numeşte **acromatică**.
- Lumina in care predomina anumite lungimi de undă se numeşte **cromatică.**

- Lumina acromatică provenită de la o sursă este percepută ca albă.
- Lumina acromatică provenită de la un obiect este percepută ca albă, neagră sau ca o nuanţă de gri în funcţie de proprietăţile fizice ale suprafeţei obiectului.
- Obiectele care reflectă acromatic mai mult de 80% din lumina incidentă albă apar ca albe. Cele care reflectă acromatic mai puţin de 3% din lumina incidentă apar ca negre.

Proprietatile luminii(2)

- Culoarea unui obiect, perceputa de ochiul uman, depinde atât de distribuţia lungimilor de undă în lumina care cade pe obiect cât şi de caracteristicile fizice ale suprafetei obiectului: reflecta/absoarbe anumite lungimi de unda.

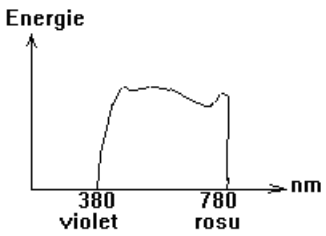
➢ Energia electromagnetică nu are culoare!

➢ **Culoarea este rezultatul unui proces psiho-fiziologic**.

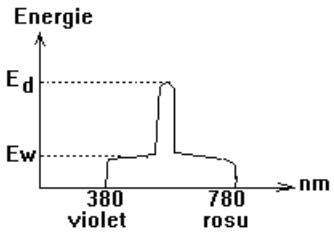
- **Definiţia psiho-fiziologică a unei culori cuprinde: nuanța, luminozitatea (stralucirea), saturatia**
  - –**Nuanţa:** roşu, galben, verde, etc. – determinată de lungimea de undă dominantă a distribuţiei spectrale a luminii.
  - –**Luminozitatea sau strălucirea** - reprezintă intensitatea luminii.
    - –Luminozitatea: caracteristică a unui obiect emiţător de lumină
    - –Strălucirea: caracterizează un obiect neemiţător, care reflectă lumina.

Proprietatile luminii(3)

- **Saturaţia sau puritatea -** o măsură a amestecului de alb într-o culoare pură;
  - permite să se facă distincţie între roşu şi roz, între albastru şi bleu, etc;
  - o culoare pură are saturaţia 100%;
  - lumina acromatică are saturaţia 0%.



Lumina acromatica



Lumina cromatica

Culoarea este data de lungimea de unda dominanta.

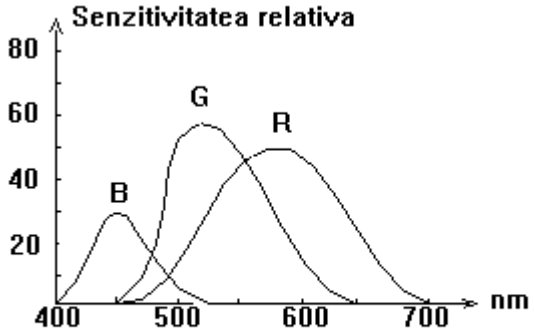
Ew=0 – culoarea are puritatea 100% : monocromatica

(largime de banda de un nanometru)

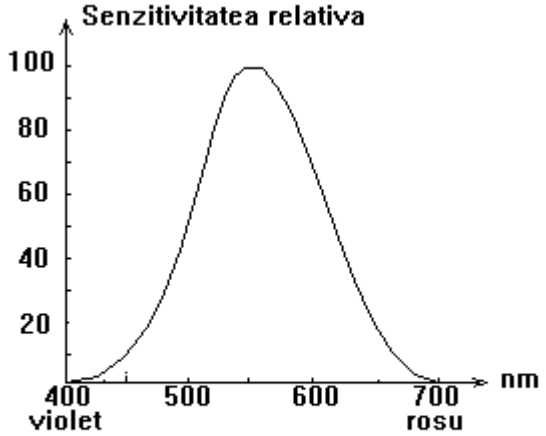
– Cea mai cunoscută dintre teoriile privind formarea culorilor în sistemul ochi-creier este aceea conform căreia în retina ochiului uman există trei tipuri de conuri, fiecare tip fiind sensibil la una dintre culorile roşu, verde şi albastru: teoria tricromaticitatii (Tristimulus theory)

Teoria tricromaticitatii (1)

- Culorile rosu, verde si albastru se numesc culori primare.
- Cf. teoriei tricromaticitatii, orice culoare care poate fi perceputa de ochiul uman se obtine printr-un amestec aditiv al celor 3 culori primare.
- Prin amestecul lor în proporţii egale se obţine alb.



Raspunsul ochiului la cele 3 culori primare



Curba de luminozitate
Suma celor 3 curbe de senzitivitate

Senzitivitatea maximă corespunde luminii cu lungimea de undă în jur de 550 nm, percepută ca galben-verde.

Teoria tricromaticitatii (2)

- **Două culori care prin amestec produc lumină albă se numesc complementare.**

- Culorile complementare culorilor primare sunt : cian (Cyan), magenta (Magenta), galben (Yellow).

- Amestecând două culori primare in proportii egale se obţine culoarea complementară celei de a treia:

  R **G** **B**

  C M Y

- De exemplu, albastru+verde=cian, roşu+verde=galben, roşu+albastru= magenta.

- Culorile **roşu, verde şi albastru** se numesc şi **"primitive aditive"** deoarece ele permit formarea de culori prin **adunarea** lor în diferite proporţii.

- Acest mod de definire a culorilor corespunde echipamentelor emiţătoare de lumină (dispozitive de vizualizare cu ecran).

Teoria tricromaticitatii (3)

- Culorile **cian, magenta, galben** se numesc **"primitive extractive"**.
- Se obtin prin **extragerea** culorilor primare din lumina alba:

cian = alb-roşu, magenta = alb-verde, galben = alb-albastru

R G B
C M Y

– **Se folosesc pentru a defini culorile reflectate de documentele imprimate**: pigmenţii existenţi în cernelurile tipografice absorb din lumina incidenta componentele complementare acelora ale pigmenţilor.

Ex: pigmentul de culoare magenta absoarbe din lumina incidentă lumina verde, iar cel de culoare galben, lumina albastra.

➢ O suprafaţă care conţine pigmenţi magenta şi galben va reflecta (sau transmite) lumină roşie.

– **C, M, Y permit specificarea de nuanţe prin extragerea culorilor primare în diferite proporţii din alb. Scăzându-le în proporţii egale din alb se obţin diferite nuanţe de gri.**

Modele de culoare folosite în sinteza imaginilor (1)

- Orientate către echipamente
  - se bazează pe culorile primare folosite de echipamente pentru redarea culorilor:

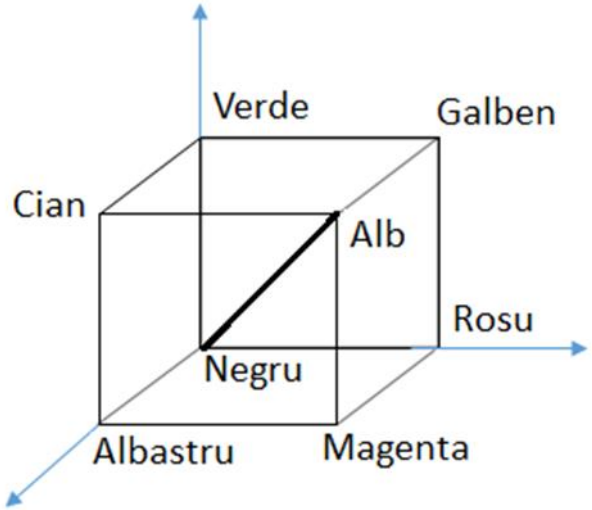
    RGB, CMY
  - YIQ: pentru transmisia imaginilor TV color

- Orientate către utilizator
  - se bazează pe proprietăţile psiho-fiziologice ale culorilor :
  - HSV şi HLS.

Modele de culoare(2)

- Un model de culoare specifică un sistem de coordonate 3D şi un subspaţiu al culorilor în sistemul de coordonate respectiv.
- Fiecare culoare se reprezintă printr-un punct în subspaţiul culorilor.

Modelul RGB (Red, Green, Blue)



Rosu: 1,0,0

Verde: 0,1,0

Albastru: 0,0,1

Alb: 1,1,1; Negru: 0,0,0

Cian: 0, 1, 1 - culoarea complementara culorii Rosu !

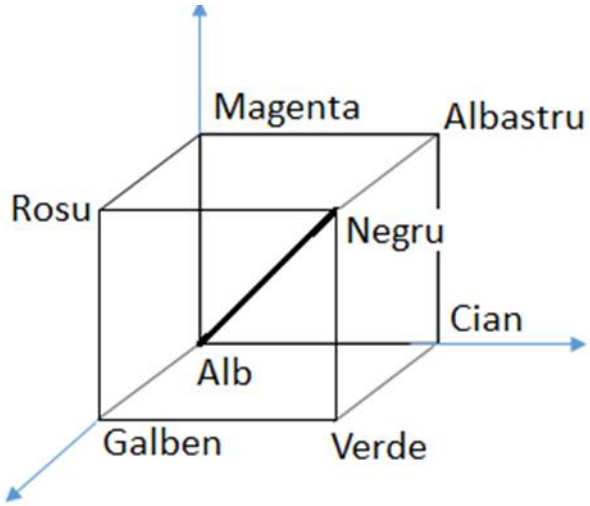
Pe diagonala Negru-Alb: nuante de gri - obtinute prin amestec in proportii egale intre R,G,B

Modele de culoare(3)

Modelul CMY (Cyan, Magenta, Yellow)

CMYK (Cyan, Magenta, Yellow, Key) → key = negru





Cian: 1,0,0 (Alb – Rosu)

Magenta: 0, 1, 0

Galben: 0,0,1

Alb: 0,0,0

Negru: 1,1,1 - (R,G,B) sunt extrase in proportie maxima din lumina alba

Rosu: 0, 1, 1 - Complementara culorii Cian!

Unele dispositive de imprimare includ culoarea negru intre culorile primare C,M,Y: CMYK

Coversia RGB ←→ CMY

[C M Y] = [1 1 1] - [R G B]
[R G B] = [1 1 1] - [C M Y]

Culorile reflectate (RGB):

Cian: 1,1,1 – 1,0,0 = 0,1,1
Magenta: 1,1,1 – 0,1,0 = 1,0,1
Negru: 1,1,1 – 1,1,1 = 0,0,0

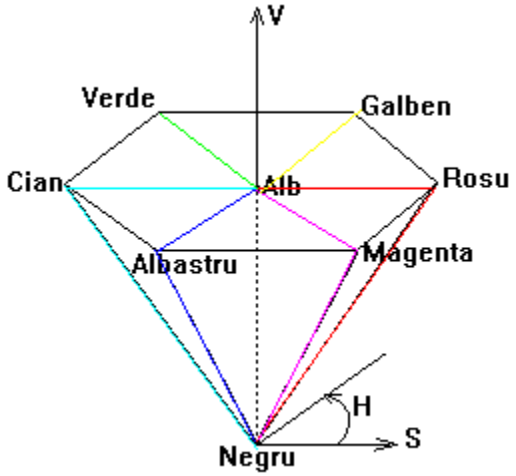
Modele de culoare(4)

Modelul HSV (Hue, Saturation, Value)

- Model de culoare orientat catre utilizator



- Artiştii specifică culorile prin tente, nuanţe şi tonuri



0<= V<= 1, 0<= S <= 1, 0<= H <= 360

V=1 – baza hexaconului

S=0 - pe axa hexaconului;

S=1 - pe frontiera bazei hexaconului: culorile pure

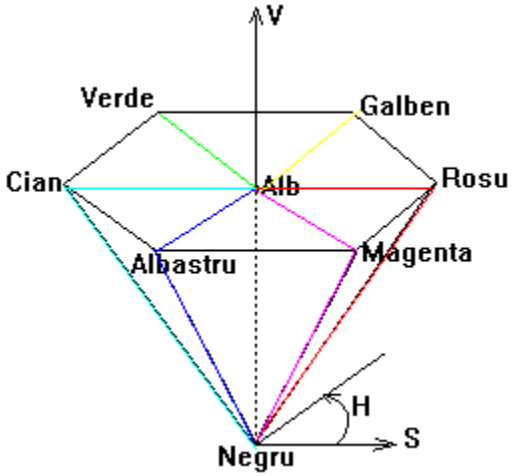
Rosu: (0, 1, 1)

Culorile complementare – la 180 grade pe baza hexaconului

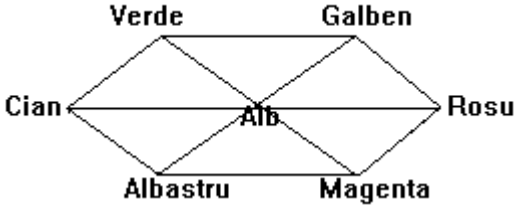
Cian: (180, 1, 1)

- Hexaconul din figura reprezinta subspatiul culorilor in modelul HSV

Modele de culoare(5)



- Saturaţia intr-un punct este definită prin distanţa de la axa hexaconului la punctul respectiv.
- Combinaţia (S=0,V=1) corespunde culorii alb iar prin (S=0, 0<=V<=1) se reprezintă nivelurile de gri.
- Pentru S=0 valoarea lui H este nesemnificativă.
- Culorile pure sunt reprezentate prin V=1 şi S=1.

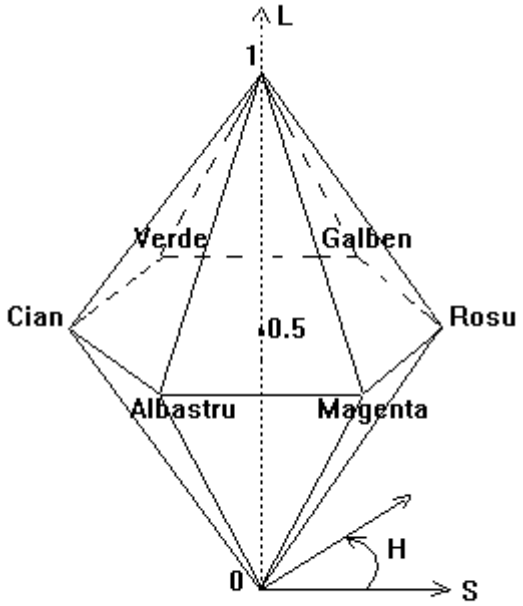


- Baza hexaconului corespunde suprafeţei văzute atunci când se priveşte cubul RGB dinspre vârful (1,1,1) de-a lungul diagonalei principale.

- Fiecare plan de V constant din spaţiul HSV corespunde vederii unui subcub din cubul RGB.
- Diagonala principală a cubului RGB devine axa principală în spaţiul HSV.
- Această interpretare sta la baza algoritmilor de conversie între modelele RGB şi HSV.

Modele de culoare(6)

Modelul HLS (Hue, Lightness, Saturation)



Modelul HLS

Culorile primare cu saturaţie maximă şi complementarele lor sunt reprezentate prin S=1, L=0.5.

Conversii RGB → HSV/HLS

max = sup(R, G, B) min = inf(R, G, B)

V = max

S= (max-min)/max daca max ≠ 0

0 daca max =0

H = (G – B) / (max – min) daca R = max

(B – R) / (max – min) + 2 daca G = max

(R – G) / (max – min) + 4 daca B = max

daca H<0 H = H +6

H = H x 60

RGB → HSV

L = (max-min)/2

S = (max – min)/(max + min) daca L<=0.5

= (max – min)/ (2 – max – min) daca L > 0.5

H = (G – B) / (max – min) daca R = max

(B – R) / (max – min) + 2 daca G = max

(R – G) / (max – min) + 4 daca B = max

daca H<0 H = H +6

H = H x 60

RGB → HLS

Interpolarea în spatiul culorilor

- Rezultatul interpolării între două culori depinde de modelul de culoare în care sunt specificate.
- Fie două culori specificate în modelul RGB, C1=(1,0,0) şi C2=(0,1,0). Le interpolăm cu ponderi egale în modelele RGB şi HSV:
- Interpolare in modelul RGB:

  C=t*C2 + (1-t)*C1 unde t=0.5, se obţine

  C=(0.5, 0.5, 0), care convertită în HSV ne dă (60, 1, 0.5)
- Interpolare in modelul HSV:

  C1 se reprezintă în HSV prin (0, 1, 1) iar

  C2 prin (120, 1, 1)

  C= 0.5*(0,1,1) + 0.5*(120,1,1) = (60, 1, 1)

Modele de reflexie

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2018-2019

Modele empirice pentru calculul reflexiei luminii

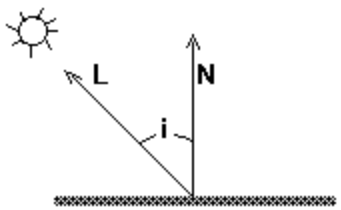
- Caracteristicile luminii reflectate intr-un punct al suprafaţei unui obiect depind de :
  - lungimile de undă conţinute în lumina incidentă,
  - direcţia şi geometria sursei luminii incidente,
  - orientarea suprafeţei in punctul respectiv,
  - proprietăţile materialului din care este construită suprafaţa.

- Expresia care modelează intensitatea luminii reflectate într-un punct al unei suprafeţe este definită pentru o suprafaţă necolorată şi o **lumină incidentă monocromatică**, deci caracterizată printr-o anumită lungime de undă, λ.
- In cazul general, **lumina reflectată nu este monocromatică**, de aceea pentru calculul său expresia ar trebui să fie evaluată continuu pe întregul domeniu al spectrului de modelat.
- In practică, expresia se evaluează separat pentru cele trei culori primare, R, G, B.

Modelarea reflexiei difuze a luminii provenite de la o sursa

Aproximarea reflexiei difuze intr-un punct al unei suprafete 3D

- Lumina reflectată difuz de o suprafaţă este dispersată regulat în toate direcţiile.
- **Legea lui Lambert** defineşte reflexia luminii provenite de la o **sursă punctiformă**, de către un difuzor perfect:



Reflexia difuza

$Id = Isursa * kd * \cos(i) \qquad 0 <= i <= \pi/2$

N – normala la suprafata in punctul considerat

Isursa – este intensitatea luminii incidente (provenita de la sursa de lumina)

Kd – este coeficientul de difuzie a luminii incidente, dependent de materialul suprafetei: suprafata reflecta/absoarbe/transmite lumina in anumite proportii: $0 <= kd <= 1$

Id este intensitatea luminii reflectate difuz de suprafata

- Dacă i este mai mare ca π/2, suprafaţa nu primeşte lumină de la sursă (sursa de lumină se află în spatele suprafeţei).

Modelarea reflexiei luminii ambiante

Lumina ambianta

- Modeleaza lumina provenita de la celelelte obiecte ale scenei 3D: este o sursa de lumina distribuita uniform in spatiu.
- Lumina ambianta este reflectata la fel in price punct al unei suprafete. Intensitatea sa depinde de intensitatea luminii ambiante si de proprietatile suprafetei.
- Reflexia luminii ambiante este modelata prin:

  Ia*ka, unde

  Ia– este intensitatea luminii ambiante iar

  0<= ka<= 1 este coeficientul de difuzie a luminii ambiante, dependent de materialul suprafetei.

Atunci:

Id – intensitatea luminii reflectate difuz intr-un punct al unei suprafete

Id = reflexia luminii ambiante + reflexia difuza a luminii de la sursa =

Ia * ka + Isursa * kd* cos(i)

$0 <= i <= \pi/2$

Modelarea atenuarii luminii de la o sursa

Sursa directionala: sursa aflata la distanta foarte mare (infinit) de scena 3D

- Vectorul L este acelasi in orice punct al suprafetei
- Pentru 2 suprafete paralele, cu aceleasi proprietati de material, rezulta aceeasi Id → daca proiectiile suprafetelor se suprapun in imagine, ele nu se vor distinge.

Intensitatea luminii descreşte proporţional cu inversul pătratului distanţei de la sursa de lumină la suprafata.

Id = Ia* ka + fat * Isursa* kd* cos(i), unde fat = $1/d^2$ este funcţia de atenuare a luminii de la o sursa
d este distanţa de la sursă la punctul de pe suprafaţă considerat.

- Corecţia nu satisface cazurile în care sursa este foarte îndepărtată: atenuare prea mare
- Dacă sursa este la distanţă foarte mică de scenă, atenuarile obţinute pentru două suprafeţe aflate la distante putin diferite de sursa, vor fi mult diferite.
- In practica se foloseste factorul de atenuare: fat = $\min(1/(c1 + c2*d + c3*d^2), 1)$
  c1, c2 şi c3 sunt trei constante care se asociază sursei de lumină.
- Constanta c1 se alege astfel încât numitorul să nu devină prea mic atunci când sursa este foarte apropiată. Valoarea funcţiei este limitată la 1 pentru a se asigura atenuarea.

Culoarea luminii reflectate difuz (1)

- Lumina incidenta poate contine mai multe lungimi de unda: → reflectate in mod diferit de o suprafata

  Csursa = [IsursaR, IsursaG, IsursaB] – culoarea luminii sursei

  Ca = [IaR, IaG, IaB] – culoarea luminii ambientale

- O suprafata reflecta in mod diferit componentele R, G, B ale luminii incidente.

❖ Ex: o suprafata care absoarbe din lumina incidenta componentele de verde (G) si albastru (B),

- Daca lumina incidenta este alba, suprafata va fi vazuta ca rosie:

  [1,1,1] – [0,1,1] = [1,0,0]

- Daca lumina incidenta este rosie, suprafata va fi vazuta tot ca rosie:

  [1,0,0] – [0,1,1] = [1,0,0] (componenetele G si B au valoarea zero – nu pot fi extrase)

- Daca lumina incidenta este verde, suprafata va fi vazuta ca neagra (componenta verde a fost absorbita):

  [0,1,0] – [0,1,1] = [0,0,0]

➢ Rezulta: coeficientii de difuzie a luminii de catre o suprafata depind de componentele luminii incidente si de proprietatile suprafetei

Culoarea luminii reflectate difuz (2)

ka = [kaR, kaG, kaB] – coeficientii de difuzie a componentelor R, G, B din lumina ambientala

kd = [kdR, kdG, kdB] – coeficientii de difuzie a componentelor R, G, B din lumina provenita de la sursa

- Expresia luminii reflectate difuz se evalueaza separat pentru cele 3 componente ale luminii incidente, R,G,B:

  $Id\lambda = Ia\lambda * ka\lambda + fat * Isursa\lambda * kd\lambda * \cos(i)$, unde λ: lungimea de unda

sau:

- Cd = [IdR, IdG, IdB] - culoarea luminii reflectate difuz

  Cd = [IaR *kaR + fat*IsursaR *kdR *cos(i),

  IaG *kaG + fat*IsursaG *kdG *cos(i),

  IaB *kaB + fat*IsursaB *kdB *cos(i)]

- ka (kaR, kaG, kaB) si kd (kdR, kdG, kdB) sunt caracteristici ale materialului suprafetei, numite: culoarea ambiantala si culoarea difuza a materialului.

Modelarea reflexiei difuze

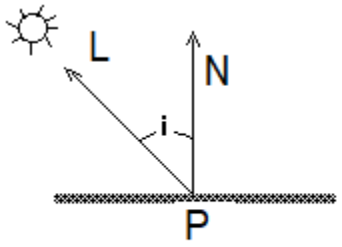
- Stiind că

  cos(i)=L· N / (|L|·|N|) = Lu·Nu

  unde: N este normala la suprafata in P

  L este vectorul din P catre sursa de lumina

  (L· N) – produsul scalar al vectorilor L, N



Reflexia difuza in punctul P

- Rezulta ecuatia care modeleaza lumina reflectata difuz:

❖ $Id\lambda = Ia\lambda * ka\lambda + fat * Isursa\lambda * kd\lambda * (Lu \cdot Nu)$

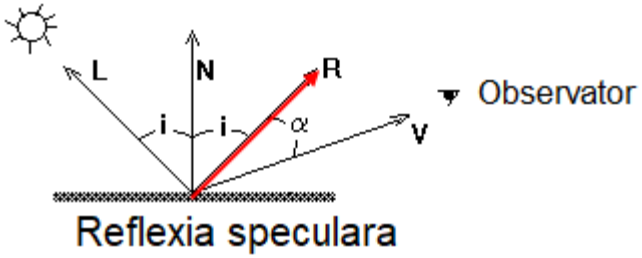
Cd = [IdR, IdG, IdB] – culoarea luminii reflectate difuz

Pentru a include si cazul in care $i > \pi/2$ (lumina de la sursa nu ajunge in punctul considerat):

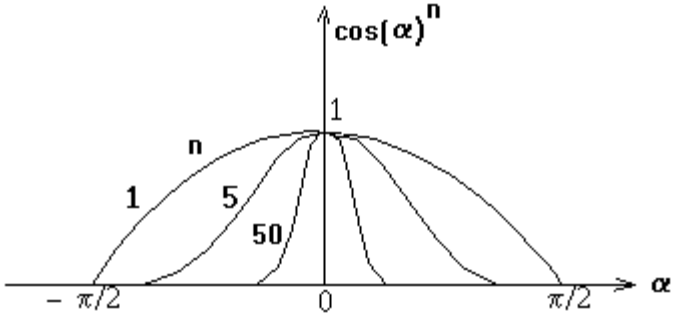
$I\lambda = Ia\lambda * ka\lambda + fat * Isursa\lambda * kd\lambda * \max((Lu \cdot Nu), 0)$

Modelarea reflexiei speculare (1)

- Un reflector perfect, de exemplu o oglindă, reflectă lumina numai într-o singură direcţie, R, care este simetrică cu L faţă de normala la suprafaţă → numai un observator situat exact pe direcţia respectivă va percepe raza reflectată:



Reflexia speculara



Functia de distributie spatiala a luminii reflectate specular

• Pentru materialele imperfect reflectante cantitatea de lumină care ajunge la observator depinde de distribuţia spaţială a luminii reflectate specular:

- la suprafeţele netede (ex. metale) distribuţia este dreaptă şi focalizată;
- la suprafeţele cu rugozităţi (ex. hartia) ea este dispersată.
- se aproximeaza prin $\cos(\alpha)^n$ (modelul Bui-Tuong Phong) unde n este exponentul de stralucire al materialului.

Modelarea reflexiei speculare (2)

Modelul Phong pentru aproximarea reflexiei speculare intr-un punct al unei suprafete 3D:

$$I_{s\lambda} = I_{sursa\lambda} * w(i,\lambda) * \cos(\alpha)^n$$

w(i, λ) este funcţia de reflectanţă, i - unghiul de incidenţă iar λ - lungimea de undă a luminii incidente

In practica, w(i, λ) este înlocuită cu o constantă determinată experimental, numită **coeficientul de reflexie speculară al materialului, notat** $k_{s\lambda}$.

- Modelul practic al reflexiei speculare:

$$I_{s\lambda} = I_{sursa\lambda} * k_{s\lambda} * \cos(\alpha)^n$$

$\cos(\alpha) = R \cdot V / (|R| \cdot |V|) = Ru \cdot Vu$

Rezulta:

$$I_{s\lambda} = I_{sursa\lambda} * fat * k_{s\lambda} * (Ru \cdot Vu)^n$$

Pentru a include si cazul in care α = 90:

$$I_{s\lambda} = I_{sursa\lambda} * fat * k_{s\lambda} * \max((Ru \cdot Vu)^n, 0)$$

Modelarea reflexiei speculare (3)

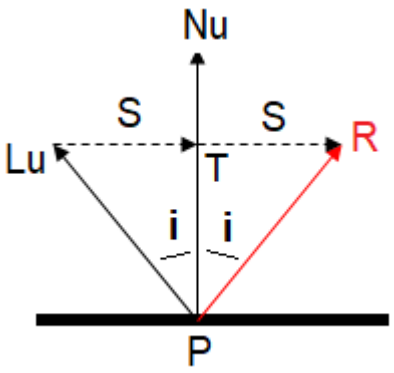
Reflexia speculara nu poate avea loc daca in punctul considerat nu se primeste lumina de la sursa:

$Is\lambda = lum * Isursa\lambda * fat * ks\lambda * \max((Ru \cdot Vu)^n, 0)$

$lum = 1$ daca $(Lu \cdot Nu) > 0$ $\quad (0 <= i < \pi/2)$

$= 0$ altfel

Calculul directiei reflexiei speculare



- Vectorul R este simetricul vectorului L faţă de N.
- Proiectia scalara a vectorului Lu pe directia normalei este marimea segmentului PT: $|Lu|\cos(i) = \cos(i) = Lu \bullet Nu$
- Proiectia vectoriala a lui Lu pe directia normalei este **vectorul PT**, care are lungimea segmentului PT si directia normalei:

$(Lu \bullet Nu) * Nu / |Nu| = (Lu \bullet Nu) * Nu$

$R = Lu + 2 * S$, $PT = Lu + S \rightarrow S = PT - Lu$

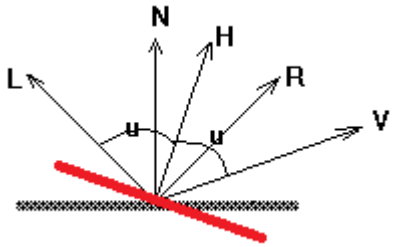
$R = Lu + 2(PT - Lu) = Lu + 2 * (Lu \bullet Nu) * Nu - 2 * Lu = 2 * (Lu \bullet Nu) * Nu - Lu$

$R = 2 * (Lu \bullet Nu) * Nu - Lu$

Modelarea reflexiei speculare (4)

O altă formulare a modelului Phong pentru reflexia speculara foloseste vectorul median, notat cu H în figura. El face unghiuri egale cu L şi cu V.

- Dacă suprafaţa ar fi orientată astfel încât normala sa să aibă direcţia lui H, atunci observatorul ar percepe lumina specular maximă (deoarece ar fi pe direcţia razei reflectate specular).



- Termenul care exprimă reflexia specular este în acest caz:

$I_{s\lambda} = I_{sursa\lambda} * f_{at} * k_{s\lambda} * (N_u \bullet H_u)^n$, unde $H_u = (L_u + V_u) / |(L_u + V_u)|$

Atunci când sursa de lumină şi observatorul sunt la infinit, utilizarea termenului Nu•Hu este avantajoasă deoarece Hu este constant (acelasi in toate punctele unei suprafate).

Modelul de iluminare locala(1)
- lumina reflectata intr-un punct al unei suprafete -

Modelul de iluminare locala (numit şi Modelul de reflexie Phong)

Combina culorile datorate:

reflexiei luminii ambientale – culoarea_ambientala,

reflexiei difuze a luminii de la o sursa / surse – culoarea_difuza

reflexiei speculare a luminii de la o sursa/ surse – culoarea_speculara

❖ Reflexia luminii într-un punct al unei suprafeţe determina culoarea sa în acel punct.

$I\lambda = Ia\lambda * ka\lambda + fat * Isursa\lambda [kd\lambda * max((Lu \bullet Nu),0) + lum * ks\lambda * max((Ru \bullet Vu)^n ,0)]$

lum = 1 daca (Lu • Nu) > 0 (0<= i < π/2)

= 0 altfel

Modelul de iluminare locala(2)

- Dacă scena 3D este luminată de m surse de lumină:

$$I\lambda = Ia\lambda * ka\lambda + \sum_{1<=i<=m} fat_i * Isursa\lambda_i * [kd\lambda * max((Lu_i \bullet Nu),0) + lum_i * ks\lambda * max((Ru_i \bullet Vu)^n,0)]$$

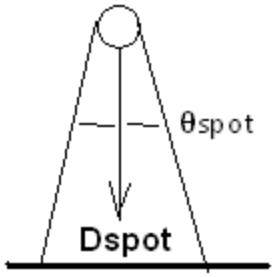
- La cele 3 componente se poate adauga culoarea_emisa de suprafata. Rezulta:

  Culoarea intr-un punct al unei suprafete
  = culoarea_emisa + culoarea_ambiantala + culoarea_difuza + culoarea_speculara

- **Culoarea emisa** :
  - lumina emisa de o suprafata
  - independenta de sursele de lumina
  - aceeasi in orice punct al suprafetei
  - nu lumineaza obiectele din jur
  - culoarea_emisa = Ke, o constanta

Efectul de spot

- In modelul de iluminare locala, sursa de lumina este considerata punctiforma. Lumina produsa de o sursa punctiforma este omnidirectionala.
- Efectul de spot simuleaza o sursa de lumina care produce un con de lumina.
- Numai suprafetele aflate in conul de lumina primesc lumina de la sursa.

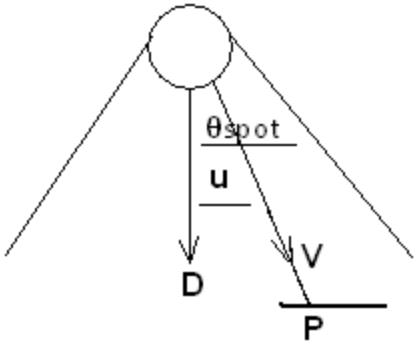


Conul de imprastiere a luminii se calculeaza folosind:

-Pozitia spotului (sursei de lumina)

-Directia spotului

-Imprastierea luminii spotului (θspot)



Un punct P al unei suprafete primeste lumina de la spot daca u< θspot:

$$\cos(u) > \cos(\theta spot)$$

sau

$$Vu \bullet Du > \cos(\theta spot)$$

Modele de "shading"

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2018-2019

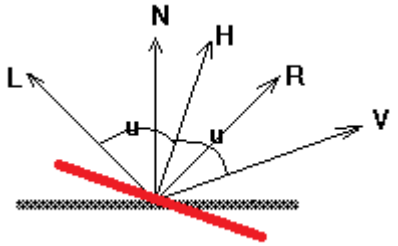
Calculul culorii fragmentelor la redarea suprafetelor 3D (Shading models) (1)

- In vederea vizualizarii, suprafeţele 3D sunt descompuse in faţete poligonale/triunghiulare conectate. Poligoanele procesate in banda grafica pot fi feţele unui corp poliedral (fețe plate) sau pot proveni din descompunerea unei suprafete oarecare (fețele sunt petice de suprafata curba).
- La momentul rasterizarii nu se cunoaste din ce suprafata face parte un poligon.

Modelul LAMBERT(1)
(Iluminare constantă sau plată)

- Toate fragmentele unui poligon sunt afisate cu aceeasi culoare, calculata folosind modelul de iluminare locala, de regula considerand numai componenta difuza:

$$Id\lambda = Ia\lambda * ka\lambda + fat * Isursa\lambda * [kd\lambda * \max((Nu \bullet Lu),0) + lum * \max((Nu \bullet Hu)^{n},0)]$$



Modelul LAMBERT(2)

Modelul se bazează pe următoarele presupuneri:

1. Poligonul face parte din suprafaţa de vizualizat şi nu este o aproximare a unui petic de suprafaţă curbă: vectorul Nu este acelasi in orice punct al poligonului.

2. Sursa de lumină este la infinit: produsul scalar (Nu • Lu) este atunci constant pe întreaga suprafaţă a poligonului.

3. Observatorul este la infinit: (Nu • Hu) este constant pe suprafaţa poligonului;

Dacă ultimele două cerinţe nu sunt satisfăcute, se poate adopta o convenţie de calcul al vectorilor L şi V pentru un întreg poligon. De exemplu, cei doi vectori pot fi calculaţi în centrul poligonului.

Dacă prima cerinţă nu este îndeplinită, aproximarea unei suprafeţe curbe prin reţeaua de faţete poligonale este vizibila deoarece intensităţile calculate pentru faţete adiacente cu orientare diferită sunt diferite.

Modelul LAMBERT(3)

Indicatii de implementare

```
//vertex shader
layout(location = 0) in vec3 v_position;
layout(location = 1) in vec3 v_normal;
uniform vec3 light_dir; // directia sursei de lumina
uniform vec3 eye_dir;//directia observatorului
flat out vec3 color; // culoarea varfului “provocator” va fi folosita la afisarea
    // fragmentelor; in mod implicit ultimul varf al primitivei (ex.: al 3-lea varf pt un triunghi)
void main() {
 //calculeaza culoarea varfului aplicand modelul de iluminare in pozitia varfului
 color = ambient + diffuse + specular;
 ……..……}
//fragment shader
flat in vec3 color; // culoarea varfului “provocator”
void main() {
gl_FragColor = color;
}
```

Efectul de banda Mach

- Efectul de banda Mach (descoperit de Mach în 1865) este o iluzie optica datorata filtrarii componentei de luminozitate a imaginii formate pe retina, de catre sistemul vizual uman.
- Efectul măreşte contrastul dintre zone adiacente cu intensitate usor diferita, la frontiera dintre zone, declanşând detecţia frontierei in sistemul vizual uman.
- Atunci cand o suprafaţa curba (de exemplu o sfera) este descompusa in faţete poligonale, este de dorit ca laturile faţetelor sa nu fie vizibile in imaginea sferei.
- In cazul folosirii modelului Lambert pentru calculul culorilor fragmentelor, datorita folosirii unei singure normale pentru toata suprafaţa unei faţete, culorile faţetelor adiacente vor fi usor diferite, iar efectul de bandă Mach măreşte percepţia schimbării de intensitate pe laturile faţetelor adiacente.

Modelul Gouraud (1)

- La momentul rasterizarii unui poligon, fiecarui vârf al poligonului îi este asociata o culoare.
- Culorile fragmentelor poligonului se obtin prin interpolarea liniara a culorilor din varfuri.

1. Culoarea într-un varf (ieşire din vertex shader: out vec3 color);

- se calculeaza aplicand modelul de iluminare locala in varf (cu N, L, V in cordonate globale).

2. Normala unui vârf, necesara în modelul de iluminare locala, trebuie să fie transmisă la vertex shader din programul de aplicaţie (este intrare pentru vertex shader):

- poate fi calculata exact din ecuatia analitica a suprafetei pe care se afla varful
- poate fi aproximata ca medie aritmetica a normalelor la feţele adiacente în varf

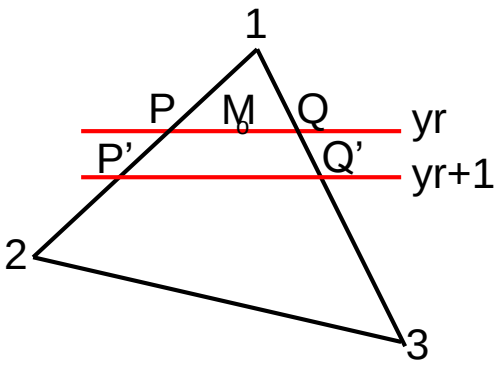
2. La rasterizarea unui poligon, procesorul de rasterizare calculeaza culoarea fiecarui fragment astfel:

- prin interpolarea liniara a culorilor varfurilor, pentru fragmentele de pe laturi
- prin interpolare liniara intre culorile din capetele segmentului interior pe care se afla fragmentul.

Culoarea fragmentului este intrare pentru programul fragment shader (in vec3 color).

Modelul Gouraud (2)

Interpolarea liniara a culorilor



$y = y1 + t\,(y2 - y1) \rightarrow tP = (yr - y1)/(y2 - y1)$

$IP = I1 + tP\,(I2 - I1) = I1 + (I2 - I1)\,(yr-y1) / (y2-y1) = I1 + (yr-y1) * \mathbf{C_{1-2}}$

$C_{1-2} = (I2 - I1) / (y2 - y1)$ - constanta pentru latura 1-2

$IQ = I1 + (I3 - I1)\,(yr-y1) / (y3-y1) = I1 + (yr - y1) * \mathbf{C_{1-3}}$

$C_{1-3} = (I3 - I1) / (y3 - y1)$ - constanta pentru latura 1-3

$x = xP + t(xQ - xP) \rightarrow tM = (xM - xP)/(xQ - xP)$

$IM = IP + tM\,(IQ - IP) = IP + (IQ - IP)\,(xM - xP) / (xQ - xP) = IP + (xM-xP)* \mathbf{C_{P-Q}}$

$C_{P-Q} = (IQ - IP) / (xQ - xP)$ - constanta pentru segmentul P-Q

Calculul incremental al culorilor

$IP' = I1 + (I2 - I1)\,(yr+1 - y1) / (y2 - y1) = I1 + (yr+1 - y1) * \mathbf{C_{1-2}} = IP + \mathbf{C_{1-2}}$

$IP' = IP + C_{1-2}$

analog, **$IQ' = IQ + C_{1-3}$**

M' (xM + 1, ys)

$IM' = IP + (xM +1 - xP) * \mathbf{C_{P-Q}} \rightarrow$ **$IM' = IM + CP-Q$**

Modelul Gouraud (3)

Aprecieri asupra modelului Gouraud

- Este integrat în algoritmul de rasterizare a poligoanelor
- Calcule simple pentru culoarea unui fragment (calcul incremental) → rapid
- Nu permite calculul luminii reflectate specular pentru fragmentele interioare unui poligon (intensitatile fragmentelor interioare nu pot fi mai mari decat cele din varfuri)
- Reduce efectul de banda Mach, dar nu il elimina:
  - Culorile pe latura de adiacenta a 2 poligoane sunt aceleasi
  - Nu tine cont de tangentele la fetele adiacente pe o latura (culorile de pe laturi se calculeaza folosind numai culorile din varfuri)
  - Efectul de banda Mach poate fi observat in vecinatatea siluetei unei suprafeţe 3D si a zonelor de curbura mare.

Modelul Phong (1)

- La momentul rasterizarii unui poligon, fiecarui vârf îi este asociata normala vârfului.
  - Normala vârfului este transmisa din programul de aplicatie, ca si pentru modelul Gouraud.
  - Normala si pozitia vârfului, in coordonate globale, sunt ieşiri din vertex shader:
  - Procesorul de rasterizare interpoleaza liniar pozitiile şi normalele varfurilor, transmitand la fragment shader pozitia si normala fragmentului.
- In fragment shader se calculeaza culoarea fragmentului aplicand modelul de iluminare locala in pozitia fragmentului

La rasterizarea unui poligon se calculeaza o normala pentru fiecare fragment astfel:

- prin interpolarea liniara a normalelor varfurilor, pentru fragmentele de pe laturi (la fel ca in modelul Gouraud, pentru culori);
- prin interpolare liniara intre normalele capetelor segmentului interior pe care se afla fragmentul (la fel ca in modelul Gouraud, pentru culori).

Modelul Phong (2)

Dezavantaje fata de modelul Gouraud

❑ Componentele Nx, Ny, Nz ale normalei unui fragment se obţin (in algoritmul de rasterizare) printr-un calcul incremental analog cu cel folosit pentru calculul culorilor in modelul Gouraud dar, pentru folosirea lor în calculul culorii, normala trebuie să fie normalizată în fragment shader:

$$Nu = N/|N|, \text{ unde } |N| = (Nx^2 + Ny^2 + Nz^2)^{0.5}$$ (se foloseste o functie GLSL)

❑ De asemenea, vectorii L (catre sursa de lumina) si V (catre observator), folositi in modelul de iluminare locala la nivel de fragment, sunt calculati si normalizati in programul fragment shader pentru a fi folositi in calculul culorii fragmentului.

➢ Calcule mai complexe la nivel de fragment decât in modelul Gouraud

Avantaje

❑ Permite redarea reflexiei speculare in orice fragment al unui poligon

❑ Reduce mult efectul de banda Mach

Transparenta
în modelele de iluminare locala

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2018-2019

MODELAREA TRANSPARENTEI (1)

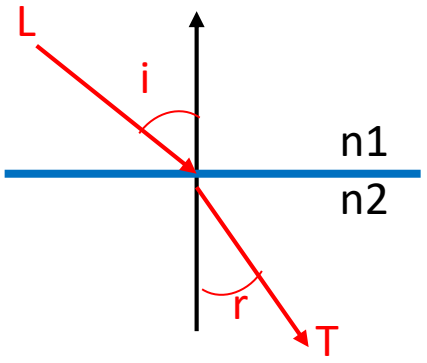
Unele obiecte ale scenei sintetizate pot fi construite din materiale transparente sau translucide.

Transmisia luminii prin obiectele transparente este specular, în timp ce prin cele translucide este difuză.

- Atunci când lumina trece dintr-un mediu într-altul (de exemplu, din aer în apă), direcţia sa se modifică datorită refracţiei.
- Relaţia dintre unghiul razei incidente, i, şi cel al razei refractate, r, este dată de legea lui Snell:

sin(r)/sin(i) = n1/n2

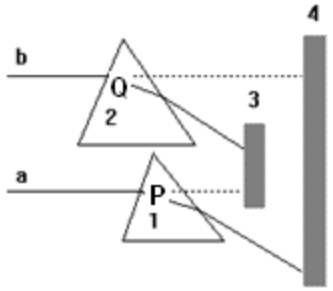
n1 şi n2 sunt indicii de refracţie ai celor două medii (materiale) traversate de lumină.



MODELAREA TRANSPARENTEI (2)

Indicele de refracţie al unui material este dependent de lungimea de undă a luminii incidente şi chiar de temperatură, dar în modelele de sinteza a imaginilor el este considerat constant.

Culoarea vizibila intr-un punct al unei suprafete transparente provine de la obiectul aflat pe directia razei transmise.



3 si 4 sunt poligoane opace

1 si 2 sunt poligoane transparente, aflate in fata poligoanelor 3 si 4

a si b sunt raze de lumina provenind de la surse, incidente in P si Q

Tinand cont de refractie:

- Culoarea in P este data de poligonul 4
- Culoarea in Q este data de poligonul 3

Neglijand refractia (T are directia razei incidente)

- Culoarea in P este data de poligonul 3
- Culoarea in Q este data de poligonul 4

MODELAREA TRANSPARENTEI (3)

- Refracţia produce, de asemenea, o distorsionare a obiectelor, asemănătoare cu aceea produsă de o proiecţie perspectivă: → dacă se doreşte obţinerea de imagini realiste, trebuie să se ţină cont de refracţie.

- Multe metode practice de modelare a transparenţei ignoră refracţia, astfel încât obiectele vizibile printr-o suprafaţă transparentă sunt cele aflate pe direcţia razei incidente.

Motivul ignorării:

- reducerea volumului de calcule;
- obţinerea realismului fotografic în totalitate (fara deformare)

Atunci când suprafaţa vizibilă într-un pixel este transparentă, culoarea în care va afişat pixelul se poate obţine combinând culoarea suprafeţei vizibile cu aceea a suprafeţei aflată imediat în spatele său (sau a fondului), folosind următoarea formulă de interpolare:

MODELAREA TRANSPARENTEI (4)

$I_\lambda = (1-kt_1)*I_{\lambda 1} + kt_1*I_{\lambda 2}$

$I_{\lambda 1}$ este culoarea suprafetei vizibile în pixel

$I_{\lambda 2}$ este culoarea suprafetei din spatele celei vizibile, care se proiecteaza in acelasi pixel

Coeficientul de transmisie, kt_1, este o măsură a transparenţei suprafeţei vizibile în pixel,

$0 <= kt_1 <= 1$

$kt_1 = 0$ → suprafaţa vizibilă este opacă şi deci pixelul va fi afişat în culoarea sa, $I_{\lambda 1}$

$k_{t1} = 1$ → suprafaţa vizibilă este perfect transparentă şi nu contribuie la culoarea pixelului.

Pixelul va fi afişat în culoarea suprafeţei din spate, $I_{\lambda 2}$

Dacă $k_{t1}=1$ şi suprafaţa din spatele celei vizibile este la rândul său transparentă, metoda de calcul se aplică recursiv, până când se întâlneşte o suprafaţă opacă sau fondul.

Aproximarea liniară din model nu dă rezultate bune pentru suprafeţele curbe: în apropierea laturilor siluetei unei suprafeţe curbe (de exemplu, o vază sau o sticlă) grosimea materialului reduce transparenţa.

MODELAREA TRANSPARENTEI (5)

Solutia propusa de Kay: kt se calculeaza in functie de normala la suprafata in punctul considerat:

$k_t = kt_{min} + (kt_{max} - kt_{min})(1-(1-N_z)^m)$

unde

kt_{min} şi kt_{max} reprezintă transparenţa minimă şi cea maximă a suprafeţei,

N_z este componenta z a normalei normalizate la suprafaţă în punctul pentru care se calculează kt,

m este un exponent ce caracterizează transparenţa. Valorile uzuale pentru m sunt 2 şi 3.

$k_t = 1 \rightarrow k_t = kt_{max}$

$k_t = 0 \rightarrow k_t = kt_{min}$

- **Majoritatea algoritmilor de determinare a vizibilităţii suprafetelor la afişarea scenelor 3D pot fi adaptaţi pentru a îngloba transparenţa.**

- In algoritmii care afişează poligoanele scenei 3D în ordinea “din spate în faţă” (back to front), de ex. algoritmul Pictorului şi BSP, $I_{\lambda 1}$ corespunde poligonului care se rasterizeaza la un moment dat iar $I_{\lambda 2}$ este valoarea existentă în memoria imagine pentru pixelul considerat.

MODELAREA TRANSPARENTEI (6)

- Adăugarea efectului de transparenţă în algoritmul Z-Buffer este mai dificilă, deoarece poligoanele sunt rasterizate în ordinea în care sunt transmise în banda grafică, neţinându-se cont de distanţa lor faţă de observator.

-Se poate realiza folosind mai multe memorii buffer-imagine si generand imaginea in mai multe etape, imaginea finala obtinandu-se prin combinarea imaginilor poligoanelor opace cu cele ale poligoanelor transparente

In OpenGL, transparenta poate fi simulata prin amestecul dintre culoarea fragmentului curent si cea a pixelului in care se afiseaza, folosind modelul (R, G, B, A);

A – opacitatea: 0 – transparent; 1 – opac

Se folosesc functiile:

glEnable(GL_ALPHA_TEST) – activeaza testul alfa

MODELAREA TRANSPARENTEI (7)

glAlphaFunc(func, ref) – specifica functia de comparare a valorii A a fragmentului cu valoarea de referinta, ref; exemplu:

func = GL_GREATER – fragmentul este afisat daca opacitatea sa este > decat valoarea ref

glEnable(GL_BLEND) - activeaza amestecul culorilor

glBlendFunc(..) – specifica modul de amestec al culorilor. Exemplu:

glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA):

noua_culoare_pixel = Culoare_fragment * A_fragment + Culoare_pixel * (1 – A_fragment)

- Operatia de amestec este o operatie raster: efectuata de procesorul de rasterizare dupa testul de adancime, inainte de actualizarea valorii pixelului in bufferul imagine (frame buffer).

INTRODUCEREA CETEI IN IMAGINI

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2018-2019

MODELAREA CEȚEI(1)

Ceaţa adaugă realism imaginii şi ajută în eliminarea defectelor produse de decuparea la nivelul planului din spate al volumului vizual:

- Fără ceaţă, pe măsură ce observatorul se îndepărtează de un obiect, obiectul se aproprie de planul din spate şi este decupat atunci când planul din spate îl intersectează.
- Cu ceaţă, dacă intensitatea ceţii creşte proporţional cu distanţa de la poziţia observatorului, se crează efectul de distanţă pentru obiectele îndepărtate, iar obiectele decupate la nivelul planului din spate nu mai sunt vizibile.

Cfog – culoarea de ceaţă

Clocal – culoarea calculată într-un punct folosind modelul de iluminare locala

$0 <= \Phi <= 1$ – factorul de ceaţă, proporţional cu distanţa de la observator la punct

($\Phi=1$: opacitate)

Culoarea punctului, afectata de ceata: **$C_{loc_fog} = (1-\Phi)*C_{local} + \Phi*C_{fog}$**

MODELAREA CETEI(2)

Fie dist distanta de la observator la un punct din scena vizualizata.

Factorul de ceaţă liniar

- Actioneaza intre 2 distante definite in sistemul coordonatelor observator, z0 si z1, masurate pe directia in care priveste observatorul.

  $\Phi = 0$ pentru dist < z0

  $\Phi = (dist - z0) / (z1 - z0)$ pentru z0 <= dist <= z1

  $\Phi = 1$ pentru dist > z1

Factorii de ceaţă exponentiali

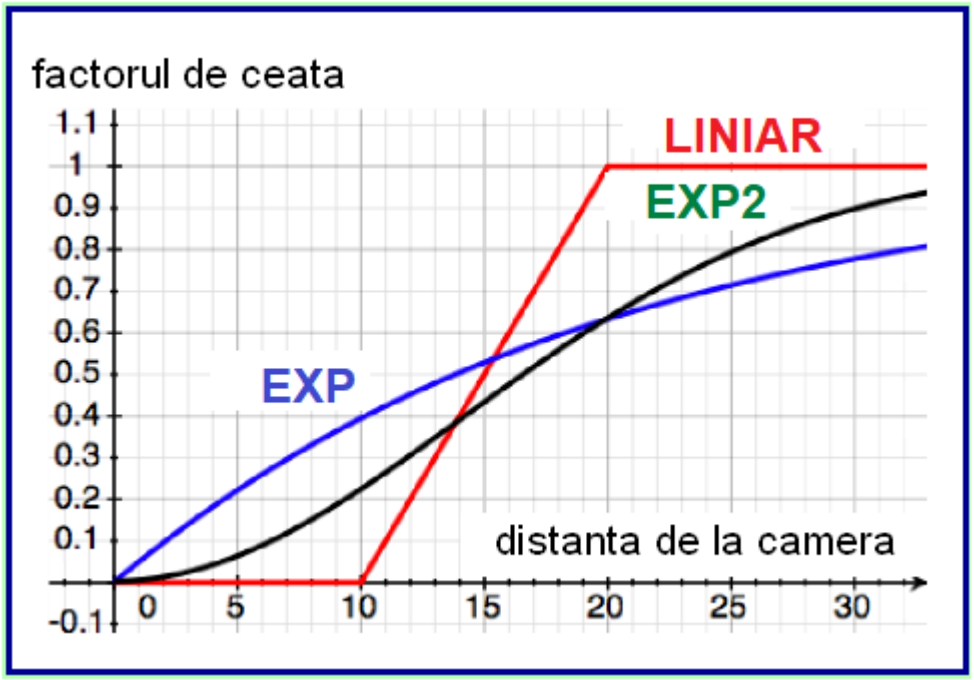
$\Phi = e^f$

EXP: $f = (- fog_density * dist)$

EXP2: $f = (- fog_density * dist)^2$

- fog_density: densitatea cetii (un numar real, de ex. 0.05) - modeleaza atenuarea luminii datorata cetii

MODELAREA CETEI(3)



z0 = 10; z1 = 20; fog_density = 0.3

INTRODUCEREA CETEI IN IMAGINI (1)

Includerea cetei in calculul culorii se poate efectua:

- In vertex shader (modelul Gouraud):

- se calculeaza **Φ** folosind distanta (dist) de la observator la pozitia varfului transformata in coordonate observator

- se calculeaza culoarea varfului afectata de ceata: **Cloc_fog = (1-Φ)*Cvrf + Φ*Cfog**

- **Cloc_fog** este iesire din vertex shader

- culorile afectate de ceata ale varfurilor sunt interpolate de GPU la nivel de fragmente

- culoarea de ceață interpolata, **Cloc_fog,** este intrare pentru fragment shader

- In fragment shader(modelul Phong):

Varianta (1)

- se transmite de la vertex shader pozitia varfului in coordonate observator

- pozitiile varfurilor in coordonate observator sunt interpolate de GPU la nivel de fragmente

- in fragment shader se calculeaza **Φ** folosind distanta (dist) de la observator la pozitia fragmentului in coordonate observator

INTRODUCEREA CETEI IN IMAGINI (2)

```
// Calculul culorii de ceata in fragment shader- Varianta (1)

//Vertex shader
layout(location = 0) in vec3 in_position;
layout(location = 1) in vec3 in_normal;
uniform mat4 model_matrix, view_matrix, projection_matrix;

out vec3 world_pos;// pozitia varfului in coord globale
out vec3 world_normal;//normala transformata in coord globale
out vec4 view_pos; //pozitia varfului in coord observator

void main(){
  world_pos = (model_matrix * vec4(in_position,1)).xyz;
  world_normal = normalize(mat3(model_matrix) * in_normal);
  view_pos = view_matrix * model_matrix * vec4(in_position,1);
  gl_Position = projection_matrix * view_pos;
 }
```

```
//fragment shader
layout(location = 0) out vec3 out_color;
uniform vec3 light_position;
uniform vec3 eye_position;
uniform int fogSelector; //0 linear; 1 exponential; 2 exponential square
uniform vec3 Cfog = vec3(0.5, 0.5,0.5);//culoarea cetii
……..
in vec3 world_pos;// pozitia interpolata a fragmentului in coord globale
in vec3 world_normal;/ normala in coord globale interpolata
in vec4 view_pos;// pozitia interpolata a fragmentului in coord observator
void main(){
//Calculeaza Cfragment aplicand modelul de iluminare locala in pozitia fragmentului(world_pos)
   Cfragment = ….
//Calculeaza distanta de la pozitia interpolata a fragmentului la observator
dist = length(view_pos); //vectorul (0,0,0) → view_pos
if(fogSelector == 0)//linear fog
{  fogFactor = (dist - 20)/(80 - 20); // z0 =20; z1=80
  fogFactor = clamp( fogFactor, 0.0, 1.0 );//restrange fogFactor la intervalul 0-1
  out_color = (1-fogFactor)*Cfragment + fogFactor*Cfog
} ……..}
```

INTRODUCEREA CETEI IN IMAGINI (3)

```
// Calculul culorii de ceata in fragment shader- Varianta (2)
//Vertex shader
out vec3 world_pos;// pozitia varfului in coord globale
out vec3 world_normal;//normala transformata in coord globale
//fragment shader
layout(location = 0) out vec3 out_color;
uniform vec3 light_position;
uniform vec3 eye_position;
uniform vec3 Cfog = vec3(0.5, 0.5,0.5);//culoarea cetii
in vec3 world_pos;// pozitia interpolata a fragmentului in coord globale
in vec3 world_normal;// normala in coord globale interpolata
void main(){
//Calculeaza Cfragment  aplicand modelul de iluminare locala in pozitia fragmentului (world_pos)
   Cfragment = ….
// Calculeaza pozitia fragmentului in coordonate observator: frg_pos → frg_eye_pos
//Calculeaza distanta de la pozitia fragmentului, frg_eye_pos, la observator
   dist = length(frg_eye_pos);
//Calculeaza fogFactor folosind dist
…….
```

INTRODUCEREA CETEI IN IMAGINI (4)

Calculeaza pozitia fragmentului in coordonate observator: frg_pos → frg_eye_pos

//transformarea in viewport

gl.FragCoord.x = (width/2)frg_xd + ox , ox = px+ width/2

gl.FragCoord.y = (height/2)frg_yd + oy, oy = py + height/2

gl.FragCoord.z = ((f-n)/2)frg_zd + (n+f)/2 = frg_zd/2 + 0.5 // implicit: n=0, f=1

Rezulta →

frg_xd = (gl.FragCoord.x – ox)/(width/2)

frg_yd = (gl.FragCoord.y – oy)/(height/2)

frg_zd = (gl.FragCoord.z – 0.5)*2

//transformarea de normalizare

frg_ xd = frg_ xc/frg_wc; frg_ yd = frg_ yc/frg_wc; frg_ zd = frg_ zc/frg_wc; frg_ wc = -eye

//pozitia fragmentului in coordonate de decupare

frg_clip_pos = [frg_xc, frg_yc, frg_zc, fgr_wc]^T

//transformarea de proiectie

frg_clip_pos = projection_matrix * frg_eye_pos



//pozitia fragmentului in coordonate observator

frg_eye_pos = (projection_matrix)$^{-1}$ *frg_clip_pos

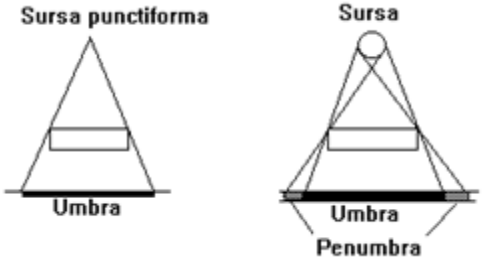
Redarea umbrelor în imagini

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2018-2019

INTRODUCEREA UMBRELOR IN IMAGINI(1)

- Atunci când un observator priveşte o scenă 3D luminată de o sursă de lumină dintr-o poziţie diferită de aceea a sursei de lumină, va vedea umbrele produse de obiectele scenei.
- Umbrele au o contribuţie însemnată la realismul imaginii, îmbunătăţind percepţia profunzimii.



In general, se considera numai umbra pura: calcule mai simple.

Volumul de calcule depinde şi de poziţia sursei de lumină:

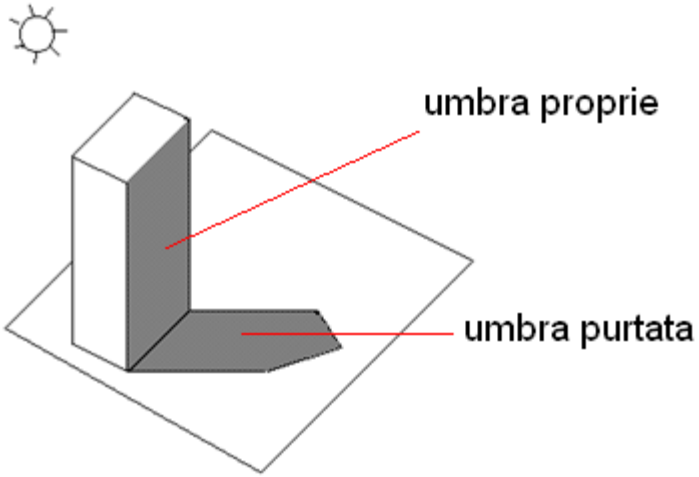
- Dacă sursa este la infinit calculele sunt mai simple.
- In cazul unei surse situate la distanţă finită dar în afara volumului vizual, este necesară o proiecţie perspectivă din poziţia sursei.
- Cazul cel mai dificil este acela în care sursa este situată în volumul vizual.

INTRODUCEREA UMBRELOR IN IMAGINI(2)

- Problema determinării umbrelor este similară aceleia de determinare a părților nevizibile ale unei scene 3D: partile scenei 3D care nu sunt vizibile din poziţia sursei de lumină sunt în umbră.

  → Crearea unei imagini cu umbre presupune rezolvarea de două ori a problemei suprafeţelor nevizibile: o dată privind scena din poziţia sursei de lumină, a doua oară privind-o din poziţia observatorului.

- Sunt două tipuri de umbre: **umbre proprii** şi **umbre purtate**.



POLIGOANE DETALIU DE UMBRA (1)

–Umbrele proprii sunt generate de obiectul însuşi, care împiedică lumina să ajungă la unele dintre feţele sale: feţele umbrite de umbra proprie sunt feţele auto-obturate atunci când scena este văzută din poziţia sursei de lumină.

–Umbra purtată este umbra pe care un obiect o produce pe alte părţi ale scenei, la care lumina nu ajunge din cauza obiectului.

Poligoane detaliu

❑ Umbrele purtate se pot determina proiectând în scenă, din poziţia sursei, toate poligoanele (feţele) neumbrite de umbra proprie.

- Rezultă un set de **poligoane de umbră**, care se adaugă la reprezentarea scenei.

❑ Atât fețele umbrite de umbra proprie cat şi poligoanele de umbra se folosesc ca poligoane-detaliu la redarea scenei 3D.

❑ Poligoanele detaliu sunt poligoane coplanare cu poligoanele scenei, care se folosesc în calculul culorii fragmentelor.

POLIGOANE DETALIU DE UMBRA (2)

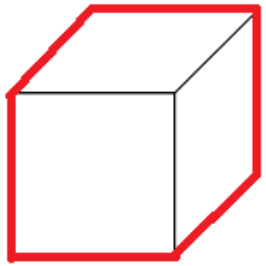
- Poligoanele de umbra au o culoare specifica umbrei pure.
- Poligoanele de umbra se trimit in banda grafica dupa poligoanele scenei, care sunt vizibile din poziţia observatorului, cu testul de adancime dezactivat (fragmentele poligoanelor de umbra suprascriu unele dintre fragmentele vizibile) .

- Pot fi generate mai multe vederi ale scenei fără a recalcula umbrele, deoarece umbrele depind numai de poziţia sursei (surselor) de lumină.

- Numărul de poligoane de umbră este mai mic,

dacă în loc să se proiecteze fiecare faţă luminată de sursă,

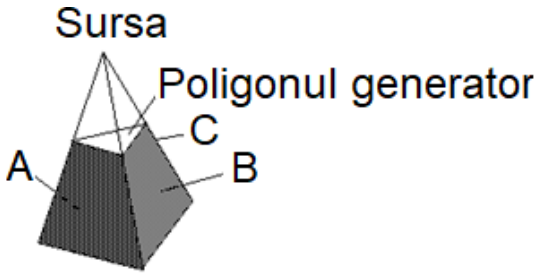
se proiectează silueta fiecărui obiect văzută din poziţia sursei de lumină.



- Calculul poligoanelor de umbra este destul de complicat, de aceea in prezent se folosesc alte metode pentru introducerea umbrelor in imagini.

VOLUME DE UMBRA (1)

- Sursa de lumină este considerată punctiformă iar scena 3D alcatuita din poligoane.
- Un **volum de umbră** este definit de o sursă de lumină şi un poligon luminat (vizibil din poziţia sursei de lumină), pe care-l vom numi **poligonul generator**.



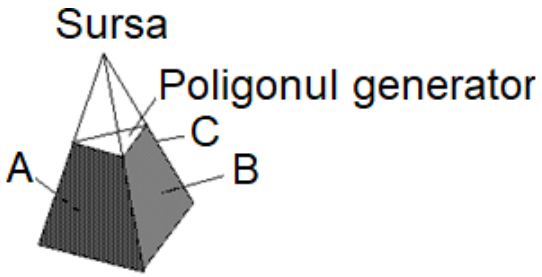
- Volumul de umbra determinat de o sursă şi un poligon generator este delimitat de o faţă care reprezintă poligonul generator scalat. Această faţă este situată la o distanţă faţă de sursă dincolo de care intensitatea luminii sursei este neglijabilă, deci orice punct aflat dincolo de această limită este umbrit.
- Volumul de umbră poate fi decupat la marginile volumului vizual.

VOLUME DE UMBRA (2)

❑ Feţele laterale ale volumului sunt numite **poligoane de umbră**. Un poligon de umbră este determinat de o latură a poligonului generator şi cele două drepte care pleacă din sursa de lumină, fiecare trecând printr-un vârf al laturii.

❑ Normalele la feţele laterale punctează înspre exteriorul volumului.

❑ Poligoanele de umbră se folosesc pentru determinarea umbririi produse de poligonul generator în scenă.



Notăm cu :

– PUV poligoanele de umbră care sunt vizibile din poziţia observatorului (A şi B în figura) şi cu

– PUN poligoanele de umbră care nu sunt vizibile din poziţia observatorului (poligonul C).

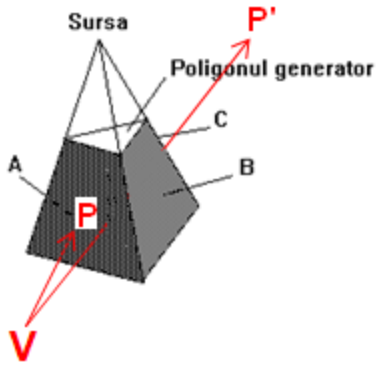
➢ Clasificarea poligoanelor de umbra in PUV si PUN se poate face pe baza normalelor la poligoane si a pozitiei observatorului (back face culling).

VOLUME DE UMBRA (3)

Fie

– un punct P al unei suprafeţe din scena 3D şi

– VP vectorul din poziţia observatorului (V) în punctul P.



Punctul P este umbrit dacă numărul de poligoane de tip PUV intersectate de vectorul VP este mai mare decât numărul de poligoane de tip PUN intersectate de vector.

Acesta este singurul caz, atunci când punctul V nu este în umbră.

❑ In general, pentru a determina dacă un punct este în umbră, se poate folosi un contor în care iniţial se memorează numărul de volume de umbră care conţin poziţia observatorului.

– Se asociază poligoanelor de tip PUV valoarea +1 iar celor de tip PUN valoarea -1.

– Atunci când vectorul VP traversează un poligon de umbră, se adună la contor valoarea asociată poligonului.

– Punctul P este umbrit dacă valoarea contorului este pozitivă în P.

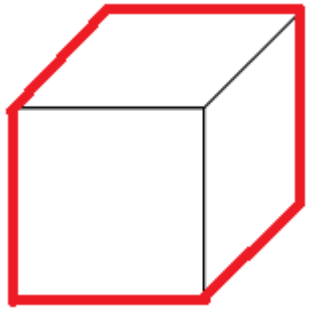
VOLUME DE UMBRA (4)

- Volumul de calcule presupus de acest algoritm poate fi redus dacă în loc să se calculeze volumul de umbră pentru fiecare poligon vizibil din poziţia sursei, se calculează un singur volum de umbră pentru o suprafaţă poliedrală.

In acest scop, se determină poligoanele de umbră numai pentru laturile care fac parte din silueta suprafeţei, văzută din poziţia sursei de lumina.

- ❑ Silueta unei suprafeţe, corespunzătoare unui punct de observare, este un set conectat de laturi care aparţin poligoanelor vizibile din punctul de observare.
- ❑ O latură de siluetă este fie o latură de margine a unei suprafeţe deschise, fie o latură care separă un poligon vizibil de unul nevizibil.

- Pentru determinarea laturilor de siluetă, este necesar să se folosească o structură de date care reflectă adiacenţa poligoanelor. Cunoscându-se poligonul adiacent pe fiecare latură a fiecărui poligon vizibil din poziţia sursei, se pot determina rapid laturile de siluetă.



VOLUME DE UMBRA (5)

Implementarea metodei folosind stencil-buffer(1)

- Buffer-ul de marcaje (stencil-buffer) este prezent pe placile grafice actuale (in memoria GPU), alaturi de buffer-ul imagine (frame-buffer) si buffer-ul de adancime (z-buffer).

– este o memorie de numere intregi asociata pixelilor imaginii, de regula un byte/pixel;

– poate fi utilizat pentru a implementa diferite operatii grafice, de regula in combinatie cu z-buffer, cum este si cazul implementarii volumelor de umbra:

– valorile din stencil-buffer pot fi incrementate/ decrementate automat pentru fiecare fragment care trece/ nu trece testul de adancime.

- Exista mai multe metode de a utiliza stencil-buffer pentru implementarea metodei volumelor de umbre.

VOLUME DE UMBRA (6)

Implementarea metodei folosind stencil-buffer(2)

- In metoda prezentata in continuare, se creaza in stencil-buffer o masca a umbrelor din imagine: orice pixel de umbra, (x,y), va fi reprezentat in stencil-buffer printr-o valoare $\neq 0$.

- **Pixelii de umbra sunt afisati in culorile fragmentelor vizibile, calculate cu formula.**

Cfragment = Cemisiva + Cambianta = ke + ka*globalAmbient

ke, ka – proprietati de material

– Se presupune ca s-au calculat poligoanele de umbra.

– Conturul fiecarui poligon de umbra trebuie sa fie orientat in sens trigonometric atunci cand este privit din exteriorul volumului de umbra. In acest fel, poligoanele PUV sunt cele “din faţa” observatorului, iar poligoanele PUN sunt cele “din spate”.

VOLUME DE UMBRA (7)

Implementarea metodei folosind stencil-buffer(3)

Consideram cazul in care observatorul nu este in umbra.

Pasul 1

Initializeaza z-buffer si stencil-buffer; // nu trebuie programate, acestea sunt
// initializate in mod automat pentru fiecare cadru imagine

Activeaza scrierea in z-buffer si dezactiveaza scrierea in stencil-buffer ; //implicite

*** Rasterizeaza scena (folosindu-se algoritmul z-buffer) fara lumini**

Efectul:

– in frame-buffer se obtine imaginea scenei in umbra (fara lumini). Culoarea fiecarui pixel este:

Cpixel = culoarea emisiva + culoarea ambientala;

La sfarsitul acestui pas, z-buffer contine coordonatele z ale fragmentelor vizibile in imagine.

VOLUME DE UMBRA (8)

Implementarea metodei folosind stencil-buffer(4)

Pasul 2

glDepthMask(GL_FALSE); // Dezactiveaza scrierea in z-buffer

glColorMask(GL_FALSE, GL_FALSE, GL_FALSE, GL_FALSE); // Dezactiveaza scrierea in frame-buffer

glFrontFace(GL_CCW);// feţele “din faţă” sunt orientate trigonometric

glCullFace(GL_BACK);//se vor elimina feţele din spate (nu sunt rasterizate poligoanele PUN)

glStencilFunc(GL_ALWAYS, 0x0, 0xff);// se activeaza scrierea in stencil buffer

glStencilOp(GL_KEEP, GL_INCR, GL_KEEP); //se incrementeaza stencil buffer pt toate
// fragmentele care trec testul de adancime

***Rasterizeaza poligoanele de umbra**

Efectul:

Sunt rasterizate poligoanele de umbra de tip PUV.

Pentru fiecare fragment rezultat, f, se face testul:

daca f.z < z-buffer[f.y][f.x] // fragmentul f in fata fragmentului vizibil in pixelul (x,y)

atunci stencil-buffer[f.y][f.x]++;

// se numara poligoanele PUV aflate intre observator si obiectele scenei

VOLUME DE UMBRA (9)

Implementarea metodei folosind stencil-buffer(5)

Pasul 3

glCullFace(GL_FRONT);//se vor elimina feţele din faţă (nu sunt rasterizate poligoanele PUV)

glStencilOp(GL_KEEP, GL_DECR, GL_KEEP); //se decrementeaza stencil buffer pt toate
// fragmentele care trec testul de adancime

***Rasterizeaza poligoanele de umbra**

Efectul:

Sunt rasterizate poligoanele de umbra de tip PUN.

Pentru fiecare fragment rezultat, f, se face testul:

daca f.z < z-buffer[f.y, f.x] // f este in faţa fragmentului vizibil in pixelul (x,y)

atunci stencil-buffer[f.y, f.x]--; // se numara poligoanele PUN aflate intre observator
// si obiectele scenei

In stencil-buffer s-a obtinut o masca cu “gauri” in punctele care primesc lumina:

valorile din stencil-buffer sunt diferite de zero pentru pixelii care trebuie afisati in umbra.

VOLUME DE UMBRA (10)

Implementarea metodei folosind stencil-buffer(6)

Pasul 4

```
glEnable(GL_STENCIL_TEST);// Activeaza testul stencil
glStencilOp(GL_KEEP, GL_KEEP, KEEP); // Dezactiveaza scrierea in stencil-buffer
glStencilFunc(GL_EQUAL, 0x0, 0xff); // Specifica testul stencil
glDepthMask(GL_TRUE); // Activeaza scrierea in z-buffer
glColorMask(GL_TRUE, GL_TRUE, GL_TRUE, GL_TRUE); // Activeaza scrierea in frame
                                                 // buffer
```

***Rasterizeaza scena (folosindu-se algoritmul zbuffer) cu lumini**

Efectul:

Se rasterizeaza scena modificandu-se in frame-buffer numai culorile pixelilor (x,y) pentru care stencil-buffer [y][x] =0 (fragmentul nu este in umbra)

VOLUME DE UMBRA (11)

Implementarea metodei folosind stencil-buffer(7)

Cazul general:

Se initializeaza stencil-buffer cu o valoare reprezentand numarul de volume de umbra care contin pozitia observatorului.

Aprecieri asupra metodei volumelor de umbra

Pozitive:

- Este generală şi poate fi aplicată pentru orice scena, pentru oricate surse de lumina.
- Produce forme de umbre exacte.

Negative:

- Complexa din punct de vedere al timpului de calcul:

  se rasterizeaza scena de 2 ori si poligoanele de umbra o singura data dar in 2 pasi!

- Sursele de lumina sunt considerate punctiforme, de aceea nu pot fi obtinute penumbre.

Introducerea umbrelor în algoritmul z-buffer - shadow mapping -

Williams [WiLL78] a propus o metodă de generare a umbrelor bazată pe rasterizarea de 2 ori a scenei, cu execuţia algoritmului Z-Buffer:

- In prima etapă se construieşte buffer-ul Z al scenei văzută din poziţia sursei de lumina (în transformarea de vizualizare a scenei – funcția lookAt - se consideră poziţia sursei).

  Vom nota acest buffer cu ZS. El contine coordonatele z ale fragmentelor care primesc lumina de la sursa. Mai este numit si « **shadow map** », de unde si denumirea sub care este cunoscuta metoda in prezent.

- In etapa a doua se construieşte imaginea văzută din poziţia observatorului, utilizand buffer-ul ZS pentru determinarea punctelor aflate in umbra.

In cazul mai multor surse de lumină, se utilizează câte un buffer ZS pentru fiecare sursă.

Implementarea moderna a metodei este numita “shadow mapping”.

Introducerea umbrelor in algoritmul z-buffer

Etapa I.

Se calculeaza matricea de transformare a varfurilor, MVPS, cu observatorul in pozitia sursei de lumina

Se transmite MVPS la vertex shader (ca variabila uniforma).

Se cere folosirea unui buffer ZS ca buffer de adancime, in locul buffer-ului Z implicit

Se cere rasterizarea scenei:

1. In vertex shader se transforma varfurile primitivelor folosind MVPS
2. pentru fiecare primitiva a scenei

    pentru fiecare fragment f(x,y,z) rezultat din rasterizarea primitivei

        dacă z < ZS[y][x] atunci

            ZS[y][x] = z //fragmentul este vizibil în pixelul (x,y) din poziţia sursei

La sfarsitul acestei etape, in ZS sunt memorate coordonatele z ale fragmentelor care primesc lumina de la sursa.

Introducerea umbrelor în algoritmul z-buffer

Etapa II.

Se calculeaza matricea de transformare a varfurilor MVPO, cu observatorul in pozitia sa.

Se transmit la vertex shader (ca variabile uniforme) MVPO si MVPS.

Se cere rasterizarea scenei:

1. In vertex shader se calculeaza varful transformat cu MVPO (gl_Position) si MVPS (poz_sursa)

poz_sursa = pozitia varfului transformata cu MVPS este variabila de iesire din vertex shader

2. pentru fiecare primitiva a scenei

pentru fiecare fragment f(x,y,z) rezultat din rasterizarea primitivei

dacă z < Z-Buffer[y][x] atunci //fragment vizibil în pixelul (x,y), din poziţia observatorului

{Z-Buffer[y][x] = z; frame-Buffer[y][x] = Culoare fragment (întoarsa de fragment shader);}

In fragment shader

poz_sursa este variabilă de intrare pt fragm shader (pozitie fragment transformata cu MVPS)

Culoare_fragment = culoare calculata folosind modelul de iluminare locală;

dacă ZS[poz_sursa.y][poz_sursa.x] < poz_sursa.z atunci //fragmentul f(x,y,z) este în umbră

Culoare_fragment = factor_umbra* Culoare_fragment (0< factor_umbra<1)

Redarea suprafetelor 3D folosind texturi

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2018-2019

Texturi (1)

- Se folosesc pentru a reda detaliile suprafeţelor.
- Sunt doua tipuri de texturi:

1. Texturi reprezentate prin imagini care se aplica pe o suprafaţă netedă, suprafaţa ramanand neteda si dupa aplicarea texturii

➢ Imaginea se aplica (« mapeaza ») pe suprafaţă in timpul generarii imaginii suprafetei



➢ Imaginea mediului inconjurator aplicata pe suprafaţa unui obiect prin simularea reflexiei şi a refracţiei mediului inconjurator de catre suprafaţa obiectului.



Texturi (2)

2. Texturi prin care se redau detalii ale geometriei suprafetei.

Se folosesc diferite metode:

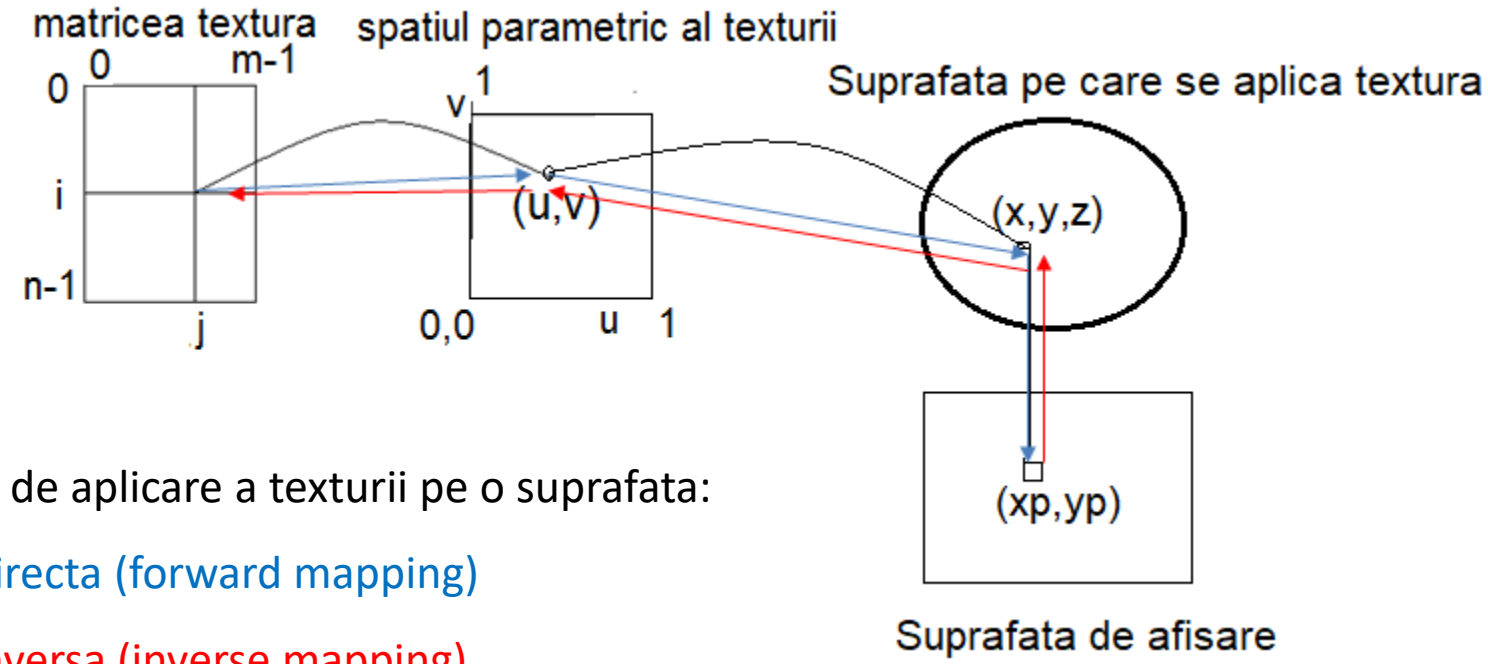
- perturbarea normalelor in punctele suprafetei
- metode fractale
- metode spectrale (functia textura este definita pentru componentele ei pe diferite frecvente)
- si altele



Redarea detaliilor geometrice prin perturbarea normalei la suprafata.

Maparea unei imagini textura pe o suprafata 3D

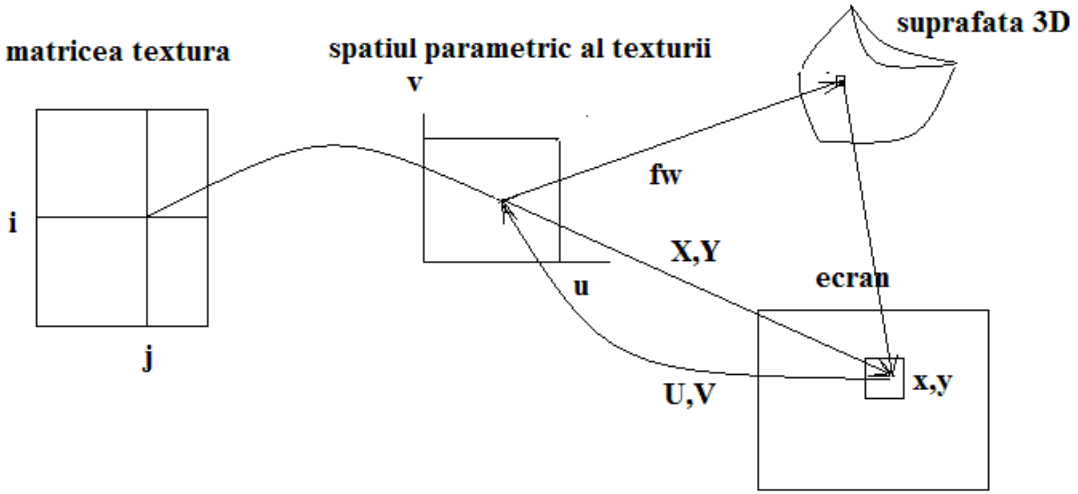
- Imaginea textura este memorata intr-o matrice ale carei elemente sunt culori si sunt adresate prin intermediul unui spatiu parametric (u,v), 0 <= u,v =< 1, care este pus in corespondenta suprafetei 3D pe care se aplica textura.
- Elementele matricei textura sunt numite « texeli ».



Sunt 2 metode de aplicare a texturii pe o suprafata:

1) maparea directa (forward mapping)
2) maparea inversa (inverse mapping)

Maparea unei imagini textura pe o suprafata 3D



1) maparea directa (forward mapping): se calculeaza pentru fiecare texel pixelul in care se afiseaza.

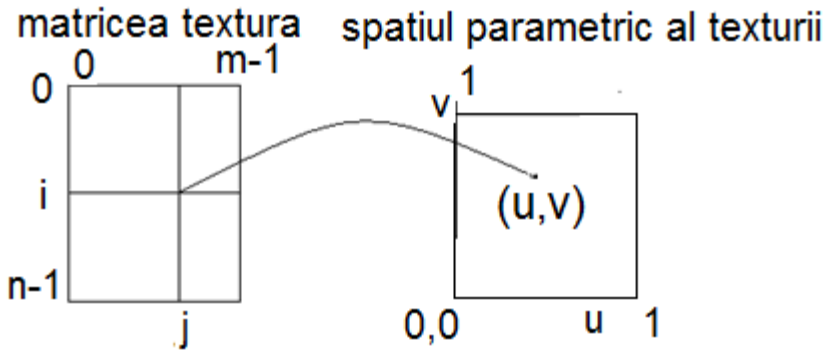
- adresa texel: (i,j) → adresa (u,v) → punct de pe suprafaţa 3D (x,y,z) → adresa pixel, (xp, yp)
- afisare pixel (xp, yp) in culoarea texelului textura[i][j]
- metoda este rar utilizata

2) maparea inversa (inverse mapping): se calculeaza pentru fiecare fragment (pixel) texelul/texelii din care se obtine culoarea textura.

- adresa pixel: (xp,yp) → (x,y,z) → adresa (u,v) → adresa texel, (ir,jr)
- afiseaza fragment folosind culoarea obtinuta prin filtrarea texturii.

Spatiul parametric al texturii

Corespondența dintre matricea textura si spatiul parametric al texturii



Maparea directa

Adresa unui texel in spatiul (u,v)

(i, j) ⟹ u = j / (m-1)

v = (1 -(i / (n-1))

Maparea inversa

Accesarea matricei textura din spatiul (u,v)

(u,v) ⟹ jr = u * (m-1)

ir = (1-v) * (n-1

(ir, jr) - numere reale

- accesarea matricei textura - prin numere intregi

⇩ Filtrarea texturii

2 posibilitati:

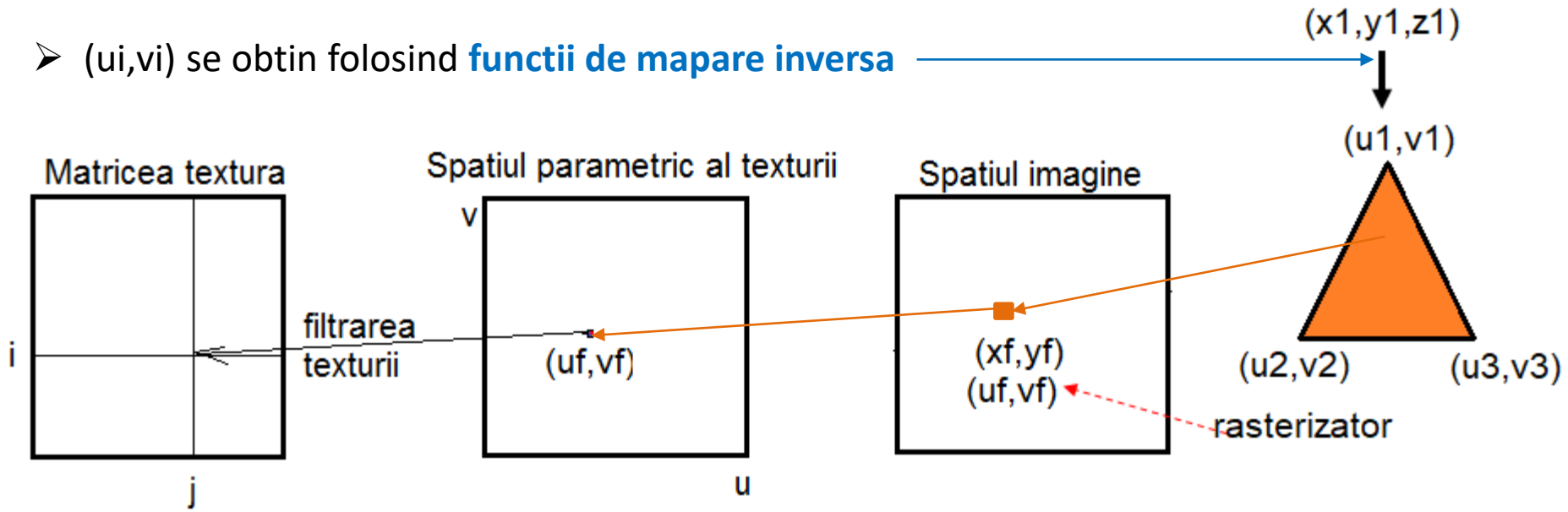
1) culoare = textura[i][j] j= round(jr), i = round(ir)

2) culoare = interpolare_biliniara (textura, ir, jr)

Maparea inversa (1)

Este metoda uzuala de mapare a texturilor pe suprafetele 3D

- Pentru fiecare fragment, (xf,yf), rezultat din rasterizarea unei primitive care are atasata o textura, rasterizatorul calculeaza adresa (uf,vf), pe care o trimite la fragment shader.
- In fragment shader se obtine o culoare din textura folosindu-se (uf,vf)
- (uf,vf) se obtine prin interpolarea coordonatelor (ui,vi) ale varfurilor primitivei
- (ui,vi) se obtin folosind **functii de mapare inversa**



Maparea inversa (2)

- Corespondenta intre punctele suprafetei 3D pe care se aplica textura si punctele spatiului (u,v) este stabilita prin functii de forma:

  $u=f_u(x,y,z)$; $v=f_v(x,y,z)$

  $f_w^{-1} = (f_u(x,y,z), f_v(x,y,z))$ este numita **functie de mapare inversa**

- Corespondenta (x,y,z) ←→ (u,v) se stabileste in programul de aplicatie pentru varfurile primitivelor grafice in care este descompusa suprafata 3D.
- In timpul rasterizarii primitivelor, coordonatele (u,v) ale varfurilor sunt interpolate rezultand coordonatele (uf,vf) corespunzatoare fiecarui fragment
- Coordonatele (uf,vf) sunt intrari pentru fragment shader si se folosesc pentru accesarea texturii.

- Pot fi stabilite functii de mapare inversa numai pentru suprafeţe 3D care pot fi definite intr-un spatiu parametric (t1,t2).
- Corespondenta se stabileste intre spatiul (t1,t2) si (u,v).

Functii de mapare standard(1)

Functiile de mapare inversa uzuale se numesc **funcţii de mapare standard.**

1. Maparea plana (maparea pe un plan)

Este definita de urmatoarele ecuatii:

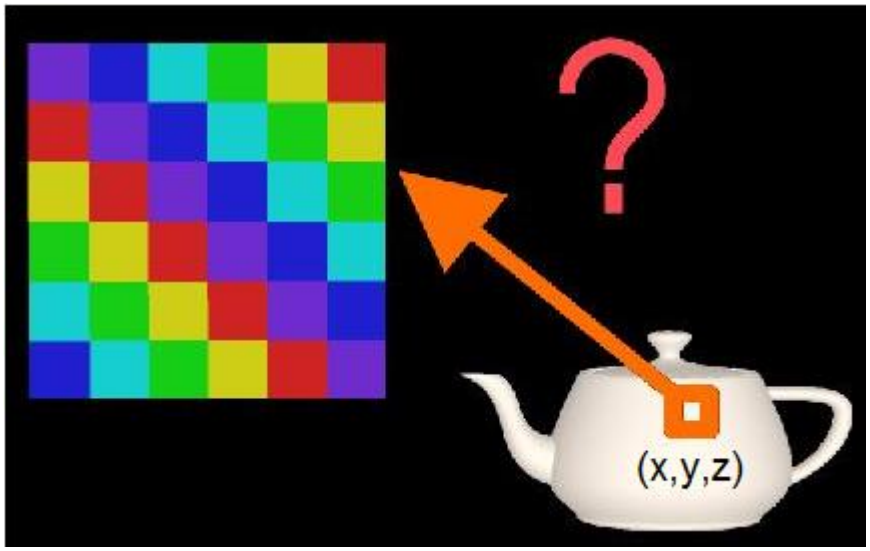
$u = su*x - ou \qquad x \in X,\ y \in Y$

$v = sv*y - ov \qquad 0 <= u, v <= 1$

unde : su, sv sunt factorii de scalare pentru transformarea spatiului (X, Y) in (u,v) iar

(ou,ov) este originea texturii: determina modul in care se aseaza textura pe suprafaţă.

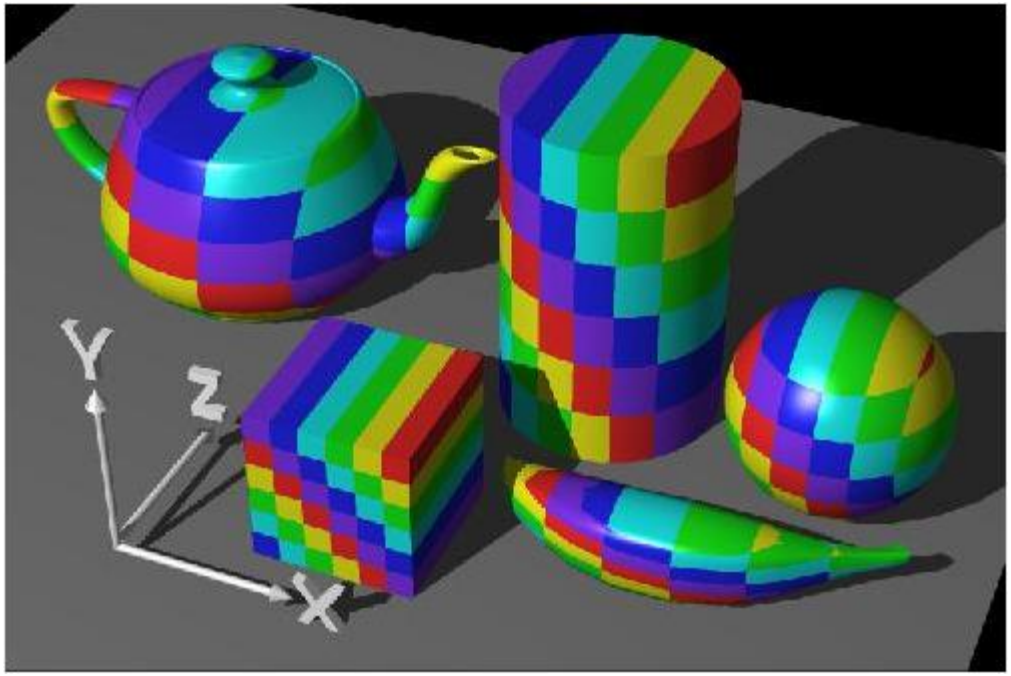
(u, v) ← (x, y) ← (x, y, z) - se renunta la coordonata z: suprafata 3D devine o suprafata plana





http://www.siggraph.org/education/materials/HyperGraph/mapping/r_wolfe/r_wolfe_mapping_2.htm

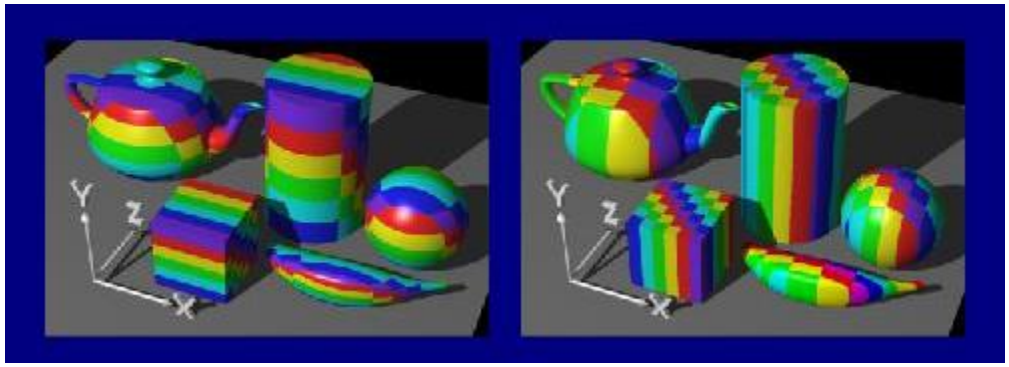
Functii de mapare standard(2)



Exemple de mapari plane

Maparea plana prin renuntarea la coordonata z: toate punctele suprafetei cu aceleasi coordonate (x,y) au aceeasi culoare.

http://www.siggraph.org/education/materials/HyperGraph/mapping/r_wolfe/r_wolfe_mapping_2.htm

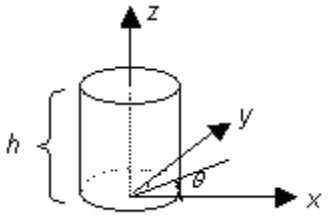


Maparea plana prin renuntarea la coordonata x, respectiv y.

Functii de mapare standard(3)

2. Maparea cilindrica

- Stabileste corespondenta dintre coordonatele (x,y,z) ale unui punct de pe o suprafata cilindrica si spatiul parametric al texturii, (u, v)



Ecuatiile parametrice ale unei suprafete cilindrice:

x=r*cos(θ)
y=r*sin(θ) 0< θ< 2Π, 0< w <1
z=w*h

- Se calculeaza coordonatele cilindrice ale punctului (x, y, z)

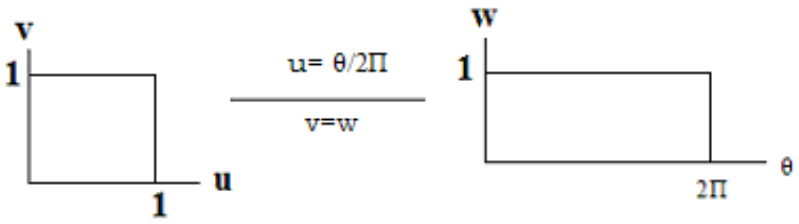
-Se pune in corespondenţă spatiul parametric al texturii, (u,v), spatiului parametric al suprafetei cilindrice:

x = r * cos(θ)
y = r* sin(θ)
z = h*w

$\Rightarrow$

$\theta = arctg(\frac{x}{y})$

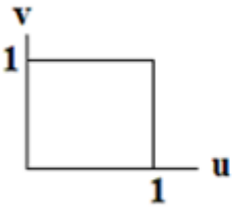
$w = \frac{z}{h}$



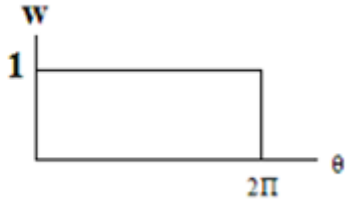
Functii de mapare standard (4)

$\theta = \text{arctg}(\frac{x}{y})$

$w = \frac{z}{h}$



u = θ/2Π ; v=w



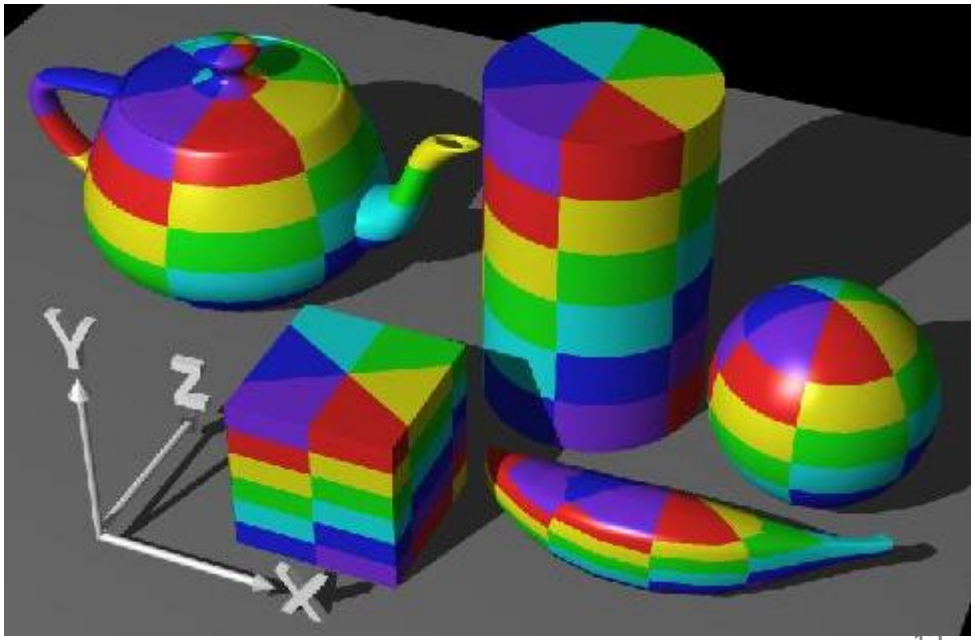
u = (1/2π)arctg(y/x)
v = z/h

Functia de mapare cilindrica

Textura este “înfaşurată” în jurul obiectului



Transformarea patratelor texturii la y= ymin si y= ymax



Functii de mapare standard (5)

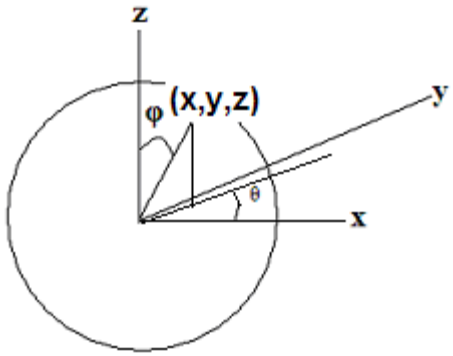
3. Maparea sferica

- Stabileste corespondenta dintre coordonatele (x,y,z) ale unui punct de pe o suprafata sferica si spatiul parametric al texturii, (u, v)

- Ecuatiile parametrice ale sferei

$$\begin{cases} x = r\cos\theta\sin\varphi \\ y = r\sin\theta\sin\varphi, cu \\ z = r\cos\varphi \end{cases} \begin{cases} 0 < \theta < 2\pi \\ 0 < \varphi < \pi \end{cases} \Rightarrow \begin{cases} \theta = arctg\left(\frac{y}{x}\right) \\ \varphi = \arccos\left(\frac{z}{r}\right) \end{cases}$$

Coord. sferice ale punctului (x,y,z)



Rezulta

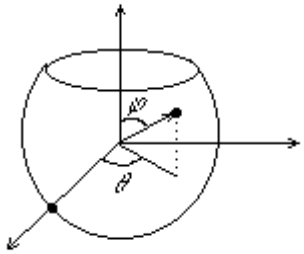
$u = \frac{\theta}{2\pi}$ sau $\begin{cases} u = \frac{1}{2\pi} arctg\left(\frac{y}{x}\right) \\ v = \frac{1}{\pi}\arccos\left(\frac{z}{r}\right) = \frac{1}{\pi}\arccos\frac{z}{\sqrt{x^2+y^2+z^2}} \end{cases}$

$v = \frac{\varphi}{\pi}$

Functia de mapare sferica

Functii de mapare standard (6)

Maparea pe un sector de sfera



$0 <= \theta <= \pi/2$

$\pi /4 <= \varphi <= \pi /2)$

Functia de mapare

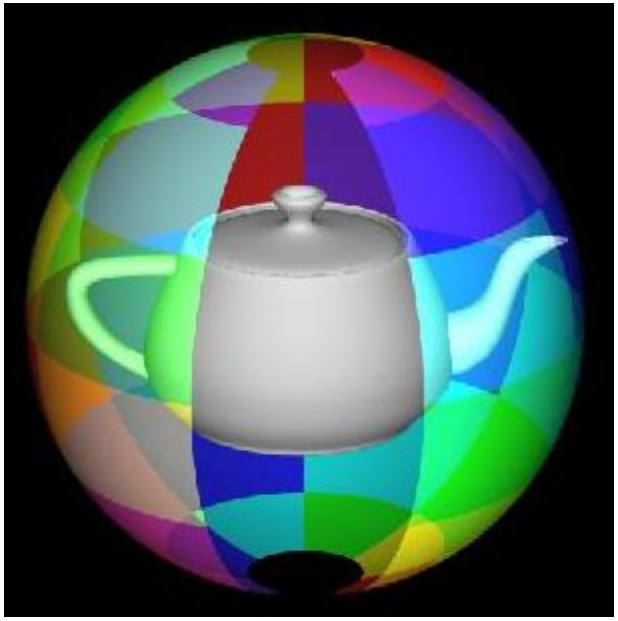
$u = \theta/(\pi/2)$

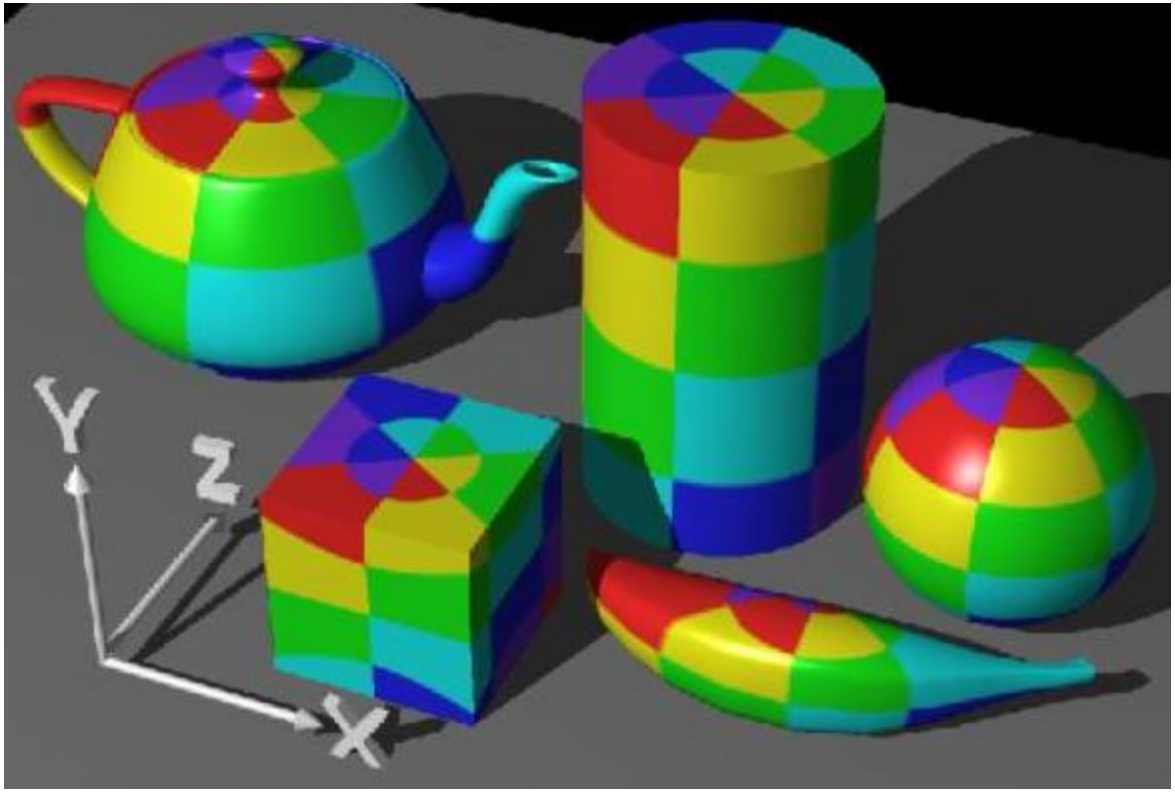
$v = ((\pi /2) – \varphi) / (\pi /4)$

Functii de mapare standard (7)

Exemple de mapari sferice

Maparea sferica “întinde” patratele texturii la nivelul ecuatorului si le “comprimă” proportional cu aproprierea de poli.





http://www.siggraph.org/education/materials/HyperGraph/mapping/r_wolfe/r_wolfe_mapping_2.htm

Maparea unei texturi pe o suprafata oarecare

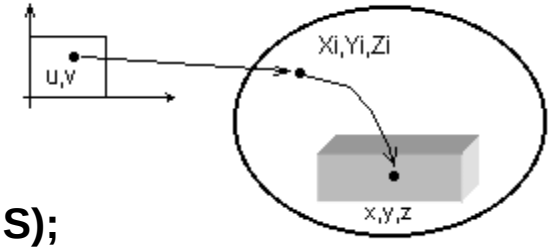
- Nu orice suprafata poate fi modelata analitic prin ecuatii parametrice a.i. sa se poata stabili corespondenta intre varfurile primitivelor in care a fost descompusa suprafata si spatiul parametric al texturii (u,v).
- Exemplu: o suprafata oarecare reprezentata printr-o retea de triunghiuri (mesh)
- Daca mesh-ul aproximeaza o sfera, atunci se pot calcula coordonatele (u,v) atasate varfurilor folosind functia de mapare sferica
- Cum atasam coordonatele (u,v) varfurilor mesh-ului, daca nu putem folosi una dintre functiile de mapare (mesh-ul reprezinta/aproximeaza o suprafata oarecare) ?

  → **folosim maparea in 2 pasi**

  → tehnică de mapare independenta de forma obiectului pe care se aplica textura. Metoda a fost definita de Bier si Sloan (1986).

Maparea în 2 pasi (1)

- Maparea texturii pe suprafata de texturat se realizeaza folosind o suprafata intermediara, pentru care exista o functie de mapare.
- Suprafata intermediara incadreaza suprafata de texturat.
- Aplicarea texturii este descompusa in doua operatii,



al caror efect este :

- **mularea texturii pe suprafata intermediara (mapare S);**
- **mularea suprafetei intermediare pe suprafata de texturat (mapare O).**

a) **maparea S este o mapare inversa:** fu si fv sunt functii de mapare inversa:
u = fu(xi, yi, zi)
v = fv(xi, yi, zi)
unde (xi,yi,zi) este un punct de pe suprafata intermediara

b) (x, y, z) → **mapare O** → (xi, yi, zi)

unde (x,y,z) este un punct de pe suprafata de texturat

Maparea în 2 pasi (2)

Maparea S

Se folosesc 4 tipuri de suprafete intermediare :

1. un plan cu o orientare oarecare;
2. sfera;
3. cilindrul;
4. fetele unui cub.

Alegerea tipului suprafetei intermediare este dependenta de forma obiectului pe care trebuie sa fie aplicata textura si de efectul dorit.

Maparea O

Se poate folosi pentru a calcula coordonatele (u,v) ale varfurilor unui mesh.

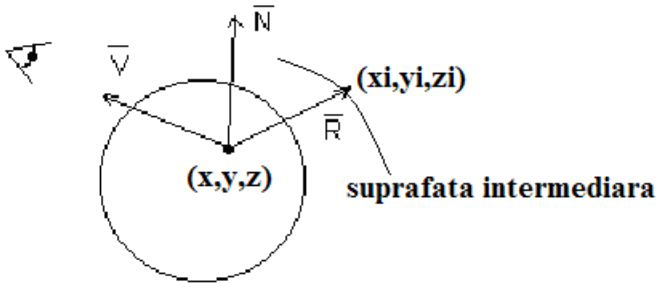
Autorii metodei au definit 4 tipuri de mapari O:

Maparea în 2 pasi (3)

Mapari O

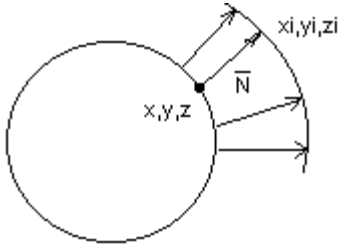
(x, y, z) – un punct al suprafetei, pentru care se doreste calculul coordonatelor (u,v)

(xi, yi, zi) – punctul corespunzator lui (x,y,z) pe suprafata intermediara



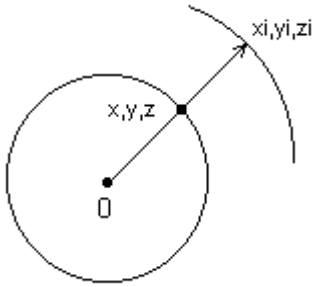
1. (xi, yi, zi), se obtine intersectand suprafata intermediara cu vectorul reflectat al vederii in punctul (x,y,z). R este simetric cu V fata de N.

Metoda este dependenta de pozitia observatorului (se foloseste la «maparea mediului inconjurator»)

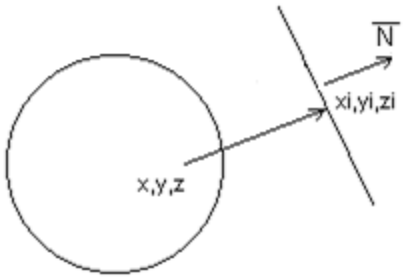


2. (xi,yi,zi) este intersectia suprafetei intermediare cu normala la obiect în (x,y,z).

Maparea in 2 pasi(4)



3. (xi,yi,zi) este intersectia suprafetei intermediare cu dreapta care trece prin (x,y,z) si centroidul obiectului.



4. xi,yi,zi) este intersectia suprafetei intermediare cu dreapta care trece prin (x,y,z) si are directia normalei la suprafata intermediara.

Acest tip de mapare O este corelat cu o mapare S pe un plan sau pe fetele unui cub.

Combinatia « mapare S pe cilindru » - « mapare O de tip 4 » este numita « shrinkwrap ».

Filtrarea texturii(1)

- O textura este definita pe un spatiu discret, fiecare texel fiind adresat prin 2 numere intregi, in timp ce coordonatele textura sunt valori reale. In general, o pereche (uf,vf), care trebuie folosita la colorarea unui fragment, nu corespunde unei adrese de texel in matricea textura.
- **Metoda de calcul a culorii dintr-o textura, folosind coordonatele (uf,vf), se numeste "filtrarea texturii".**

Cazul 0<= uf, yf <= 1

Fie (if, jf) adresa coresp. perechii (uf,vf) in spatiul texturii: if = (1-vf)*(n-1), jf = uf*(m-1)

Sunt doua moduri standard de filtrare:

1. Se considera culoarea texelului cu adresa cea mai apropiata de adresa (if,jf) in spatiul texturii:
   - rapid dar poate produce defecte in imagine
2. Se efectueaza o interpolare biliniara intre culorile celor mai apropiati texeli in spatiul texturii:
   - scade viteza de calcul a culorii fragmentului
   - poate produce un efect de incetosare

Filtrarea texturii(2)

Interpolarea biliniara intre culorile din matricea textura

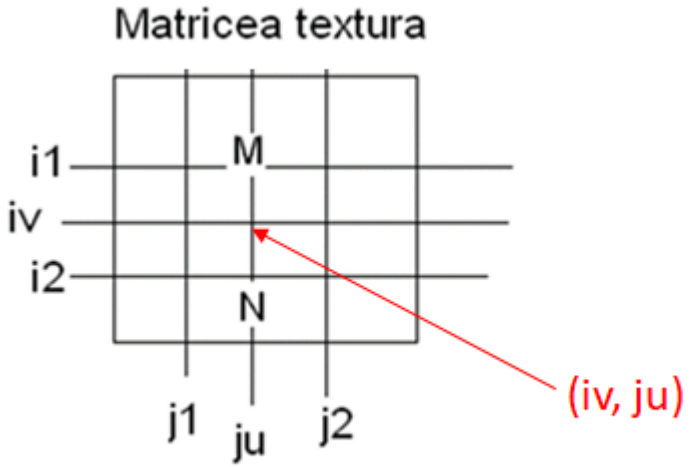
- Daca (ju, iv) sunt adresele reale corespunzatoare valorilor (uf,vf) in spatiul textura,

  iv = (1-vf)*(n-1); ju = uf*(m-1)

  n- numarul de linii ale matricei textura; m – numarul de coloane

  i1< iv < i2, i2 = i1 + 1 j1< ju < j2, j2 = j1 + 1

- Culoarea extrasa din matricea textura se calculeaza astfel:



i1, i2, j1, j2 sunt indici de linie/coloana in matricea textura

M(i1,ju); N(i2,ju);

C(i,j) – culoarea texelului de indici (i,j)

Cul(M) = (j2-ju)*C(i1,j1) + (ju-j1)*C(i1,j2)

Cul(N) = (j2-ju)*C(i2,j1) + (ju-j1)*C(i2,j2)

Culoare_textura = (i2-iv)*Cul(M) + (iv-i1)*Cul(N)

Filtrarea texturii (3)

Cazul uf, vf in afara domeniului [0,1]

- Coordonatele (ui, vi) asociate varfurilor unei primitive pot fi in afara domeniului [0,1].

Rezulta:

coordonatele (uf, vf) ale unui fragment pot fi in afara domeniului [0,1].

- Coordonatele (uf, vf) rezultate in procesul de rasterizare sunt transformate in coordonate

0<= u, v <=1

- Cele doua moduri standard de transformare a coordonatelor textura in domeniul [0,1] sunt denumite “clamping” si “repeating”: ele permit utilizarea eficienta a texturilor si obtinerea de efecte interesante.

Clamping (Comprimare)

Coordonatele (uf,vf) sunt transformate in (u, v) astfel:

(u, v) = (min(max(0,uf),1),min(max(0,vf),1)

Deci: daca uf sau vf <0, atunci u , respectiv v =0; daca uf sau vf >1, atunci u, respectiv v=1

daca 0<= uf, vf <=1, atunci u, v = uf, vf.

Filtrarea texturii (4)

Clamping (Comprimare)



Aplicarea acestei transformari are ca efect propagarea culorii din imaginea textura de-a lungul directiei in care coordonata textura depaseste limita de 0 sau de 1.

Filtrarea texturii (5)

Repeating

- Pentru accesarea texturii se calculeaza (u,v) cu formula:

  (u,v) = (uf- [uf]), vf-[vf]),

  unde [w] reprezinta cel mai mare intreg mai mic sau egal cu w.

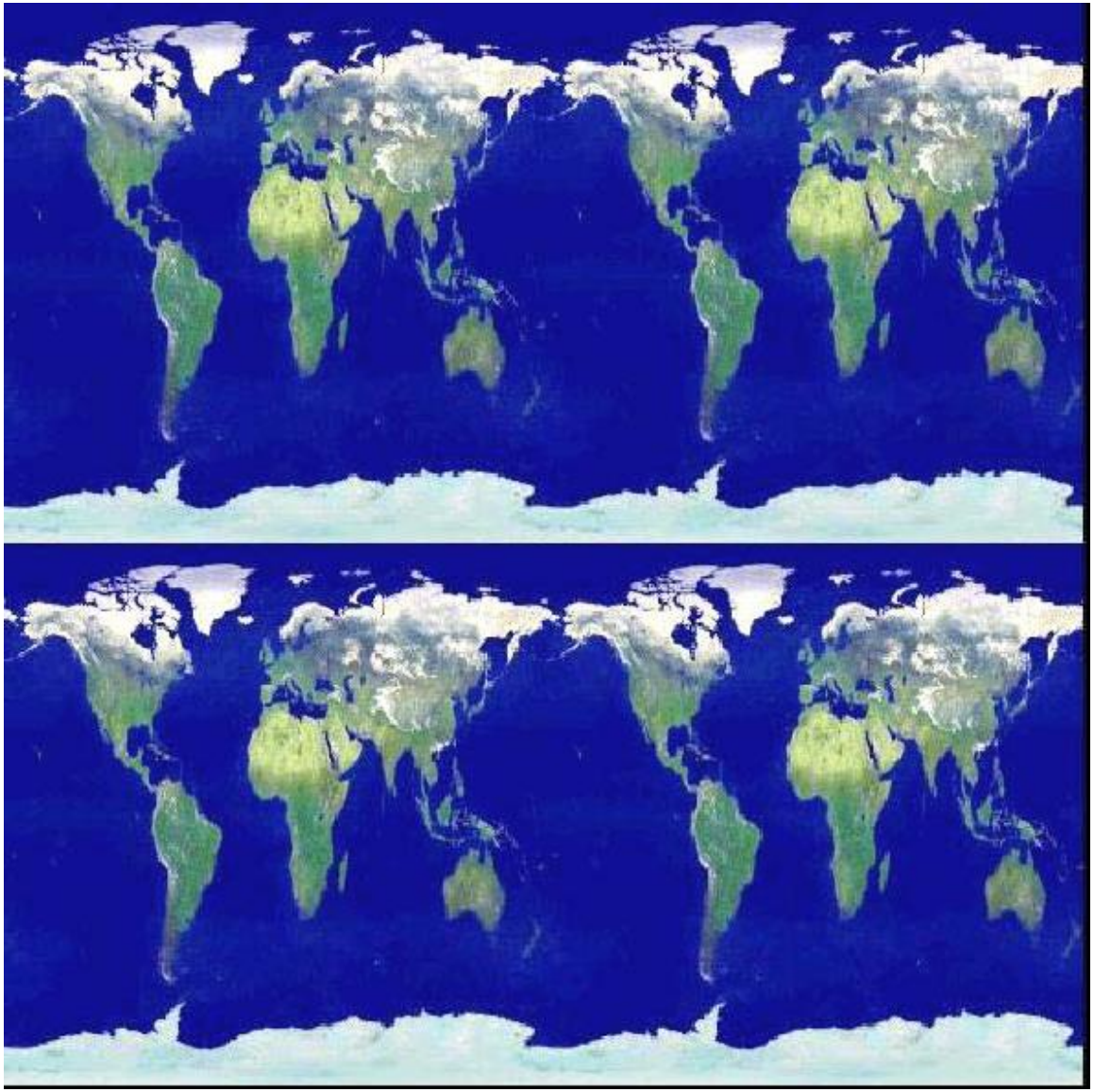
  De exemplu, (-0.1, 1.5) se transforma in (-0.1 – (-1), 1.5 – (1)) = (0.9, 0.5).

- **Acest mod de transformare permite repetarea unei texturi, producand un efect periodic.**
- La utilizarea acestui mod este necesar ca laturile stanga/dreapta, respectiv sus/jos ale imaginii textura se se potriveasca, astfel incat sa nu se observe marginile texturii.

- Daca pentru una dintre coordonate se utilizeaza transformarea “comprimare” iar pentru cealalta “repetare”, textura se numeste “cilindrica”.
- Daca pentru ambele coordonate se utilizeaza modul “repetare”, textura se numeste “toroidala”

Filtrarea texturii (6)

Repetare



Transparenta texturii

- Imaginea textura poate avea o componenta aditionala, numita “canalul alfa”, prin care se controleaza transparenta sau opacitatea texturii aplicate.
- Fiecare texel este in acest caz reprezentat prin 4 componente: (R,G,B,A), unde A reprezinta opacitatea.
  - daca A = 1, atunci texelul este complet opac si fragmentul va fi afisat in culoarea sa;
  - daca A=0, texelul este complet transparent; nu se foloseste in calculul culorii fragmentului.
- Pentru 0<= A <= 1, culoarea texelului moduleaza culoarea fragmentului, determinata pe baza geometriei (modelul Gouraud sau modelul Phong).
- Interfetele de programare OpenGL si Direct3D permit specificarea modului de combinare intre culoarea unui fragment determinata pe baza geometriei primitivei si culoarea rezultata din filtrarea texturii (texturilor), folosind operatori care se aplica culorilor (RGB) si operatori care se aplica opacitatilor A.
- Combinarea dintre culoarea determinata pe baza geometriei si culoarea texturii poate fi programata in “fragment shader”.

Iluminarea globala
-Ray Tracing-

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2018-2019

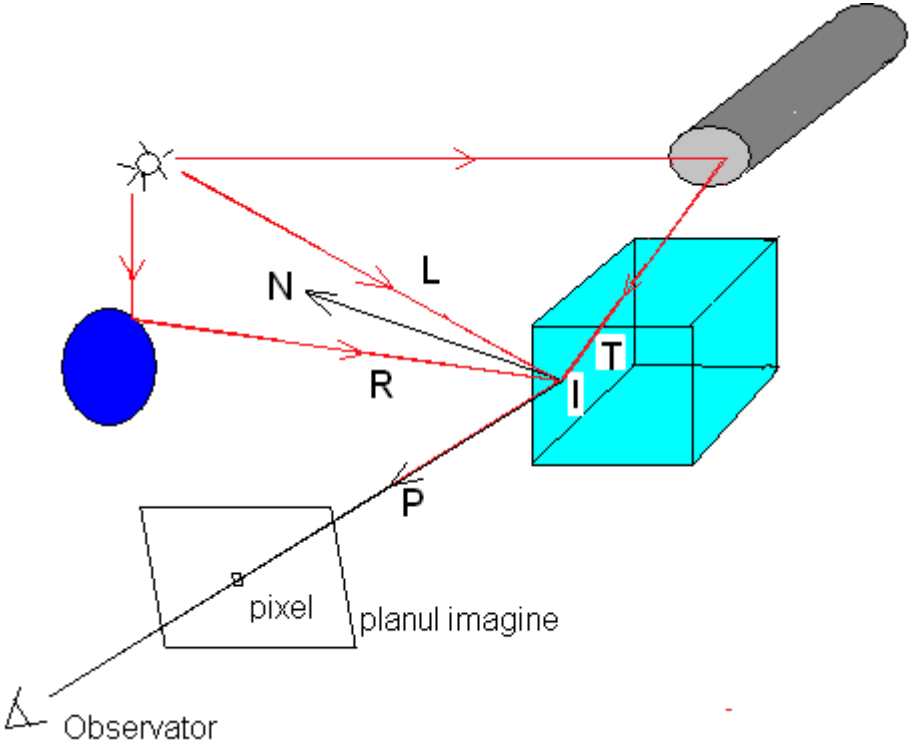
Iluminarea globala in Ray tracing(1)

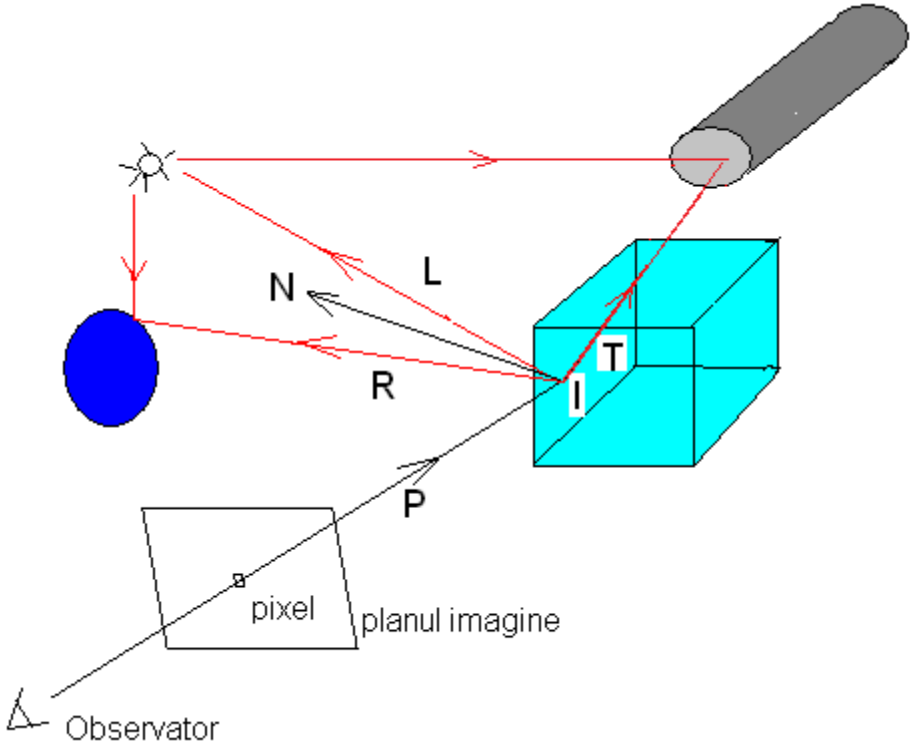
Imbina:

- Eliminarea partilor nevizibile ale scenelor 3D
- Calculul reflexiei luminii
- Calculul refractiei luminii (transparenta)
- Calculul umbrelor
- Interactiunea globala a reflexiei si refractiei luminii la nivelul intregii scene 3D

- In algoritmul Ray Tracing:
  - Imaginea unei scene 3D NU se obtine prin rasterizarea primitivelor grafice in care a fost descompusa scena
  - Imaginea este calculata intersectand scena 3D cu raze care pleaca din pozitia observatorului, fiecare trecand printr-un pixel din planul imaginii
- Culoarea unui pixel este calculata in punctul de intersectie al razei cu scena, care este cel mai apropiat de observator.

Iluminarea globala in Ray tracing(2)





Directia razelor de lumina in realitate.

Pixelul este colorat in culoarea punctului I.
P este directia pe care lumina din punctul I ajunge la observator.

In calculul culorii pixelului se considera raze cu directia inversa celor din realitate

Algoritmul Ray tracing

Imaginea se calculeaza pixel cu pixel, pornind de la spatiul imagine.

Considerand o singura sursa de lumina:

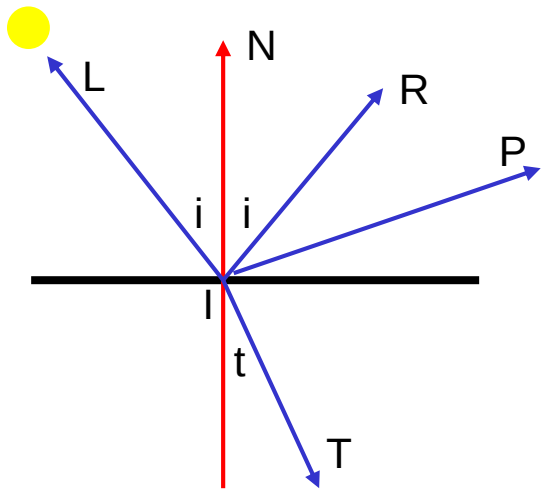
Pentru fiecare pixel al imaginii:

- Se calculeaza raza primara, P, care porneste din pozitia observatorului si trece prin centrul pixelului, in planul imaginii
- Se determina punctul de intersectie, I, al razei primare cu cel mai apropiat obiect de observator
- **Daca** raza nu intersecteaza nici un obiect al scenei,
  - Se afiseaza pixelul in culoarea fondului

  altfel
  - Se calculeaza culoarea obiectului in punctul I, tinand cont de:
    - Reflexia luminii provenita direct de la sursa de lumina, raza L
    - Lumina provenita in I de la alte obiecte pe directia reflexiei speculare, raza R
    - Lumina provenita in I de la alte obiecte pe directia razei transmise, raza T
  - Afiseaza pixelul in culoarea obtinuta prin combinarea contributiei celor trei raze

Ray tracing – calculul razelor

Calculul razelor care contribuie la culoarea luminii in punctul de intersectie cu raza primara



P: raza primara – din I catre observator

L: raza din I catre sursa de lumina

R: raza reflectata specular, simetrica, fata de N, cu L

$R = 2(N \cdot L) \cdot N - L$

T: raza transmisa, calculata pe baza legii lui Snell:

$n1/n2 = \sin(t)/\sin(i)$, n1,n2: indicii de refractie

$T = L*(n1/n2) – (\cos(t) + (n1/n2)*(L \cdot N))*N$

✓*Razele sunt considerate infinit subtiri*

✓*Reflexia speculara si refractia au loc fara imprastiere (sunt perfect focalizate)*

➢**Efect**: obiectele din imaginea produsa sunt de regula stralucitoare, producand reflexii multiple focalizate.

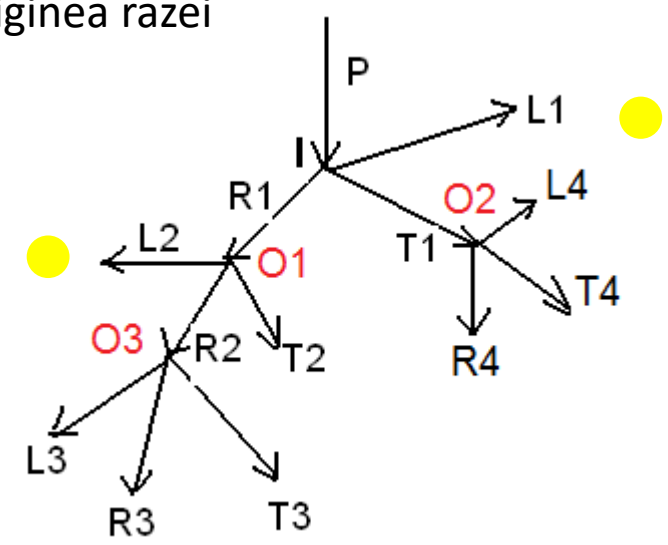
Ray tracing – arborele de raze

Algoritmul Ray tracing este recursiv:

Lumina provenita in punctul I prin reflexie speculara de la un obiect O1 (de-a lungul razei R1) sau prin transmisie de la un obiect O2 (de-a lungul razei T1), poate fi compusa din:

- Reflexia luminii provenita direct de la o sursa, de catre O1/O2 – raza L2/L4
- Reflexia speculara a altor obiecte de catre O1/O2 – raza R2/R4
- Transmisia luminii pe O1/O2 provenita de la alte obiecte ale scenei – raza T2/T4

➢ Pentru fiecare raza se considera punctul de intersectie cu scena, care sete cel mai apropiat de originea razei



Arborele de raze

➢*Pentru obtinerea culorii pixelului, se evalueaza arborele de raze de la frunze catre radacina*

Ray tracing- calculul reflexiei luminii(1)

Calculul culorii luminii în punctul I:

$$I_\lambda(I) = I_{local\lambda}(I) + Ks*R_\lambda(I) + Kt*T_\lambda(I)$$

unde:

λ – reprezinta lungimea de unda: expresia se evalueaza pentru R,G,B

$I_{local\lambda}(I)$ – reprezinta reflexia luminii provenite direct de la sursele de lumina din scena 3D (calculata folosind modelul de iluminare locala)

Ks – este coeficientul de reflexie speculara al materialului obiectului

$R_\lambda(I)$ – reprezinta reflexia speculara a luminii provenite de la alte obiecte ale scenei in punctul I: se obtine prin evaluarea arborelui de raze

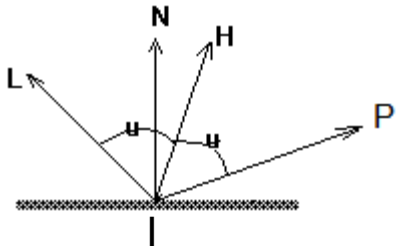
Kt – este coeficientul de transmisie, specific materialului obiectului

$T_\lambda(I)$ - reprezinta lumina provenita prin transmisie (refractie) in punctul I: se obtine prin evaluarea arborelui de raze

0<= ks, kt <= 1

Ray tracing- calculul reflexiei luminii(2)

Modelul de iluminare locala (reflexia lumii in punctul I) :



$I_{local\lambda}(I) = Ia_{\lambda} * Ka + Isursa_{\lambda} * fat * s * lum * [kd * (N \cdot L) + Ks * (N \cdot H)^{n}]$

Iaλ – reprezinta intensitatea luminii ambiante

Ka – este coeficientul de difuzie a luminii ambiante, specific materialului obiectului, 0<= ka <= 1

$Isursa_{\lambda}$ – reprezinta intensitatea luminii provenite de la sursa

L – este versorul directiei din punctul I catre pozitia sursei de lumina

N – este normala in punctul I (versor)

H – este versorul directiei bisectoare a unghiului dintre L si vectorul din I catre observator (raza P)

fat – este factorul de atenuare a luminii de la sursa, proportional cu distanta de la sursa la punctul I

0<= s <= 1,

s = 0, daca lumina de la sursa nu ajunge in punctul I: vectorul L intersecteaza un obiect opac al scenei

s = 1, daca vectorul L nu intersecteaza un alt obiect al scenei

0 < s <1, daca vectorul L intersecteaza un obiect transparent (sau mai multe)

Punctul I primeste lumina de la sursa daca:

- produsul scalar (N·L)>0 (lum =1, altfel =0)
- raza L nu intersecteaza un obiect opac al scenei

➢ Daca I nu primeste lumina de la sursa, iluminarea locala se reduce la componenta ambianta.

Ray tracing- calculul reflexiei luminii(3)

- Daca in scena 3D exista mai multe surse de lumina, fiecare poate contribui in mod diferit la $I_{local\lambda}(I)$:

$$I_{local\lambda}(I) = Ia_{\lambda} * Ka + \sum_{i=1,n} Isursa_{i,\lambda} * fat_i * s_i * lum_i * [kd * (N \cdot L_i) + Ks * (N \cdot H_i)^n]$$

- Pentru evaluarea componentelor $R_{\lambda}(I)$ si $T_{\lambda}(I)$ din calculul culorii in I,

$$I_{\lambda}(I) = I_{local\lambda}(I) + Ks * R_{\lambda}(I) + Kt * T_{\lambda}(I)$$

se coboara in arborele de raze pana la un numar pre-specificat de nivele (energia luminoasa scade destul de repede!).

Algoritmul Ray tracing

Algoritmul Ray tracing

Pentru fiecare pixel al imaginii

```
{    *calculeaza raza primara, P;//din pozitia observatorului prin centrul suprafetei pixelului
     *culoare_pixel = TraseuRaza(P, 1); // culoarea in punctul de intersectie al razei P cu scena
     *afiseaza pixelul in culoare_pixel;
}
```

Culoare TraseuRaza**(Raza R, int n)** //apelata recursiv

```
{    // n este nivelul in arborele de raze
     // intoarce culoarea in punctul de intersectie al razei R cu scena
     *calculeaza intersectiile razei R cu obiectele scenei;
     daca (nu exista intersectii), atunci
               return  (culoare_fond);
     altfel
               *fie I punctul de intersectie cel mai apropiat de observator si O obiectul intersectat;
               *calculeaza normala, N, in punctul I;
               return  CuloarePunct(O, R, I, N, n); // culoarea in I
}
```

Algoritmul Ray tracing

```
Culoare CuloarePunct(Obiect O, Raza R, Punct I, Normala N, adancime_arbore n)
{    Culoare  culoare;
     culoare = culoare_ambianta;
     pentru fiecare sursa de lumina S executa
          *calculeaza vectorul L, din I catre S
           daca ((Nu·Lu) > 0 si vectorul L nu intersecteaza un obiect opac al scenei) atunci
             *calculeaza contributia sursei S la culoare, CS, folosind modelul de iluminare locala
              culoare = culoare + CS
     daca (n<=nivel_max) atunci // n>nivel_max: iesirea din recursivitate
          daca (obiectul O produce reflexii speculare) atunci
               *calculeaza raza reflectata in punctul I, RS;
                culoare = culoare + Ks* TraseuRaza(RS, n+1); // se coboara in arbore
          daca (obiectul O este transparent) atunci
               *calculeaza raza transmisa (refractata) in punctul I, RT;
                culoare = culoare + Kt* TraseuRaza(RT, n+1); //se coboara in arborele de raze
     return (culoare)
}
```

Algoritmul Ray tracing

Deficienţele algoritmului

1. **Efecte de aliasing**: razele sunt convergente si infinit subtiri, esantionarea spatiului 3D este neuniforma
   - Se produc defecte la marginile suprafetelor (aliasing: marginile au aspect zimţat)
   - Obiectele mici si subtiri pot sa apara si sa dispara din imagine, la schimbarea pozitiei observatorului

   Imbunatatire:

   – marirea rezolutiei esantionarii spatiului 3D; ex. 4 raze primare/pixel → creste complexitatea

2. **Complexitatea computationala**
   - Buna din punct de vedere teoretic:

     - pentru o raza/pixel este de O(p*n), unde p este nr. de pixeli - constant (independent de scena) = O(n), unde n este nr. de obiecte din scena
   - Creste liniar cu numarul de raze/pixel
   - Complexitatea calculelor de intersectie depinde de geometria obiectelor intersectate

   Imbunatatire:

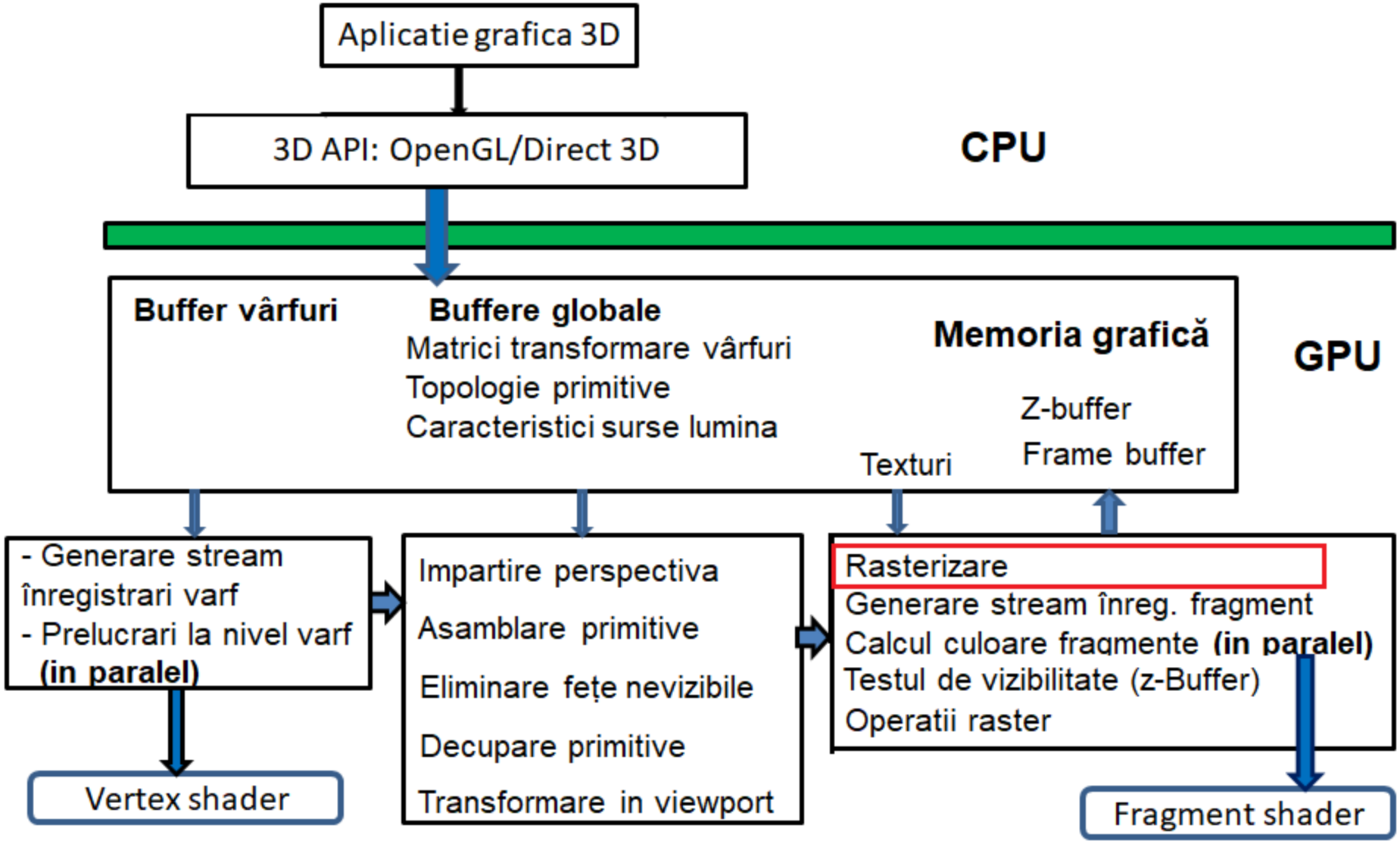
   – Reducerea calculelor de intersectie, prin folosirea de volume incadratoare ale obiectelor si gruparea obiectelor din scena 3D

   – Complexitatea se poate reduce la O(log (n))

Rasterizarea primitivelor grafice

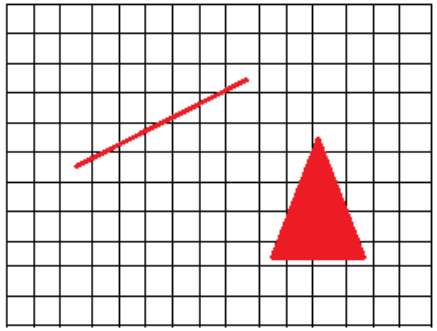
Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2018-2019


Aplicatie grafica 3D
3D API: OpenGL/Direct 3D
CPU
Buffer vârfuri
Buffere globale
Matrici transformare vârfuri
Topologie primitive
Caracteristici surse lumina
Memoria grafică
Z-buffer
Frame buffer
Texturi
GPU
- Generare stream înregistrari varf
- Prelucrari la nivel varf (in paralel)
Vertex shader
Impartire perspectiva
Asamblare primitive
Eliminare fețe nevizibile
Decupare primitive
Transformare in viewport
Rasterizare
Generare stream înreg. fragment
Calcul culoare fragmente (in paralel)
Testul de vizibilitate (z-Buffer)
Operatii raster
Fragment shader

Rasterizarea (1)

- Singurele primitive grafice acceptate de placile grafice care suporta versiunile noi ale bibliotecii OpenGL(3.x→) sunt:
  - **punctul**, definit prin coordonatele sale (x,y,z);
  - **linia**, definita prin coordonatele extremitatilor sale, (x1,y1,z1) si (x2,y2,z2); specificate in aceasta ordine, **linia este un vector cu originea in (x1,y1,z1) si varful in (x2,y2,z2);**
  - **suprafaţa triunghiulară**, definita prin coordonatele 3D ale varfurilor sale.
- Suprafata de afisare este un spatiu discret, alcatuit din celule de afisare (pixeli) adresate prin coordonate intregi, (0,0) <= (x,y) <= (xmax,ymax)
- Pentru afisare, este necesara descompunerea primitivelor in fragmente care se afiseaza in celulelele suprafetei de afisare.



- **Rasterizarea:**
  - operatia prin care o primitiva grafica este descompusa in fragmente care se afiseaza in pixelii suprafetei de afisare;
  - are ca rezultat o aproximare in spatiul discret a primitivelor definite analitic.

Rasterizarea (2)

Algoritmii de rasterizare integreaza, la nivel de fragment:

1. Calculul coordonatelor (x,y) ale pixelului in care se afiseaza fragmentul
2. Calculul coordonatei z a fragmentului
3. Calculul culorii fragmentului (in fragment shader)
4. Testul de vizibilitate (adancime) si actualizarea culorii pixelului (x,y)

```
daca z< Z-buffer[y][x] atunci
{ Z-buffer[y][x] = z
*actualizeaza culoarea pixelului (x,y) in buffer-ul imagine folosind culoarea fragmentului
}
```

Deoarece in algoritmii de rasterizare este important modul de calcul al coordonatelor (x,y), vom include pasii 2,3 si 4 in functia

```
putpixel (int x, int y);
```

- Pasul 4 este acelasi pentru orice fragment, indiferent de tipul primitivei din care provine.

Rasterizarea vectorilor (1)

La rasterizarea unui vector:

1. Pixelii (x,y) in care se afiseaza fragmentele rezultate din rasterizare sunt determinati astfel incat sa fie cei mai apropiati de adresele punctelor de pe vectorul analitic.

2. Coordonata z a fiecarui fragment se obtine prin interpolarea liniara a coordonatelor z ale varfurilor vectorului (prin calcul incremental – vezi “algoritmul z-buffer”).

3. Culoarea fragmentului se obtine prin interpolarea liniara a culorilor varfurilor vectorului (prin calcul incremental); nu este uzual sa se calculeze culoarea unui fragment de vector in “fragment shader” (fragment shader intoarce ca rezultat culoarea primita de la rasterizator)

- Algoritmii de rasterizare a vectorilor se deosebesc prin pasul 1.

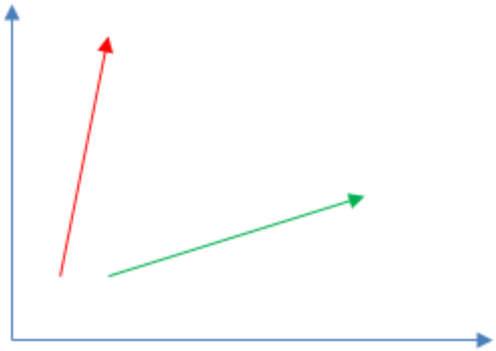
Rasterizarea vectorilor (2)

Ecuatia unui vector: y = m*x + b

m = (y2-y1)/(x2-x1) şi b=y1 - m*x1

Consideram urmatorul algoritm de rasterizare a unui vector:

```
putpixel(x1,y1);
for(x=x1+1; x<x2; x++)
    {y=m*x+b ;
     putpixel(x, (int)(y+0.5));
    }
putpixel(x2,y2);
```



Dezavantaje algoritm:

- Nu se tine cont de panta dreptei: vectorii cu panta mare sunt aproximati prin câțiva pixeli!
- Calculul fiecarei adrese de pixel de pe traseul vectorului contine operatii cu numere reale

Algoritmul Digital Differential Analyser (DDA)

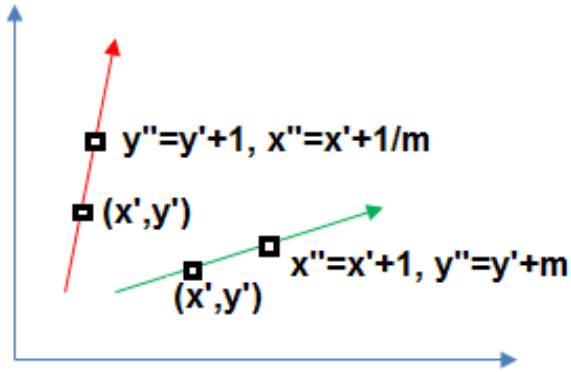
- Ține cont de panta vectorului
- Coordonatele pixelilor de pe traseul vectorului se obtin printr-un calcul incremental (eficient)

Fie (x', y') si (x'',y'') - 2 puncte succesive de pe vector

- (y'' – y') / (x'' – x') = (y2 – y1) / (x2 – x1) = m

  |m <1| : se incrementeaza x : x'' = x' +1 → y'' = y' + m

  | m >1| : se incrementeaza y: y'' = y' + 1 → x'' = x' +1/m




```
void DDA(int x1, int y1, int x2, int y2)
{double m, absm, rx, ry; int x, y;
//vectorul se genereaza de la (x1,y1) la (x2,y2)
 if(x1==x2) //vector vertical
  { if(y1>y2) {y=y1; y1=y2; y2=y;}
    for(y=y1; y<=y2; y++)  putpixel(x1,y);
   return;
 }
```

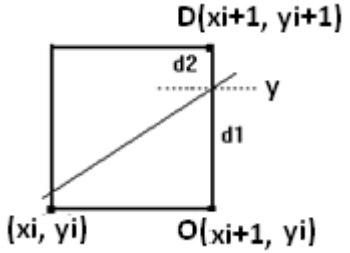
Algoritmul DDA(2)

```
if(y1==y2) //vector orizontal
  {if(x1>x2)
     {x=x1; x1=x2; x2=x;}
   for(x=x1; x<= x2; x++)  putpixel(x,y1);
   return;
  }
 m=(double)(y2-y1)/(x2-x1); absm=abs(m);
 if(absm<=1 && x1>x2  ||  absm>1 &&  y1>y2)
     {x=x1; x1=x2; x2=x; y=y1; y1=y2; y2=y;}
 putpixel(x1, y1);
 if( absm<=1)
     for(x=x1+1, ry=y1; x<x2; x++)
       {ry+=m;  putpixel(x, (int)(ry+0.5)); } // y = y + m
else
     {m=1/m;
      for(y=y1+1, rx=x1; y<y2; y++)
       {rx+=m;  putpixel((int)(rx+0.5), y); } // x = x +1/m
     }
 putpixel(x2, y2);
}
```

Dezavantaj: calcule cu numere reale

Algoritmul Bresenham (1)

- Contine numai operatii cu numere intregi. **Este definit pentru vectori din primul octant**
- Un vector face parte din"primul octant" daca panta sa, m, satisface conditia 0<m<1.
- Coordonatele pixelilor de pe traseul vectorului se obtin prin calcul incremental.
- Pentru fiecare valoare a lui x, de la xmin_vector la xmax_vector, se alege acel punct al spaţiului discret care este mai apropiat de punctul de pe vectorul teoretic.



- Fie m=(y2-y1)/(x2-x1) panta vectorului,
- (x_i, y_i) ultimul punct al spaţiului discret ales la executia algoritmului
- d1 distanţa de la punctul de pe vectorul teoretic, (x_i+1, y), la punctul $O(x_i+1, y_i)$
- d2 distanţa de la punctul de pe vectorul teoretic la punctul $D(x_i+1, y_i+1)$.
- O si D sunt adrese de pixeli (puncte ale spatiului discret)

➢ Următorul punct al spatiului discret ales pentru aproximarea vectorului va fi:

- O dacă d1<d2, D în caz contrar.
- Dacă d1=d2 se poate alege oricare dintre cele două puncte.

Algoritmul Bresenham(2)

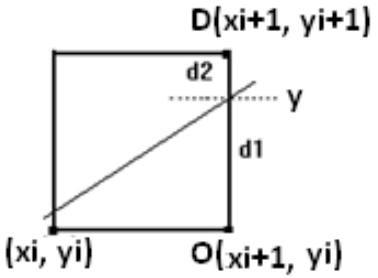
- Exprimăm difereţa d1-d2:

  $y=m*(x_i+1)+b$ este ordonata punctului de pe vectorul teoretic

  $d1= y - y_i= m*(x_i+1)+b-y_i$

  $d2= y_i + 1-y= y_i+1-m*(x_i+1)-b$

  $d1- d2= 2*m*(x_i+1)-2*y_i+2*b -1$



- Se înlocuieşte m cu dy/dx apoi se înmulţeşte în ambele părţi cu dx. Rezultă:

  $t_i= (d1-d2)*dx= 2*dy*(x_i+1)-2*dx*y_i+2*b*dx-dx = 2*dy*x_i - 2*dx*y_i + 2*b*dx - dx + 2*dy$

- t_i reprezintă eroarea de aproximare în pasul i: pe baza ei se alege urmatorul punct al spatiului discret

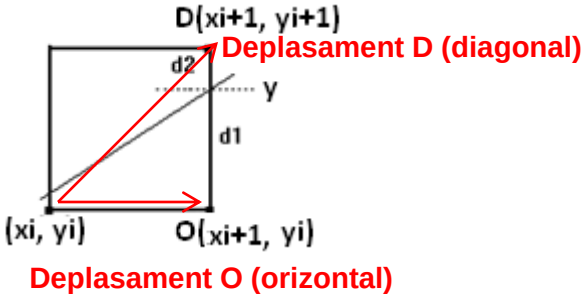
  Notăm cu (x_{i+1},y_{i+1}) punctul care se va alege în pasul i.

- Expresia erorii de aproximare în pasul i+1 este:

  $t_{i+1} = 2*dy*x_{i+1} - 2*dx*y_{i+1} + 2*b*dx - dx +2*dy$

Algoritmul Bresenham(3)

$t_i=(d1-d2)*dx = 2*dy*x_i - 2*dx*y_i + 2*b*dx-dx + 2*dy$, (dx>0)



(1) Dacă $t_i <= 0$, se alege punctul O, deci $x_{i+1} = x_i+1$ şi $y_{i+1} = y_i$

Rezultă:

$t_{i+1} = 2*dy*(x_i+1) - 2*dx*y_i + 2*b*dx - dx +2*dy$

sau **$t_{i+1} = t_i +2*dy$**

(2) Dacă $t_i > 0$, se alege punctul D, deci $x_{i+1} = x_i + 1$ şi $y_{i+1} = y_i +1$

Rezultă:

$t_{i+1} = 2*dy*(x_i+1) - 2*dx*(y_i+1) + 2*b*dx - dx +2*dy$

sau **$t_{i+1} = t_i + 2*dy - 2*dx$**

- Valoarea variabilei de test se obtine prin calcul incremental: adunarea unei constante intregi

Eroarea de aproximare pentru primul pas: $x_i = x1$ şi $y_i = y1$:

$t_1 = 2*dy*x1 - 2*dx*y1 + 2*dy – dx + 2*dx(y1 - (dy/dx)*x1)$

sau **$t_1 = 2*dy-dx$**

Algoritmul Bresenham(4)

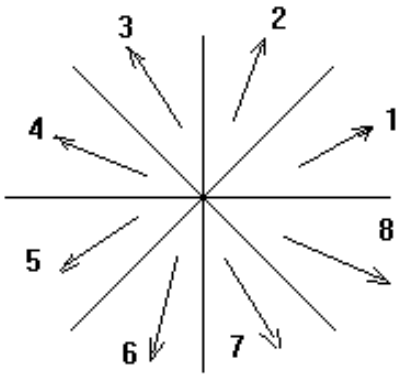
Implementare in C:

```
void Bres_vect(int x1, int y1, int x2, int y2)
{ //pentru vectori cu panta cuprinsă între 0 şi 1
  int dx, c1, c2, x, y, t;
  dx=x2-x1;
  c1=(y2-y1)<<1;  // 2*dy
  c2=c1-(dx<<1);  // 2*dy – 2*dx
  t=c1-dx;  // 2*dy - dx
  putpixel(x1, y1);
  for(x=x1+1, y=y1; x<x2; x++)
   { if(t<0) t+= c1; //deplasament  O
      else  { t+= c2; y++;} // deplasament D
    putpixel(x,y);
   }
  putpixel(x2,y2);
}
```

Algoritmul Bresenham- generalizare(1)

Generalizarea algoritmului Bresenham pentru vectori de orice panta

- Vectorii definiţi în planul XOY pot fi clasificaţi, pe baza pantei, în opt clase geometrice, numite **“octanţi“**
- Un vector care apartine unui octant O are 7 vectori simetrici in ceilalti 7 octanti



dx=x2 – x1 şi dy=y2 – y1

octantul 1: dx>0 şi dy>0 şi dx>=dy;

octantul 2: dx>0 şi dy>0 şi dx<dy;

octantul 3: dx<0 şi dy>0 şi abs(dx)<dy;

octantul 4: dx<0 şi dy>0 şi abs(dx)>=dy;

octantul 5: dx<0 şi dy<0 şi abs(dx)>=abs(dy);

octantul 6: dx<0 şi dy<0 şi abs(dx)<abs(dy);

octantul 7: dx>0 şi dy<0 şi dx<abs(dy);

octantul 8: dx>0 şi dy<0 şi dx>=abs(dy);

Algoritmul Bresenham –generalizare(2)

Notam cu:

+h, deplasamentul orizontal spre dreapta (în sensul crescător al axei x),

-h, deplasamentul orizontal spre stânga (în sensul descrescător al axei x),

+v, deplasamentul vertical în sus (în sensul crescător al axei y),

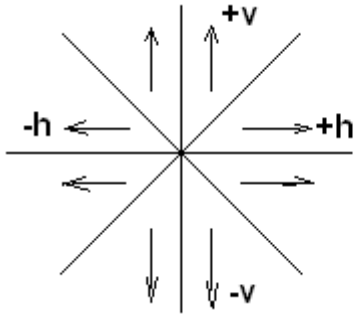
-v, deplasamentul vertical în jos (în sensul descrescător al axei y),

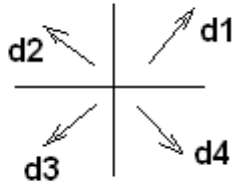
d1, deplasamentul diagonal dreapta-sus,

d2, deplasamentul diagonal stânga-sus,

d3, deplasamentul diagonal stânga-jos,

d4, deplasamentul diagonal dreapta-jos.





Octant	1	2	3	4	5	6	7	8
Deplas O	+h	+v	+v	-h	-h	-v	-v	+h
Deplas D	d1	d1	d2	d2	d3	d3	d4	d4

Corespondenţa între alegerea curentă într-un pas al algoritmului Bresenham (punctul O sau punctul D) şi deplasamentul echivalent în fiecare octant:

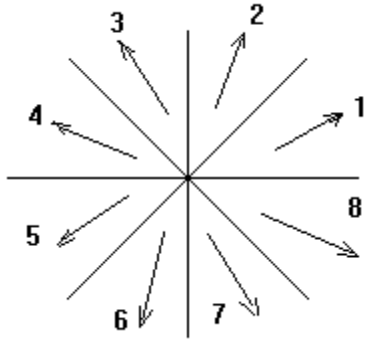
Algoritmul Bresenham – generalizare(3)

Algoritmul Bresemham generalizat - implementare in C

```
void Bres_general(int x1, int y1, int x2, int y2)
{ int x, y, i, oct, dx, dy, absdx, absdy, c1, c2, t;
  if(x1==x2) //vertical
  { if(y1>y2){y=y1; y1=y2; y2=y;}
     for(y=y1; y<=y2;y++)
       putpixel(x1,y);
     return;
  }
 if(y1==y2) //orizontal
   {if(x1>x2) {x=x1; x1=x2; x2=x;}
     for(x=x1; x<= x2; x++)
       putpixel(x,y1);
    return;
   }
```

Algoritmul Bresenham – generalizare (4)

```
dx= x2-x1; dy= y2-y1; absdx= abs(dx); absdy= abs(dy);
if(dx>0)//oct=1,2,7,8
   { if(dy>0)// oct=1,2
       if(dx>=dy) oct=1; else oct=2;
     else
       if(dx>=absdy) oct=8; else oct=7;
   }
else//3,4,5,6
   { if(dy>0)// oct=3,4
       if(absdx>=dy) oct=4; else oct=3;
     else
       if(absdx>=absdy) oct=5; else oct=6;
   }
// Numărul de paşi la execuţia algoritmului generalizat este maxim(abs(dx), abs(dy))
// Dacă abs(dy) > abs(dx), se inversează rolul variabilelor dx şi dy în calculul constantelor c1 şi c2
if(absdy>absdx) //  adresele de pe traseul vectorului se obtin prin incrementarea lui y
    {x=absdx; absdx=absdy;absdy=x;}
 c1= absdy<<1;  c2= c1-(absdx<<1); t= c1-absdx;
```



```
 putpixel(x1,y1);
 for(i=1, x=x1, y=y1; i<absdx; i++)
 { if(t<0) // deplasament O
    {t+=c1;
     switch(oct)
     {case 1:  case 8: x++; break; // +h
      case 4:  case 5: x--;break; // -h
      case 2:  case 3: y++;break; //+v
      case 6:  case 7: y--;break; // -v
     }
    }
   else
    {t+=c2 // deplasament D
     switch(oct)
     {case 1: case 2: x++; y++; break; // d1
      case 3: case 4: x--; y++; break; // d2
      case 5: case 6: x--; y--; break; // d3
      case 7: case 8: x++; y--; break; // d4
     }
   }
   putpixel(x,y);
 } putpixel(x2,y2);
}
```

Algoritmul Bresenham – generalizare (5)

Generarea liniilor întrerupte

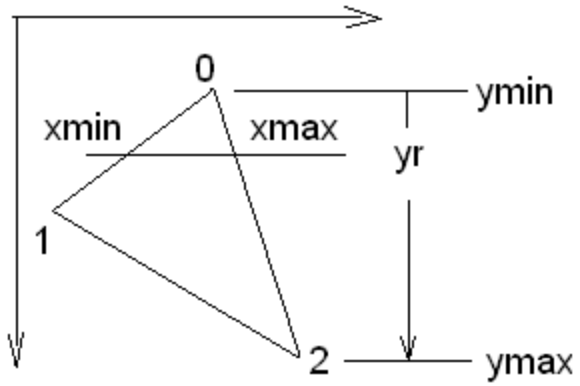
```
void Bres_gen(int x1, int y1, int x2, int y2, int şablon)
{  // sablon: intreg pe 16 biti cu valori 0 si 1
    int val, bit, biţi[16] ;
    .............................
    for(int i=0,val=1; i<16; i++)
    { if (şablon & val) biţi[i] = 1; else biţi[i] = 0;   // se extrag biţii sablonului in vectorul biți
      val = val << 1;
    }
    ....................................
   if(biţi[0]]) putpixel(x1,y1);
   for(i=1, x=x1, y=y1, bit=1; i<absdx; i++)
  { .....................................
     if(biţi[(bit++) % 16]) putpixel(x,y);
  }
  if(biţi[bit % 16) putpixel(x2,y2);
}
```

Rasterizarea suprafetelor triunghiulare

La rasterizarea unei suprafete triunghiulare

1. Pixelii (x,y) in care se afiseaza fragmentele rezultate din rasterizare sunt determinati intersectand suprafata triunghiului cu liniile raster de la ymin_triunghi la ymax_triunghi;
2. Fiecare segment dintr-o linie raster, care este interior suprafetei triunghiulare, este descompus in fragmente.
3. Coordonatele z ale fragmentelor se obtin prin calcul incremental (vezi curs “algoritmul z-buffer”).
4. Pentru calculul culorii fiecarui fragment este apelat programul “fragment shader”.
5. Sunt afisate fragmentele care trec testul de vizibilitate.

Rasterizarea suprafetelor triunghiulare - algoritmul -



Bool SupTri (int x0, int y0, int x1, int y1, int x2, int y2)

{ 1. Se sorteaza varfurile triunghiului crescator dupa coordonata y, a.i. y0 = ymin;

2. Daca triunghiul este degenerat (varfurile sunt coliniare), return false;

3. Se orienteaza conturul in sens trigonometric (0-1-2: sens trigonometric);

4. Pentru toate liniile raster de la ymin-triunghi la ymax-triunghi

se calculeaza extremitatile segmentelor interioare suprafetei triunghiului, si se memoreaza in 2 tablouri, XMIN, XMAX;

5. Se afiseaza pixelii segmentelor interioare (de la ymin_triunghi la ymax_triunghi);

return true;

}

Pasii 1 si 2

1. Se sorteaza varfurile triunghiului crescator dupa coordonata y, a.i. y0 = ymin

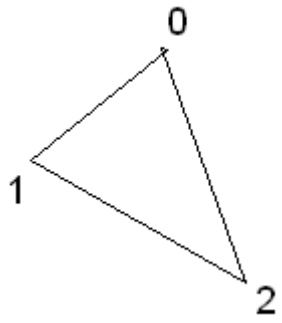
```
if( y1 < y0) { t= y1; y1 = y0; y0 =t; t= x1; x1 = x0; x0 =t;}

if( y2 < y0) { t= y2; y2 = y0; y0 =t; t= x2; x2 = x0; x0 =t;}
```

2. Daca triunghiul este degenerat (varfurile sunt coliniare), return false

$$\det = \begin{vmatrix} x1 - x0 & y1 - y0 \\ x2 - x0 & y2 - y0 \end{vmatrix}$$

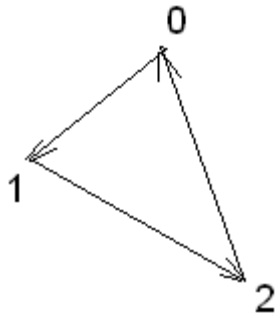
$$= (x1 - x0)(y2 - y0) - (x2 - x0)(y1 - y0)$$




```
if(det==0) return false
```

Pasul 3: orientarea conturului

3. Se orienteaza conturul in sens trigonometric (0-1-2: sens trigonometric)



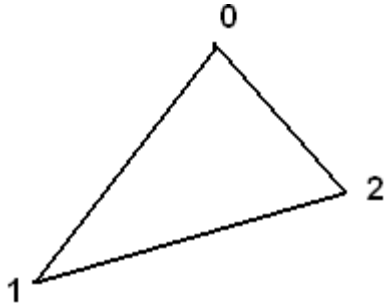
dx0 = x1 – x0; dx1 = x2 – x0; dy0 = y1 – y0; dy1 = y2 – y0;

x1 < x0 , x2 > x0

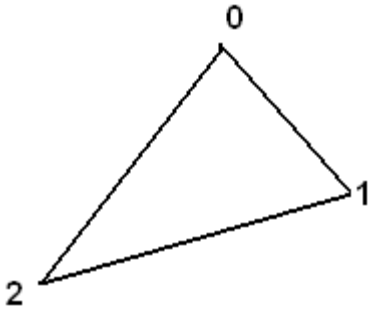
dx0 < 0, dx1 > 0, dy0 > 0, dy1 > 0

det = dx0*dy1- dx1*dy0 **< 0**

sens trigonometric,
Cazul: ymax = y2



sens trigonometric,
Cazul: ymax = y1



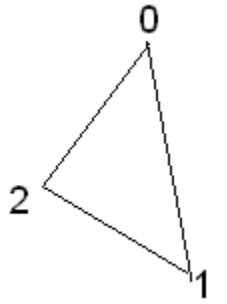
x1> x0, x2 < x0

dx0>0, dx1<0, dy0>0, dy1>0

det = dx0*dy1- dx1*dy0 **> 0**

Se inverseaza varful 1 cu 2

sens anti-trigonometric,
Cazul: ymax = y2



sens anti-trigonometric
Cazul: ymax = y1

Pasul 4: calculul extremitatilor segmentelor interioare(1)

4. Se calculeaza extremitatile segmentelor interioare suprafetei triunghiului

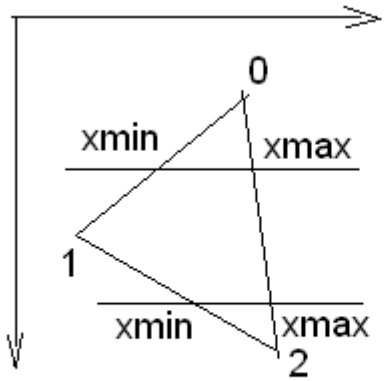
- Coordonatele xmin, xmax ale extremitatilor segmentelor interioare suprafetei triunghiului se memoreaza in 2 tablouri:

  int XMIN[H], XMAX[H]; // H este numarul de linii imagine

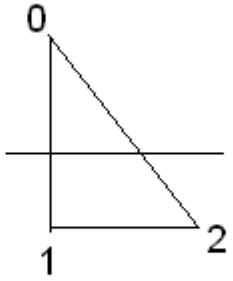
- Calculul extremitatilor este efectuat in functiile **CalculXmin()** si **CalculXmax()**;

Cazuri:

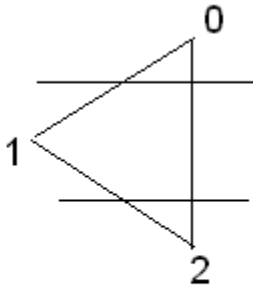
cazul I.



y0 < y1 < y2



x0 = x1



x0 = x2

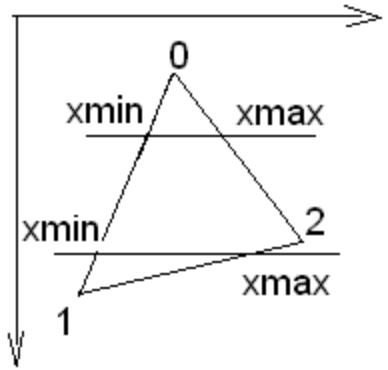
CalculXmin(x0, y0, x1, y1);

CalculXmin(x1, y1, x2, y2);

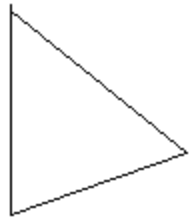
CalculXmax(x0, y0, x2, y2);

Pasul 4: calculul extremitatilor segmentelor interioare (2)

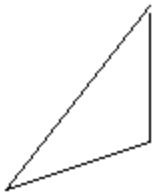
cazul II.



y0 < y2 < y1



x0 = x1



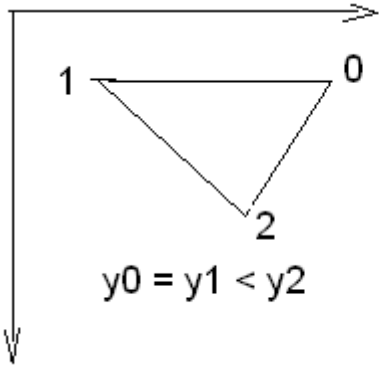
x0 = x2

CalculXmin(x0, y0, x1, y1);

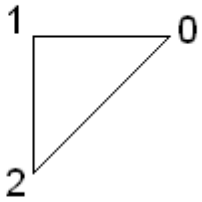
CalculXmax(x0, y0, x2, y2);

CalculXmax(x2, y2, x1, y1);

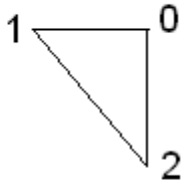
cazul III.



y0 = y1 < y2



x1 = x2

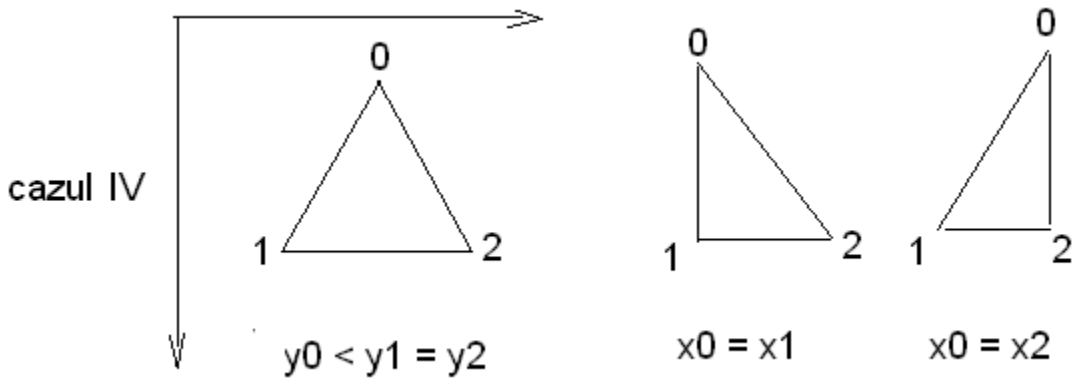


x0 = x2

CalculXmin(x1, y1, x2, y2);

CalculXmax(x0, y0, x2, y2);

Pasul 4: calculul extremitatilor segmentelor interioare (3)



CalculXmin(x0, y0, x1, y1);

CalculXmax(x0, y0, x2, y2);

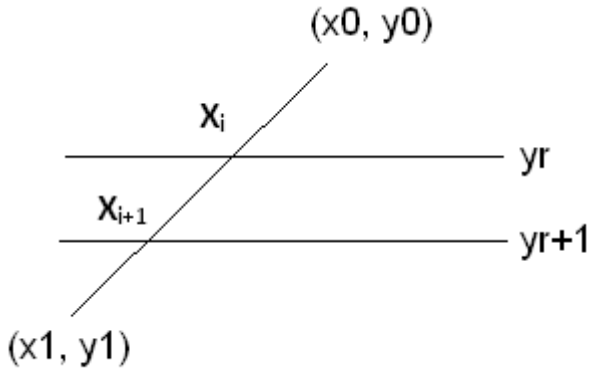
Calculul coordonatelor xmin, xmax

m = (y1 - y0)/ (x1 – x0) - panta laturii

$(yr +1 – yr)/ (x_{i+1} – x_i) = m \rightarrow x_{i+1} = x_i + (1/m)$

dx = 1.0/m - constanta a laturii

$x_{i+1} = x_i + dx \rightarrow$ calcul incremental



Pasul 4: calculul extremitatilor segmentelor interioare (4)

```
int  XMIN[H], XMAX[H];

void Calcul Xmin(int x0, int y0, int x1, int y1)

{ // y0 < y1

  int y;

  if( x0==x1)// latura verticala
   {for(y =y0; y<=y1;y++)
              XMIN[y] = x0;
    return;
   }

float dx = (float)(x1 – x0) / (y1 – y0);

XMIN[y0] = x0; XMIN[y1] = x1;

 for(y = y0 +1; y<y1; y++)

      XMIN[y] = XMIN[y-1] + dx;

}
```

```
void CalculXmax(int x0, int y0, int x1, int y1)

{ - similara cu functia CalculXmin

  - memoreaza punctele succesive de pe latură

    in tabloul XMAX

}
```

Pasul 5: afisarea pixelilor suprafetei triunghiulare

5. Se afiseaza pixelii interiori triunghiului, intre xmin si xmax pe fiecare linie raster

```
ymax = y1; if (y2>y1 )ymax = y2;
for(int y= y0; y<= ymax; y++) // pt fiecare linie raster care intersecteaza suprafata triunghiului
    for( int x = XMIN[y]; x <= XMAX[y]; x++)
        putpixel(x, y); // afisare fragment interior suprafetei triunghiului
```

Ray Tracing- optimizari

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2018-2019

Imbunatatirea calitatii imaginii(1)

Marirea numarului de raze primare

1) Uniforma

4 raze primare/pixel, care trec prin colturile suprafetei pixelului

- Culoarea pixelului se determina ca medie a culorilor obtinute cu cele 4 raze primare
- Razele primare utilizate pentru un pixel contribuie si la culoarea pixelilor adiacenti
- Pentru o imagine de mxn pixeli: (m+1)x(n+1) raze→ numarul de raze creste cu (m+n+1)

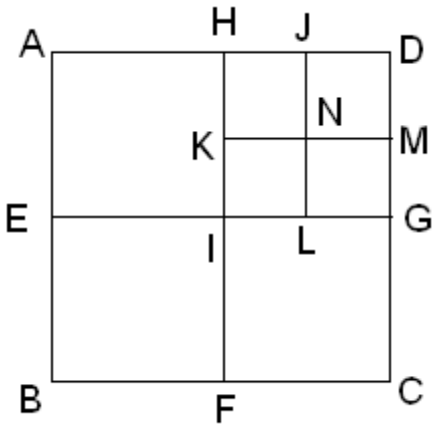
2) **Adaptiva** (Whitted) – subdivizarea adaptiva a suprafetei pixelului

- Se creste rezolutia esantionarii spatiale numai in zonele in care este necesara operatia de anti-aliasing:
- Daca diferenta dintre culorile celor 4 raze primare pentru un pixel este mare, se subdivizeaza suprafata pixelului in 4 subzone
- Se aplica acelasi criteriu de comparatie intre culorile celor 4 raze primare corespunzatoare unei subzone
- Subdivizarea se continua recursiv pana la un nivel maxim prestabilit sau pana cand diferenta dintre cele 4 culori scade sub un prag dat
- Culoarea pixelului se obtine ca o medie ponderata a culorilor subzonelor in care a fost divizata suprafata pixelului

Imbunatatirea calitatii imaginii(2)

Imbunatatirea calitatii imaginii(2)

Exemplu de subdivizare adaptiva



CA, CB, CC, CD, CE, CF, CG, CH, CI, CJ, CK, CL, CM, CN

- sunt culorile obtinute cu razele primare care trec prin punctele respective

Culoare pixel (A-B-C-D) =

¼((CA+CE+CI+CH)/4 + (CE+CB+CF+CI)/4 + (CI+CF+CC+CG)/4 +

¼((CH+CK+CN+CJ)/4 + (CK+CI+CL+CN)/4 + (CJ+CN+CM+CD)/4 +

(CN+CL+CG+CM)/4))

Reducerea complexitatii computationale

Reducerea calculelor de intersectie

1. **Utilizarea de volume incadratoare la nivel de obiect**

   Se testeaza intersectia raza-volum incadrator in loc de raza-obiect:

   - Daca raza nu intersecteaza volumul incadrator → nu se va calcula intersectia cu obiectul

2. **Divizarea scenei in volume incadratoare**

   Se divizeaza scena in volume egale pana la o anumita rezolutie spatiala sau se divizeaza scena adaptiv.

   Se testeaza intersectia razei cu volumele in care a fost divizata scena.

3. **Reprezentarea scenei printr-o ierarhie de volume incadratoare (BVH)**

   Scena este reprezentata printr-un arbore in care frunzele sunt obiectele scenei iar nodurile interne sunt grupari logice de obiecte ale scenei.

   Fiecare nod are un volum incadrator.

Utilizarea de volume incadratoare (1)

Utilizarea de volume incadratoare la nivel de obiect/ grup de obiecte

- Permite evitarea calculelor de intersectie cu obiecte ale scenei care pot avea geometrie complexa
- Permite eliminarea din calculele de intersectie a unui intreg grup de obiecte care nu este intersectat de raza

Volume incadratoare:

- Sfera, paralelipipedul cu fetele paralele cu planele principale, elipsoidul, cilindrul.
- Calculul intersectiei raza-volum incadrator trebuie sa fie mai simplu decat calculul intersectiei cu obiectul (de ex. o retea poligonala)
- Volumul incadrator al unui obiect se alege in functie de forma obiectului

Utilizarea de volume incadratoare (2)

Calcule de intersectie raza-volum incadrator

Ecuatia razei: r(t) = P0 + t*D

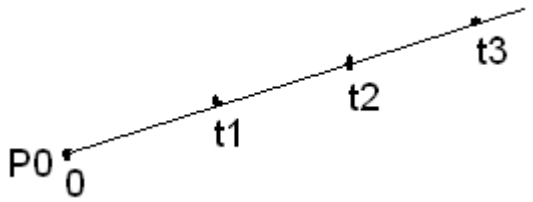
P0 – este originea razei

D – este directia razei

$x = x0 + t*dx$

$y = y0 + t*dy$

$z = z0 + t*dz$



Punctul de intersectie cel mai apropiat de originea razei este r(tmin)

P0 poate fi:

- pozitia observatorului (raza primara)
- un punct de intersectie cu scena (raza secundara)

1) Intersectia cu sfera

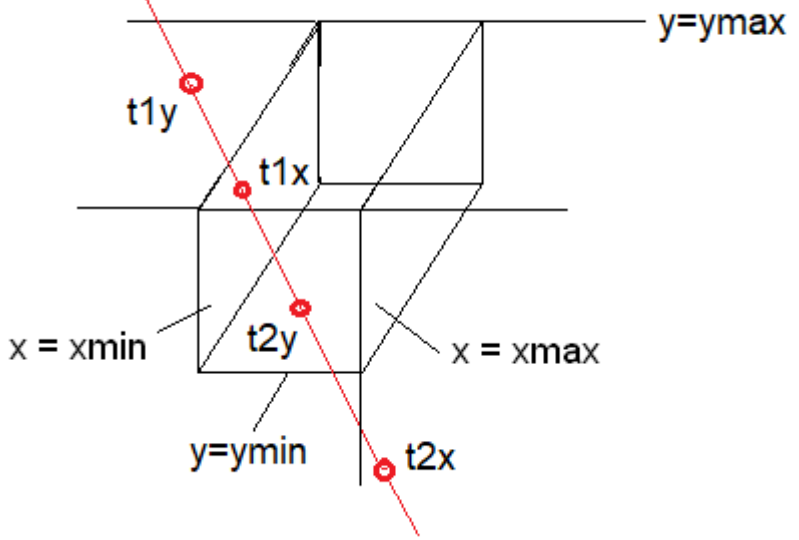
$(x-a)^2 + (y-b)^2 + (z-c)^2 = r^2$

- Se inlocuiesc in ec sferei x, y, z, cu cele din ecuatia razei
- Rezulta o ecuatie de grad 2 in t
- Se calculeaza discriminantul D, al ecuatiei:
  - D<0 – raza nu intersecteaza sfera
  - Daca sfera este obiect al scenei (intereseaza punctul de intersectie):
    - D=0 – raza este tangenta la sfera
    - D>0 – se calculeaza radacinile, t1, t2 → punctul de intersectie cel mai aropiat de originea razei este r(tmin(t1,t2))

Utilizarea de volume incadratoare (3)

2) Intersectia cu un paralelipiped cu fetele paralele cu planele principale (1)

Raza intra in volum prin fața x = xmin și iese prin fața y = ymin.



Planele care delimiteaza volumul:

x= xmin, x = xmax, y=ymin, y = ymax, z= zmin, z= zmax

Intersectiile cu x=xmin si x=xmax

xmin = x0 + t*dx → t1x = (xmin – x0)/dx

xmax = x0 + t*dx →t2x =(xmax – x0)/dx

tix = min(t1x, t2x), tex = max(t1x, t2x)

Analog se calculeaza intersectiile cu planele y=ymin si y=ymax:

tiy = min(t1y, t2y), tey = max(t1y, t2y)

tix=min(t1x,t2x) = t1x
tex=max(t1x,t2x) = t2x
tiy=min(t1y,t2y) = t1y
tey=max(t1y,t2y) = t2y
max(tix,tiy) = max(t1x,t1y) = t1x
min(tex,tey) = min(t2x,t2y) = t2y

t1x<t2y → se calculeaza intersectiile cu z=zmin si z=zmax

daca max(tix,tiy) > min(tex,tey) raza nu intersecteaza volumul

altfel, se calculeaza intersectiile cu z=zmin si z= zmax

daca max(tix, tiy, tiz) < min(tex, tey, tez)

raza intersecteaza volumul

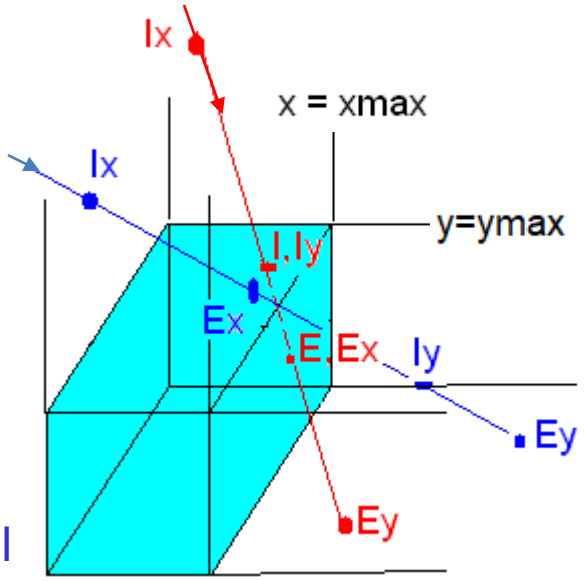
Utilizarea de volume incadratoare (4)

2) Intersectia cu un paralelipiped cu fetele paralele cu planele principale (2)

Punctele de intrare si iesire din volum sunt:

I = r(max(tix, tiy, tiz))

E = r(min(tex,tey,tez)



Raza albastra intersecteaza prelungirile fețelor x=xmin, x=xmax

max(tix, tiy) = tiy , min(tex,tey)=tex

max (tix, tiy) > min(tex,tey) → raza nu intersecteaza volumul

Raza rosie intra in volum prin fața y=ymax si iese prin fața x=xmax

max(tix, tiy) = tiy, min(tex,tey) = tex

max(tix,tiy) < min(tex,tey) → se calculeaza (tiz, tez)

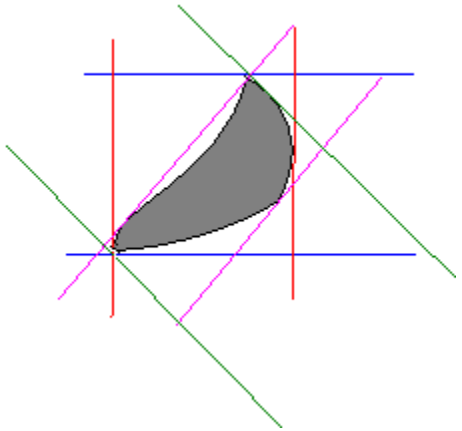
max (tix,tiy,tiz) = tiy, min (tex,tey,tez) = tex

Punctele de intersectie cu volumul: I = r(tiy); E = r(tex)

Utilizarea de volume incadratoare (5)

3) Intersectia cu un volum incadrator poliedru convex

Volumul incadrator (Kay, Kajiya[1986]) este un poliedru convex format din intersectiile a 4 perechi de plane paralele, inclinate la 0, 45, 90, 135 grade fata de orizontala.



Ec. unui plan: A*x + B*y + C*z + D =0

- Fie t11 si t12 - valorile lui t pentru intersectiile cu o pereche de plane paralele

t1min = min(t11,t12) corespunde punctului de intersectie mai apropiat de originea razei

- Fie t21, t22 - valorile lui t pentru intersectiile cu urmatoarea pereche de plane paralele

Daca max (t1min, t2min) > min (t1max, t2max) raza nu intersecteaza volumul

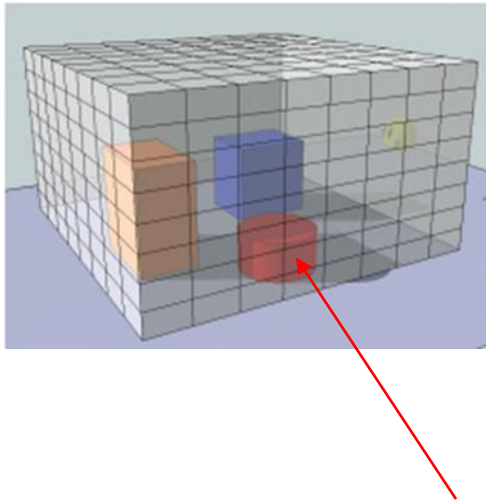
altfel, se continua cu intersectia urmatoarei perechi de plane

➢ **Punctele de intersectie cu volumul: I = r(max($t_i$min)), E = r(min($t_i$max))**

Divizarea scenei in volume incadratoare (1)

Divizarea regulata a scenei

- Se porneste de la paralelipipedul incadrator al scenei, care se divizeaza recursiv in opt subvolume egale, pana la o anumita rezolutie.
- Subvolumele finale sunt numite **voxeli**.
- La divizare nu se tine cont de structura scenei
- Fiecarui voxel ii este asociata lista de obiecte pe care le contine.
- Un obiect se poate afla in mai multi voxeli (liste)



Intersectia unei raze cu scena:

- Raza este intersectata numai cu obiectele din voxelii traversati de raza
- Traseul razei prin volumul de voxeli poate fi calculat eficient printr-un algoritm DDA 3D (extensie pentru 3D a algoritmului DDA pentru rasterizarea vectorilor).

Divizarea scenei in volume incadratoare (2)

Voxelul de start: prin care raza primara intra in volum/ din care pleaca o raza secundara

Voxel_curent = voxel de start; gata = false; in_volum = true;

cât timp (!gata && in_volum)

{ **daca** voxelul curent nu este gol (lista de obiecte nu este vida) **atunci**

{*se intersecteaza raza cu obiectele continute in voxelul curent;

daca exista intersectii **atunci**

{ *se retine punctul de intersectie cel mai apropiat de originea razei;

gata = true;

continue;

}

}

// voxelul curent este gol sau nu exista intersectii cu obiectele voxelului curent

*determina urmatorul voxel de pe traseul razei;

daca raza iese din volum **atunci**

in_volum = false;

altfel voxel_curent = urmatorul voxel de pe traseul razei;

}

Divizarea scenei in volume incadratoare (3)

Divizarea adaptiva a scenei

- Se porneste de la paralelipipedidul incadrator al scenei, care se divide in 8 subvolume egale, in mod recursiv, divizarea unui subvolum terminandu-se daca el nu contine nici un obiect al scenei sau contine un singur obiect.
- Scena se reprezinta printr-un arbore octal, fiecare nod fiind asociat unui subvolum.
- Pentru intersectia razei cu scena se fac teste de intersectie pornind din nodul radacina spre frunze.
- **Daca subvolumul unui nod nu este intersectat de raza,**

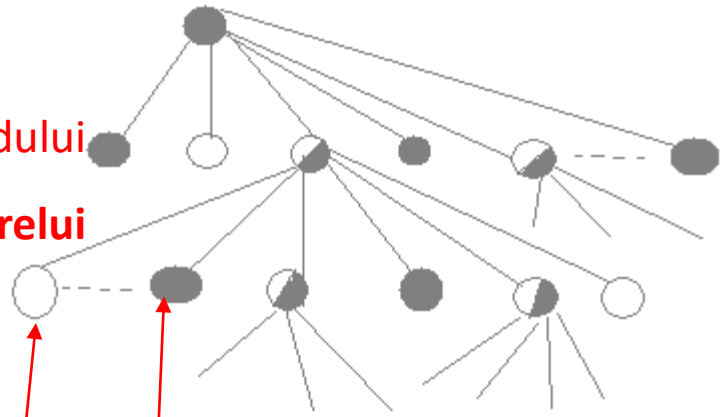
  atunci (niciunul dintre subvolumele din subarborele nodului nu va fi intersectat) **se revine pe nivelul anterior al arborelui**

  altfel daca nodul current este frunza **atunci**

  calculeaza intersectia razei cu obiectul din nod

  altfel se coboara in arbore

Arborele octal al scenei



- subvolum gol
- subvolumul contine un singur obiect

Scena – ierarhie de volume incadratoare

Bounding Volume Hierarchy – BVH

- Scena 3D este reprezentata printr-un arbore (arborele scenei) in care fiecare nod are atasat un volum incadrator pentru un grup de obiecte: vezi cursul 5 (BVH).
- Radacina are atasat volumul incadrator al intregii scene
- Fiecare frunza are atasat volumul incadrator al unui obiect (sau mai multor obiecte), celelalte noduri au atasate volume ce incadreaza grupuri de obiecte.
- Pentru intersectia razei cu scena este parcurs arborele de la radacina spre frunze
- **Daca raza nu intersecteaza volumul incadrator al nodului curent atunci**

(- nici unul dintre obiectele din volumul nodului nu este intersectat

- nici unul dintre obiectele din subarborele nodului nu este intersectat)

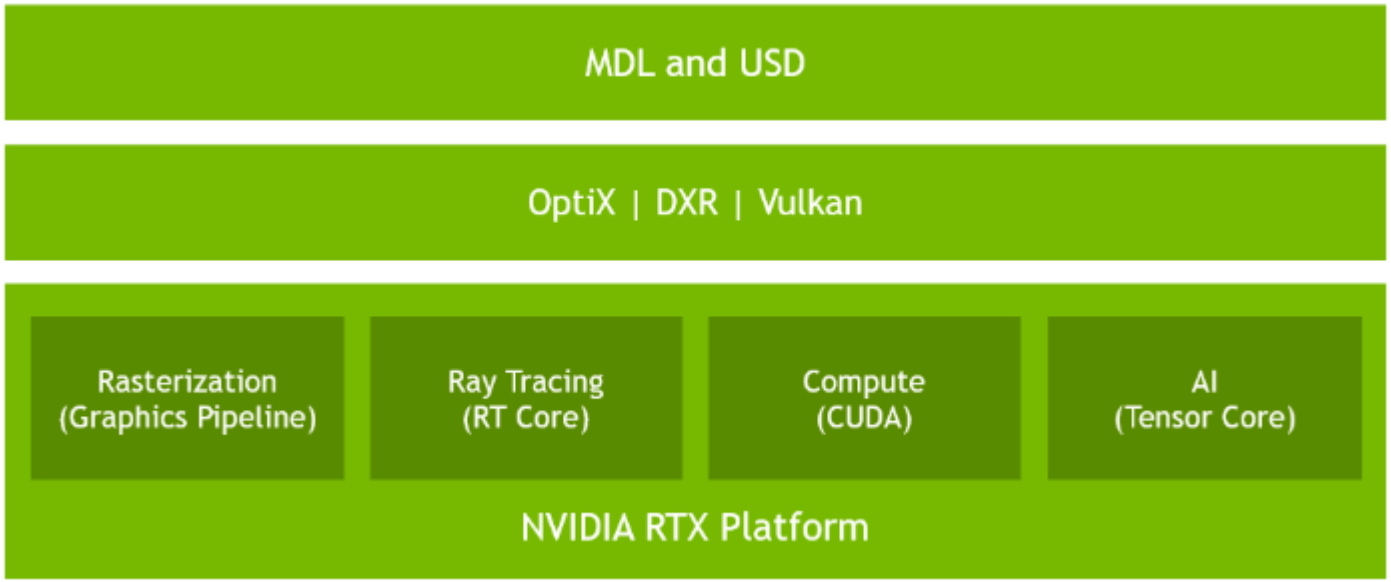
se revine pe nivelul anterior al arborelui

altfel daca nodul este frunza atunci

calculeaza intersectia razei cu obiectul/obiectele din nod

altfel se coboara in arbore

Platforma NVIDIA RTX permite implementarea algoritmului Ray-tracing folosind API-uri si kituri software de dezvoltare dedicate: OptiX, Microsoft DXR şi Vulkan



https://developer.nvidia.com/rtx

Rasterizarea suprafetelor poligonale 2D

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2018-2019

Suprafete poligonale 2D

- Definite prin lista coordonatelor varfurilor.

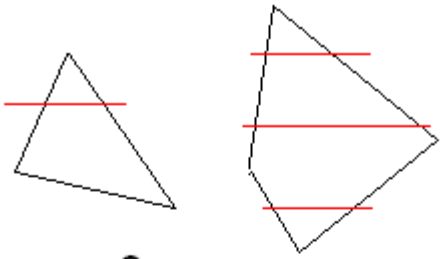
Rasterizarea unei suprafețe poligonale 2D: descompunerea suprafetei în fragmente care se afiseaza in pixeli.

- **Necesita calculul adreselor pixelilor in care se afiseaza fragmentele**
- In cazul unei suprafețe poligonale 3D transformată în spațiul de afişare, algoritmul de rasterizare include:
  - Calculul coordonatei z pentru fiecare fragment (explicat in cursul despre Z-buffer)
  - Calculul culorii fragmentului în fragment shader
  - Testul de adancime

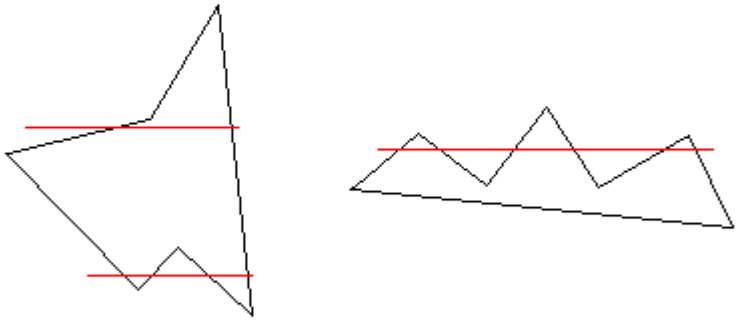
  Aceste operatii le consideram incluse in functia putpixel (int x, int y)
- Pentru o suprafață poligonală definită într-un sistem grafic 2D vom folosi pentru afişarea unui fragment funcția setpixel (in x, int y, RGB culoare).

Rasterizarea suprafetelor poligonale (1)

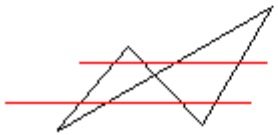
O suprafață triunghiulară este un caz particular de suprafață poligonala.



Convexe



Concave



Autointersectant

Forma generală a algoritmului:

pentru toate liniile raster care intersecteaza suprafata poligonului

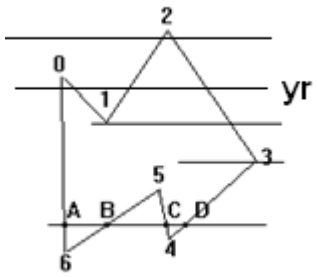
- calculeaza punctele de intersectie dintre linia raster si laturile poligonului
- coloreaza pixelii de pe segmentele interioare poligonului de pe fiecare linie raster

Probleme

1) O linie raster nu intersecteaza toate laturile poligonului: nu trebuie calculate intersectiile cu aceste laturi.

2) Intersectia dintre o linie raster si un poligon concav produce mai multe segmente de pixeli interiori.

Rasterizarea suprafetelor poligonale (2)



1) O linie raster nu intersecteaza toate laturile poligonului.

➢ se determina pentru fiecare linie raster setul laturilor intersectate:

setul laturilor active (ymin_latura <= yr < ymax_latura)

2) Intersectia dintre o linie raster si un poligon concav produce mai multe segmente de pixeli interiori. Pentru determinarea segmentelor de pixeli interiori de pe o linie raster, yr:

➢ Se ordonează punctele de intersecţie crescător după abscisă: x1< x2< x3 <x4 .

Exemplu: xA< xB< xC< xD.

➢ Segmentele de pixeli interiori suprafetei poligonului sunt:

(x1, yr) – (x2,yr) şi (x3-yr) – (x4,yr). Exemplu: (A-B, C-D).

❖ Punctele de pe segmentele (x1, yr) – (x2,yr), (x3-yr) – (x4,yr), ..,respecta **definitia punctului interior unui poligon**: "***un punct este interior unui poligon dacă orice semidreaptă dusă din punctul respectiv intersectează poligonul într-un număr impar de puncte*** "

Rasterizarea suprafetelor poligonale (3)

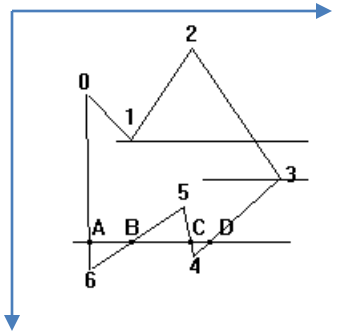
❖ **Cazuri particulare: liniile raster care traverseaza varfurile poligonului:**

- daca fiecare varf apartine la 2 laturi, atunci, o semidreapta orizontala dusa dintr-un punct interior poligonului

  -va produce 3 puncte de intersectie daca traverseaza varful 1

  -va produce 2 puncte de intersectie daca traverseaza varful 3

- Varfurile 0, 1, 2, 4, 5, 6 sunt varfuri de minim / maxim local



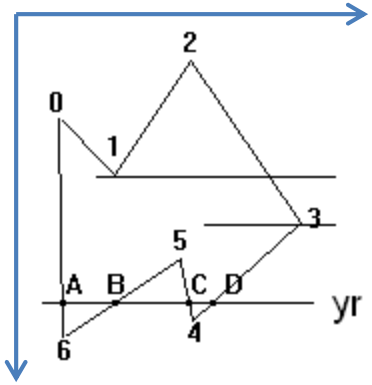
Conventie:

- la traversarea unui varf de minim / maxim local trebuie sa rezulte 0 sau 2 puncte de intersectie
- la traversarea oricarui alt varf trebuie sa rezulte un singur punct de intersectie

Implementarea conventiei: se considera fiecare latura un interval inchis la un capat si deschis la celalalt, de exemplu: [ymin_latura – ymax_latura). Atunci:

-un vârf de maxim local nu aparţine nici uneia dintre laturile care-l formează iar un vârf de minim local aparţine ambelor laturi.

Rasterizarea suprafetelor poligonale (4)



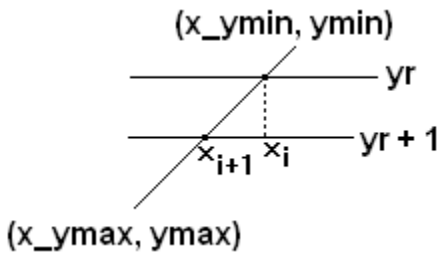
Algoritmul de rasterizare

1) Se crează lista laturilor poligonului, ordonată crescător după ordonatele minime ale laturilor: 1-2, 2-3, 0-1, 6-0, 3-4, 4-5, 5-6

2) for(yr = ymin_poligon; yr< ymax_poligon; yr++)

- Determina setul laturilor active pe linia y=yr; exemplu: 6-0, 3-4, 4-5, 5-6.
- Calculează intersecţiile dreptei y=yr cu laturile active, **in ordinea din lista laturilor**; pentru exemplul din figura punctele de intersectie vor fi: A, D, C, B.
- Ordonează punctele de intersecţie crescător după abscisă: A, B, C, D.
- Colorează pixelii de pe segmentele de dreaptă interioare suprafetei poligonului (A-B, C-D).

Rasterizarea suprafetelor poligonale (5)



Calculul intersectiilor liniilor raster cu laturile poligonului

$m = (ymax - ymin)/(x_ymax - x_ymin) = (yr+1 - yr)/(x_{i+1} - x_i)$

$x_{i+1} = x_i + 1/m$ sau $x_{i+1} = x_i + dx$

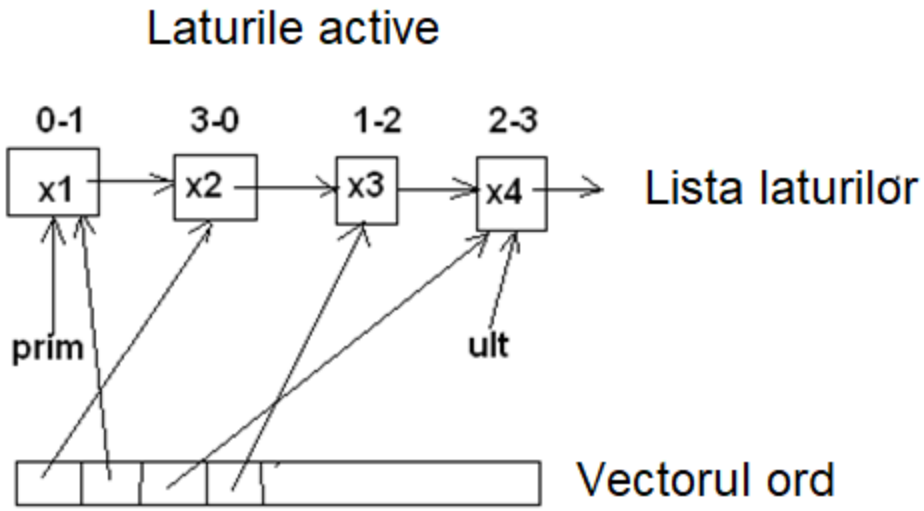
x1 = x_ymin – dx (inainte ca latura sa devina activa)

Implementare-1

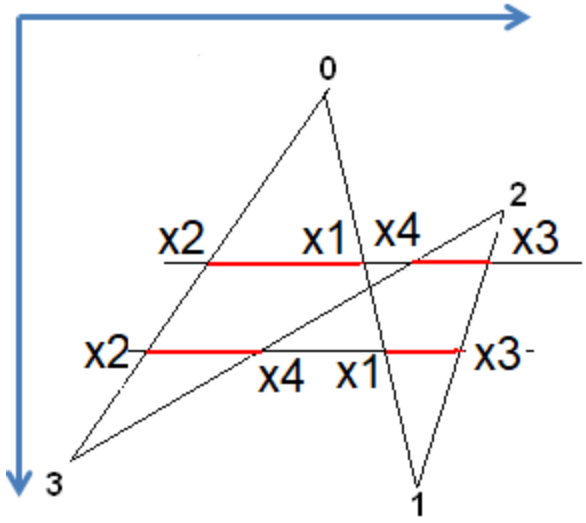
Reprezentarea unei laturi in lista de laturi:

- ymax_latură şi ymin_latură, folosite în testul de latură activă;
- xi - abscisa ultimului punct de intersecţie al laturii;
  initial, xi = abscisa varfului de ymin_latura - dx
- dx constanta folosită în calculul incremental al absciselor punctelor de intersecţie.

Implementare-1 (2)




```
typedef struct {int x,y;} virf;
struct latura{  int ymax,ymin;
                float xi,dx;
                struct latura * urm;
              };
typedef struct latura latura;
latura * prim,*ult;
latura **ord;
```

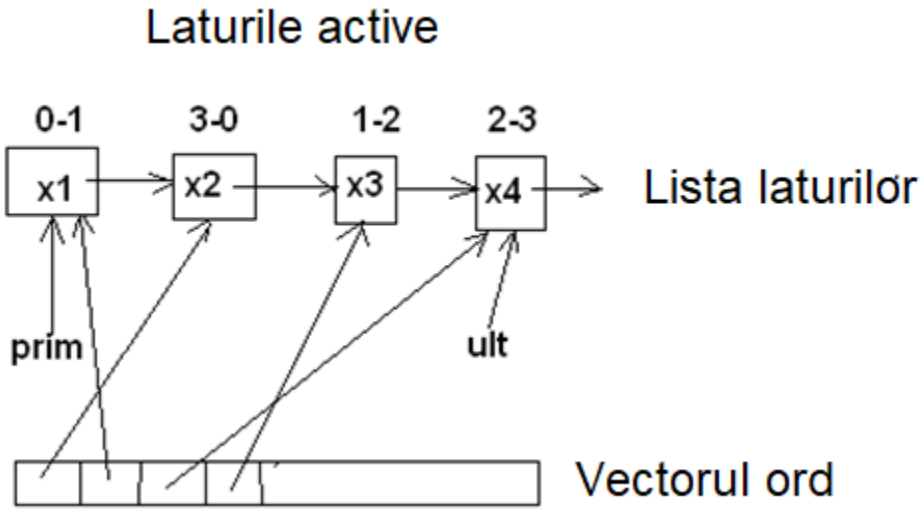


- prim si ult delimiteaza setul laturilor active in lista laturilor
- vectorul ord contine pointeri catre punctele de intersectie in ordinea crescatoare a absciselor

➢ Vectorul ord trebuie ordonat pentru fiecare linie raster,

chiar daca nu s-a schimbat setul laturilor active.

Implementare-1 (3)



Determina lista laturilor active
pentru linia raster curenta

Initial: ult=prim;

int set_activ(int yr)

{

* Adaugă laturile care devin active
(avans **ult** in lista laturilor cat timp yr ==ymin_latura)
* Elimină din lista laturilor, intre prim si ult, laturile care au devenit inactive:
(yr == ymax_latura)

return 1 daca s-a modificat setul laturilor active,
0 altfel;

}

Implementare-1 (4)

int colorare_poligon(int nrv, varf poly[])

{ // **nrv** este numarul de vârfuri ale poligonului; **poly** este vectorul vârfurilor

latura **ord; latura * k; int I, k, modif, x;

* **Creaza lista de laturi ordonata crescator dupa ymin_laturi**;

ult = prim;

// Genereaza interiorul poligonului prin intersectii cu linii raster, de la ymin_poligon la

// ymax_poligon

for(yr=ymin_poligon; yr < ymax_poligon; yr++)

{ modif=set_activ(yr); // calculeaza setul laturilor active

*** Calculeaza punctele de intersecţie cu laturile active (memorate in celulele listei de laturi intre prim si ult): xi = xi + dx**

if(modif) // s-a modificat setul laturilor active

*** Memoreaza in vectorul ord pointeri catre laturile active**

Implementare-1 (5)

// Ordoneza vectorul ord a.i. sa puncteze catre punctele de intersecţie cu laturile active in ordinea crescatoare a absciselor

```
do  {o=0;
      for(i=0; i<nrint-1; i++) //  nrint este numarul de puncte de intersectie
           if(ord[i]->xi > ord[i+1]->xi) {k=ord[i]; ord[i]=ord[i+1]; ord[i+1]=k;o=1;}
 }while(o);
```

// Coloreaza pixelii interiori poligonului de pe linia curenta

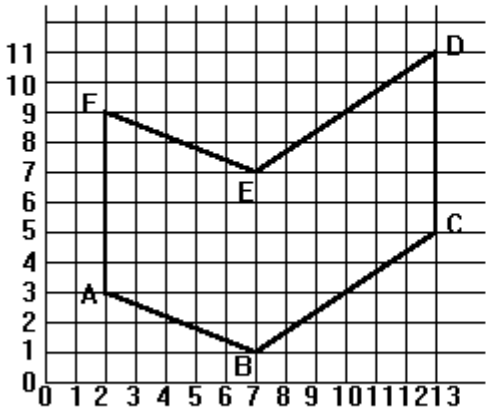
```
for(i=0; i<nrint-1; i+=2)
  for(x = (int)(ord[i]->xi+0.5); x<=(int)(ord[i+1]->xi+0.5); x++)
  { putpixel(x,yr)  sau setpixel(x, yr, culoare(x,yr)); // afiseaza fragmentul
}//for yr
return 1;
```

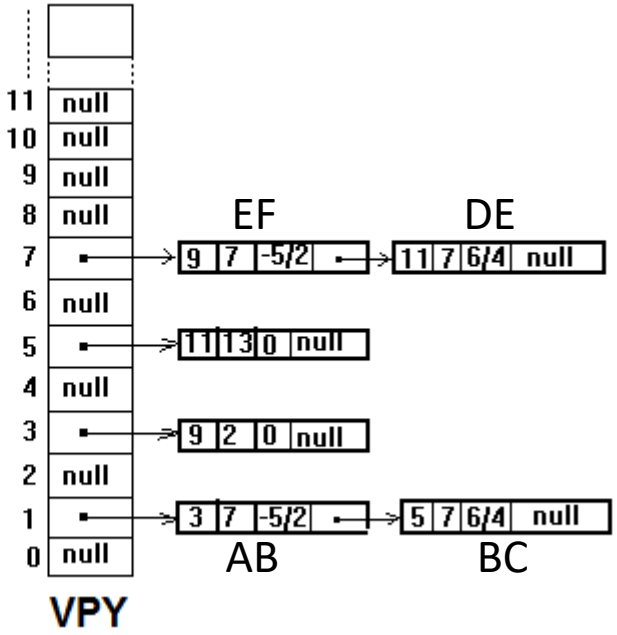
setpixel (int x, int y, RGB culoare) – scrie in frame buffer culoarea pixelului (x,y)
-pentru un poligon definit intr-un sistem grafic 2D

RGB **culoare**(int x, int y) – intoarce culoarea calculata pentru fragmentul afisat in pixelul (x,y)

Implementare-2 (1)

Poligonul ABCDEFA in spatiul ecran





- **VPY**: vector de pointeri cu un număr de intrări egal cu numărul de linii imagine.
- **VPY[i]**: pointer către lista laturilor care au ordonata minimă (ymin) egală cu **i**
- Pentru fiecare latură se memorează: ordonata maximă , abscisa vârfului de ordonată minimă şi incrementul (1/m) folosit în calculul punctelor de intersecţie.
- Lista laturilor cu acelaşi ymin este ordonată crescător după abscisa vârfului cu ordonata ymax.

Implementare-2 (2)

Lista laturilor active (LLA) este reprezentată separat, printr-o listă înlănţuită, în care laturile sunt memorate în ordinea crescătoare a absciselor punctelor de intersecţie cu linia curenta.

Actualizare LLA pentru linia raster curenta, yr:

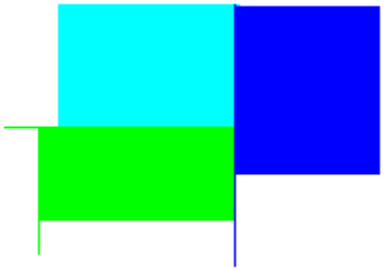
- Se adauga la LLA lista de laturi punctata de VPY[yr]: laturi care devin active
- Se elimina din LLA acele laturi pentru care yr == ymax_latura
- Se sorteaza LLA crescator dupa x-intersectie (calculat incremental pornind de la x-ymin)

Comparatie cu implementare-1

- Lista laturilor se construieste mai rapid dar consuma mai multa memorie (în majoritatea cazurilor numai un număr mic dintre intrările vectorului VPY sunt folosite)
- Actualizarea listei de laturi active consuma mai mult timp.

Generarea pixelilor strict interiori

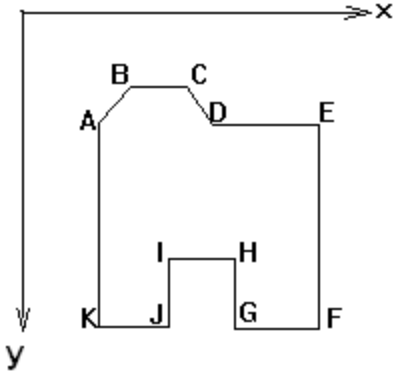
Trebuie evitata suprascrierea pixelilor de pe laturile comune poligoanelor .



Conventie:

-Pixelii de pe laturile de xmin si ymin apartin poligonului (se afiseaza)

-Pixelii de pe laturile de xmax si ymax nu se afiseaza



Datorita conventiei folosite pentru traversarea varfurilor poligonului (Implementare 1):

- Liniile orizontale care unesc 2 varfuri de minim local vor fi afisate
- Liniile orizontale care unesc 2 varfuri de maxim local nu vor fi afisate

➢In acest fel se respecta conventia pentru laturile orizontale.

➢Se afiseaza: BC, DE, IH

Generarea pixelilor strict interiori

Pentru laturile neorizontale:

Pentru punctele de xmin si xmax ale segmentelor interioare poligonului:

- daca fiecare segment interior poligonului este un interval inchis in punctul de abscisa minima si deschis in punctul de abscisa maxima, atunci se respecta conventia.

```
for(x = (int)(ord[i]->xi+0.99999); x<=(int)(ord[i+1]->xi-0.000001); x++)
    setpixel(x, yr, culoare(x,yr));
```

❖ Generarea numai a pixelilor strict interiori nu dă rezultate bune atunci când unghiul dintre două laturi este foarte mic. Este posibil ca între cele două laturi să fie afişat un singur pixel sau chiar niciunul.

Generarea interiorului unui poligon folosind un şablon

Modificarea implementarii anterioare pentru generarea interiorului folosind un sablon

```
int sablon[8][8]={255,0,0,0,0,0,0,255, 0,255,0,0,0,0,255,0, 0,0,255,0,0,255,0,0,
                  0,0,0,255,255,0,0,0,0,0,0,255,255,0,0,0, 0,0,255,0,0,255,0,0,
                  0,255,0,0,0,0,255,0,255,0,0,0,0,0,0,255
                }; //matrice de intensităţi

int hs=8, ls=8, lin, x;//hs – inaltimea sablonului; ls – latimea sablonului
// sablonul se multiplica incepand din (x=0, y=0): originea imaginii

void colorare_poligon(int nrv, virf poly[] )
{……..
 for(yr=ymin_poligon; yr < ymax_poligon; yr++)
 { ……
   lin=yr%hs; // linia din matricea sablon folosita la colorarea poligonului pe linia raster yr
   for(i=0;i<nrint-1;i+=2) // nrint: numarul de intersectii dintre linia raster si suprafata
                           // poligonului
     for(x=(int)(ord[i]->xi+0.99999; x<=(int)(ord[i+1]->xi+0.000001); x++)
           setpixel(x,yr,sablon[lin][x%ls]);//afisare pixel in culoarea din sablon[lin][x%ls]
  }
```

Generarea suprafetelor 2D definite prin contur sau prin interior

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2018-2019

Suprafete definite prin contur sau interior

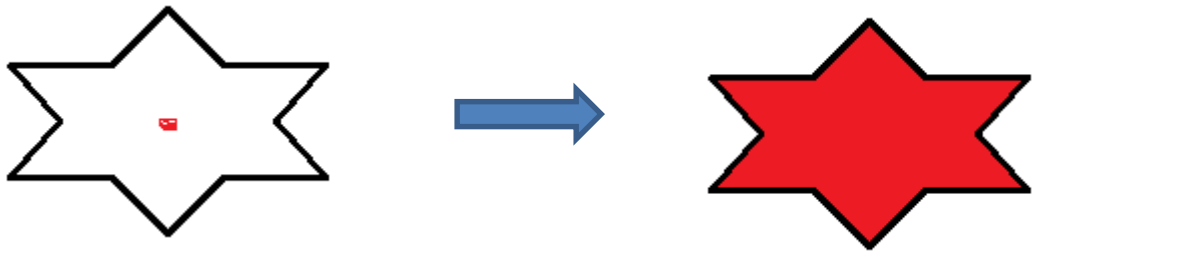
Suprafetele definite prin contur sau interior sunt suprafete 2D oarecare.

(1) Suprafete definite prin contur

- **se dau:** culoarea pixelilor care alcătuiesc conturul suprafeţei şi un punct interior;

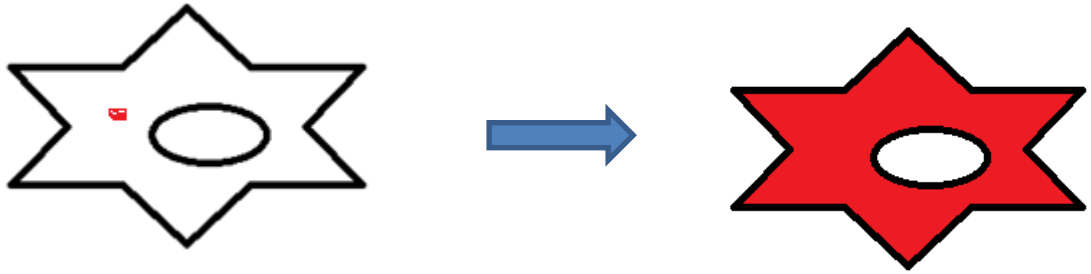
- la generarea suprafetei, conturul trebuie sa fie inscris in memoria imagine (frame buffer);

- pornind din punctul interior cunoscut, se modifică culoarea tuturor pixelilor interiori conturului în culoarea sau conform şablonului care sunt date.



Culoare contur: negru

Culoarea de umplere: rosu



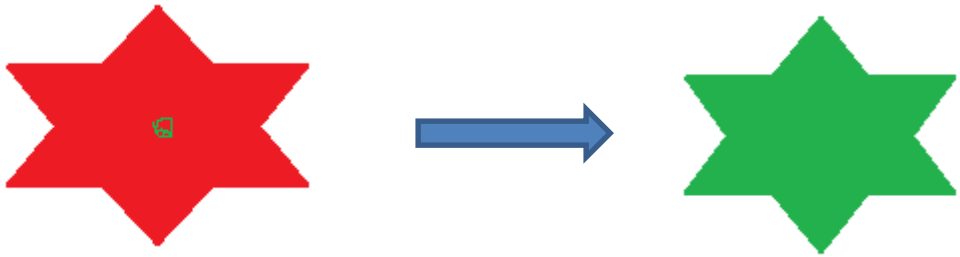
Suprafete definite prin contur sau interior

(2) **Suprafete definite prin interior**

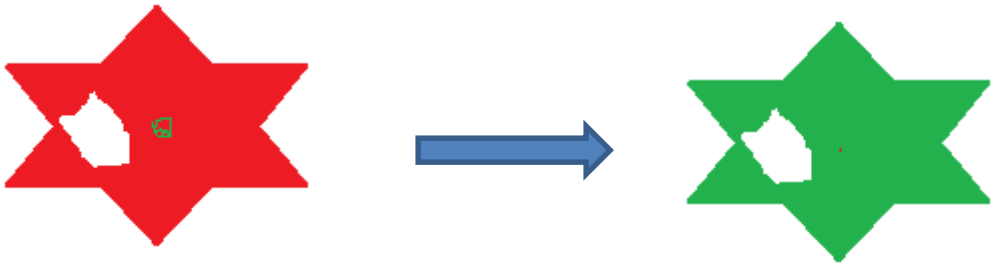
- **se dau:** culoarea pixelilor interiori suprafeţei şi un punct interior;

- la generarea suprafetei, pixelii interiori suprafetei trebuie sa aiba culoarea data (sa fie inscrisi in memoria ecran) ;

- pornind din punctul interior cunoscut, se modifică culoarea tuturor pixelilor suprafeţei în culoarea sau cu şablonul care sunt date.



Culoarea pixelilor suprafetei: rosu Culoarea de umplere: verde



Algoritmi flood fill (1)

Există două tipuri de algoritmi folosiţi la generarea suprafeţelor definite prin contur sau prin interior:

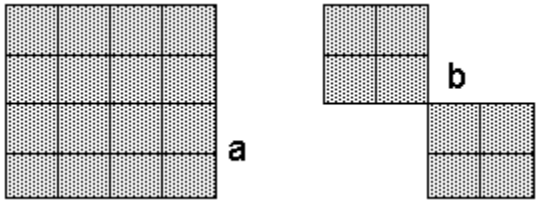
(1) Algoritmi recursivi, bazaţi pe verificarea culorii punctelor vecine punctului curent (algoritmi **"flood fill"**) si modificarea culorii lor pana la atingerea conturului/marginii suprafetei;

(2) Algoritmi bazaţi pe parcurgerea liniilor raster care traversează suprafaţa.

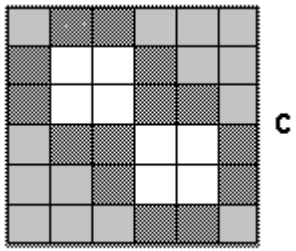
In algoritmii **flood fill** se tine cont de tipul suprafetei: **conexa de ordin 4 sau conexa de ordin 8:**

— Fiecare pixel al unei suprafeţe conexe de ordinul 4 poate fi atins, pornind dintr-un punct interior, printr-o combinaţie de deplasări în numai patru direcţii: stânga, dreapta, sus, jos (suprafata **a** din figura).

— Un pixel al unei suprafeţe conexe de ordinul 8 poate fi atins printr-o combinaţie de deplasări orizontale, verticale şi diagonale (suprafetele **b** si **c** din figura)



Suprafete definite prin interior



Suprafata definita prin contur

Algoritm flood fill (2)

Functie de colorare a unei suprafete conexe de ordin 4, definita prin contur.

```
void sup_conex_4(int x, int y, int cul_interior, int cul_contur)
{
   int culoare = getpixel(x,y); // se citeste culoarea pixelului (x,y) din memoria ecran

   if(culoare != cul_contur && culoare != cul_interior)
  {
    setpixel(x,y,cul_interior); // scrie cul_interior pt pixelul (x,y)
    sup_conex_4(x+1, y, cul_interior, cul_contur);
    sup_conex_4(x -1, y, cul_interior, cul_contur);
    sup_conex_4(x, y+1, cul_interior, cul_contur);
    sup_conex_4(x, y-1, cul_interior, cul_contur);
  }
}
```

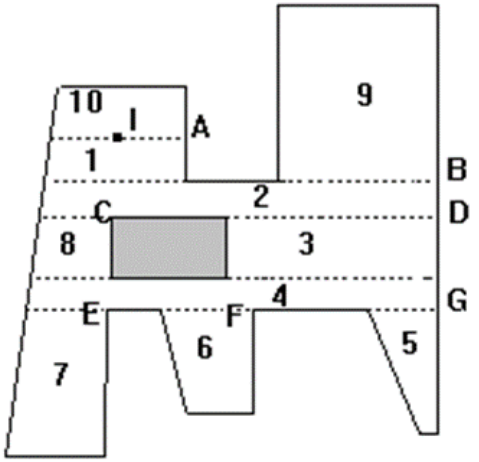
- Algoritmul necesită un spaţiu mare de memorare pentru stiva program.
- De aceea, pentru generarea suprafeţelor definite prin contur sau prin interior sunt preferaţi algoritmii bazaţi pe parcurgerea liniilor imagine care traverseaza suprafata.

Generarea unei suprafete prin parcurgerea liniilor raster(1)

- Consideram suprafata definita prin contur.
- O suprafaţă poate fi convexă sau concavă şi poate avea găuri.

➢ Deci, o linie orizontală care o traversează poate întâlni conturul suprafeţei de mai multe ori -> exista mai multe segmente de pixeli interiori pe o linie

- O secvență de pixeli situaţi pe aceeaşi linie raster, care nu au culoarea conturului şi nici culoarea de umplere, formează un **interval**; de exemplu, E, F şi G sunt extremităţile din dreapta a trei intervale de pe aceeaşi linie raster.



- Fie **I(xi,yi) punctul interior dat**. El este **primul punct de start pentru colorarea suprafetei.**
- Punctele de start pentru colorare se memoreaza intr-o stiva.

Generarea unei suprafete prin parcurgerea liniilor raster(2)

Se introduce in stiva punctul interior dat, I.

Cat timp stiva nu este vida:

(1) Se extrage din vârful stivei punctul de start pentru colorare, (xi,yi).

(2) Pornind din punctul de start, se colorează toţi pixelii din dreapta sa până când se ajunge la contur, apoi toţi pixelii din stânga sa până când se ajunge la contur.

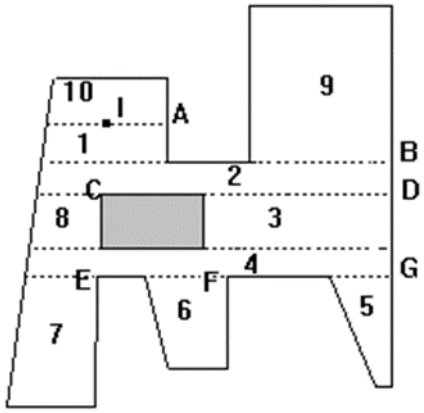
Se memorează extremitatea din dreapta a intervalului colorat, xmax, şi extremitatea sa stângă, xmin.

(3) Se examinează pixelii xmin < (x,y-1) < xmax (de pe linia de deasupra celei curente), pentru a se determina intervalele de colorat pe linia respectiva.

Se memorează extremitatea din dreapta a fiecărui interval, în stivă.

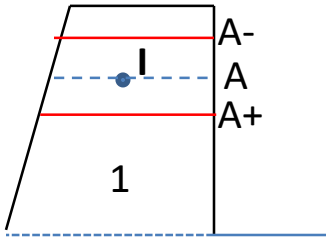
(4) Se examinează pixelii xmin < (x,yi+1) < xmax (de pe linia de sub cea curenta), pentru a se determina intervalele de colorat.

Se memorează extremitatea din dreapta a fiecărui interval, în stivă.

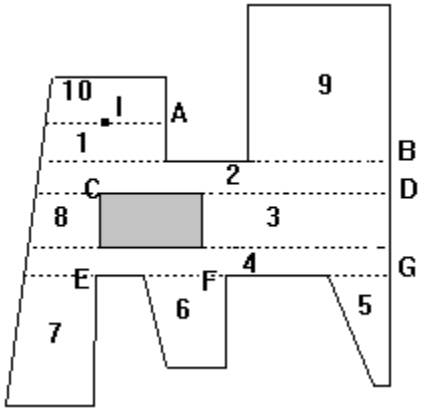


Deoarece ultimul punct introdus în stivă este de pe linia de sub cea curentă, in iteratia urmatoare prelucrarea se continua cu această linie

Generarea unei suprafete prin parcurgerea liniilor raster(4)



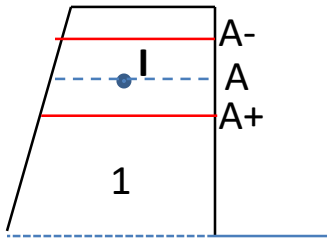
- Colorarea poligonului din figura începe din punctul I.
- După colorarea liniei $y=y_I$:
  - se parcurge linia de deasupra si se memoreaza in stiva punctul A-
  - se parcurge linia de sub cea curenta si se memoreaza in stiva punctul A+
- Se extrage din stiva A+ si se coloreaza pixelii de la linia $y= y_I +1$
- Colorarea continua cu liniile din zona 1.
- La prima linie din zona 2 se schimbă xmax; la parcurgerea liniei de deasupra sa se memoreaza B in stiva.

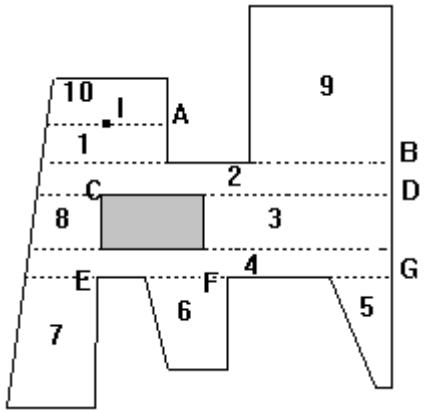


- Atunci când se examinează pixelii de pe linia ce conţine latura de sus a dreptunghiului interior poligonului se vor memora în stivă punctul C şi apoi punctul D.
- Punctul D este extras imediat din stivă, devenind punct de start. Parcurgând linia $y=y_D$, spre stânga, se va întâlni latura din dreapta a dreptunghiului interior, modificându-se xmin.

Generarea unei suprafete prin parcurgerea liniilor raster(4)

- Colorarea se continuă cu liniile din zona 3, apoi cu cele din zona 4
- Pe prima linie de sub zona 4 se determina 3 intervale de pixeli necolorati: se memoreaza in stiva extremitatile lor din dreapta - E, F si G
- Se extrage punctul G din stiva şi colorarea se continua cu liniile din zona 5.
- Când se ajunge la limita de jos a zonei 5, colorarea nu poate continua nici pe linia de deasupra nici pe cea de sub ea: nu se adauga punct in stiva.
- In iteratia urmatoare punctul din vârful stivei este F, de aceea colorarea se continuă cu zona 6, apoi cu zona 7.
- Se extrage din stiva punctul B
- Se coloreaza liniile din zona 9
- Se extrage din stiva punctul A-
- Se coloreaza liniile din zona 10





```
#define DIM_STIVA 100
#define Error -1

int Colorare_supraf(int culoare_umplere, int culoare_contur, int xinit, int yinit)
{
  int stiva[DIM_STIVA];
  int vf_stiva=1, int xi, yi, x, xmax, xmin, in_int, culoare;
  stiva[0]=xinit; // push(xinit);
  stiva[1]=yinit; // push(yinit);

  while (vf_stiva > 0)  //!StivaGoala()
  {
      // Extragere punct de start din stiva
      yi=stiva[vf_stiva--]; //pop(&yi);
      xi=stiva[vf_stiva--]; //pop(&xi);

     // Colorarea liniei curente
     putpixel(xi,yi,culoare_umplere);
     for(x=xi+1; getpixel(x,yi)!=culoare_contur; x++)
         setpixel(x,y+i,culoare_umplere);
     xmax=x;
     for(x=xi-1; getpixel(x,yi)!=culoare_contur; x--)
        setpixel(x,yi,culoare_umplere);
    xmin=x;

    // Determina intervalele de pe linia de deasupra
     for(x=xmin+1, in_int=0; x<xmax; x++)
     {   culoare=getpixel(x,yi-1);
         if (culoare!=culoare_contur && culoare!=culoare_umplere)
              { if(!in_int) in_int=1;}
               else
                 if (in_int)
                 { if (vf_stiva==DIM_STIVA-2) return Error;
                   stiva[++vf_stiva]=x-1;  //push(x-1);
                   stiva[++vf_stiva]=yi-1; //push(yi-1);
                   in_int=0;
                 }
     }
   if (in_int) {if (vf_stiva==DIM_STIVA-2) return Error;
                  stiva[++vf_stiva]=x-1; //push(x-1);
                  stiva[++vf_stiva]=yi-1; //push(yi-1);
                 }
```

```
   // Determina intervalele de pe linia de dedesubt
    for(x=xmin+1, in_int=0; x<xmax; x++)
    {    culoare=getpixel(x,yi+1);
         if (culoare!=culoare_contur && culoare!=culoare_umplere)
         {  if(!in_int) in_int=1;}
            else
             if (in_int)
              { if (vf_stiva==DIM_STIVA-2) return Error;
                stiva[++vf_stiva]=x-1; //push(x-1);
                stiva[++vf_stiva]=yi+1;//push(yi+1);
                in_int=0;
               }
    }

 if (in_int) { if (vf_stiva==DIM_STIVA-2) return Error;
                  stiva[++vf_stiva]=x-1; // push(x-1);
                  stiva[++vf_stiva]=yi+1;//push(yi+1);
                 }
}
return 0;
}
```

Generarea cercurilor în spatiul discret

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2018-2019

Aproximarea cercurilor in spatiul discret(1)

- Cercurile si elipsele nu sunt primitive grafice in OpenGL, deci nu pot fi transmise in banda grafica.
- Cercurile si elipsele sunt primitive 2D definite in planul XOY.
- Pentru afisare ele sunt descompuse in primitive de tip vector sau primitive de tip punct:
  - le putem asocia coordonata z si transmite in banda grafica sub forma unei secvente de vectori sau puncte 3D
  - mai eficient: aproximarea cercurilor prin vectori sau puncte se poate face in “Geometry shader”
- Fie (xc,yc,0) coordonatele centrului unui cerc/elipsa. Pentru afisarea sa vom folosi functiile:

  point (int x, int y, RGB culoare);

  - functia genereaza o primitiva “point” avand coordonatele (x,y,0) si culoarea specificata.

  line (int x1, int y1, int x2, int y2, RGB culoare)

  - functia genereaza o primitiva “line” cu varfurile (x1,y1, 0) - (x2,y2,0) si culoarea specificata asociata ambelor varfuri.

Aproximarea cercurilor in spatiul discret(1)

1) Folosind ecuatia cercului in coordonate carteziene

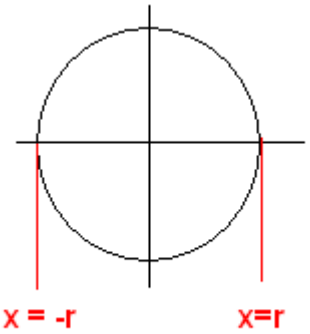
$(x - xc)^2 + (y - yc)^2 = r^2$

Generarea cercului cu centrul in (xc,yc) se reduce la generarea cercului centrat in origine, plus o translatie cu (xc, yc) a coordonatelor.

$x^2 + y^2 = r^2$

Se incrementeaza x de la –r la r si se calculeaza y din ecuatia cercului:

```
point (xc - r, yc, culoare);
point (xc + r, yc, culoare);
for (x = -r +1; x< r; x++)
{   y = sqrt(r^2 – x^2);
    point (xc + x, yc + y, culoare);
    point (xc + x, yc - y, culoare);
}
```

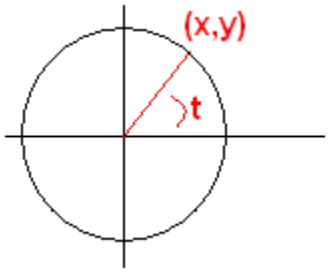


Dezavantaje:

- Spatierea inegala a punctelor de pe cerc afişate
- Calcule complexe cu numere reale

Aproximarea cercurilor in spatiul discret(2)

2) Folosind ecuatia cercului in coordonate polare



$x = xc + r^* \cos(t)$

$y = yc + r^* \sin t(t)$ $\quad 0 <= t <= 2\pi$

Se genereaza cercul centrat in origine si se aplica punctelor rezultate o translatie cu (xc, yc):

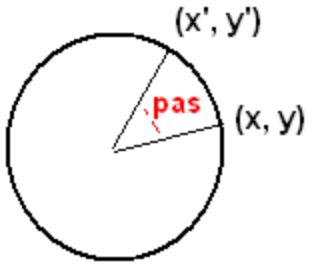
$x = r^* \cos(t)$

$y = r^* \sin t(t)$ $\quad 0 <= t <= 2\pi$

Cercul poate fi aproximat pintr-o polilinie:

- daca se calculeaza N puncte de pe cerc, egal distantate, t creste cu pasul $2\pi/N$

Calculul punctelor succesive de pe cerc, pe baza unei relatii de recurenta:



- (x, y): punctul curent

- (x', y'): punctul urmator

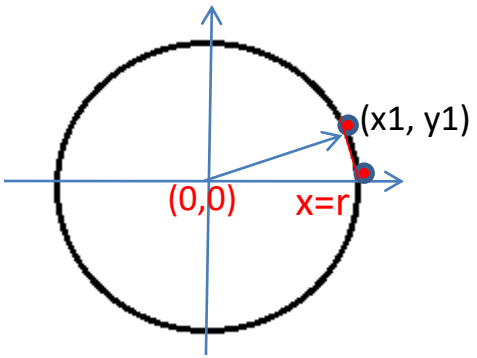
$x' = r^* \cos(t + pas) = r(\cos(t) * \cos(pas) - \sin(t)* \sin(pas)) = x*\cos(pas) - y*\sin(pas)$

$y' = r^* \sin(t + pas) = r(\cos(t) * \sin(pas) + \sin(t)* \cos(pas)) = x*\sin(pas) + y*\cos(pas)$

Aproximarea cercurilor in spatiul discret(3)

Avantajul calculului prin recurenta: nu este necesar sa se calculeze sin si cos in fiecare iteratie!

```
// Aproximarea unui cerc printr-o linie poligonala
void Cerc( int xc, int yc, int r, int N, RGB culoare)
{ double pas = 2*M_PI /N ; //pasul unghiular
  double u, x, y, x1,y1, s, c;
  c=cos(pas); s=sin(pas); // sin si cos se calculeaza o singura data
  for( int  i = 1, x=r, y=0; i<N; i++)
  { x1 =x*c - y*s; // x'
    y1 =x*s + y*c;  // y'
    line(xc+(int)(x+0.5), yc+(int)(y+0.5), xc+(int)(x1+0.5), yc+(int)(y1+0.5), culoare);
    x=x1 ; y= y1 ;
}
line(xc+(int)(x+0.5), yc+(int)(y+0.5),xc+r,yc, culoare);
}
```



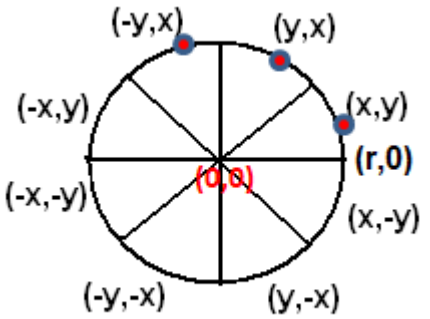
Aproximarea cercurilor in spatiul discret(4)

Simetria punctelor de pe cerc

- **Fiecare punct de pe cerc are 7 puncte simetrice pe cerc:**

  - Este suficient sa se calculeze adresele punctelor dintr-un singur octant al cercului, celelalte puncte de pe cerc find calculate prin simetrie

```
// Afiseaza punctele simetrice de pe cerc
void punct_simetric(int xc,int yc,int x,int y, RGB culoare)
{ //(x,y) – punct de pe cercul centrat in origine
  point(xc+x,yc+y,culoare);
  point(xc+x,yc-y,culoare);
  point(xc-x,yc-y,culoare);
  point(xc-x,yc+y,culoare);
  point(xc+y,yc+x,culoare);
  point(xc+y,yc-x,culoare);
  point(xc-y,yc-x,culoare);
  point(xc-y,yc+x,culoare);
}
```



Obs:

Simetricele punctului (r,0): (r, 0), (-r, 0), (-r,0), (0,r), (0,-r), (0,-r), (0,r)

→ pixelii (xc+r,yc), (xc-r,yc), (xc,yc+r), (xc,yc-r) vor fi suprascrisi

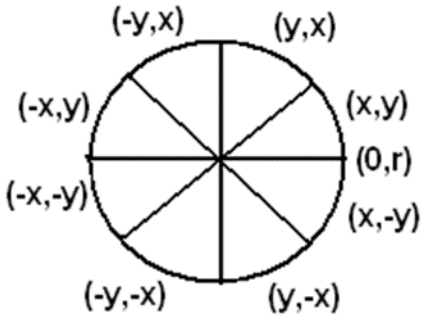
→ nu va fi apelata functia pentru punctul (r,0)

Aproximarea cercurilor in spatiul discret(5)

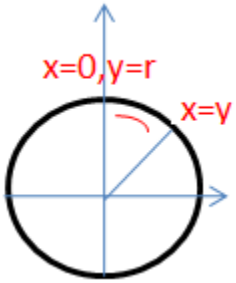
- Pentru afisarea punctelor succesive de pe circumferintă, pasul unghiular trebuie sa fie aprox. (1/ raza).

// Calculeaza numai punctele din primul octant si afiseaza toate punctele simetrice

```
void Cerc_sim( int xc, int yc, int r, RGB culoare)
{ double pas, u, x, y, xx, s, c;
  pas=1.0/r; // pas unghiular pentru obtinerea de puncte succesive de pe cerc
  c=cos(pas); s=sin(pas);
  point(xc+r,yc,culoare);
  point(xc-r,yc,culoare);
  point(xc,yc+r,culoare);
  point(xc,yc-r,culoare);
  for(u=pas, x=r, y=0; u<M_PI_4; u+=pas)
  { xx=xx;
    x =xx*c - y*s;
    y =xx*s + y*c;
    punct_simetric(xc, yc,(int)(x+0.5), (int)(y+0.5), culoare);
  }
}
```

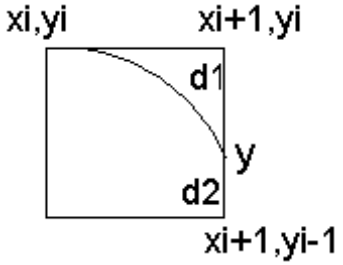


Algoritmul Bresenham pentru aproximarea cercurilor în spatial discret(1)



- Consideram cercul cu centrul în originea sistemului de coordonate si raza r.
- In cadrul algoritmului **se calculeaza numai punctele din octantul al 2-lea,** celelalte obtinandu-se prin simetrie.
- Se calculeaza punctele de pe cerc incepand cu (x=0,y=r) pana la (x=y), prin incrementarea lui x.
- Fie (x_i,y_i) ultimul punct al spatiului discret, ales pentru aproximarea cercului.

  Urmatorul punct va fi unul dintre (x_i+1,y_i) si (x_i+1,y_i-1):



Fie y ordonata punctului de pe cercul teoretic, cu abscisa x_i+1.

$$y^2 = r^2-(x_i+1)^2$$

Fie d1 si d2 distantele de la cele 2 puncte ale spatiului discret la punctul de pe cercul teoretic.

Intereseaza semnul diferentei (d1-d2):

- daca $(y_i - y) > (y- (y_i-1))$ atunci

$$(y_i^2 - y^2) > (y^2 - (y_i-1)^2)$$

$$d1' = y_i^2 - y^2 = y_i^2 - r^2 + (x_i+1)^2$$

$$d2' = y^2 - (y_i-1)^2 = r^2 - (x_i+1)^2 - (y_i-1)^2$$

Algoritmul Bresenham (2)

- Notam cu t_i eroarea de aproximare în pasul curent, pe baza careia se va alege urmarorul punct:

$t_i = d1' - d2' = y_i^2 + 2*(x_i+1)^2 + (y_i-1)^2 - 2*r^2$

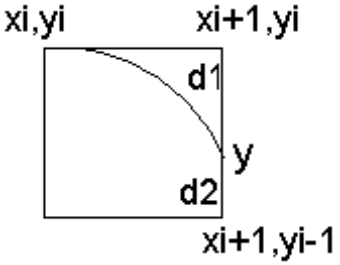
- Se obtine o relatie de recurenţă pentru calculul erorii de aproximare in pasul urmator:

$t_{i+1} = y_{i+1}^2 + 2*(x_{i+1}+1)^2 + (y_{i+1}-1)^2 - 2*r^2$

(1) Daca **$t_i<0$** ($d1' < d2'$) atunci $x_{i+1}=x_i+1$ si $y_{i+1}=y_i$. Deci,

$t_{i+1} = y_i^2 + 2*((x_i+1)+1)^2 + (y_i-1)^2 - 2*r^2$

sau **$t_{i+1} = t_i + 4*x_i + 6$**



(2) Daca **$t_i >= 0$** atunci $x_{i+1} = x_i + 1$ si $y_{i+1} = y_i - 1$. Deci,

$t_{i+1} = (y_i-1)^2 + 2*((x_i+1)+1)^2 + ((y_i-1)-1)^2 - 2*r^2$

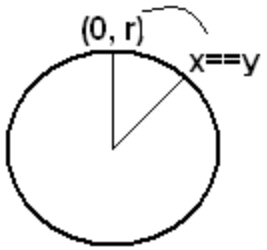
sau **$t_{i+1} = t_i + 4*(x_i - y_i) + 10$**

Algoritmul Bresenham (3)

- Valoarea erorii de aproximare în primul pas:

$t_i = y_i^2 + 2*(x_i+1)^2 + (y_i-1)^2 - 2*r^2$: $x_1=0$, $y_1 = r$. Rezultă, **t1 = 3-2*r**

```
void Bres_cerc(int xc, int yc, int r, RGB culoare)
{ int x,y,t;
  t=3-(r<<1);
 point(xc,yc+r,culoare); point(xc,yc-r,culoare);
 point(xc+r,yc,culoare); point(xc-r,yc,culoare);
  for(x=1,y=r; x<y; x++)
  {
    if(t<0)
      t+=6+(x<<2);
    else
     { t+=10+((x-y)<<2); y--;}
    punct_simetric(xc,yc,x,y,culoare);
  }
// pentru x==y se afiseaza 4 puncte simetrice
 point(xc+x,yc+y,culoare); point(xc+x,yc-y,culoare);
 point(xc-x,yc+y,culoare); point(xc-x,yc-y,culoare);
}
```



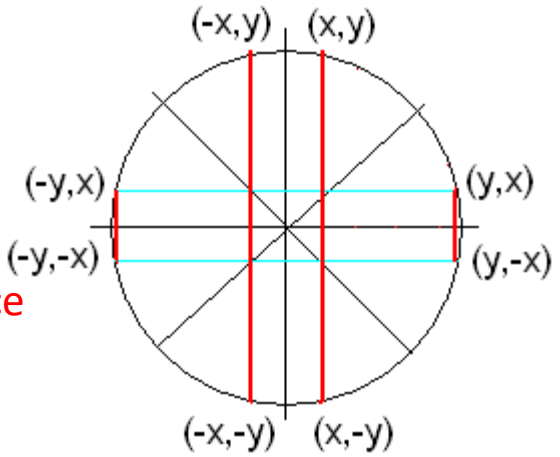
Generarea unei suprafete circulare

EXERCITIU:

- Modificati algoritmul Bresenham pentru generarea cercurilor astfel incat sa se obtina o suprafata circulara.

```
void Bres_cerc(int xc, int yc, int r, RGB culoare)
{ int x,y,t;
  t=3-(r<<1);
  line(xc, yc+r, xc, yc-r, culoare); point(xc-r, yc, culoare); point(xc+r, yc, culoare);
  for(x=1, y=r; x<y; x++)
  {
    if(t<0)
      t+=6+(x<<2);
    else
     { t+=10+((x-y)<<2); y--;}
   line_simetric(xc,yc,x,y,culoare);//traseaza 4 linii simetrice
  }
 point(xc+x,yc+y,culoare); point(xc+x,yc-y,culoare);
 point(xc-x,yc+y,culoare); point(xc-x,yc-y,culoare);
}
```

De ce linii verticale si nu orizontale?



Decuparea vectorilor

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2018-2019

Decuparea primitivelor grafice

Decuparea: necesara pentru obiectele partial vizibile in imagine

Intr-un sistem grafic 2D:

— Decupare la frontiera unui dreptunghi din spatiul de afisare, avand laturile paralele cu cele ale sistemului de coordonate carteziene 2D (viewport)

— Decupare la frontiera unui poligon oarecare din spatiul de afisare

— Decuparea se poate efectua:

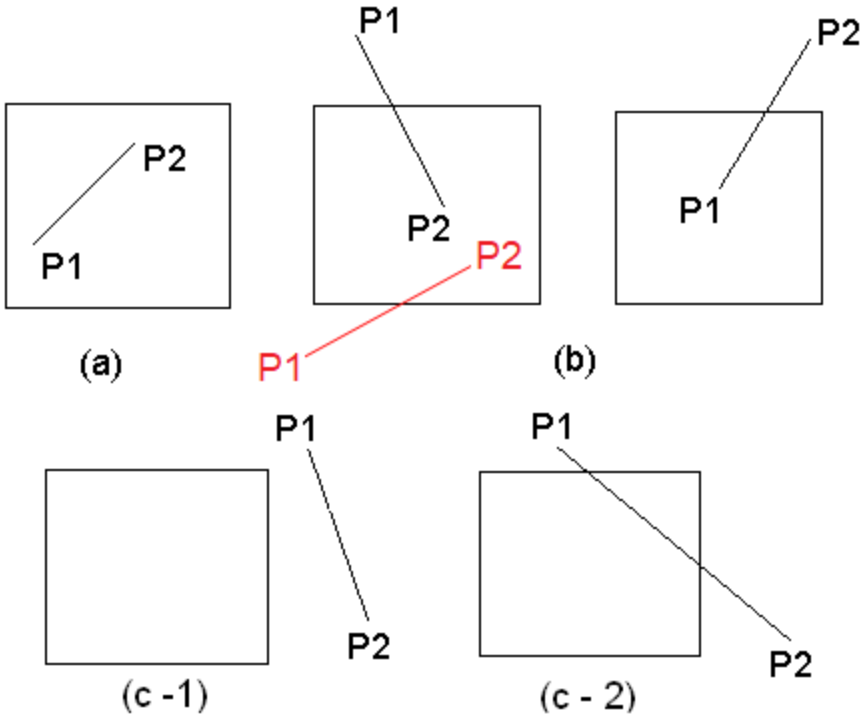
- Analitic, prin intersectia primitivelor grafice cu frontiera zonei de decupare (dreptunghi/poligon oarecare), inainte de rasterizare
- La momentul rasterizarii primitivelor, prin testul de apartenenta a fiecarui fragment la dreptunghiul/poligonul de decupare – de ex. pentru cercuri si elipse

— Pentru linii si poligoane, decuparea se efectueaza analitic (mai eficient).

Intr-un sistem grafic 3D bazat pe OpenGL:

— Decuparea se efectueaza la frontiera volumului vizual canonic, in coordonate de decupare (dupa transformarea de proiectie, inainte de transformarea in poarta de afisare).

Decuparea vectorilor 2D(1)



Pozitia unui vector fata de dreptunghiul de decupare

Exemplu: intersectie P1-P2 cu dreptunghiul neţinând cont de poziţiile punctelor P1, P2 faţă de dreptunghiul de decupare:

1. Calcul intersectie cu latura din stanga
2. Calcul intersectie cu latura de sus
3. Calcul intersectie cu latura din dreapta
4. Calcul intersectie cu latura de jos

1, 2, 3 – calcule inutile

Optimizari: evitarea calculelor de intersectie inutile

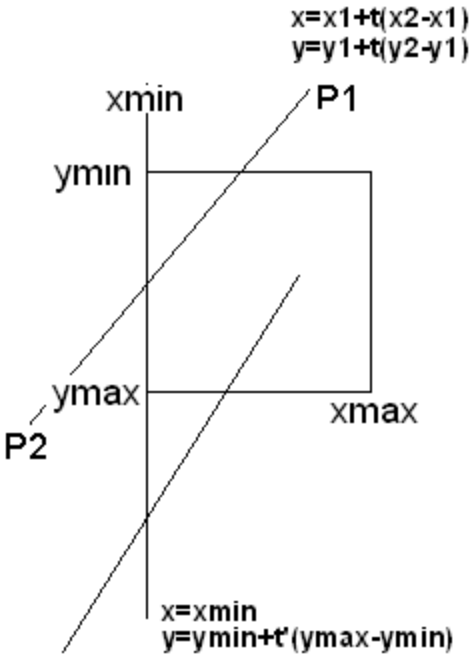
- Evitarea calculelor de intersectie in cazurile de tip (c-1) :

  - cum se diferentiaza cazurile (c-1) de cazurile (c-2) fara a efectua calcule de intersectie?

- Determinarea laturii/laturilor intersectate in cazurile de tipul (b) si (c-2), fara calcule de intersectie.

Decuparea vectorilor 2D(2)

Calculul intersectiilor cu laturile dreptunghiului de decupare



Intersectia segmentului P1-P2 cu latura x = xmin a dreptunghiului

Ec. parametrica a vectorului P1-P2:

P(t) = P1 + (P2-P1)*t 0<= t <=1 intre P1-P2

Punctul de intersectie cu dreapta x=xmin are abscisa xmin:

1) xmin=x1+t(x2-x1) → t=(xmin-x1)/(x2-x1)

2) este 0<=t<=1 ???

nu: punctul de intersectie se afla pe prelungirea segmentului

da: punctul de intersectie se afla pe segmentul P1-P2

- se calculeaza yi=y1+(xmin-x1)*(y2-y1)/(x2-x1)
- este ymin<= yi <= ymax ??

da: exista intersectie intre vectorul P1-P2 si latura x=xmin

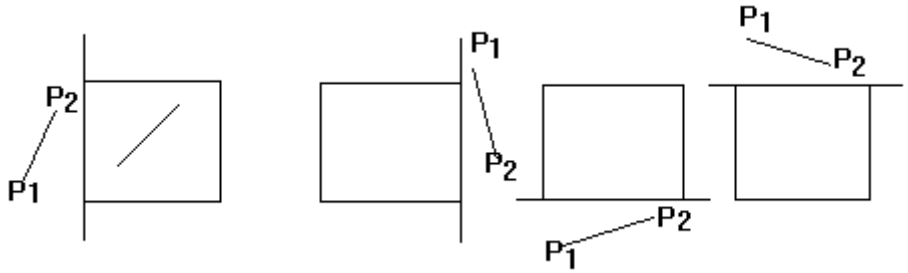
nu: punct de intersectie cu prelungirea laturii – calcul inutil

Algoritmul Cohen-Sutherland (1)

1001	1000	1010
0001	0000	0010
0101	0100	0110

- Punctele din plan sunt codificate prin 4 biti, b3b2b1b0, in functie de pozitia lor fata de dreptunghiul de decupare. De exemplu, se face conventia:
- b0=1: pentru punctele din stanga dreptunghiului,
- b1=1: dreapta, b2=1: sub dreptunghi, b3=1: deasupra dreptunghiului

Fie **cod1** codul binar al punctului P1 si **cod2** codul binar al punctului P2.



daca (cod1 ==0 && cod2==0) segment este acceptat trivial;

altfel

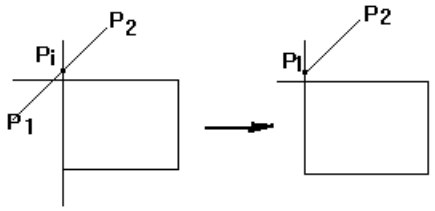
daca ((cod1 & cod2) != 0) segmental este rejectat trivial
(cele 2 coduri au acelasi bit =1)
P1 si P2 de aceeasi parte a dreptunghiului
→ segmentul nu intersecteaza dreptunghiul

altfel: se intersecteaza segmentul P1-P2 cu dreptunghiul de decupare

Algoritmul Cohen-Sutherland (2)

- Se alege latura de intersectie cu segmentul P1-P2 pe baza codului varfului P1, care trebuie sa fie situat in afara dreptunghiului de decupare.
- Daca cod1 = 0 → se interschimba P1 cu P2

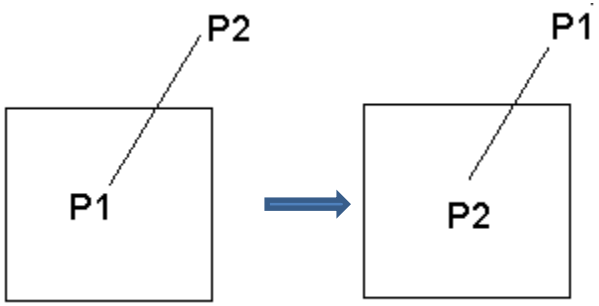
cod1 & 1 != 0
P1 in stanga dreptunghiului



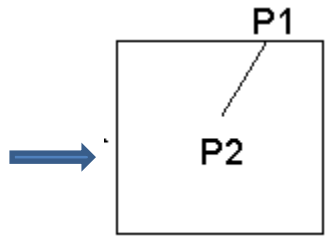
- Se calculeaza intersectia cu x=xmin
- Se muta P1 in punctul de intersectie
- In iteratia urmatoare cod1&cod2 != 0

  → segmentul este rejectat

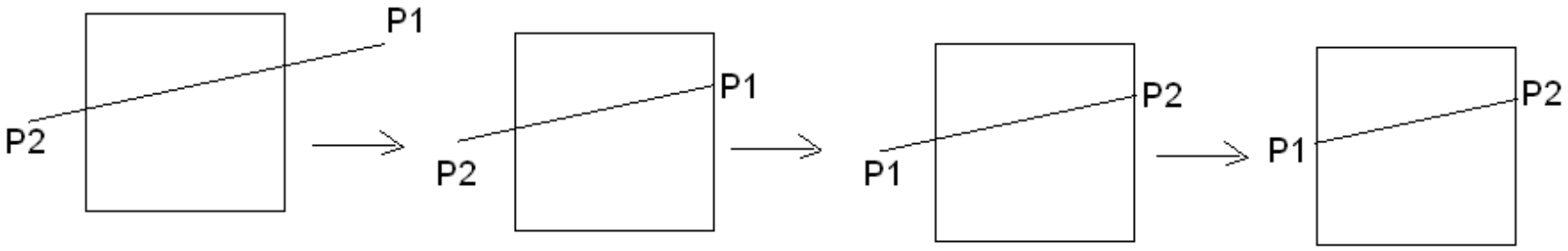
cod1 = 0



cod1 & 8 != 0
P1 deasupra dreptunghiului



Algoritmul Cohen-Sutherland (3)



Calculul punctelor de intersectie

$P(t) = P1 + (P2-P1)*t \rightarrow x(t) = x1 + t(x2-x1);\ y(t) = y1 + t(y2-y1)$

Intersectia cu latura situata pe dreapta y = ymax

ymax=y1+t(y2-y1) de unde, t=(ymax-y1)/(y2-y1)

x=x1+(ymax-y1)*(x2-x1)/(y2-y1) → **xi=x1+(ymax-y1)/m** m: panta segmentului P1-P2

Intersectia cu latura situata pe dreapta x = xmin

xmin=x1+t(x2-x1), deci t=(xmin-x1)/(x2-x1)

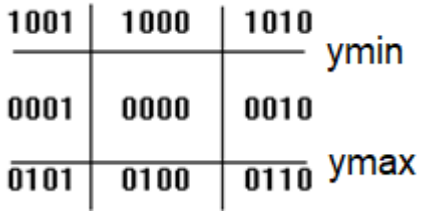
y=y1+(xmin-x1)*(y2-y1)/(x2-x1) → **yi=y1+(xmin-x1)*m**

Algoritmul Cohen-Sutherland (4)

```
int xmin, ymin, xmax, ymax;  // colturile dreptunghiului de decupare

// functia codif calculeaza codul binar al unui punct din plan, pe baza pozitiei sale fata de
     dreptunghiul de decupare

int codif(int x, int y)
{
  int cod =0;
  if(x < xmin)  cod =1;
  if(x > xmax) cod=2;
  if(y > ymax)  cod |=4;  //b2=1
  if(y < ymin) cod |=8;  //b3=1
  return  cod;
}
```



Algoritmul Cohen-Sutherland (5)

```
int Cohen_Suth(int x1i, int y1i, int x2i, int y2i, int* x1d, int* y1d, int* x2d, int* y2d)
{ // primeste coord. capetelor vectorului de decupat si intoarce coord. partii din vector
  // inclusa in dreptunghi, daca exista
  int acceptat,rejectat,vertical;
  float m, x1=x1i, y1=y1i, x2=x2i, y2=y2i;
  int cod1, cod2, t;
  acceptat=rejectat=vertical=0;
  if (x1i != x2i)  //vectorul nu este vertical
      m=(y2-y1)/(x2-x1);
  else
      vertical=1;
 do
   { cod1=codif(x1,y1);  cod2=codif(x2,y2);
     if(cod1==0 && cod2==0) { acceptat=1; break;}
     if((cod1 & cod2) !=0 )  { rejectat=1; break;}
```

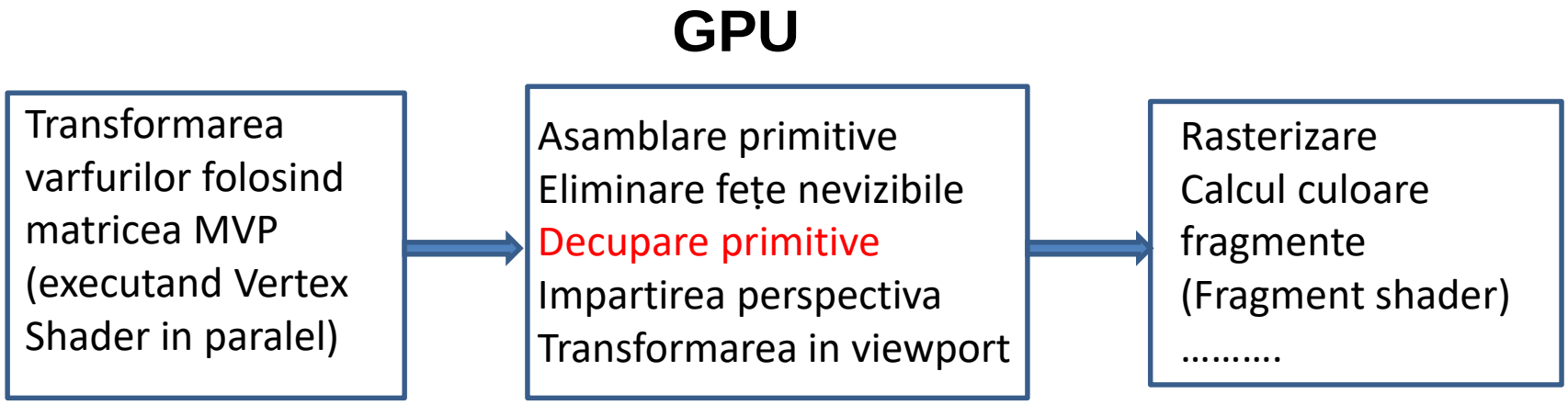
Algoritmul Cohen-Sutherland (6)

```
if(cod1==0) // P1 este in interiorul dreptunghiului de decupare
    {t=x1;x1=x2;x2=t; t=y1;y1=y2;y2=t; cod1=cod2;} //interschimbare capete
if(cod1 & 1)  //P1 in stanga dreptunghiului
     { y1+=(xmin-x1)*m;  x1=xmin;}  // punctul de intersectie devine P1 pentru iteratia urmatoare
  else
   if(cod1 & 2) //P1 in dreapta dreptunghiului
     { y1+=(xmax-x1)*m;  x1=xmax; }
      else
     if(cod1 & 4) // P1 sub dreptunghi
       { if (!vertical)   x1+=(ymax-y1)/m;
         y1=ymax;
       }
     else // P1 deasupra dreptunghiului
       {if (!vertical) x1+=(ymin-y1)/m;
        y1=ymin;
       }
  }   while ( acceptat==rejectat); // cat timp vectorul nu este acceptat sau rejectat
```

Algoritmul Cohen-Sutherland (7)

```
 if(acceptat)
  {*x1d=x1;*y1d=y1;*x2d=x2;*y2d=y2;
   return 1;
  }
  return 0;
 }
}
```

Decuparea în banda grafica (1)



- Decuparea se efectueaza asupra primitivelor 3D, la frontiera volumului vizual canonic
- Din decupare pot rezulta noi varfuri
- Varfurile primitivelor pot avea asociate atribute: culoare, normala, coordonate textura
- Pentru varfurile rezultate din decupare se calculeaza atribute prin interpolarea atributelor asociate varfurilor, folosind valoarea variabilei parametrice (t) în punctul de decupare.

Decuparea în banda grafica (2)

Transformarea de proiectie:

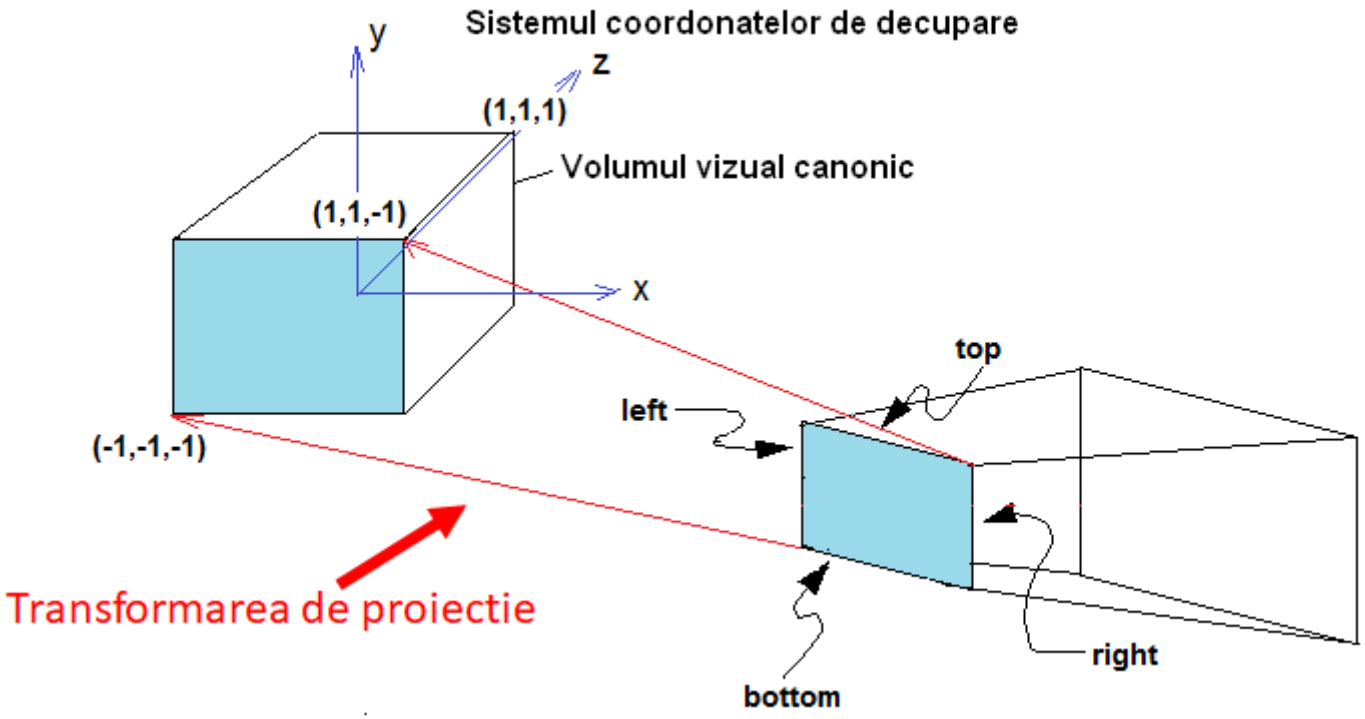
$[xc\ yc\ zc\ w]^T = P \bullet [xe\ ye\ ze\ we]^T$

(clip coordinates) ← (eye coordinates)

Coordonate omogene → împartirea perspectiva → coordonate carteziene:

xd = xc/w , yd = yc/w , zd= zc/w, w = -ze

(xd, yd, zd): coordonate dispozitiv normalizate (**N**ormalized **D**evice **C**oordinates)



Decuparea în banda grafica (3)

Decuparea: înainte sau dupa împarţirea perspectivă?

1. Inainte: decupare in coordonate omogene
2. Dupa: decupare in coordonate carteziene (NDC)

- Problema: împartirea perspectiva

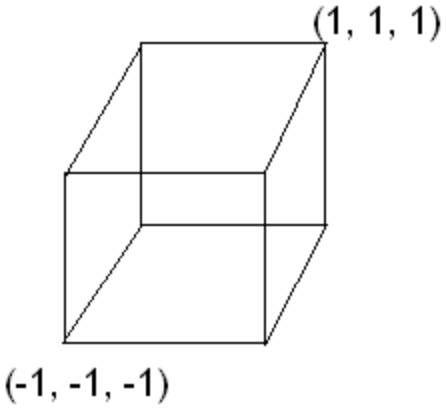
  xd = xc/w , yd = yc/w , zd= zc/w

  w=- ze = 0 pentru proiectia perspectiva daca punctul transformat se afla in acelasi plan cu camera (centrul de proiectie): ze = 0

Problema nu apare in cazul 1, caci varfurile / primitivele din planul camerei sunt eliminate la decupare, ele nefiind incluse in volumul vizual (near>0).

In cazul 2, la conversia in NDC trebuie tratat cazul special w=0.

Generalizarea algoritmului Cohen-Sutherland pentru decuparea vectorilor 3D(1) (decuparea în coordonate carteziene)



Faţă de volumul vizual canonic, un punct din spatiu se poate afla: in interior, in stanga, in dreapta, sub volum, deasupra volumului, in fata, in spate:

→pentru codificarea pozitiei unui punct din spatiu fata de volumul vizual canonic sunt necesari 6 biti.

De ex. se poate face conventia:

b0 = 1 pentru puncte (x,y,z) cu x < -1

b1 = 1 pentru puncte (x,y,z) cu x > 1

b2 = 1 pentru puncte (x,y,z) cu y < -1

b3 = 1 pentru puncte (x,y,z) cu y > 1

b4 = 1 pentru puncte (x,y,z) cu z < -1

b5 = 1 pentru puncte (x,y,z) cu z > 1

daca: cod(P1) ==0 && cod(P2)==0
    segment acceptat trivial

daca (cod(P1) & cod(P2)) != 0
    segment rejectat trivial
altfel
    se intersecteaza segmentul cu volumul

Generalizarea algoritmului Cohen-Sutherland pentru decuparea vectorilor 3D(2)

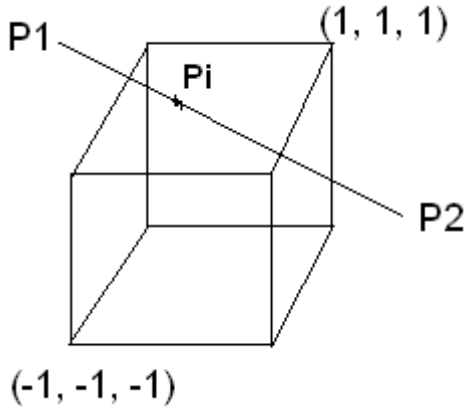
Intersectiile segmentului cu volumul vizual

Ec. parametrice ale segmentului

$x = x1 + t(x2 – x1)$

$y = y1 + t(y2 –y1)$

$z = z1 + t(z2 – z1)$



daca (cod(P1) & 8 !=0) //P1 deasupra volumului

Se efectueaza intersectia cu planul y= 1:

$1 = y1 + t(y2 – y1) \rightarrow tinters = (1-y1)/(y2-y1)$

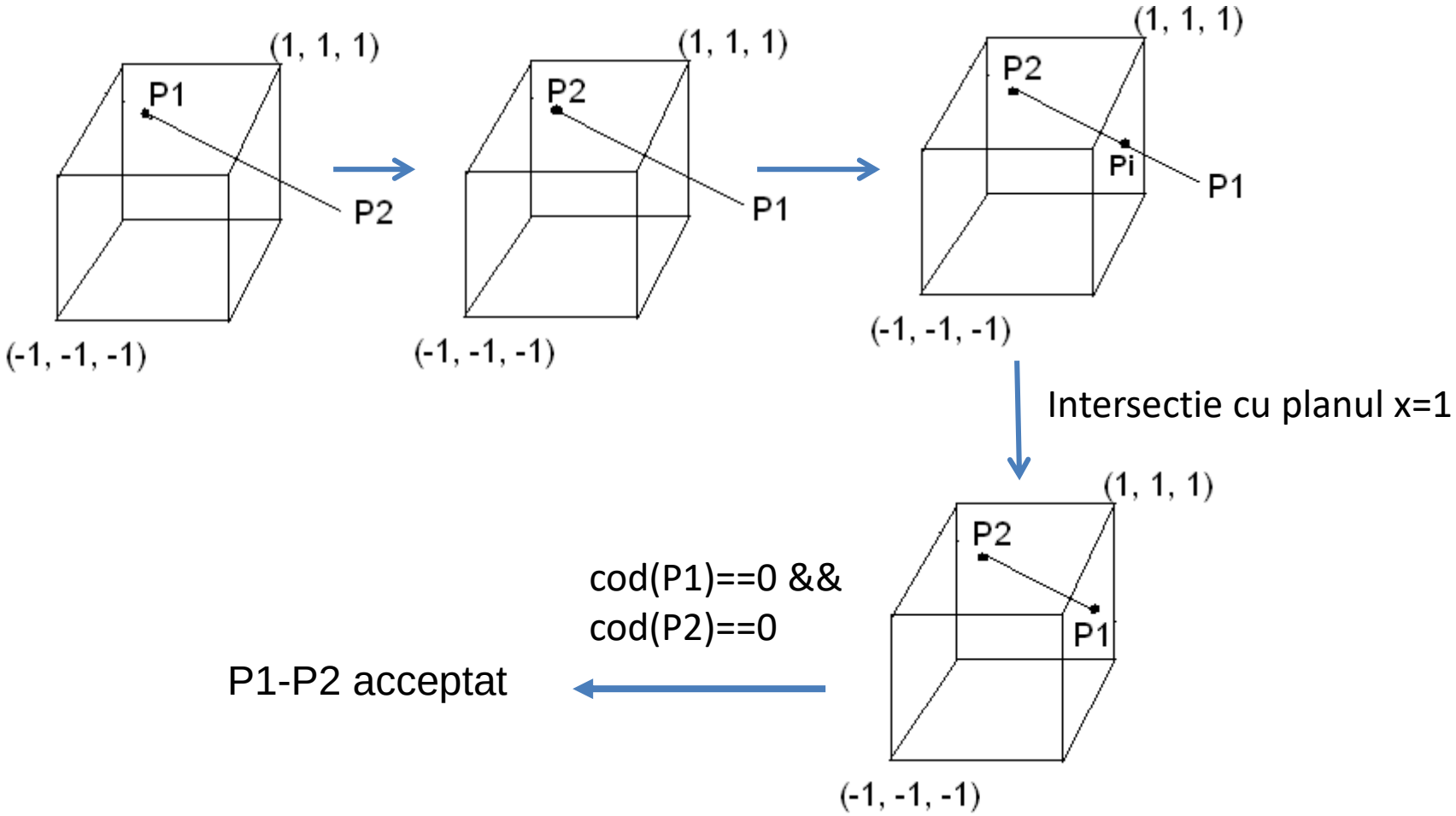
Calculeaza punctul de intersectie si muta P1 in punctul de intersectie

$x1 = x1 + tinters(x2 – x1)$

$y1 = 1;$

$z1 = z1 + tinters(z2 – z1)$

Generalizarea algoritmului Cohen-Sutherland pentru decuparea vectorilor 3D(3)



Algoritmul Liang-Barsky (1)

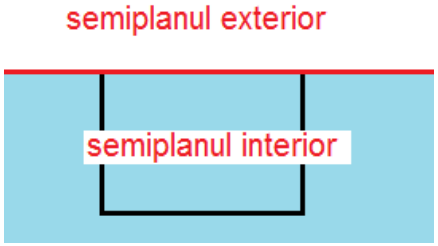
- Cyrus si Beck (1978): **algoritm de decupare parametrica**
  - a unui vector 2D la frontiera unui dreptunghi sau a unui poligon convex oarecare;
  - a unui vector 3D la frontiera unui poliedru convex oarecare
  - mai eficient decat algoritmul Cohen-Sutherland atunci cand majoritatea liniilor intersecteaza dreptunghiul/poliedrul de decupare.
- Liang si Barsky (1984): algoritm de decupare parametrica mai rapid decat algoritmul Cyrus-Beck pentru cazurile in care zona de decupare este un dreptunghi sau un paralelipiped aliniat cu axele sistemului de coordonate carteziene 2D/3D.

Decupare parametrica: se determina punctele de intersectie dintre vector si frontiera zonei de decupare pe baza valorilor variabilei parametrice (t) in punctele de intersectie.

Algoritmul Liang-Barsky (2)

- Fiecare latura a zonei de decupare împarte planul în 2 semiplane:

- semiplanul interior – care contine zona de decupare
- semiplanul exterior



- Fiind dat un segment P1-P2, se va afisa numai acea parte a sa care este conţinută în semiplanul interior faţă de fiecare latură a zonei de decupare.
- Pozitia unui punct fata de o latura a zonei de decupare se determina folosind normala la latura respectiva, orientata înspre exteriorul zonei de decupare.
- Fie P(t) = P1 + (P2 – P1)*t ecuatia vectorului decupat

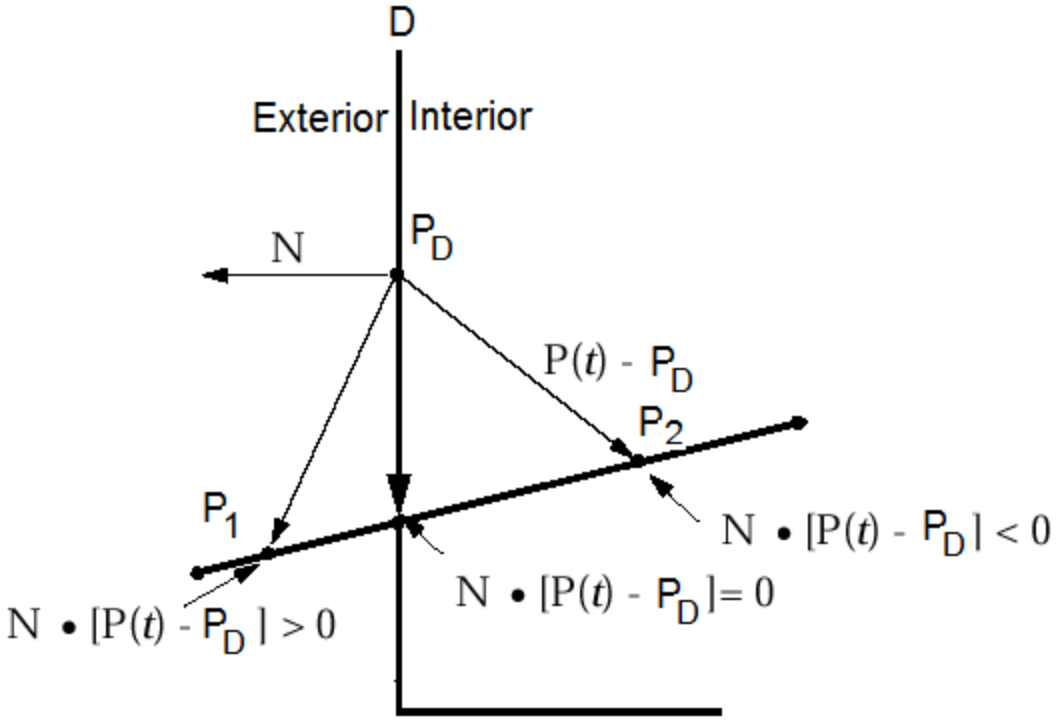
  t=0 → P1 si t=1 → P2

  D – dreapta pe care se afla latura zonei de decupare

  P_D - un punct de pe dreapta D

Algoritmul Liang-Barsky (3)

- Se poate determina pozitia unui punct P(t), de pe vectorul [P2 - P1], fata de zona de decupare examinand valoarea produsului scalar dintre N si vectorul [P(t) - P_D], unde N este normala la latura D a zonei de decupare.



Produsul scalar a 2 vectori:

V1 • V2 = |V1|*|V2|*cos(u)

0<=u<90 → V1 • V2 >0

u=90 → V1 • V2 = 0

90< u <=180 → V1 • V2 <0

Algoritmul Liang-Barsky (4)

- Valoarea lui t în punctul de intersectie cu dreapta D

  $N \bullet [P(t) - P_D] = N \bullet [P1 + (P2 - P1)*t - P_D] = 0$ (vectori perpendiculari)

  $N \bullet [P1 - P_D] + N \bullet [(P2 - P1)*t] = 0$

Rezulta:

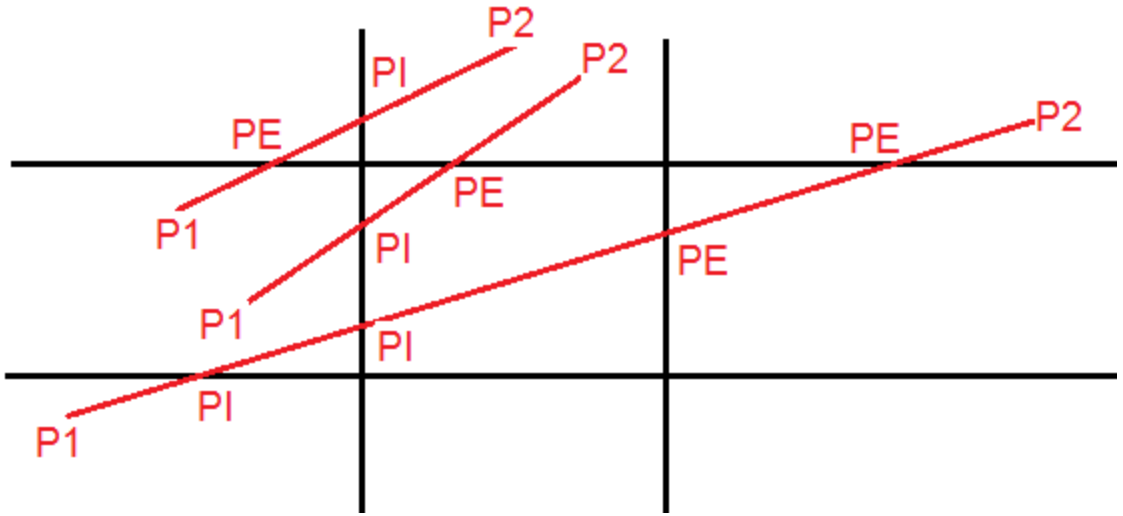
$t = - (N \bullet [P1 - P_D]) / (N \bullet [P2 - P1])$

- Daca $N \bullet [P2 - P1] = 0$ → (P1->P2) este paralel cu latura D → nu intersecteaza D.
- Altfel, se calculeaza valorile t1, t2, t3, t4, corespunzatoare intersectiilor dintre vectorul (P1->P2) si dreptele pe care se afla laturile dreptunghiului de decupare.
- Se retin numai valorile 0 <= ti <= 1 (punctul de intersectie sa fie intre P1 si P2)
- Se verifica pentru fiecare valoare ti retinuta daca punctul de intersectie se afla pe latura dreptunghiului de decupare (si nu pe prelungirea sa)

Algoritmul Liang-Barsky (5)

- Punctul de intersectie dintre vectorul (P1->P2) si o latură a dreptunghiului de decupare este clasificat in:

- Punct potential de intrare in dreptunghiul de decupare, PI, daca N • [P2 – P1] < 0
- Punct potential de iesire din dreptunghiul de decupare, PE, daca N • [P2 – P1] > 0



PI – intrare in semiplanul interior al laturii
PE – iesire din semiplanul interior al laturii

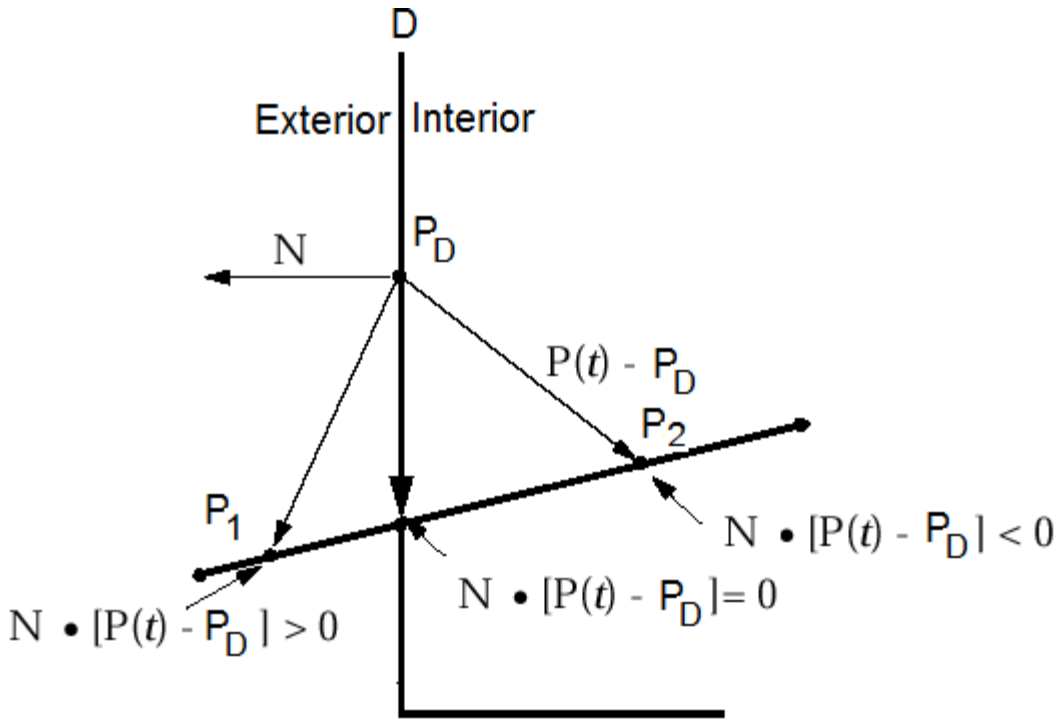
- Partea din vectorul (P1->P2) inclusa in dreptunghiul de decupare este segmentul (P(timax) -> P(temin)), cu timax >= 0 si temin <=1 si timax < temin
- Daca timax > temin → vectorul P1->P2 nu intersecteaza dreptunghiul de decupare.

Algoritmul Liang-Barsky (6)

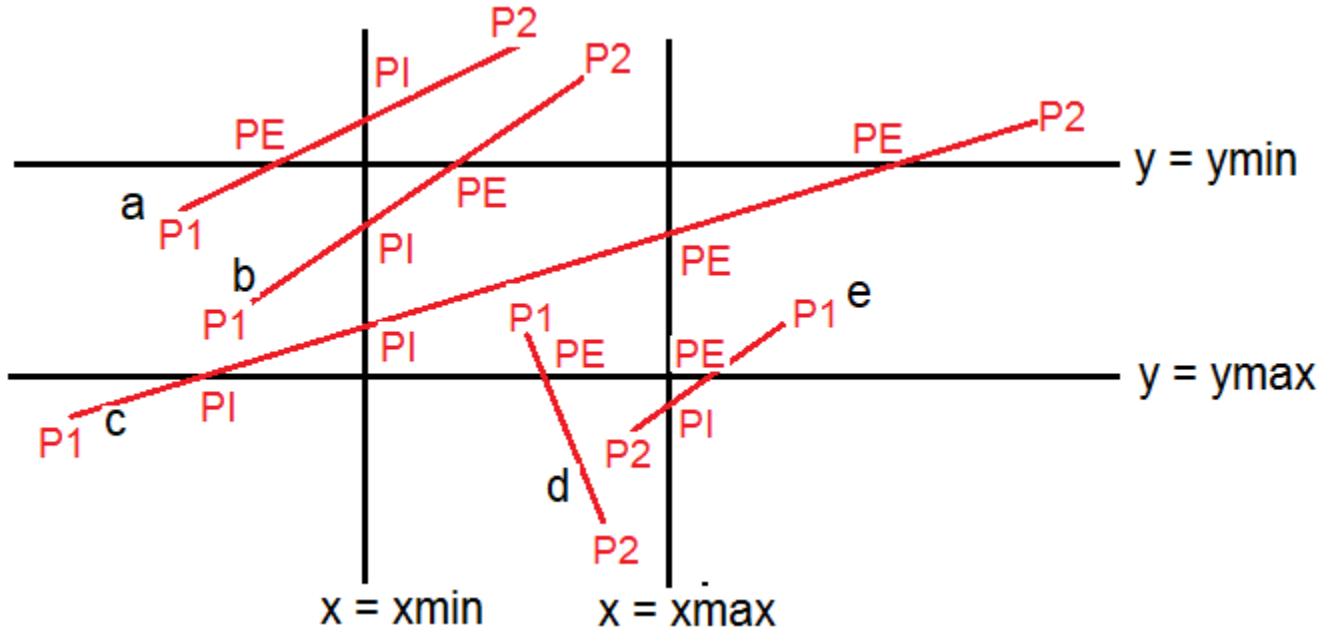
Calculul lui t pentru cele 4 laturi ale dreptunghiului de decupare:

$t = - (N \bullet [P1 – PD]) / (N \bullet [P2 – P1])$

1. D: x= xmin
   N =[-1,0]; N • [P2 – P1] = -(x2-x1) = -dx
   P_D = (xmin, y)
   P1 - P_D = [x1 – xmin, y1 –y]
   t = (xmin – x1)/dx
2. D: x= xmax
   N =[1,0]; N • [P2 – P1] = dx
   P_D = (xmax, y)
   P1 - P_D = [x1 – xmax, y1 –y]
   t = (xmax – x1)/dx
3. D: y= ymin
   N =[0,-1]; N • [P2 – P1] = -dy
   P_D = (x, ymin)
   P1 - P_D = [x1 – x, y1 –ymin]
   t = (ymin – y1)/dy
4. D: y = ymax
   analog cu D: y = ymin



Algoritmul Liang-Barsky (7)



Cazul (a)

- se calculeaza t in intersectia cu x=xmin: ti1; timax = ti1
- se calculeaza t in intersectia cu y = ymin: te1; temin = te1

  timax > temin → segmentul P1->P2 este rejectat trivial

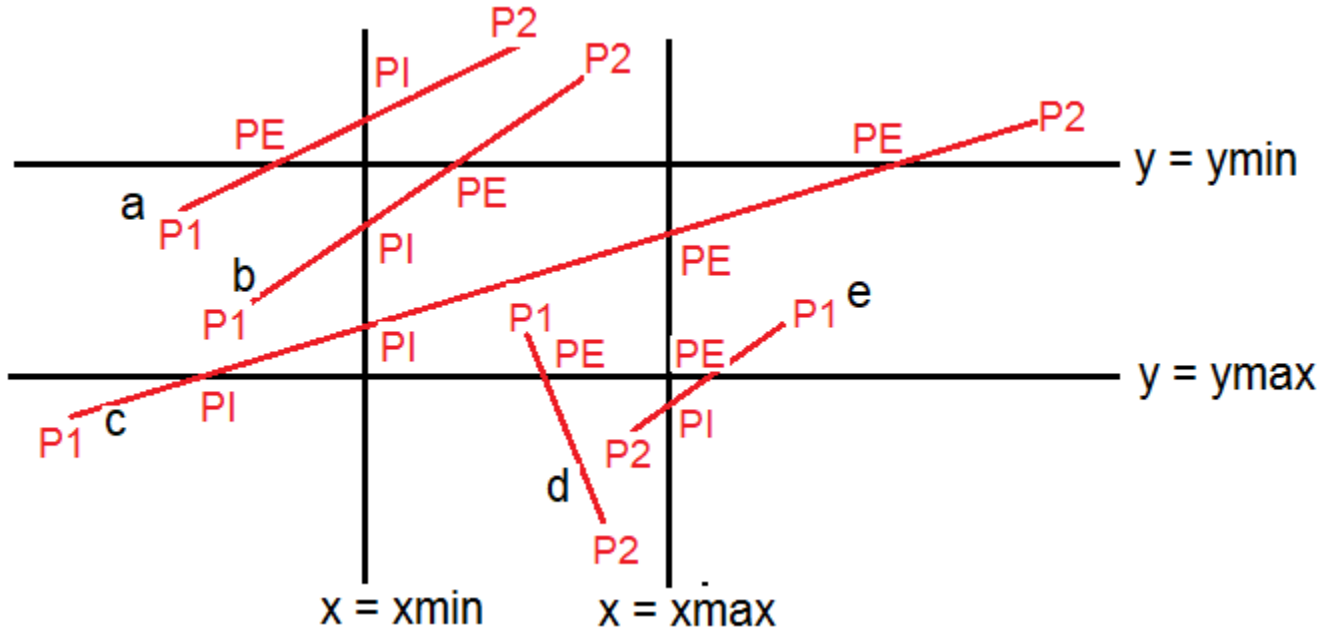
Cazul (b)

- se calculeaza t in intersectia cu x=xmin: ti1; timax = ti1
- se calculeaza t in intersectia cu y = ymin: te1; temin = te1

  timax < temin → se continua calculul lui t in intersectiile cu celelalte laturi
- in intersectia cu y=ymax t<0 iar in intersectia cu x=xmax t>1

  timax<temin →se intoarce segmentul P(timax)->P(temin)

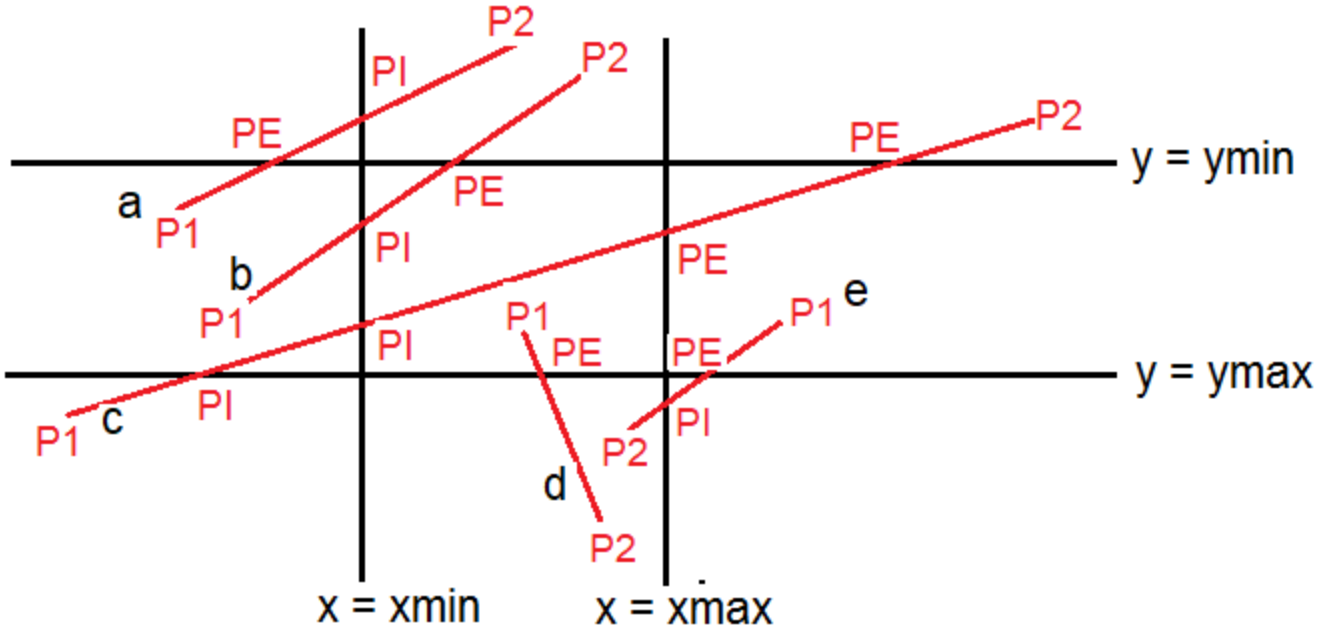
Algoritmul Liang-Barsky (7)



Cazul (c)

- se calculeaza t in intersectia cu x=xmin: ti1; timax = ti1
- se calculeaza t in intersectia cu y = ymin: te1; temin = te1
  timax < temin → se continua calculul lui t in intersectiile cu celelalte laturi
- se calculeaza t in intersectia cu x = xmax: te2; temin = te2
  timax < temin → se continua
- se calculeaza t in intersectia cu y = ymax: ti2; timax = ti1
  timax < temin → se intoarce segmentul P(timax)->P(temin)

Algoritmul Liang-Barsky (7)



Cazul (d)

- se calculeaza t in intersectia cu x=xmin: t<0, nu se retine
- se calculeaza t in intersectia cu y = ymin: t<0, nu se retine
- se calculeaza t in intersectia cu x = xmax: t>1, nu se retine
- se calculeaza t in intersectia cu y = ymax: te1; temin=te1
  → se intoarce segmentul P1->P(temin)

Cazul (e)

- se calculeaza t in intersectia cu x=xmin: t>1, nu se retine
- se calculeaza t in intersectia cu y = ymin: t<0, nu se retine
- se calculeaza t in intersectia cu x = xmax: ti1; timax = ti1
- se calculeaza t in intersectia cu y = ymax: te1; temin=te1
  timax>temin → segmentul P1->P2 este rejectat

Decuparea vectorilor - concluzii

- Algoritmul Cohen – Sutherland este eficient daca:

  - se folosesc operatii foarte simple pentru codificarea capetelor vectorilor si testarea lor

  - majoritatea vectorilor sunt acceptati sau rejectati trivial

- Decuparea parametrica (Algoritmul Liang- Barsky) este avantajoasa atunci cand majoritatea vectorilor traverseaza zona de decupare
- Ca si algoritmul Cohen-Sutherland, se generalizeaza usor pentru decuparea vectorilor 3D la frontiera volumului vizual canonic.

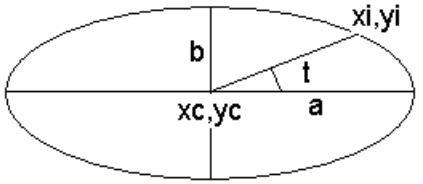
Generarea elipselor în spatiul discret

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2018-2019

Aproximarea elipselor in spatiul discret(1)

- Ecuaţiile parametrice ale elipsei sunt:



$x = xc + a*\cos(t)$

$y = yc + b*\sin(t)$ $\quad 0 <= t <= 2*\pi$

unde (xc,yc) este centrul elipsei, iar a si b sunt marimile semiaxelor elipsei.

- Se poate obţine o secvenţă de puncte care aproximează circumferinţa elipsei dându-i lui t valori de la 0 la 2π cu un pas constant.

- Fie (x_i,y_i) ultimul punct calculat de pe circumferinţa elipsei:

$x_i = xc + a*\cos(t) = xc + a*dx$ $\quad dx=\cos(t)$

$y_i = yc + b*\sin(t) = yc + b*dy$ $\quad dy = \sin(t)$

- Următorul punct, (x_{i+1},y_{i+1}), este distanţat de cel curent cu pasul unghiular:

$x_{i+1} = xc + a*\cos(t+pas) = xc + a*\ dx'$

$y_{i+1} = yc + b*\sin(t+pas) = yc + b*dy'$

$dx'=\cos(t+pas)=\cos(t)*\cos(pas) - \sin(t)*\sin(pas) = dx*c - dy*s$

$dy'=\sin(t+pas)=\cos(t)*\sin(pas) + \sin(t)*\cos(pas) = dx*s + dy*c$

unde c=cos(pas), s=sin(pas) sunt constante

Aproximarea elipselor in spatiul discret(2)

Rezultă:

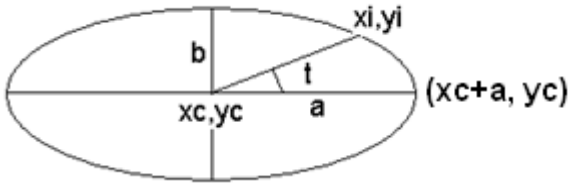
x_{i+1} = xc+a*dx' = xc + a*(dx*c - dy*s)

y_{i+1} = yc+b*dy' = yc + b*(dx*s + dy*c)

// Generarea unei elipse cu axele paralele cu axele sistemului de coordonate carteziene 2D

void Elipsa(int xc, int yc, int a, int b, int N, RGB culoare)

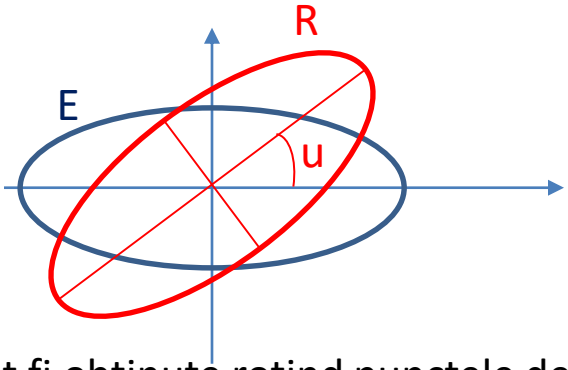
```
{ double c,s,t,dx,dy,u, pas = 2*M_PI/N; // N: numarul de puncte calculate pe circumferinta elipsei
  int x, y, x1, y1 ;
  c=cos(pas); s=sin(pas);
  x= xc +a; y = yc; // dx = cos(0) = 1; dy = sin(0) = 0
  for(i=1, dx=1, dy=0; i<N; i++)
     {   t=dx;
         dx=dx*c - dy*s; // dx'
         dy=t*s + dy*c; //dy'
        x1 = xc +(int)(a*dx+0.5); y1 = yc+(int)(b*dy+0.5);
        line(x, y, x1, y1, culoare);
        x = x1; y = y1;
     } line(x, y, xc+a,yc, culoare);
}
```



Aproximarea elipselor in spatiul discret(3)

Generarea unei elipse rotite

- Fie R, o elipsă cu centrul în originea sistemului de coordonate şi semiaxele a, b rotite cu unghiul u.
- Fie E, elipsa cu centrul în origine şi semiaxele a şi b suprapuse peste axele sistemului de coordonate.



- Punctele de pe elipsa R pot fi obţinute rotind punctele de pe elipsa E în jurul originii cu unghiul u.

- Punctul curent de pe elipsa E:

  $x_i = a*\cos(t) = a*dx$ $\quad dx=\cos(t)$

  $y_i = b*\sin(t) = b*dy$ $\quad dy = \sin(t)$

- Următorul punct, (x_{i+1},y_{i+1}), de pe elipsa E: dx1=a*dx' , dy1=b*dy‘

  $x_{i+1} = a*\cos(t+pas) = a*\ dx' = dx1$

  $y_{i+1} = b*\sin(t+pas) = b*dy' = dy1$

- Următorul punct, (x_{i+1},y_{i+1}), de pe elipsa R se obtine prin rotatia vectorului (dx1,dy1) in jurul originii :

  $x_{i+1} = dxr = dx1*\cos(u) - dy1*\sin(u)$

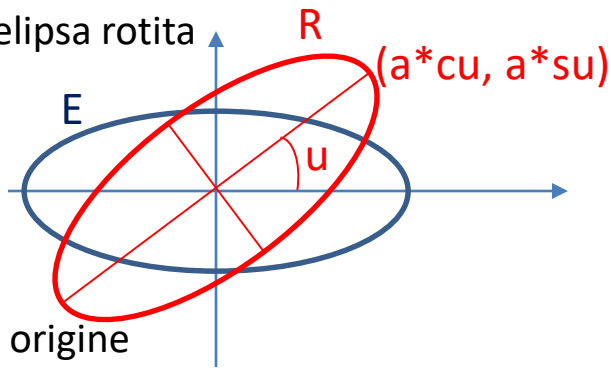
  $y_{i+1} = dyr = dx1*\sin(u) + dy1*\cos(u)$

Aproximarea elipselor in spatiul discret(4)

// Generarea unei elipse cu axele rotite fata de axele sistemului de coordonate carteziene 2D

void Elipsa_rot (int xc, int yc, int a, int b, int N, double u, RGB culoare)

```
{ double c,s,t,dx,dy,dx1,dy1,cu,su, pas = 2*M_PI/N;
  c=cos(pas);s=sin(pas);
  cu=cos(u) ;su=sin(u);
  x= xc+(int)(a*cu+0.5); y= yc+(int)(a*su+0.5); // punctul initial de pe elipsa rotita
  for(i=1, dx=1,dy=0; i< N; i++)
  {  t=dx;
     dx=dx*c-dy*s; //dx'
     dy=t*s+dy*c; // dy'
     dx1=a*dx; dy1=b*dy; // punctul de pe elipsa nerotita cu centrul in origine
     x1  = xc+ (int)(dx1*cu-dy1*su+0.5); y1 = yc + (int)(dx1*su+dy1*cu+0.5);
     line(x, y, x1, y1, culoare);
     x = x1; y = y1;
  }
  line(x, y, xc+(int)(a*cu+0.5),yc+(int)(a*su+0.5), culoare);
}
```

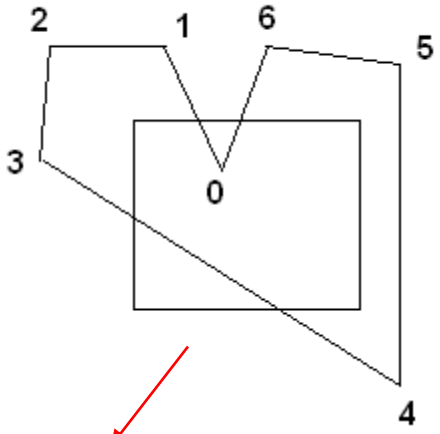


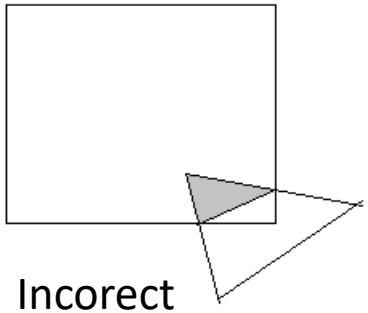
Decuparea poligoanelor

Prof. univ. dr. ing. Florica Moldoveanu

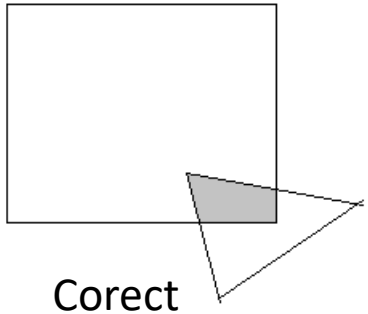
Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2018-2019

Decuparea poligoanelor 2D

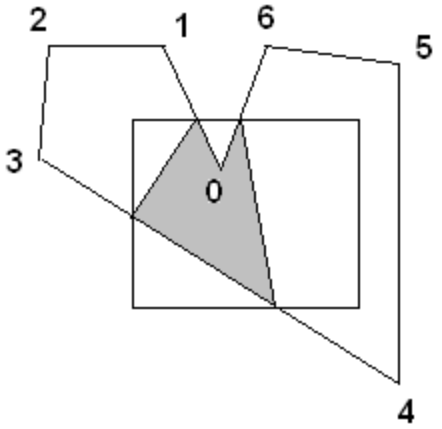


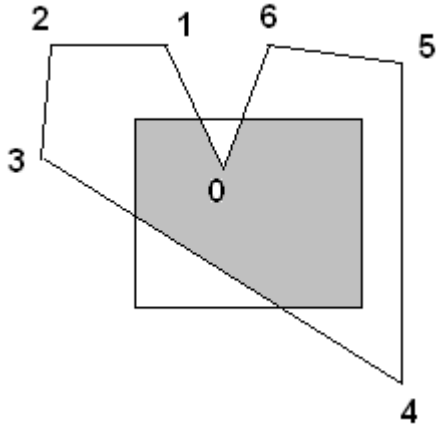


Incorect



Corect



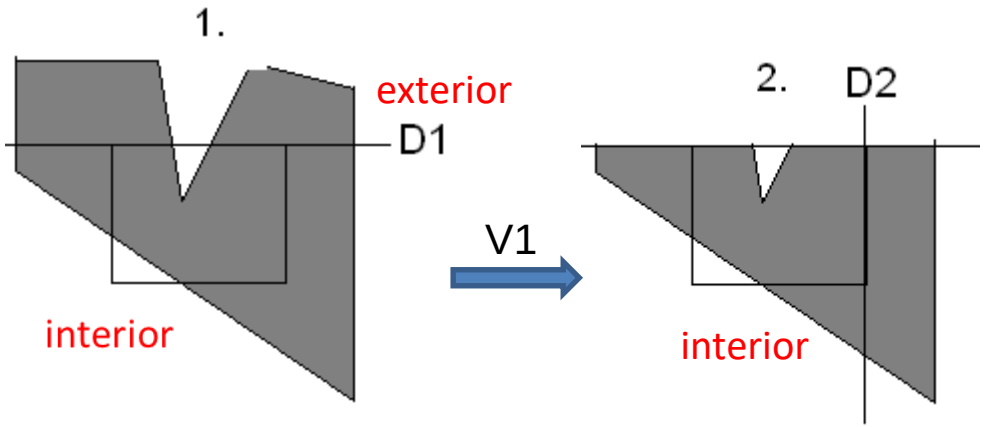


Rezultatul obtinut daca se decupeaza pe rand laturile poligonului folosind algoritmul de decupare vectori.

Rezultatul corect.

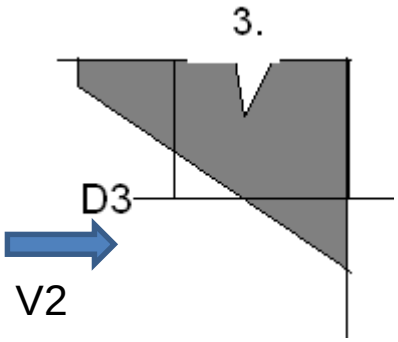
Algoritmul Sutherland-Hodgman(1)

Decuparea poligoanelor convexe față de o zona dreptunghiulară

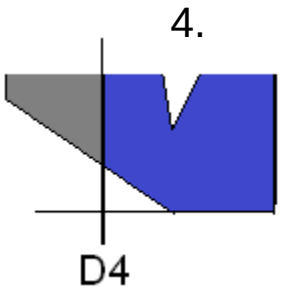


2. Se decupeaza poligonul cu varfurile din lista V1 fata de dreapta D2
   - Se creaza lista varfurilor poligonului rezultat ,V2

1. Se decupeaza laturile poligonului fata de dreapta pe care se afla o latura a zonei de decupare, D1
   - Se creaza lista varfurilor poligonului rezultat (partea din poligonul initial cuprinsa in semiplanul interior), V1



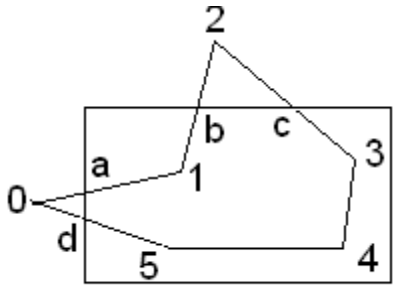
3. Se decupeaza poligonul cu varfurile din lista V2 fata de dreapta D3
   - Se creaza lista varfurilor poligonului rezultat, V3



4. Se decupeaza poligonul cu varfurile din lista V3 fata de dreapta D4
   - Se creaza lista varfurilor poligonului rezultat ,V4

Algoritmul Sutherland-Hodgman(2)

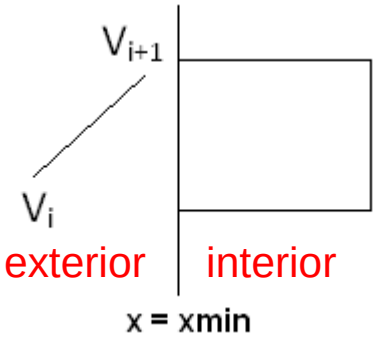
Calculul varfurilor poligonului rezultat intr-o etapa a decuparii



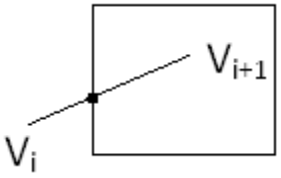
0-1: a, 1 --> a, 1
1-2: 1,b --> a, 1, 1, b
2-3: c,3 --> a, 1, 1, b, c, 3
3-4: 3,4 --> a,1, 1, b, c, 3, 3, 4
4-5: 4,5 --> a,1, 1, b, c, 3, 3, 4, 4, 5
5-0: 5,d --> a,1, 1, b, c, 3, 3, 4, 4, 5, 5, d

Lista corecta: a, 1, b, c, 3, 4, 5, d

Convenție:
- Fiecare latura a poligonului se considera un interval: [V_i -> V_{i+1})
- Numai varful V_i apartine laturii

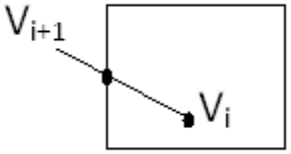


0 varfuri



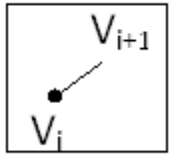
Latura intra in semiplanul interior

un varf



Latura iese din semiplanul interior

2 varfuri

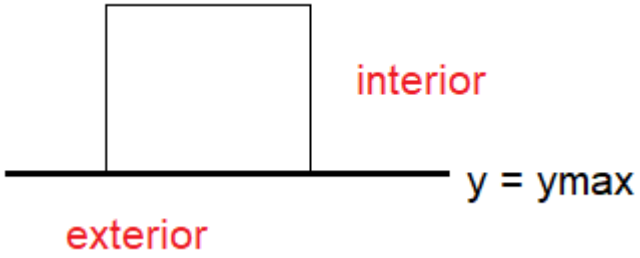


un varf

Algoritmul Sutherland-Hodgman(3)

int DecupLatura(P2D v1, P2D v2, P2d * v1d, P2D *v2d, int latura)

// decupeaza latura v1→v2, fata de dreapta pe care se afla o latura a zonei de decupare




```
{ // functia intoarce numarul de varfuri rezultate din decupare
  switch(latura)
  { case 1: // dreapta y = ymax
      if(v1.y > ymax && v2.y > ymax) return 0; // latura in semiplanul exterior al zonei de decupare
      if(v1.y <= ymax && v2.y <= ymax) { *v1d = v1; return1;} // latura in semiplanul interior
      if(v1.y <= ymax && v2.y > ymax) // latura iese din semiplanul interior
          { * v1d = v1; x_intersectie = …; v2d->x = x_intersectie; v2d->y = ymax; return 2;}
      else// v1.y > ymax && v2.y <= ymax  // latura intra in semiplanul interior
         { x_intersectie = ….; v1d->x = x_intersectie; v1d->y = ymax; return 1;}
    case 2:  …….
  }
}
```

Algoritmul Sutherland-Hodgman(4)

```
void DecupLaturi(int nv, P2D *vrf, int *nd, P2D *vrfd, int latura)
{ // decupeaza toate laturile poligonului fata de dreapta pe care se afla o latura a zonei de
     decupare
 // vrf: lista varfurilor poligonului de decupat
//  nv: numarul de varfuri ale poligonului de decupat
 // vrfd: lista varfurilor poligonului rezultat din decupare
//  nd: numarul de varfuri ale poligonului rezultat din decupare


   *nd = 0;

  for(int i =0; i< nv; i++)

      *nd+=  DecupLatura(vrf[i], vrf[i+1], &vrfd[*nd], &vrfd[*nd+1], latura);

 *nd+= DecupLatura(vrf[nv-1], vrf[0], &vrfd[*nd], &vrfd[*nd + 1], latura);

 }
```

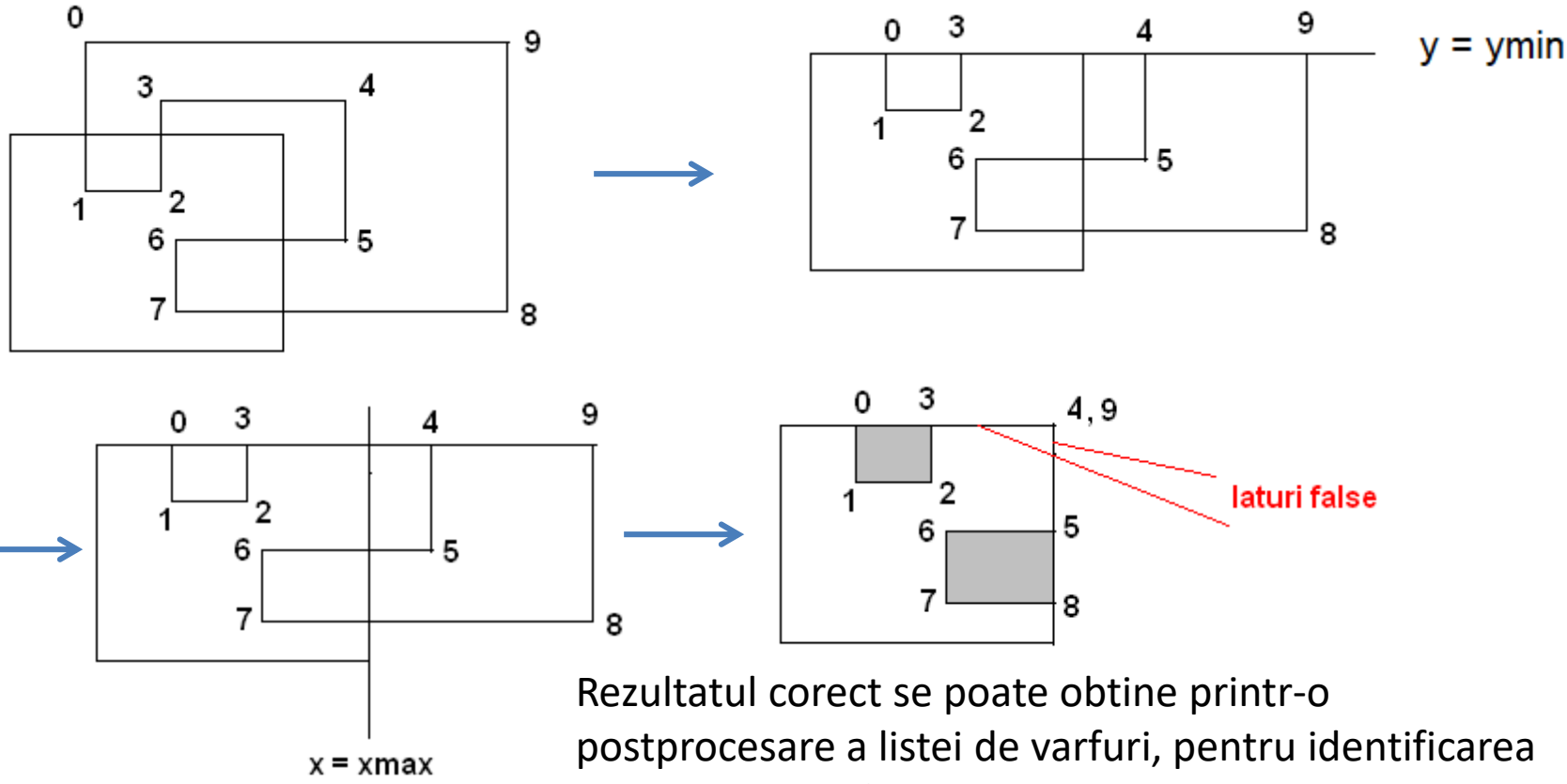
Algoritmul Sutherland-Hodgman(5)

```
void DecupPoligon(int nv, P2D * vrf, int *nd, P2D **vrfd)
{ // vrf: lista varfurilor poligonului de decupat
 // vrfd: lista varfurilor poligonului rezultat din decupare
   P2D * v1 = new P2D[2*nv]; P2D * v2 = new P2D[2*nv]; // se folosesc 2 liste de varfuri
   memcpy(v1, vrf, nv*sizeof(P2D));
  for(int latura =1 ; latura <= 4; latura+ =2) // zona de decupare un dreptunghi
  { DecupLaturi(nv, v1, nd, v2, latura);
    DecupLaturi(*nd, v2, &nv, v1, latura+1);
  }
*nd = nv;
*vrfd = new P2D[nv];
  memcpy(*vrfd, v1, nv*sizeof(P2D));
  delete v1; delete v2;
}
```

In cazul unei zone de decupare poligon oarecare, decuparea se efectueaza faţă de fiecare latura a poligonului.

Algoritmul Sutherland-Hodgman(6)

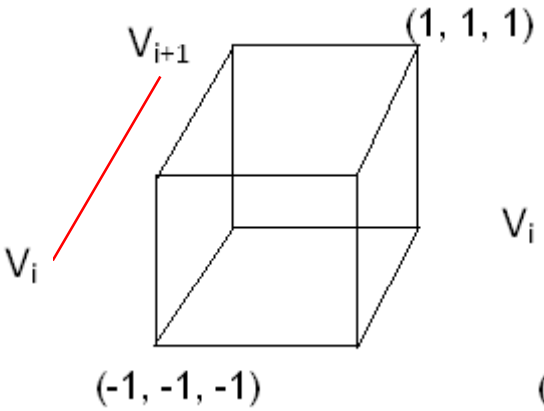
Limitarea algoritmului Sutherland-Hodgman: la decuparea poligoanelor concave, atunci cand din decupare pot rezulta mai multe poligoane. Algoritmul produce o singura lista de varfuri: un singur poligon, cu posibile laturi false.



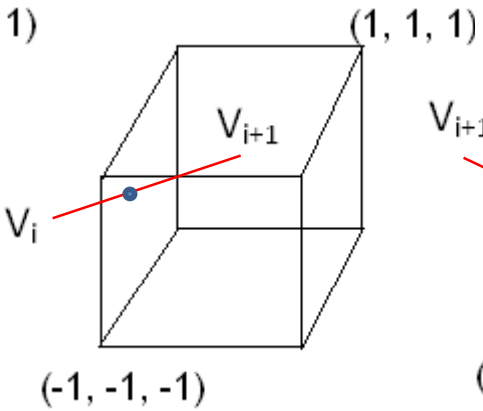
Rezultatul corect se poate obtine printr-o postprocesare a listei de varfuri, pentru identificarea ciclurilor de varfuri.

Decuparea poligoanelor 3D prin algoritmul Sutherland-Hodgman(1)

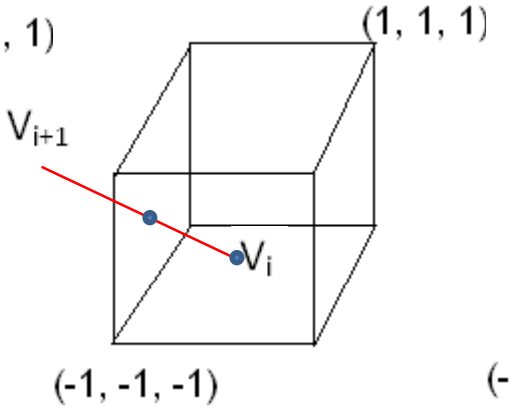
- Decuparea unui poligon 3D se efectueaza la frontiera volumului canonic.
- Se intersecteaza poligonul, pe rand, **cu cele 6 planuri** care delimiteaza volumul.
- Fiecare plan de intersctie imparte spatiul in 2 semispatii:
  - semispatiul interior – care conține volumul
  - semispatiul exterior
- Din fiecare intersectie pot rezulta noi varfuri. Exemplu: intersectia cu planul x=-1



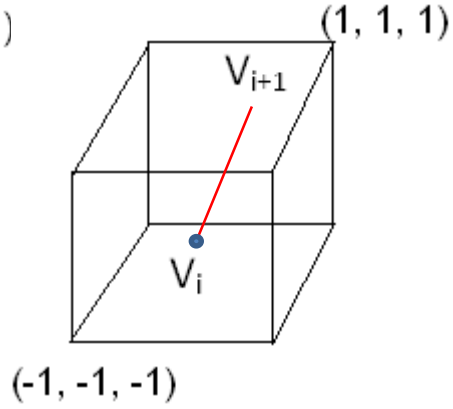
0 varfuri



1 varf: intersectia vectorului cu planul x=-1



2 varfuri: V_i si intersectia vectorului cu planul x=-1



1 varf: varful V_i

Decuparea poligoanelor 3D prin algoritmul Sutherland-Hodgman(2)

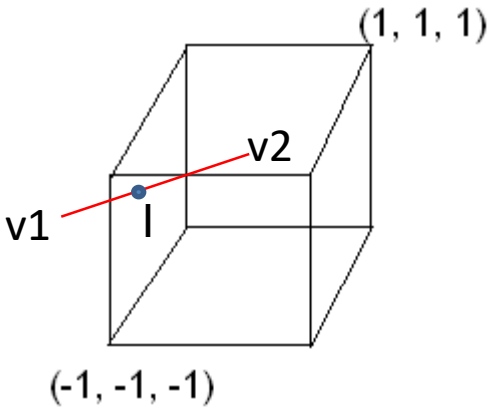
Intersectia unei laturi V1-V2 cu planul x=-1

Ec. parametrice ale laturii

$x = x1 + t(x2 - x1)$

$y = y1 + t(y2 - y1)$

$z = z1 + t(z2 - z1)$



Intersectia cu planul x=-1

$-1 = x1 + t(x2-x1)$ → tinters = $-(x1+1)/(x2-x1)$

$xi = -1$

$yi = y1 + tinters * (y2-y1)$

$zi = z1 + tinters * (z2-z1)$

Decuparea poligoanelor 3D prin algoritmul Sutherland-Hodgman(3)

```
int DecupLatura(P3D v1, P3D v2, P3d * v1d, P3D *v2d, int plan)
// decupeaza latura v1->v2 fata de planul in care se afla o față a volumului de decupare
{ // functia intoarce numarul de varfuri rezultate din decupare
  float tint;
  switch(plan)
  { case 1: // planul x = -1
    if(v1.x < -1 && v2.x < -1) return 0; // latura in semispatiul exterior

    if(v1.x >= -1 && v2.x >= -1 ) { *v1d = v1; return1;} // latura in semispatiul interior

    if(v1.x <-1 && v2.x >= -1) // latura intra in semispatiul interior
    {tint = -(v1.x+1)/(v2.x-v1.x);  v1d->x = -1; v1d->y = v1.y + tint*(v2.y-v1.y); v1d->z= v1.z + tint*(v2.z-v1.z);
    return 1;}
    else // v1.x >= -1 && v2.x  < -1) // latura iese din semispatiul interior
    { * v1d = v1; tint = -(v1.x+1)/(v2.x-v1.x);  v2d->x = -1; v2d->y = v1.y + tint*(v2.y-v1.y);
        v2d->z= v1.z + tint*(v2.z-v1.z) ; return 2;}
   case 2: …….
…… } …….}
```

Decuparea poligoanelor 3D prin algoritmul Sutherland-Hodgman(4)

void DecupLaturi(int nv, P3D *vrf, int *nd, P3D *vrfd, int plan)

```
{ // decupeaza toate laturile poligonului fata de planul in care se afla o față a volumului de
     decupare
 // vrf: lista varfurilor poligonului de decupat
//  nv: numarul de varfuri ale poligonului de decupat
 // vrfd: lista varfurilor poligonului rezultat din decupare
//  nd: numarul de varfuri ale poligonului rezultat din decupare


   *nd = 0;

   for(int i =0; i< nv; i++)

       *nd+=  DecupLatura(vrf[i], vrf[i+1], &vrfd[*nd], &vrfd[*nd+1], plan);

  *nd+= DecupLatura(vrf[nv-1], vrf[0], &vrfd[*nd], &vrfd[*nd + 1], plan);

 }
```

Decuparea poligoanelor 3D prin algoritmul Sutherland-Hodgman(5)

```
void DecupPoligon(int nv, P3D * vrf, int *nd, P3D **vrfd)
{ // vrf: lista varfurilor poligonului de decupat
 // vrfd: lista varfurilor poligonului rezultat din decupare
   P3D * v1 = new P3D[2*nv]; P3D * v2 = new P3D[2*nv]; // se folosesc 2 liste de varfuri

   memcpy(v1, vrf, nv*sizeof(P3D));

  for(int plan =1 ; plan <= 6; plan+ =2) // se decupeaza laturile poligonului fata de cele 6

      //planuri in care se afla fețele volumului de decupare

  { DecupLaturi(nv, v1, nd, v2, plan);

    DecupLaturi(*nd, v2, &nv, v1, plan+1);

  }

 *nd =nv; *vrfd = new P3D[nv];

  memcpy(*vrfd, v1, nv*sizeof(P3D));
  delete v1; delete v2;
}
```

Algoritmul Weiler – Atherton(1)

Decuparea unui poligon oarecare (poligonul subiect) fata de un poligon oarecare (poligonul de decupare).

Notam cu:

PS- poligonul subiect

PD-poligonul de decupare

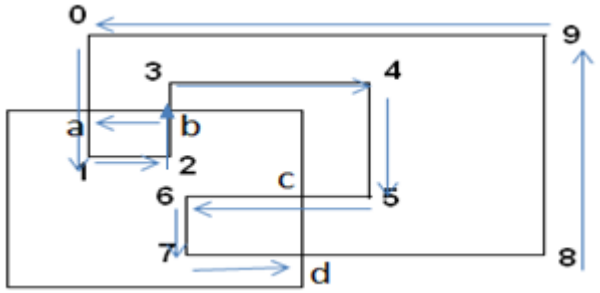
PR- poligonul rezultat – acesta poate fi alcatuit din mai multe cicluri de varfuri (poligoane)

LS – lista de varfuri a PS

LV - lista de varfuri a PR

Vcurent: varful curent din LS (indicele) –

din care începe/se continua parcurgerea conturului PS



- Se parcurg alternativ conturul PS si cel al PD, pastrand sensul de parcurgere (trigonometric).

Cicluri de varfuri rezultate din decupare:

(a, 1, 2, b, a); (c, 6, 7, d, c)

Algoritmul Weiler – Atherton(2)

Se calculeaza punctele de intersectie dintre PS si PD si se clasifica in: **puncte de intrare in PD si puncte de iesire din PD.**

Se initializeaza LV. Se alege un varf al PS exterior PD. Acesta este Vstart (varful din care incepe parcurgerea conturului poligonului PS).

Vcurent ← Vstart; gata = false.

repeta

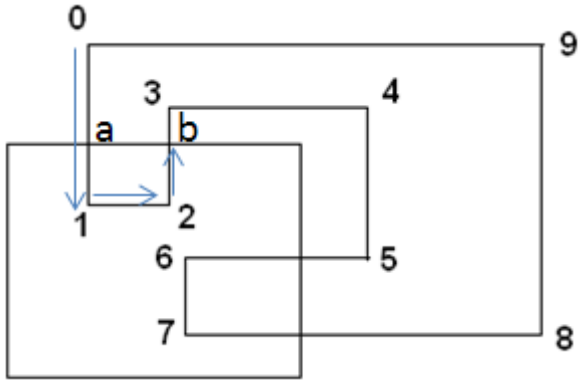
*Se parcurge conturul poligonului PS incepand din Vcurent,
pana cand se ajunge in Vstart
sau intr-un punct Pi de intrare in PD.

daca Vcurent == Vstart

gata = true; break;

altfel

1. Se adauga Pi in LV si se continua parcurgerea pe conturul PS adaugand la LV varfurile PS intalnite pe parcurs, pana intr-un punct ,Pe, de iesire din PD. Se adauga Pe la LV.



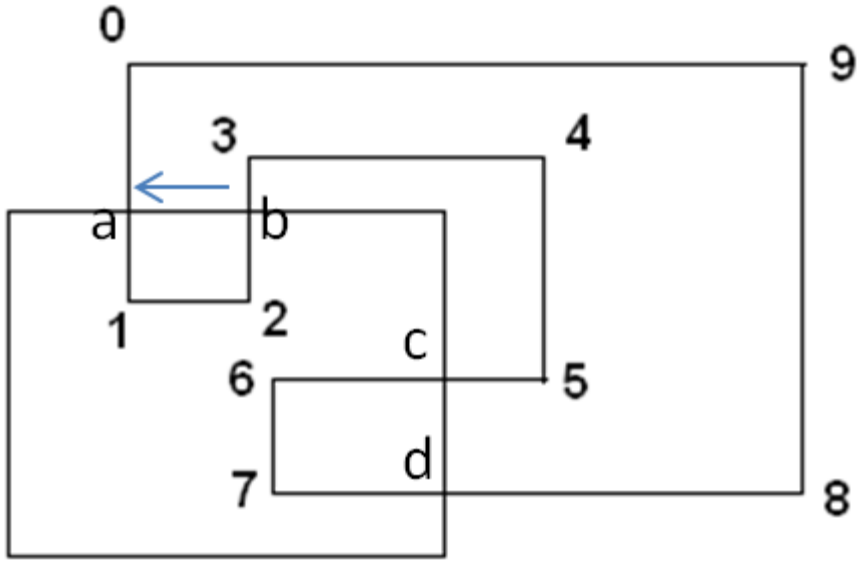
LV: a, 1, 2, b

Algoritmul Weiler – Atherton(3)

2. Vcurent ← vârful exterior al laturii PS care intersecteaza PD in Pe. // vrf. 3

3. Se continua parcurgerea pe conturul PD din Pe pana cand se ajunge intr-un punct, Pi, de intersectie cu PS. Se adauga in LV varfurile PD intalnite pe parcurs si punctul Pi.

4. Daca s-a format un ciclu de varfuri, se memoreaza ciclul de varfuri si se initializeaza LV.
altfel, Vcurent ← Pi (se inlocuieste in LS vrf curent cu Pi).



LV: a, 1, 2, b, a

↓

Ciclul de varfuri c1: a, 1, 2, b, a

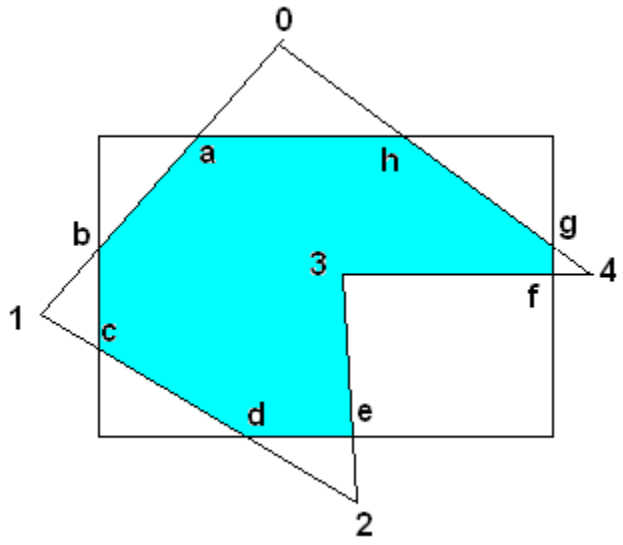
↓

LV: vida

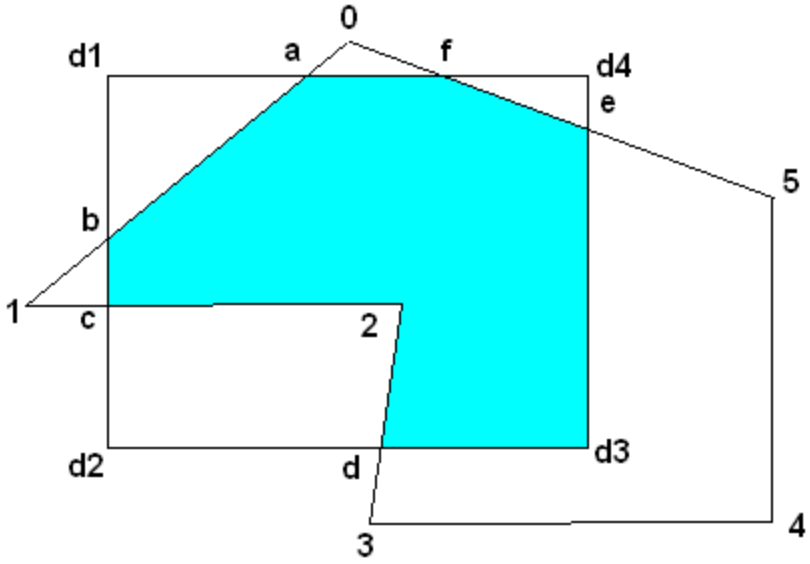
pana(gata)

In iteratiile urmatoare:
-se continua parcurgerea pe PS din varful 3 si se formeaza ciclul de varfuri c2: c, 6, 7, d, c
-se continua parcurgerea pe PS din vârful 8 pana se ajunge in Vstart.

Algoritmul Weiler – Atherton(4)



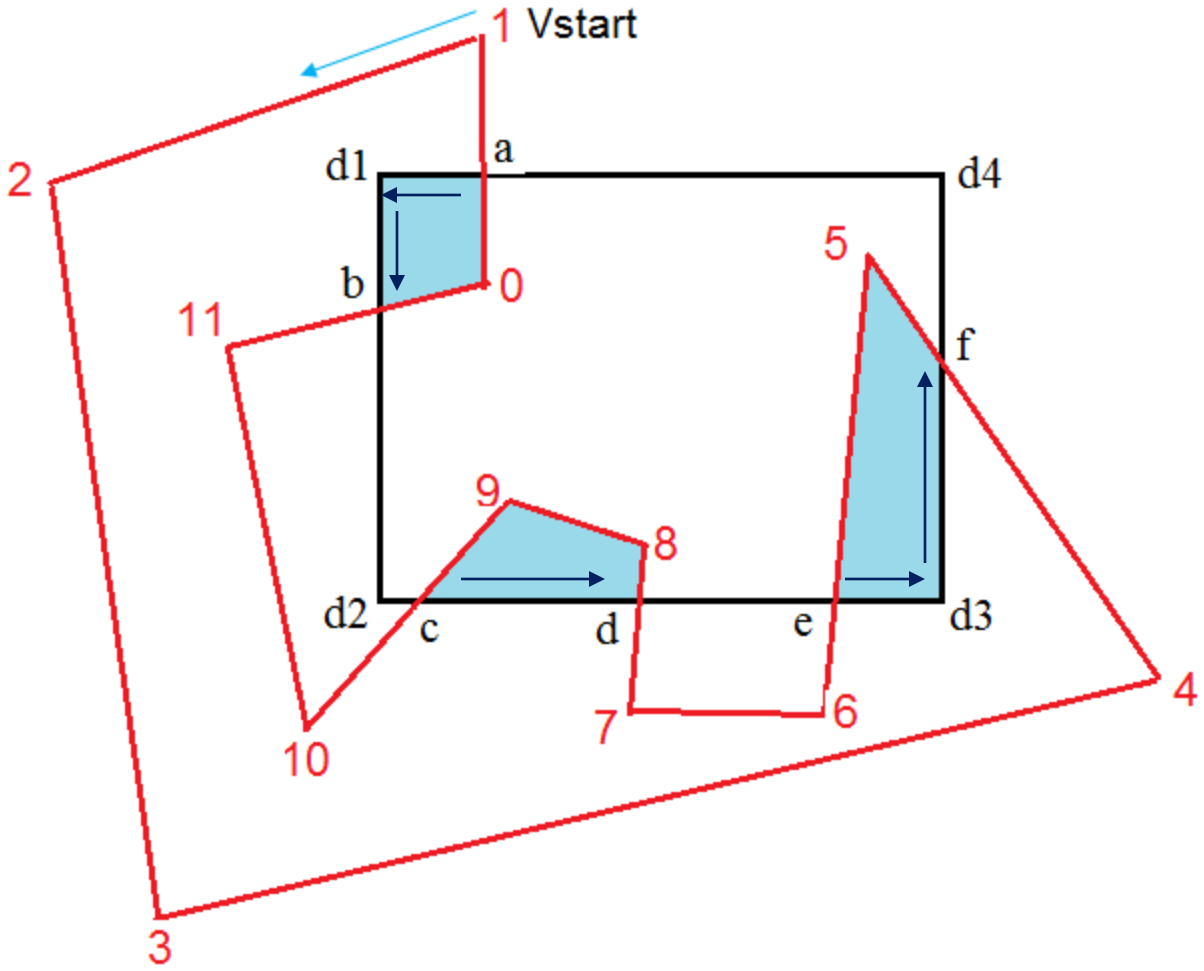
Se porneste din varful 0
LV: a, b (Vcurent ← 1)
a, b, c (Vcurent ← c: se inlocuieste in LS vrf 1 cu c)
a, b, c, d (Vcurent ← 2)
a, b, c, d, e (Vcurent ← e: se inlocuieste in LS vrf 2 cu e)
a, b, c, d, e, 3, f (Vcurent ← 4)
a, b, c, d, e, 3, f, g (Vcurent ← g: se inlocuieste in LS vrf 4 cu g)
a, b, c, d, e, 3, f, g, h (Vcurent ← 0)
a, b, c, d, e, 3, f, g, h, a: s-a format un ciclu de laturi
LV: vida
Se reia din Vcurent: Vcurent == Vstart



Se porneste din varful 0

LV: a, b (Vcurent ← 1)
a, b, c (se inlocuieste in LS vrf. 1 cu c)
a, b, c, 2, d, d3, e (se inlocuieste in LS vrf. 3 cu e)
a, b, c, 2, d, d3, e, f, a

Algoritmul Weiler – Atherton(5)



Vstart ← 1
LV: f,5,e (Vcurent ← 6)
LV: f,5,e,d3, f → s-a format un ciclu de varfuri:
c1: f,5,e,d3, f
- se initializeaza LV
LV: d,8,9,c (Vcurent ← 10)
LV: d,8,9,c,d → s-a format un ciclu de varfuri:
c2: d,8,9,c,d
- se initializeaza LV

LV: b,0,a (Vcurent ←1)
LV: b,0,a,d1,b → s-a format un ciclu de varfuri:
c3: b,0,a,d1,b
- se initializeaza LV
Vcurent == Vstart